

Hailo Media Library User Guide

Release 1.11.0
30 March 2026

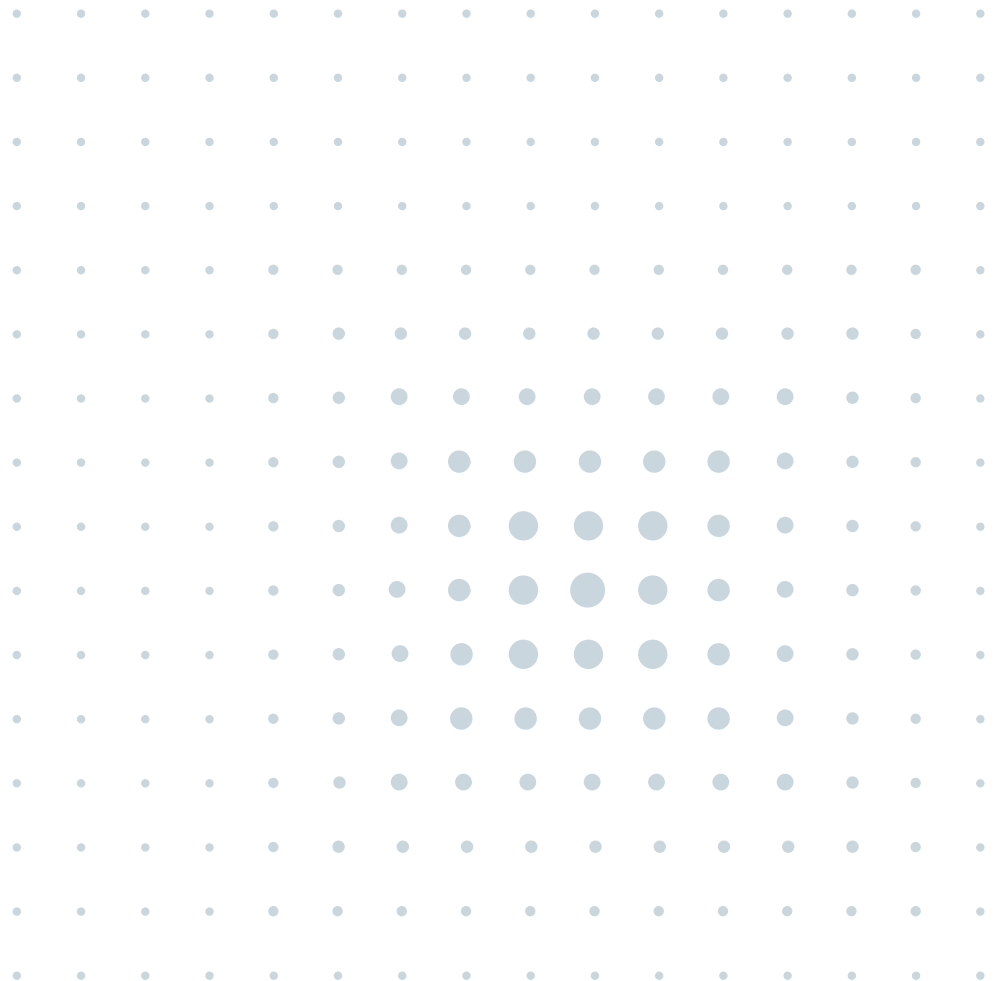


Table of Contents

1	Getting Started with Hailo Media Library	2
1.1	Prerequisites	2
1.2	Hailo Media Library Package Content	2
1.3	Hailo Media Library Overview	2
1.4	Hailo Media Library APIs	4
1.5	AI Vision Reference Code	4
1.6	Supported Image Sensors	8
2	Configurations	9
2.1	Media Library Configurations	9
2.2	Profile Configurations	10
2.3	Frontend Configurations	23
2.4	Encoder-OSD Configurations	35
2.5	Tuning Encoder	44
3	GStreamer Elements	52
3.1	Hailo Frontendbinsrc	52
3.2	Hailo Encoder	57
4	GStreamer API Elements	61
4.1	Overview	61
4.2	Architecture	61
4.3	Reference Code Examples	61
5	Changelog	65
6	Cross-Compiling with Meson	72
6.1	Prerequisites	72
6.2	Compiling Hailo Media Library	72
6.3	Compiling Hailo AI Analytics	73
6.4	Compiling Hailo Postprocess	73
6.5	Troubleshooting	74
7	Using The Encoder	75
7.1	Introduction	75
7.2	Encoder Workflow (Low Level)	75
7.3	Encoder Limitations	77
7.4	Initialization and Release of the Encoder	77
7.5	Configuration of the Encoder	79
7.6	Stream Production	83
7.7	Interlaced Coding	84
7.8	Video Pre-processor Usage	85
7.9	Video Frame Storage Format	86
7.10	GOP Structure Configuration	90
7.11	Encode H264	92
8	Media Library C++ API Reference	95
8.1	Buffer Pool	95
8.2	Electronic Image Stabilization (EIS)	97
8.3	MediaLibrary	101
8.4	Frontend	112
8.5	Encoder	116

8.6	OSD - On Screen Display	123
8.7	Media Library Types	136
8.8	Media Library Logger	167
8.9	Digital Image Stabilization	169
8.10	Privacy Mask	170
8.11	Motion Detection	178
8.12	Analytics Database	181
8.13	Snapshot Feature	186
8.14	Sensor Registry	190
9	Low Level API Reference	197
9.1	DSP Utils	197
9.2	Encoder (low level) API-Manual	201
10	AI Analytics API	245
10.1	Buffer	245
10.2	Queue	253
10.3	Stage	259
10.4	Pipeline	273
10.5	PipelineBuilder	283
10.6	Available Stages	290
11	Reference Pipelines and Applications	342
11.1	Analytic Draw Client	342
11.2	Single Stream Processing Reference Application	345
11.3	Single Stream Object Detection Reference Application	348
11.4	Face Landmarks Reference Application	351
11.5	Dynamic Privacy Mask Case Study	354
11.6	Dual Sensor Processing Reference Application	357
11.7	Dual Sensor Face Landmarks Application	360
11.8	GStreamer Vision Analytics Reference Application	363
12	Debugging and Visualization	367
12.1	Pipeline Viewer	367

Disclaimer and Proprietary Information Notice

Copyright

© 2026 Hailo Technologies Ltd ("Hailo"). All Rights Reserved.

No part of this document may be reproduced or transmitted in any form without the expressed, written permission of Hailo. Nothing contained in this document should be construed as granting any license or right to use proprietary information for that matter, without the written permission of Hailo.

This version of the document supersedes all previous versions.

General Notice

Hailo, to the fullest extent permitted by law, provides this document "as-is" and disclaims all warranties, either express or implied, statutory or otherwise, including but not limited to the implied warranties of merchantability, non-infringement of third parties' rights, and fitness for particular purpose.

Although Hailo used reasonable efforts to ensure the accuracy of the content of this document, it is possible that this document may contain technical inaccuracies or other errors. Hailo assumes no liability for any error in this document, and for damages, whether direct, indirect, incidental, consequential or otherwise, that may result from such error, including, but not limited to loss of data or profits.

The content in this document is subject to change without prior notice and Hailo reserves the right to make changes to content of this document without providing a notification to its users.

1. Getting Started with Hailo Media Library

The Hailo-15™ is a family of AI Vision Processor Units (VPU) implemented as System-on-Chip (SoC) solutions, purpose-built for next-generation smart cameras. It combines Hailo's patented, field-proven AI inference architecture with advanced computer vision pipelines, delivering superior image quality and enabling real-time, high-performance video analytics. To expose and control these capabilities, Hailo provides the Hailo Media Library — a dedicated software package that interfaces directly with the Hailo-15™ vision subsystem, offering unified and fine-grained control over video capture, video encoding, on-screen display (OSD), and image correction and enhancement. This enables developers to dynamically configure camera behavior and optimize performance across diverse applications.

1.1. Prerequisites

Before starting, ensure the following are available:

- Hailo-15™ SBC (or another board featuring the Hailo-15™) flashed with the latest Hailo-15™ software
- A host machine with an internet connection
- Git and GStreamer installed on the host machine

1.2. Hailo Media Library Package Content

This software package includes the following components:

- Media Library – for interfacing with Hailo-15 resources from C++ applications
- GStreamer plugin - for interfacing with Hailo-15 resources
- Additional utilities and configurations files
- AI vision and AI Analytics reference code examples. Refer to sections 1.5 and 9
- Reference pipelines. Refer to sections 10
- Reference applications. Refer to section 10

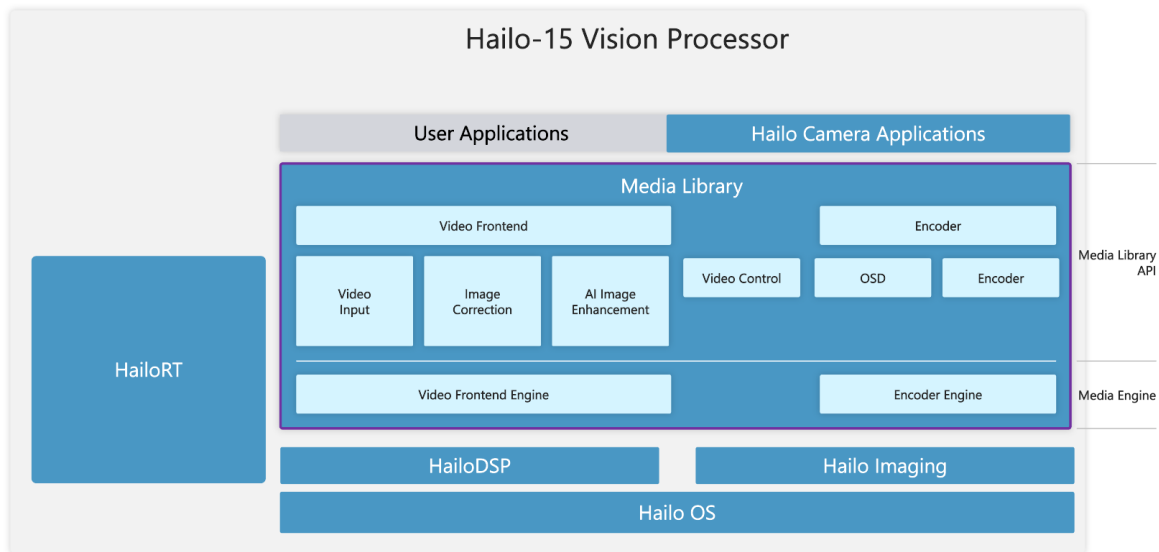
1.3. Hailo Media Library Overview

Note: Media Library is only supported on Hailo-15.

The Hailo Media Library provides unified media control and handling for the Hailo-15™ VPU, exposing an integrated API that manages the Vision subsystem, including video capture, video encoding, on-screen display (OSD), and image correction and enhancement features. It serves as an extension to native media interfaces, allowing developers to continue using v4l2, HailoRT, and GStreamer, while providing a simpler, consolidated interface that centralizes control of the Hailo-15™ vision capabilities in one place.

In addition, the Media Library includes analytics APIs that simplify the use of HailoRT for AI inference and vision-based analytics.

The Hailo Media Library includes two components: Hailo Media Library Frontend ("hailofrontend") and Media Library Encoder-OSD ("hailoencodebin").



1.3.1. Media Library Frontend (hailofrontend)

Provides advanced video preprocessing and image enhancement through a developer-friendly C++ API:

- Advanced image correction: Lens shading, distortion correction, defog, and other image enhancements.
- Digital and optical zoom: Precise zoom control for one or multiple streams.
- Stream duplication and scaling: Flexible stream routing and resizing.
- Low-light enhancement: AI-assisted denoising and exposure optimization.
- Neural network-based image enhancement (future): AI-driven IQ improvements.
- 3A and IQ control: managing auto-exposure, auto-focus, auto-white balance, and other image quality parameters.

Warning: When using Media Library Front-end, Multi-process is not supported

1.3.2. Media Library Encoder & OSD (hailoencodebin)

Handles video encoding and on-screen display (OSD) overlays via an integrated C++ API:

- Stream encoding: Single or multiple streams, configurable resolution and bit depth.
- Supported formats: H.264 (AVC), H.265 (HEVC), JPEG.
- Dynamic OSD overlays: Add text, images, timestamps, or graphics in real time.
- Integration with hailofrontend: Works seamlessly with AI-enhanced streams for encoding and display.

1.4. Hailo Media Library APIs

The Hailo Media Library is available in both GStreamer API and C++ API, offering the same functionality and control in each. This flexibility allows customers to choose the API that best fits their existing infrastructure and development preferences.

1.4.1. GStreamer API

GStreamer is a powerful multimedia framework used to build complex media processing pipelines. It supports a wide range of multimedia formats and operations, including audio and video processing, streaming, and more. It is suitable for customers who:

- Are already familiar with or have existing projects using GStreamer.
- Prefer to leverage the multimedia capabilities and extensive plugin ecosystem provided by GStreamer.
- Wish to integrate Hailo's AI vision processing capabilities seamlessly into their existing GStreamer-based media applications.

1.4.2. C++ API

The C++ API provides direct access to the Hailo Media Library, allowing developers to integrate Hailo's AI vision processing capabilities into their applications. It is suitable for customers who:

- Are experienced with C++ and prefer to use it for their application development.
- Have existing C++ codebases and wish to integrate Hailo's functionality.
- Require the performance and control provided by a compiled language like C++.

1.4.3. Choosing the Right API

Both APIs offer the same level of control and functionality, so the decision should be based on which API aligns better with your team's expertise and your project's architecture.

When deciding between the GStreamer API and the C++ API, consider the following factors:

- **Existing Infrastructure:** If your project or organization already utilizes GStreamer, choosing the GStreamer API can streamline integration and development.
- **Development Preference:** If your development team is more comfortable with C++, the C++ API might be the more natural choice.
- **Community and Plugins:** GStreamer has a wide community and is considered one of the standards for media pipelines, offering a vast array of pre-existing plugins that can enhance and simplify your development process.

1.5. AI Vision Reference Code

Hailo AI Vision is comprehensive C++ and GStreamer reference implementations for smart cameras based on Hailo-15 devices. They combine vision-pipeline elements with pre-trained AI tasks to demonstrate best practices for leveraging Hailo-15 hardware-acceleration, achieving low latency, efficient memory utilization, and robust real-time performance.

These examples serve as reference code for developing end-to-end AI vision solutions on Hailo-15. They provide practical reference designs for integrating the Hailo-15 software stack into complete video and AI pipelines, supporting evaluation, demos, and application development

1.5.1. AI Vision Reference Code C++ API

These reference code examples demonstrate how to use the Hailo Media Library via the C++ API with the following capabilities:

- Integration with the Media Library vision frontend
- Integration with hardware-accelerated encoders (H.264, H.265)
- Computer Vision tasks such as Image cropping and resizing (tiling) via DSP acceleration
- On-screen display (OSD) overlays using hardware-accelerated DSP
- DMA buffer utilization for efficient memory management
- Video streaming to remote servers via RTP
- High Dynamic Range (HDR) processing
- AI-ISP - Advanced 12-bit AI-based video denoising for enhanced image quality in extremely low-light conditions

Refer to the sub-sections below for detailed descriptions. These examples are located at:

```
/home/root/app/examples
```

These reference code examples focus on demonstrating core Media Library API features and functionality, providing practical blueprints for common use cases such as dynamic configuration management, on-screen display overlays, and runtime profile switching. They serve as starting points for developers learning to work with the Media Library's C++ API.

Each reference code example:

- Demonstrates a specific feature or use case
- Includes complete, working source code
- Provides clear prerequisites and setup instructions
- Explains expected behavior and output
- Uses the common utilities framework for consistency

Profile Switching

Overview

This reference code example demonstrates how to dynamically switch between different imaging profiles (Daylight, Lowlight, High_Dynamic_Range) at runtime using the Media Library API. Profiles allow you to configure different image quality settings optimized for various lighting conditions and use cases.

Prerequisites

- Hailo-15 device with camera sensor connected
- Valid `medialib_config.json` configuration file with multiple profiles defined
- Camera sensor properly calibrated for target profiles

What You Will Learn

- How to initialize the Media Library with a configuration file
- How to switch between imaging profiles programmatically
- How to set up encoded output streams to files
- How to use the common examples utilities for pipeline management
- How to implement graceful shutdown with signal handling

Usage

```
./profile_switching <config.json> <iterations> <profile1> [profile2] [profile3] ...
```

Example:

```
./profile_switching /usr/bin/medialib_config.json 5 Daylight Lowlight High_Dynamic_
↪Range
```

This will cycle through the specified profiles, switching every 10 seconds for 5 iterations.

Expected Behavior

The application will:

1. Initialize the media library with the provided configuration
2. Connect frontend to encoders and start the pipeline
3. Cycle through the specified profiles in order
4. Wait 10 seconds between profile switches
5. Write encoded H.264 output to `/var/volatile/tmp/profile_switching_sink*.h264`
6. Exit gracefully after completing all iterations

OSD

Overview

This reference code example demonstrates how to create and dynamically update on-screen display (OSD) text overlays using the Media Library API. The reference code example shows how to add text overlays to encoded video streams and update their content in real-time.

Prerequisites

- Hailo-15 device with camera sensor connected
- Valid `medialib_config.json` configuration file
- TrueType font file available at `/usr/share/fonts/ttf/LiberationMono-Bold.ttf`

What You Will Learn

- How to access the OSD blender for encoder streams
- How to create text overlays with custom positioning and styling
- How to dynamically update overlay text content at runtime
- How to enable and manage OSD overlays on multiple encoder streams
- How to use the common examples utilities for pipeline management
- How to implement graceful shutdown with signal handling

Usage

```
./osd_example <config.json> <duration_seconds>
```

Example:

```
./osd_example /usr/bin/medialib_config.json 30
```

This will run the OSD example for 30 seconds, updating the overlay text every second.

Expected Behavior

The application will:

1. Initialize the media library with the provided configuration
2. Connect frontend to encoders and start the pipeline
3. Create a centered text overlay displaying the current iteration count
4. Update the overlay text every second
5. Write encoded H.264 output with overlays to `/var/volatile/tmp/osd_example_sink*.h264`
6. Exit gracefully after the specified duration

OSD Configuration

The example creates a text overlay with the following properties:

- **ID:** `status`
- **Position:** Centered horizontally at 50%, 10% from top
- **Text:** `Iteration: N` (updated each second)
- **Colors:** White text on black background
- **Font:** Liberation Mono Bold, 40pt
- **Alignment:** Center

Refer to sections 9 and 10 for additional reference code, reference pipelines and reference applications.

Refer to the Hailo-15 Vision Processor Software Package Release Notes for detailed information.

1.5.2. AI Vision Reference Code Examples Using GStreamer API

Reference code examples for the use of Hailo Media Library via GStreamer API elements for building video capture and encoding pipelines, are embedded in the [GStreamer API Elements](#) section. These examples are located at `hailo-analytics/apps/case_studies/gst_vision_encoder/`.

1.6. Supported Image Sensors

The Hailo Media Library supports various image sensors to enable flexibility across different camera applications and use cases.

For a complete list of approved CMOS sensors for Hailo-15, refer to the Hailo-15 Approved Vendor List document. You may also review the sensor registry directory at:

```
hailo-media-library/media_library/src/isp/sensor_registry
```

Note: Throughout this User Guide, the IMX678 sensor is referenced in most examples and demonstrations. However, the Media Library functionality described is applicable to all supported sensors, with configuration adjustments as needed for specific sensor characteristics.

2. Configurations

2.1. Media Library Configurations

The media library configuration is defined in a JSON structure. This configuration allows users to define profiles and their associated settings.

The JSON snippet below represents the media library configuration:

```
{
  "version": "2.0.0",
  "backup_folder_path": "/etc/imaging/cfg/backups/app_backup",
  "default_profile": "Daylight",
  "profiles": [
    {
      "name": "Daylight",
      "config_file": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
↪daylight/daylight_profile.json"
    },
    {
      "name": "High_dynamic_range",
      "config_file": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
↪HDR/HDR_profile.json"
    },
    {
      "name": "Lowlight",
      "config_file": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
↪Lowlight/lowlight_r0225c8_profile.json"
    },
    {
      "name": "Infrared",
      "config_file": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
↪IR/IR_profile.json"
    }
  ]
}
```

Here is a breakdown of the JSON structure:

2.1.1. Version

- **version:** Specifies the version of the configuration. Example: "2.0.0". The field is an internal field used by Hailo to track changes in the structure of the configuration file and ensure suitable backward compatibility. This field is not expected to be modified by the user.

2.1.2. Default Profile

- **default_profile:** The name of the default profile. Example: "Daylight". The field specifies the default profile to be used by the media library. The field denotes the profile used by the system during startup. It is recommended for the default profile to be one that doesn't use any AI feature in the system.

2.1.3. Profiles

- **profiles:** A list of profiles, where each profile contains:
 - **name:** The name of the profile. Example: "Daylight".
 - **config_file:** The path to the configuration file for the profile. Example: "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/daylight/daylight_profile.json".
- The name provided in the "default_profile" field must be present as one of the profiles in the list.
- During application startup, all profiles in the list will be validated.

2.1.4. Backup Folder Path

- **backup_folder_path:** The directory path where the profiles backup is stored. Example: "/etc/imaging/cfg/backups/app_backup".

2.1.5. Restrictions

- The *default_profile* must match one of the profile names in the *profiles* list.
- The *config_file* paths must be valid and accessible.

2.1.6. API Reference

Media Library Configurations.

2.2. Profile Configurations

2.2.1. Profile Structure and Differences

The profile contains all the configurations of an application including configurations previously present in the *front-end_config.json*, *encoder_config.json*, *3aconfig.json* and sensor entry.

This document describes the profile JSON structure used to configure the media library. The profile points to all the files that configure the system as a one stop shop for all the configurations of the system.

The JSON snippet below represents the webserver daylight profile configuration:

```
{
  "version": "1.0.0",
  "sensor_config": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
  ↪ daylight/webserver_daylight/sensor_config.json",
  "application_settings": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/
  ↪ profiles/daylight/webserver_daylight/application_settings.json",
  "stabilizer_settings": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/
  ↪ profiles/daylight/webserver_daylight/stabilizer_settings.json",
  "iq_settings": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
  ↪ daylight/webserver_daylight/iq_settings.json",
  "encoded_output_streams": [
    {
      "stream_id": "sink0",
      "encoding": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/
      ↪ daylight/webserver_daylight/encoder_sink0.json",
      "osd": "/etc/imaging/cfg/hailo15h/imx678/theia_sl1410m/4k/profiles/daylight/
      ↪ webserver_daylight/osd_sink0.json",
    }
  ]
}
```

(continues on next page)

(continued from previous page)

```

    "masking": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/profiles/
    ↪daylight/webserver_daylight/masking_sink0.json"
  }
]
}

```

Here is a breakdown of the JSON structure:

Metadata

The profile and all referenced configuration files include a top-level metadata field. This field is generated automatically and validated by the Media Library during runtime.

The metadata provides:

- The target architecture (hailo15h or hailo15l)
- A content hash of the file
- A generation timestamp
- An optional description field

Runtime validation ensures that:

- A configuration file cannot be executed on the wrong architecture.
- A configuration file cannot be modified without re-hashing it.
- If validation fails, the Media Library prevents the pipeline from starting.

Example

```

{
  "metadata": {
    "architecture": "hailo15h",
    "description": "",
    "content_hash":
    ↪"8698f18a59334dafef6853657fdb0016569f57246c7d18af4557c83d0482abec",
    "generation_timestamp": "2025-11-16T12:18:55.676282+00:00"
  },
  "sensor_config": "...",
  "application_settings": "...",
  "stabilizer_settings": "...",
  "iq_settings": "...",
  "encoded_output_streams": [ "..." ]
}

```

Allowed Architectures

Only the following architectures are valid:

- hailo15h
- hailo15l

Runtime Enforcement

The Media Library validates metadata during initialization:

- If the architecture does not match the running device, initialization fails.
- If a file's content_hash does not match its actual content, initialization fails.

This protects against accidental edits, stale files, and misconfigured systems.

Rehashing Configuration Files

To make any modification permanent, files must be rehashed. This is done using the Profile Manager tool.

Version

The "version" field indicates the version of the profile configuration structure, as well as the profiles and all their underlying components. the version can only be updated using the Profile Manager tool.

Sensor Configuration

The sensor_config file is used by the ISP media server to configure and load the sensor correctly.

```
{
  "version": "1.0.0",
  "input_video": {
    "resolution": {
      "width": 3840,
      "height": 2160,
      "framerate": 30
    }
  },
  "sensor_configuration": {
    "name": "imx678",
    "drv": "HAILO_IMX678.drv",
    "mode": 0,
    "pixel_mode": 0,
    "sensor_only": 0,
    "af_i2c_bus": -1,
    "af_i2c_addr": "0x0"
  },
  "sensor_calibration_file": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/
  ↪profiles/daylight/webserver_daylight/calib.json"
}
```

input_video

This section describes the video received from the sensor and which will then be sent to the media library.

- source_type: Type of the input video source. Possible values:
 - V4L2SRC - Linux camera/capture devices (default).
 - APPSRC - Raw-frame application-based sources.
- source: The source file path of the input video when using APPSRC.
- sensor_id: The sensor id, which represents the sensor index, used to identify the sensor when using V4L2SRC. Possible values:

- SENSOR_0 (default)
- SENSOR_1 (used for dual-sensor devices)
- resolution: A resolution for the input video.
 - width: The width of the video resolution.
 - height: The height of the video resolution.
 - framerate: The framerate of the video.

Note:

- This example assumes the input video comes from a V4L2 device.
- When using APPSRC, a valid source file path must be provided

sensor_configuration

This section describes the sensor configuration parameters previously included in the sensor0_entry.cfg file, which include the current sensor mode, driver and additional parameters needed in order to configure the sensor driver itself.

- name: The name of the sensor (e.g., imx678).
- drv: The driver file for the sensor (e.g., HAILO_IMX678.drv).
- mode: Linked to a specific sensor calibration (Applies to all Hailo-15 flavours) and dewarp configuration (applies only to Hailo-15L). The sensor might support multiple modes for different scenarios.
- pixel_mode: The sensor pixel mode, typically set to 0
- sensor_only: disable all ISP modules and get an output that looks like the sensor's input
- af_i2c_bus: for future use, not currently in used.
- af_i2c_addr: for future use, not currently in used.
- sensor_calibration_file: Initial image quality calibration parameters that are loaded during startup, for more information please refer to the tuning and calibration guide.

Application_settings

The application_settings file contains settings related to the stream zoom, resolution, orientation and other advanced application capabilities the media library provides.

Application Input Streams

The application_input_streams field contains settings related to the output video stream from media library.

```
{
  "application_input_streams": {
    "method": "INTERPOLATION_TYPE_BILINEAR",
    "format": "IMAGE_FORMAT_NV12",
    "grayscale": false,
    "resolutions": [
      {
        "width": 3840,
        "height": 2160,
        "framerate": 30,
        "pool_max_buffers": 20,
```

(continues on next page)

(continued from previous page)

```

        "scaling_mode": "STRETCH"
    },
    {
        "width": 1280,
        "height": 720,
        "framerate": 30,
        "pool_max_buffers": 20,
        "scaling_mode": "STRETCH"
    }
]
}
}

```

- **method:** Specifies the interpolation method used for resizing the video. Possible value: `INTERPOLATION_TYPE_BILINEAR`.
- **format:** Defines the format of the output video. Possible value: `IMAGE_FORMAT_NV12`.
- **grayscale:** Boolean indicating whether the output video should be in grayscale. Possible values: `true` or `false`.
- **resolutions:** A list of resolutions for the output video, each containing:
 - **stream_id:** (Optional) The given unique id that can be retrieved from frontend output for identification purposes, if t
 - ✱ **width:** The width of the video resolution.
 - **height:** The height of the video resolution.
 - **framerate:** The frame rate of the video.
 - **pool_max_buffers:** The number of buffers to allocate in the pool .
 - ✱ **scaling_mode:** Indicates how the image is to be resized. Options include:
 - `STRETCH`: Scale to fill the new dimensions, without maintaining aspect ratio (default).
 - `LETTERBOX_MIDDLE`: Scale to fit the new dimensions, framing with padding added evenly around it.
 - `LETTERBOX_UP_LEFT`: Scale to fit the new dimensions, framing with padding added to the bottom or to the right.
 - `SCALE_AND_CROP`: Scale to fill the new dimensions, maintaining aspect ratio and cropping the scaled image.

Note: Refer to the [Buffer pool](#) for detailed explanation about buffer pools.

Optical Zoom

The `optical_zoom` field is required for Hailo to adjust the dewarping according to the camera zoom.

```

{
  "optical_zoom": {
    "enabled": true,
    "magnification": 1.0,
    "max_dewarping_magnification": 100.0
  }
}

```

- **enabled:** Boolean indicating whether optical zoom is enabled. Possible values: `true` or `false`.

- **magnification:** Magnification factor for optical zoom.
- **max_dewarping_magnification:** Maximum dewarping magnification value. if dewarping is enabled, when the magnification factor exceeds this value, the dewarping is disabled automatically. this feature is implemented to avoid over correction with dewarping since high-magnification warping is negligible.

Digital zoom

The *digital_zoom* field contains settings related to digital zoom. Digital zoom has two modes: DIGITAL_ZOOM_MODE_ROI and DIGITAL_ZOOM_MODE_MAGNIFICATION, and the user can choose only one of them.

```
{
  "digital_zoom": {
    "enabled": false,
    "mode": "DIGITAL_ZOOM_MODE_ROI",
    "magnification": 1.0,
    "roi": {
      "x": 200,
      "y": 200,
      "width": 2800,
      "height": 1800
    }
  }
}
```

- **enabled:** Boolean indicating whether digital zoom is enabled. Possible values: *true* or *false*.
- **mode:** Mode of digital zoom. Possible values: DIGITAL_ZOOM_MODE_ROI, DIGITAL_ZOOM_MODE_MAGNIFICATION.
- **magnification:** Magnification factor for digital zoom. Max digital zoom is 31× at any resolution. In rare non-standard or telescopic setups, the zoom may cap marginally lower.
- **roi:** Region of interest for digital zoom, containing:
 - **x:** X-coordinate of the region.
 - **y:** Y-coordinate of the region.
 - **width:** Width of the region.
 - **height:** Height of the region.

Note: If DIGITAL_ZOOM_MODE_ROI mode is selected, magnification would be ignored, and if DIGITAL_ZOOM_MODE_MAGNIFICATION mode is selected, the ROI would be ignored.

Motion Detection

The *motion_detection* field contains settings related to motion detection feature.

```
{
  "motion_detection": {
    "enabled": true,
    "resolution": {
      "width": 640,
      "height": 480,
      "framerate": 30
    },
    "roi": {
```

(continues on next page)

(continued from previous page)

```

    "x": 200,
    "y": 200,
    "width": 640,
    "height": 480
  },
  "sensitivity_level": "MEDIUM",
  "threshold": 0.5
}
}

```

- **enabled:** Boolean indicating whether motion_detection is enabled. Possible values: `true` or `false`.
- **resolution:** A resolution for the motion detection stream (Includes bitmask size), containing:
 - **width:** The width of the video resolution.
 - **height:** The height of the video resolution.
 - **framerate:** The frame rate of the video.
 - **pool_max_buffers:** (Optional) The number of buffers to allocate in the pool . Default value is the maximum pool_max_buffers of output resolutions.
- **roi:** Region of interest for motion detection event to trigger, containing:
 - **x:** X-coordinate of the region.
 - **y:** Y-coordinate of the region.
 - **width:** Width of the region.
 - **height:** Height of the region.
- **sensitivity_level:** Sensitivity level of the motion detection. Possible values: `LOWEST`, `LOW MEDIUM`, `HIGH`, `HIGHEST`.
- **threshold:** Threshold for the motion detection, How much of the ROI should be changed in order to trigger motion detected. A float value between 0 and 1.

Rotation

The *rotation* field contains settings related to video rotation. The rotation will be applied to all the output resolutions.

```

{
  "rotation": {
    "enabled": false,
    "angle": "ROTATION_ANGLE_180"
  }
}

```

- **enabled:** Boolean indicating whether rotation is enabled. Possible values: `true` or `false`.
- **angle:** Angle of rotation. Possible value: `ROTATION_ANGLE_0`, `ROTATION_ANGLE_90`, `ROTATION_ANGLE_180`, `ROTATION_ANGLE_270`.

Flip

The *flip* field contains settings related to video flipping.

```
{
  "flip": {
    "enabled": false,
    "direction": "FLIP_DIRECTION_HORIZONTAL"
  }
}
```

- **enabled:** Boolean indicating whether flipping is enabled. Possible values: true or false.
- **direction:** Direction of the flip. Possible value: FLIP_DIRECTION_NONE, FLIP_DIRECTION_HORIZONTAL, FLIP_DIRECTION_VERTICAL, FLIP_DIRECTION_BOTH.

HailoRT

The *hailort* field contains settings related to the Hailo Runtime.

```
{
  "hailort": {
    "device-id": "device0"
  }
}
```

- **device-id:** ID of the Hailo device.

stabilizer_settings

The *stabilizer_settings* file contains settings related to video stabilization, including DIS and EIS.

```
{
  "version": "1.0.0",
  "dis": {
    "enabled": false,
    "minimum_coefficient_filter": 0.1,
    "decrement_coefficient_threshold": 0.001,
    "increment_coefficient_threshold": 0.01,
    "running_average_coefficient": 0.033,
    "std_multiplier": 3.0,
    "black_corners_correction_enabled": true,
    "black_corners_threshold": 0.5,
    "average_luminance_threshold": 0,
    "camera_fov_factor": 0.85,
    "angular_dis": {
      "enabled": false,
      "vsm": {
        "hoffset": 1856,
        "voffset": 1016,
        "width": 1920,
        "height": 1080,
        "max_displacement": 64
      }
    }
  },
  "debug": {
    "generate_resize_grid": false,
    "fix_stabilization": false,
    "fix_stabilization_longitude": 0.0,
    "fix_stabilization_latitude": 0.0
  }
}
```

(continues on next page)

(continued from previous page)

```

    }
  },
  "eis": {
    "enabled": false,
    "stabilize": true,
    "eis_config_path": "/home/root/apps/resources/final_calibration.json",
    "window_size": 10,
    "rotational_smoothing_coefficient": 0.0,
    "iir_hpf_coefficient": 0.997,
    "camera_fov_factor": 0.85,
    "line_readout_time": 7410,
    "hdr_exposure_ratio": 0.2,
    "min_angle_deg": 0.05,
    "force_clamp_correction_angles": false
  },
  "gyro": {
    "enabled": false,
    "sensor_name": "lsm6dsr_gyro",
    "sensor_frequency": "833.000000",
    "scale": 0.000152716
  }
}

```

DIS

The *dis* field contains settings related to Digital Image Stabilization (DIS).

```

{
  "dis": {
    "enabled": false,
    "minimun_coefficient_filter": 0.1,
    "decrement_coefficient_threshold": 0.001,
    "increment_coefficient_threshold": 0.01,
    "running_average_coefficient": 0.033,
    "std_multiplier": 3.0,
    "black_corners_correction_enabled": true,
    "black_corners_threshold": 0.5,
    "average_luminance_threshold": 0,
    "debug": {
      "generate_resize_grid": false,
      "fix_stabilization": false,
      "fix_stabilization_longitude": 0.0,
      "fix_stabilization_latitude": 0.0
    }
  }
}

```

- **enabled:** Boolean indicating whether DIS is enabled. Possible values: *true* or *false*.
- **minimun_coefficient_filter:** Minimal value of the coefficient 'k' used to filter the motion vectors (MVs). This value is a float in the range [0, 1], determining how fast changes in output from a given MV are seen. Example: $k = 0$ results in complete filtering and lack of consideration of the current MV, $k = 1$ means immediate impact.
- **decrement_coefficient_threshold:** Value to decrement k whenever the difference of succeeding motion vectors is not too large. This value is a float in the range [0, 1], recommended values are between 1/100 and 1/10.
- **increment_coefficient_threshold:** Value to increment k when large motion occurs to prevent black corners. This value is a float in the range [0, 1], recommended values are between 1/100 and 1/10.

- **running_average_coefficient:** Coefficient used to calculate the runtime average of motion vectors (MV). A float value typically set to "1 / number-of-frames-to-average". Set to 1 to disable.
- **std_multiplier:** Acceptable deviation, usually set between 2.5 and 3.5. Set to a very large value to disable.
- **black_corners_correction_enabled:** Boolean indicating whether black corners correction is enabled. *true* enables smooth stabilization with black corners, *false* avoids black corners but may violate stabilization.
- **black_corners_threshold:** Coefficient affecting filter strength when stabilizing rotation is greater than "BLKCRN_TO_K_THR * room-for-stabilization". A float value between 0 and 1, recommended values are between 0.2 and 0.5.
- **average_luminance_threshold:** Threshold for average luminance to enable or disable the stabilizer. A uint8 value in the range [0, 255]. If set to 0, the stabilizer is always enabled; if set to 255, the stabilizer is always disabled.
- **debug:** Contains debug settings:
 - **generate_resize_grid:** Boolean indicating whether to generate a grid that only resizes the input image into the output. Possible values: *true* or *false*.
 - **fix_stabilization:** Boolean indicating whether to fix the stabilized orientation to predefined values. *true* removes the impact of the stabilization filter and black-corners limitations.
 - **fix_stabilization_longitude:** Fixed stabilized longitude correlated to the first frame, in radians. A float value.
 - **fix_stabilization_latitude:** Fixed stabilized latitude correlated to the first frame, in radians. A float value.

EIS

The *eis* field contains settings related to Electronic Image Stabilization (EIS).

```
{
  "eis": {
    "enabled": false,
    "eis_config_path": "/home/root/apps/resources/final_calibration.json",
    "window_size": 10,
    "rotational_smoothing_coefficient": 0.0,
    "iir_hpf_coefficient": 0.997,
    "camera_fov_factor": 0.85,
    "line_readout_time": 7410,
    "min_angle_deg": 0.005,
    "force_clamp_correction_angles": false
  },
  "gyro": {
    "enabled": false,
    "sensor_name": "lsm6dsr_gyro",
    "sensor_frequency": "833.000000",
    "scale": 0.000152716
  }
}
```

- **eis:** Contains settings related to the EIS:
 - **enabled:** Boolean indicating whether EIS is enabled. Possible values: *true* or *false*.
 - **eis_config_path:** String path to the EIS configuration file - a JSON file containing calibration data. The file is created using the calibration tool.
 - **window_size:** Integer indicating the number of frames orientations used for motion smoothing.
 - **rotational_smoothing_coefficient:** Float between 0 and 1 indicating the strength of the smoothing algorithm. 0 means correcting all motion (including intentional smooth motion and shake), and 1 means no correction at all.

- **iir_hpf_coefficient**: Float between 0 and 1 indicating the strength of the IIR high-pass filter.
- **camera_fov_factor**: Float indicating the field of view of the output frame. A larger FOV results in less cropping on the output frame but may cause black corners to appear.
- **line_readout_time**: The time between two sequential lines in the sensor, expressed in nanoseconds. This integer value is used to adjust the stabilization for rolling shutter effects. The value should be obtained from the sensor datasheet.
- **min_angle_deg**: Float indicating the minimum angle in degrees for EIS to be applied. This value is used to filter out small movements that do not require stabilization.
- **force_clamp_correction_angles**: Boolean indicating whether to force clamping of correction angles. If set to *true*, the correction angles will be clamped to the maximum angles defined, even if the gyroscope data suggests otherwise. May cause degradation in stabilization performance due to limiting the correction angles.
- **gyro**: Contains settings related to the gyroscope used in EIS:
 - **enabled**: Boolean indicating whether the gyroscope is enabled. Possible values: *true* or *false*.
 - **sensor_name**: String indicating the name of the gyroscope sensor.
 - **sensor_frequency**: String indicating the frequency of the gyroscope sensor (*The value we recommend and mostly worked with here is "833.000000". Other possible values are: "208.000000", "416.000000"*).
 - **scale**: Float indicating the scale of the gyroscope sensor. Used to convert the raw sensor data into the degrees per seconds unit. Can be obtained from the sensors datasheet "Angular rate sensitivity".

Note: EIS *enabled* set to *true* will not work without *gyro enabled* set to *true*.

iq_settings

The `iq_settings` file contains settings related to image quality enhancements, including HDR, denoise, dewarp and all automatic algorithms previously located in the `3aconfig` file.

```
{
  "version": "1.0.0",
  "grayscale": false,
  "denoise": {
    "enabled": false,
    "sensor": "imx678",
    "method": "HIGH_QUALITY",
    "loopback-count": 1,
    "network": {
      "network_path": "/usr/lib/medialib/denoise_config/vd_m_imx678.hef",
      "y_channel": "model/input_layer1",
      "uv_channel": "model/input_layer4",
      "feedback_y_channel": "model/input_layer3",
      "feedback_uv_channel": "model/input_layer2",
      "output_y_channel": "model/conv17",
      "output_uv_channel": "model/conv14"
    }
  },
  "hdr": {
    "enabled": false,
    "dol": 2
  },
  "dewarp": {
    "enabled": false,
    "color_interpolation": "INTERPOLATION_TYPE_BILINEAR",
    "sensor_calib_path": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/
    shared/calibration/cam_intrinsics.txt",

```

(continues on next page)

(continued from previous page)

```

    "camera_type": "CAMERA_TYPE_PINHOLE"
  },
  "automatic_algorithms": {
  }
}

```

Denoise

Note: For an example of AI denoising, you can refer to the [denoise_analytics.sh](#) script in the TAPPAS repository. This script demonstrates how to use AI denoising in combination with AI analytics

The *denoise* field contains settings related to video denoising. There are three generations of AI-ISP denoise:

- **AI-ISP Gen1:** Post-ISP lowlight denoise. Processes YUV data after the ISP pipeline.
- **AI-ISP Gen2:** Pre-ISP lowlight denoise using VDM (Visidon Denoise Model). Processes raw Bayer data before the ISP pipeline.
- **AI-ISP Gen3:** Pre-ISP lowlight denoise using HDM (Hailo Denoise Model). Processes raw Bayer data with fusion and gamma feedback channels. Currently available on Hailo-15L with IMX678 sensor.

The denoise generation is determined automatically based on the profile configuration: `bayer: false` selects Gen1 (post-ISP), `bayer: true` selects Gen2 or Gen3 depending on the network channel configuration.

Post-ISP denoise (AI-ISP Gen1):

```

{
  "denoise": {
    "enabled": false,
    "sensor": "imx678",
    "method": "HIGH_QUALITY",
    "loopback-count": 1,
    "network": {
      "network_path": "/usr/lib/medialib/denoise_config/vd_m_imx678.hef",
      "y_channel": "model/input_layer1",
      "uv_channel": "model/input_layer4",
      "feedback_y_channel": "model/input_layer3",
      "feedback_uv_channel": "model/input_layer2",
      "output_y_channel": "model/conv17",
      "output_uv_channel": "model/conv14"
    }
  }
}

```

- **enabled:** Boolean indicating whether denoising is enabled. Possible values: `true` or `false`.
- **sensor:** Type of sensor used.
- **method:** Denoising method used. Possible values: `HIGH_QUALITY`, `BALANCED`, `HIGH_PERFORMANCE`.
- **loopback-count:** Number of loopbacks for denoising.
- **network:** Contains settings for the denoising network:
 - **network_path:** Path to the network configuration file.
 - **y_channel:** Y-channel input layer for the network.
 - **uv_channel:** UV-channel input layer for the network.
 - **feedback_y_channel:** Feedback Y-channel input layer for the network.

- **feedback_uv_channel**: Feedback UV-channel input layer for the network.
- **output_y_channel**: Y-channel output layer for the network.
- **output_uv_channel**: UV-channel output layer for the network.

Pre-ISP denoise (AI-ISP Gen2 / Gen3):

Pre-ISP denoise profiles use `"bayer": true` and configure the `bayer_network` field instead of `network`. Gen3 (HDM) profiles additionally include fusion and gamma feedback channels for improved image quality.

HDR

Note:

1. Only 2DOL is supported
2. HDR and Denoise are mutually exclusive and cannot be enabled at the same time.
3. HDR is supported only with input video of 4K or FHD.

The `hdr` field contains settings related to High Dynamic Range (HDR) video.

```
{
  "hdr": {
    "enabled": false,
    "dol": 2
  }
}
```

- **enabled**: Boolean indicating whether HDR is enabled. Possible values: `true` or `false`.
- **dol**: Digital overlap for HDR.

Dewarp

The `dewarp` field contains settings related to the dewarping of the video.

```
{
  "dewarp": {
    "enabled": true,
    "color_interpolation": "INTERPOLATION_TYPE_BILINEAR",
    "sensor_calib_path": "/etc/imaging/cfg/hailo15h/imx678/kit/4k/shared/
    ↪calibration/cam_intrinsics.txt",
    "camera_type": "CAMERA_TYPE_PINHOLE",
    "camera_fov": -1
  }
}
```

- **enabled**: Boolean indicating whether dewarping is enabled. Possible values: `true` or `false`.
- **color_interpolation**: Specifies the interpolation method used for color correction. Possible value: `INTERPOLATION_TYPE_BILINEAR`, `INTERPOLATION_TYPE_BICUBIC`.
- **sensor_calib_path**: Path to the sensor calibration file.
- **camera_type**: Specifies the type of camera. Possible value: `CAMERA_TYPE_PINHOLE`, `CAMERA_TYPE_FISHEYE`, `CAMERA_TYPE_INPUT_DISTORTIONS`.
- **camera_fov**: Specifies the field of view of the camera. Value `-1` indicates auto-detection.

Automatic Algorithms

The *automatic_algorithms* field contains settings related to automatic image quality algorithms running in the ISP. for more details about each algorithm please refer to the isp documentation.

Encoded Output Streams

The *encoded_output_streams* field contains settings related to the final encoding stage of the pipeline, which consists of masking, OSD and video/still encoding

```
{
  "encoded_output_streams": [
    {
      "stream_id": "sink0",
      "encoding": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/profiles/
↪daylight/webserver_daylight/encoder_sink0.json",
      "osd": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/profiles/daylight/
↪webserver_daylight/osd_sink0.json",
      "masking": "/etc/imaging/cfg/hailo15h/imx678/theia_sl410m/4k/profiles/
↪daylight/webserver_daylight/masking_sink0.json"
    }
  ]
}
```

- **encoded_output_streams:** A list of encoded output streams, each containing:
 - **stream_id:** The identifier for the output stream (e.g., *sink0*).
 - **config_path:** The path to the codec configuration file for the output stream.
 - **osd:** The path to the OSD configuration file for the output stream. for more details about the OSD configuration file please refer to the relevant section
 - **masking:** The path to the masking configuration file for the output stream. for more details about the OSD configuration file please refer to the relevant section

Restrictions

- Input and output resolutions are constant, and cannot be changed during pipeline running
- Frame rates must not exceed the input rate, and must be of a divisible fraction (input rate: 30/1, output rate alternatives: 1,2,3,5,6,10,15)
- Once the pipeline has been initialized, the denoise network cannot be changed

2.3. Frontend Configurations

2.3.1. Video Frontend

The video frontend configuration is done in a single JSON structure.

The API should receive the JSON as argument, this means the JSON is not read from file by the media library code, the example and/or gst-element should read from file and execute the api, for a C++ example see the [Frontend C++ Configuration](#) section.

The JSON snippet below represents the video front-end JSON that includes the following:

```
{
  "application_input_streams": {
    "method": "INTERPOLATION_TYPE_BILINEAR",
    "format": "IMAGE_FORMAT_NV12",
    "grayscale": false,
    "resolutions": [
      {
        "width": 3840,
        "height": 2160,
        "framerate": 30,
        "pool_max_buffers": 20,
        "scaling_mode": "STRETCH"
      },
      {
        "width": 1280,
        "height": 720,
        "framerate": 30,
        "pool_max_buffers": 20,
        "scaling_mode": "STRETCH"
      }
    ]
  },
  "application_analytics": {},
  "dewarp": {
    "enabled": true,
    "color_interpolation": "INTERPOLATION_TYPE_BILINEAR",
    "sensor_calib_path": "/etc/imaging/cfg/hailo15h/imx678/kit/4k/shared/
↪calibration/cam_intrinsics.txt",
    "camera_type": "CAMERA_TYPE_PINHOLE"
  },
  "dis": {
    "enabled": false,
    "minimum_coefficient_filter": 0.1,
    "decrement_coefficient_threshold": 0.001,
    "increment_coefficient_threshold": 0.01,
    "running_average_coefficient": 0.033,
    "std_multiplier": 3.0,
    "black_corners_correction_enabled": true,
    "black_corners_threshold": 0.5,
    "average_luminance_threshold": 0,
    "camera_fov_factor": 0.85,
    "angular_dis": {
      "enabled": false,
      "vsm": {
        "hoffset": 1856,
        "voffset": 1016,
        "width": 1920,
        "height": 1080,
        "max_displacement": 64
      }
    }
  },
  "debug": {
    "generate_resize_grid": false,
    "fix_stabilization": false,
    "fix_stabilization_longitude": 0.0,
    "fix_stabilization_latitude": 0.0
  },
  "gmv": {
    "source": "isp",
    "frequency": 0.0
  },
}
```

(continues on next page)

(continued from previous page)

```

"optical_zoom": {
  "enabled": true,
  "magnification": 1.0
},
"digital_zoom": {
  "enabled": false,
  "mode": "DIGITAL_ZOOM_MODE_ROI",
  "magnification": 1.0,
  "roi": {
    "x": 200,
    "y": 200,
    "width": 2800,
    "height": 1800
  }
},
"motion_detection": {
  "enabled": false,
  "resolution": {
    "width": 640,
    "height": 480,
    "framerate": 30
  },
  "roi": {
    "x": 0,
    "y": 0,
    "width": 640,
    "height": 480
  },
  "sensitivity_level": "MEDIUM",
  "threshold": 0.5,
  "buffer_pool_size": 8
},
"rotation": {
  "enabled": false,
  "angle": "ROTATION_ANGLE_180"
},
"flip": {
  "enabled": false,
  "direction": "FLIP_DIRECTION_HORIZONTAL"
},
"isp": {
  "isp_config_files_path": "/usr/bin"
},
"hailort": {
  "device-id": "device0"
},
"hdr": {
  "enabled": false,
  "dol": 2,
  "resolution": "4k",
  "lsRatio": 23.7,
  "vsRatio": 6.25
},
"denoise": {
  "enabled": false,
  "sensor": "imx678",
  "method": "HIGH_QUALITY",
  "loopback-count": 1,
  "network": {
    "network_path": "/usr/lib/medialib/denoise_config/vd_m_imx678.hef",
    "y_channel": "model/input_layer1",

```

(continues on next page)

(continued from previous page)

```

    "uv_channel": "model/input_layer4",
    "feedback_y_channel": "model/input_layer3",
    "feedback_uv_channel": "model/input_layer2",
    "output_y_channel": "model/conv17",
    "output_uv_channel": "model/conv14"
  }
}
}

```

Here is a breakdown of the JSON structure:

Input Video

The `input_video` field contains settings related to the input video stream for the frontend example.

```

{
  "input_video": {
    "resolution": {
      "width": 3840,
      "height": 2160,
      "framerate": 30
    },
    "sensor_id": "SENSOR_0"
  }
}

```

- **source_type**: The type of the input video source. Possible value: V4L2SRC for Linux camera/capture devices, or APPSRC for raw-frame app-based sources (the default is V4L2SRC).
- **source**: The source file path of the input video when using APPSRC as source type.
- **sensor_id**: The sensor id, which represents the sensor index, used to identify the sensor when using V4L2SRC as source type (the default is SENSOR_0).
- **resolution**: A resolution for the input video, containing:
 - **width**: The width of the video resolution.
 - **height**: The height of the video resolution.
 - **framerate**: The framerate of the video.

Note: This section assumes the input video is from a v4l2 device, however the input is dependent on the type of device used.

Note: Sensor id is mainly relevant for dual sensor devices, where its value for the input video can be either SENSOR_0 or SENSOR_1.

Application Input Streams

The `application_input_streams` field contains settings related to the Application Input Streams stream.

```
{
  "application_input_streams": {
    "method": "INTERPOLATION_TYPE_BILINEAR",
    "format": "IMAGE_FORMAT_NV12",
    "grayscale": false,
    "resolutions": [
      {
        "width": 3840,
        "height": 2160,
        "framerate": 30,
        "pool_max_buffers": 20,
        "scaling_mode": "STRETCH"
      },
      {
        "width": 1280,
        "height": 720,
        "framerate": 30,
        "pool_max_buffers": 20,
        "scaling_mode": "STRETCH"
      }
    ]
  }
}
```

- **method:** Specifies the interpolation method used for resizing the video. Possible value: `INTERPOLATION_TYPE_BILINEAR`.
- **format:** Defines the format of the output video. Possible value: `IMAGE_FORMAT_NV12`.
- **grayscale:** Boolean indicating whether the output video should be in grayscale. Possible values: `true` or `false`.
- **resolutions:** A list of resolutions for the output video, each containing:
 - **stream_id:** (Optional) The given unique id that can be retrieved from frontend output for identification purposes, if this field is omitted then output stream will be named automatically as `sink0`, `sink1`, etc.
 - **width:** The width of the video resolution.
 - **height:** The height of the video resolution.
 - **framerate:** The frame rate of the video.
 - **pool_max_buffers:** The number of buffers to allocate in the pool.
 - **scaling_mode:** Indicates how the image is to be resized. Options include:
 - ✱ `STRETCH`: Scale to fill the new dimensions, without maintaining aspect ratio (default).
 - ✱ `LETTERBOX_MIDDLE`: Scale to fit the new dimensions, framing with padding added evenly around it.
 - ✱ `LETTERBOX_UP_LEFT`: Scale to fit the new dimensions, framing with padding added to the bottom or to the right.
 - ✱ `SCALE_AND_CROP`: Scale to fill the new dimensions, maintaining aspect ratio and cropping the scaled image.

Note: Refer to the [Buffer pool](#) for detailed explanation about buffer pools.

Dewarp

The *dewarp* field contains settings related to the dewarping of the video.

```
{
  "dewarp": {
    "enabled": true,
    "color_interpolation": "INTERPOLATION_TYPE_BILINEAR",
    "sensor_calib_path": "/etc/imaging/cfg/hailo15h/imx678/kit/4k/shared/
    calibration/cam_intrinsics.txt",
    "camera_type": "CAMERA_TYPE_PINHOLE",
    "camera_fov": -1
  }
}
```

- **enabled:** Boolean indicating whether dewarping is enabled. Possible values: *true* or *false*.
- **color_interpolation:** Specifies the interpolation method used for color correction. Possible value: *INTERPOLATION_TYPE_BILINEAR*, *INTERPOLATION_TYPE_BICUBIC*.
- **sensor_calib_path:** Path to the sensor calibration file.
- **camera_type:** Specifies the type of camera. Possible value: *CAMERA_TYPE_PINHOLE*, *CAMERA_TYPE_FISHEYE*, *CAMERA_TYPE_INPUT_DISTORTIONS*.
- **camera_fov:** Specifies the field of view of the camera. Value *-1* indicates auto-detection.

DIS

The *dis* field contains settings related to Digital Image Stabilization (DIS).

```
{
  "dis": {
    "enabled": false,
    "minimun_coefficient_filter": 0.1,
    "decrement_coefficient_threshold": 0.001,
    "increment_coefficient_threshold": 0.01,
    "running_average_coefficient": 0.033,
    "std_multiplier": 3.0,
    "black_corners_correction_enabled": true,
    "black_corners_threshold": 0.5,
    "average_luminance_threshold": 0,
    "debug": {
      "generate_resize_grid": false,
      "fix_stabilization": false,
      "fix_stabilization_longitude": 0.0,
      "fix_stabilization_latitude": 0.0
    }
  }
}
```

- **enabled:** Boolean indicating whether DIS is enabled. Possible values: *true* or *false*.
- **minimun_coefficient_filter:** Minimal value of the coefficient 'k' used to filter the motion vectors (MVs). This value is a float in the range [0, 1], determining how fast changes in output from a given MV are seen. Example: *k = 0* results in complete filtering and lack of consideration of the current MV, *k = 1* means immediate impact.
- **decrement_coefficient_threshold:** Value to decrement *k* whenever the difference of succeeding motion vectors is not too large. This value is a float in the range [0, 1], recommended values are between 1/100 and 1/10.
- **increment_coefficient_threshold:** Value to increment *k* when large motion occurs to prevent black corners. This value is a float in the range [0, 1], recommended values are between 1/100 and 1/10.

- **running_average_coefficient:** Coefficient used to calculate the runtime average of motion vectors (MV). A float value typically set to "1 / number-of-frames-to-average". Set to 1 to disable.
- **std_multiplier:** Acceptable deviation, usually set between 2.5 and 3.5. Set to a very large value to disable.
- **black_corners_correction_enabled:** Boolean indicating whether black corners correction is enabled. *true* enables smooth stabilization with black corners, *false* avoids black corners but may violate stabilization.
- **black_corners_threshold:** Coefficient affecting filter strength when stabilizing rotation is greater than "BLKCRN_TO_K_THR * room-for-stabilization". A float value between 0 and 1, recommended values are between 0.2 and 0.5.
- **average_luminance_threshold:** Threshold for average luminance to enable or disable the stabilizer. A uint8 value in the range [0, 255]. If set to 0, the stabilizer is always enabled; if set to 255, the stabilizer is always disabled.
- **debug:** Contains debug settings:
 - **generate_resize_grid:** Boolean indicating whether to generate a grid that only resizes the input image into the output. Possible values: *true* or *false*.
 - **fix_stabilization:** Boolean indicating whether to fix the stabilized orientation to predefined values. *true* removes the impact of the stabilization filter and black-corners limitations.
 - **fix_stabilization_longitude:** Fixed stabilized longitude correlated to the first frame, in radians. A float value.
 - **fix_stabilization_latitude:** Fixed stabilized latitude correlated to the first frame, in radians. A float value.

EIS

The *eis* field contains settings related to Electronic Image Stabilization (EIS).

```
{
  "eis": {
    "enabled": false,
    "eis_config_path": "/home/root/apps/resources/eis_calibration.json",
    "window_size": 10,
    "rotational_smoothing_coefficient": 0.0,
    "iir_hpf_coefficient": 0.997,
    "camera_fov_factor": 0.85,
    "line_readout_time": 7410,
    "min_angle_deg": 0.005,
    "force_clamp_correction_angles": false,
  },
  "gyro": {
    "enabled": false,
    "sensor_name": "lsm6dsr_gyro",
    "sensor_frequency": "833.000000",
    "scale": 0.000152716
  }
}
```

- **eis:** Contains settings related to the EIS:
 - **enabled:** Boolean indicating whether EIS is enabled. Possible values: *true* or *false*.
 - **eis_config_path:** String path to the EIS configuration file - a JSON file containing calibration data. The file is created using the calibration tool.
 - **window_size:** Integer indicating the number of frames orientations used for motion smoothing.
 - **rotational_smoothing_coefficient:** Float between 0 and 1 indicating the strength of the smoothing algorithm. 0 means correcting all motion (including intentional smooth motion and shake), and 1 means no correction at all.

- **iir_hpf_coefficient**: Float between 0 and 1 indicating the strength of the IIR high-pass filter.
- **camera_fov_factor**: Float indicating the field of view of the output frame. A larger FOV results in less cropping on the output frame but may cause black corners to appear.
- **line_readout_time**: The time between two sequential lines in the sensor, expressed in nanoseconds. This integer value is used to adjust the stabilization for rolling shutter effects. The value should be obtained from the sensor datasheet.
- **min_angle_deg**: Float indicating the minimum angle in degrees for EIS to be applied. This value is used to filter out small movements that do not require stabilization.
- **force_clamp_correction_angles**: Boolean indicating whether to force clamping of correction angles. If set to *true*, the correction angles will be clamped to the maximum angles defined, even if the gyroscope data suggests otherwise. May cause degradation in stabilization performance due to limiting the correction angles.
- **gyro**: Contains settings related to the gyroscope used in EIS:
 - **enabled**: Boolean indicating whether the gyroscope is enabled. Possible values: *true* or *false*.
 - **sensor_name**: String indicating the name of the gyroscope sensor.
 - **sensor_frequency**: String indicating the frequency of the gyroscope sensor (*The value we recommend and mostly worked with here is "833.000000". Other possible values are: "208.000000", "416.000000"*).
 - **scale**: Float indicating the scale of the gyroscope sensor. Used to convert the raw sensor data into the degrees per seconds unit. Can be obtained from the sensors datasheet "Angular rate sensitivity".

Note: EIS *enabled* set to *true* will not work without gyro *enabled* set to *true*.

Optical Zoom

The *optical_zoom* field is required for Hailo to adjust the dewarping according to the camera zoom.

```
{
  "optical_zoom": {
    "enabled": true,
    "magnification": 1.0,
    "max_dewarping_magnification": 100.0
  }
}
```

- **enabled**: Boolean indicating whether optical zoom is enabled. Possible values: *true* or *false*.
- **magnification**: Magnification factor for optical zoom.
- **max_dewarping_magnification**: Maximum dewarping magnification value. if dewarping is enabled, when the magnification factor exceeds this value, the dewarping is disabled automatically. this feature is implemented to avoid over correction with dewarping since high-magnification warping is negligible.

Digital zoom

The *digital_zoom* field contains settings related to digital zoom. Digital zoom has two modes: DIGITAL_ZOOM_MODE_ROI and DIGITAL_ZOOM_MODE_MAGNIFICATION, and the user can choose only one of them.

```
{
  "digital_zoom": {
    "enabled": false,
    "mode": "DIGITAL_ZOOM_MODE_ROI",
    "magnification": 1.0,
    "roi": {
      "x": 200,
      "y": 200,
      "width": 2800,
      "height": 1800
    }
  }
}
```

- **enabled:** Boolean indicating whether digital zoom is enabled. Possible values: *true* or *false*.
- **mode:** Mode of digital zoom. Possible values: DIGITAL_ZOOM_MODE_ROI, DIGITAL_ZOOM_MODE_MAGNIFICATION.
- **magnification:** Magnification factor for digital zoom. Max digital zoom is 31× at any resolution. In rare non-standard or telescopic setups, the zoom may cap marginally lower.
- **roi:** Region of interest for digital zoom, containing:
 - **x:** X-coordinate of the region.
 - **y:** Y-coordinate of the region.
 - **width:** Width of the region.
 - **height:** Height of the region.

Note: If DIGITAL_ZOOM_MODE_ROI mode is selected, magnification would be ignored, and if DIGITAL_ZOOM_MODE_MAGNIFICATION mode is selected, the ROI would be ignored.

Rotation

The *rotation* field contains settings related to video rotation. The rotation will be applied to all the output resolutions.

```
{
  "rotation": {
    "enabled": false,
    "angle": "ROTATION_ANGLE_180"
  }
}
```

- **enabled:** Boolean indicating whether rotation is enabled. Possible values: *true* or *false*.
- **angle:** Angle of rotation. Possible value: ROTATION_ANGLE_0, ROTATION_ANGLE_90, ROTATION_ANGLE_180, ROTATION_ANGLE_270.

Flip

The *flip* field contains settings related to video flipping.

```
{
  "flip": {
    "enabled": false,
    "direction": "FLIP_DIRECTION_HORIZONTAL"
  }
}
```

- **enabled:** Boolean indicating whether flipping is enabled. Possible values: `true` or `false`.
- **direction:** Direction of the flip. Possible value: `FLIP_DIRECTION_NONE`, `FLIP_DIRECTION_HORIZONTAL`, `FLIP_DIRECTION_VERTICAL`, `FLIP_DIRECTION_BOTH`.

ISP

The *isp* field contains settings related to image signal processor.

```
{
  "isp": {
    "isp_config_files_path": "/usr/bin"
  },
}
```

- **isp_config_files_path:** Path to the directory containing the ISP configuration files.

HailoRT

The *hailort* field contains settings related to the Hailo Runtime.

```
{
  "hailort": {
    "device-id": "device0"
  }
}
```

- **device-id:** ID of the Hailo device.

HDR

Note:

1. Only 2DOL is supported
2. HDR and Denoise are mutually exclusive and cannot be enabled at the same time.
3. HDR is supported only with input video of 4K or FHD.

The *hdr* field contains settings related to High Dynamic Range (HDR) video.

```
{
  "hdr": {
    "enabled": false,
    "dol": 2
  }
}
```

- **enabled:** Boolean indicating whether HDR is enabled. Possible values: `true` or `false`.
- **dol:** Digital overlap for HDR.

Denoise

Note: For an example of AI denoising, you can refer to the [denoise_analytics.sh](#) script in the TAPPAS repository. This script demonstrates how to use AI denoising in combination with AI analytics

The *denoise* field contains settings related to video denoising.

```
{
  "denoise": {
    "enabled": false,
    "sensor": "imx678",
    "method": "HIGH_QUALITY",
    "loopback-count": 1,
    "network": {
      "network_path": "/usr/lib/medialib/denoise_config/vd_m_imx678.hef",
      "y_channel": "model/input_layer1",
      "uv_channel": "model/input_layer4",
      "feedback_y_channel": "model/input_layer3",
      "feedback_uv_channel": "model/input_layer2",
      "output_y_channel": "model/conv17",
      "output_uv_channel": "model/conv14"
    }
  }
}
```

- **enabled:** Boolean indicating whether denoising is enabled. Possible values: `true` or `false`.
- **sensor:** Type of sensor used.
- **method:** Denoising method used. Possible value: `HIGH_QUALITY`.
- **loopback-count:** Number of loopbacks for denoising.
- **network:** Contains settings for the denoising network:
 - **network_path:** Path to the network configuration file.
 - **y_channel:** Y-channel input layer for the network.
 - **uv_channel:** UV-channel input layer for the network.
 - **feedback_y_channel:** Feedback Y-channel input layer for the network.
 - **feedback_uv_channel:** Feedback UV-channel input layer for the network.
 - **output_y_channel:** Y-channel output layer for the network.
 - **output_uv_channel:** UV-channel output layer for the network.

Motion Detection

The *motion_detection* field contains settings related to motion detection feature.

```
{
  "motion_detection": {
    "enabled": true,
    "resolution": {
      "width": 640,
      "height": 480,
      "framerate": 30
    },
    "roi": {
      "x": 200,
      "y": 200,
      "width": 640,
      "height": 480
    },
    "sensitivity_level": "MEDIUM",
    "threshold": 0.5
  }
}
```

- **enabled:** Boolean indicating whether motion_detection is enabled. Possible values: `true` or `false`.
- **resolution:** A resolution for the motion detection stream (Includes bitmask size), containing:
 - **width:** The width of the video resolution.
 - **height:** The height of the video resolution.
 - **framerate:** The frame rate of the video.
 - **pool_max_buffers:** (Optional) The number of buffers to allocate in the pool . Default value is the maximum pool_max_buffers of output resolutions.
- **roi:** Region of interest for motion detection event to trigger, containing:
 - **x:** X-coordinate of the region.
 - **y:** Y-coordinate of the region.
 - **width:** Width of the region.
 - **height:** Height of the region.
- **sensitivity_level:** Sensitivity level of the motion detection. Possible values: LOWEST, LOW MEDIUM, HIGH, HIGHEST.
- **threshold:** Threshold for the motion detection, How much of the ROI should be changed in order to trigger motion detected. A float value between 0 and 1.

Restrictions

- Input and output resolutions are constant, and cannot be changed during pipeline running
- Frame rates must not exceed the input rate, and must be of a divisible fraction (input rate: 30/1, output rate alternatives: 1,2,3,5,6,10,15)
- Once the pipeline has been initialized, the denoise network cannot be changed

API Reference

Frontend.

2.4. Encoder-OSD Configurations

2.4.1. Encoder + OSD

The encoder + OSD configuration is done in a single JSON structure.

The API should receive the JSON as argument, this means the JSON is not read from file by the media library code, the example and/or gst-element should read from file and execute the api.

Currently only NV12 pipelines are supported for encoding.

Below JSON snippets represent the video frontend JSON and what they include.

Hailo Encoder JSON example

```
{
  "version": "1.1.0",
  "metadata": {
    "architecture": "hailo15h",
    "content_hash":
    ↪ "59360a443a5d1fac85843ef9ee69ffdfcde2e54e9711389b439a5f6b70889109",
    "description": "",
    "generation_timestamp": "2025-11-24T12:11:03.809222+00:00"
  },
  "encoding": {
    "hailo_encoder": {
      "coding_control": {
        "deblocking_filter": {
          "beta_offset": 5,
          "deblock_override": false,
          "tc_offset": -2,
          "type": "DEBLOCKING_FILTER_ENABLED"
        },
      },
      "intra_area": {
        "bottom": 0,
        "enable": false,
        "left": 0,
        "right": 0,
        "top": 0
      },
      "ipcm_area1": {
        "bottom": 0,
        "enable": false,
        "left": 0,
        "right": 0,
        "top": 0
      },
      "ipcm_area2": {
        "bottom": 0,
        "enable": false,
        "left": 0,
        "right": 0,
        "top": 0
      },
      "sei_messages": {
```

(continues on next page)

(continued from previous page)

```

        "encoder_timing_sei": true,
        "user_metadata_sei": true
    },
    },
    "config": {
        "output_stream": {
            "codec": "CODEC_TYPE_H264"
        }
    },
    "gop_config": {
        "b_frame_qp_delta": 0,
        "gop_size": 1
    },
    "monitors_control": {
        "bitrate_monitor": {
            "enable": true,
            "output_result_to_file": false,
            "period": 3,
            "result_output_path": "bitrate.txt"
        },
        "cycle_monitor": {
            "deviation_threshold": 5,
            "enable": true,
            "output_result_to_file": false,
            "result_output_path": "cycle.txt",
            "start_delay": 0
        }
    },
    "rate_control": {
        "bitrate": {
            "target_bitrate": 10000000
        },
        "gop_anomaly_bitrate_adjuster": {
            "enable": false
        },
        "intra_pic_rate": 60,
        "padding": false,
        "picture_rc": true,
        "picture_skip": false,
        "qp_smooth_settings": {
            "alpha": 0,
            "q_step_divisor": 2,
            "qp_delta": 128,
            "qp_delta_limit": 1536,
            "qp_delta_limit_step": 384,
            "qp_delta_step": 128
        },
        "quantization": {
            "qp_hdr": -1
        },
        "rc_mode": "CVBR",
        "zoom_bitrate_adjuster": {
            "mode": "DISABLED"
        }
    },
    "smart_encoder": {
        "enabled": false,
        "background_qp_delta": 10,
        "rois": []
    }
},

```

(continues on next page)

(continued from previous page)

```

    "input_stream": {
      "format": "NV12",
      "framerate": 30,
      "height": 1080,
      "width": 1920
    }
  },
  "osd": {
    "dateTime": [
      {
        "angle": 0,
        "datetime_format": "%d-%m-%Y %H:%M:%S",
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "font_size": 100,
        "id": "example_datetime",
        "rotation_policy": "CENTER",
        "text_color": [255, 0, 0],
        "x": 0.1,
        "y": 0.7,
        "z-index": 3
      }
    ],
    "image": [
      {
        "angle": 0,
        "height": 0.13,
        "id": "example_image",
        "image_path": "/home/root/apps/webserver/resources/configs/osd_hailo_
↪static_image.png",
        "rotation_policy": "CENTER",
        "width": 0.2,
        "x": 0.76,
        "y": 0.3,
        "z-index": 1
      }
    ],
    "text": [
      {
        "angle": 0,
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "font_size": 100,
        "id": "example_text1",
        "label": "HailoAI",
        "rotation_policy": "CENTER",
        "text_color": [255, 255, 255],
        "x": 0.7,
        "y": 0.05,
        "z-index": 2
      },
      {
        "angle": 0,
        "background_color": [0, 0, 0],
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "font_size": 100,
        "id": "example_text2",
        "label": "camera name",
        "rotation_policy": "CENTER",
        "text_color": [102, 0, 51],
        "x": 0.05,
        "y": 0.1,
        "z-index": 1
      }
    ]
  }
}

```

(continues on next page)

(continued from previous page)

```

    }
  ]
},
"privacy_mask": {
  "color_value": [0, 0, 0],
  "mask_type": "COLOR",
  "pixelization_size": 60
}
}

```

2.4.2. Input Stream

```

{
  "width": 1920,
  "height": 1080,
  "framerate": 30,
  "format": "NV12",
  "max_pool_size": 5
}

```

- **width**: The width of the input stream in pixels (e.g., 1920).
- **height**: The height of the input stream in pixels (e.g., 1080).
- **framerate**: The number of frames per second (e.g., 30).
- **format**: The format of the input stream (e.g., NV12).
- **max_pool_size**: (Optional) The maximum pool size of the encoder. Note that this is only supported for JPEG encoder at the moment.

2.4.3. Hailo Encoder

Config

```

{
  "config": {
    "output_stream": {
      "codec": "CODEC_TYPE_H264",
      "profile": "auto",
      "level": "auto",
      "bit_depth_luma": 8,
      "bit_depth_chroma": 8,
      "stream_type": "bytestream"
    }
  }
}

```

- **config**: Configuration for the hailo encoder.
 - **output_stream**: Output stream settings.
 - * **codec**: The codec type used. Supported values: CODEC_TYPE_H264 (H.264/AVC) and CODEC_TYPE_HEVC (H.265/HEVC).
 - * **profile**: The profile used (e.g., VCENC_H264_MAIN_PROFILE).
 - * **level**: The level used (e.g., 5.1).
 - * **bit_depth_luma**: The bit depth for luma (e.g., 8).

- * **bit_depth_chroma:** The bit depth for chroma (e.g., 8).
- * **stream_type:** The stream type (e.g., bytestream).

GOP Config

```
{
  "gop_config": {
    "gop_size": 1,
    "b_frame_qp_delta": 0
  }
}
```

- **gop_config:** Group of pictures (GOP) configuration.
 - **gop_size:** The GOP size (e.g., 1).
 - **b_frame_qp_delta:** The B-frame QP delta (e.g., 0).

Coding Control

```
{
  "coding_control": {
    "sei_messages": {
      "encoder_timing_sei": true,
      "user_metadata_sei": true
    },
    "deblocking_filter": {
      "type": "DEBLOCKING_FILTER_ENABLED",
      "tc_offset": -2,
      "beta_offset": 5,
      "deblock_override": false
    },
    "intra_area": {
      "enable": false,
      "top": 0,
      "left": 0,
      "bottom": 0,
      "right": 0
    },
    "ipcm_area1": {
      "enable": false,
      "top": 0,
      "left": 0,
      "bottom": 0,
      "right": 0
    },
    "ipcm_area2": {
      "enable": false,
      "top": 0,
      "left": 0,
      "bottom": 0,
      "right": 0
    }
  }
}
```

- **coding_control:** Coding control settings.
 - **sei_messages:** SEI messages configuration object.
 - * **encoder_timing_sei:** Enable encoder's buffering period & picture timing SEI messages (e.g., true).

- ✱ **user_metadata_sei**: Enable custom UUID + JSON metadata SEI (e.g., true).
- **deblocking_filter**: Deblocking filter settings.
 - ✱ **type**: The type of deblocking filter (e.g., DEBLOCKING_FILTER_ENABLED).
 - ✱ **tc_offset**: The TC offset value (e.g., -2).
 - ✱ **beta_offset**: The beta offset value (e.g., 5).
 - ✱ **deblock_override**: Whether deblock override is enabled (e.g., false).
- **intra_area**: Intra area settings.
 - ✱ **enable**: Whether intra area is enabled (e.g., false).
 - ✱ **top**: Top position (e.g., 0).
 - ✱ **left**: Left position (e.g., 0).
 - ✱ **bottom**: Bottom position (e.g., 0).
 - ✱ **right**: Right position (e.g., 0).
- **ipcm_area1**: IPCM area 1 settings.
 - ✱ **enable**: Whether IPCM area 1 is enabled (e.g., false).
 - ✱ **top**: Top position (e.g., 0).
 - ✱ **left**: Left position (e.g., 0).
 - ✱ **bottom**: Bottom position (e.g., 0).
 - ✱ **right**: Right position (e.g., 0).
- **ipcm_area2**: IPCM area 2 settings.
 - ✱ **enable**: Whether IPCM area 2 is enabled (e.g., false).
 - ✱ **top**: Top position (e.g., 0).
 - ✱ **left**: Left position (e.g., 0).
 - ✱ **bottom**: Bottom position (e.g., 0).
 - ✱ **right**: Right position (e.g., 0).

Rate Control

```
{
  "rate_control": {
    "rc_mode": "CVBR",
    "picture_rc": true,
    "picture_skip": false,
    "ctb_rc": false,
    "block_rc_size": 64,
    "hrd": false,
    "padding": false,
    "cvbr": 0,
    "hrd_cpb_size": 0,
    "intra_pic_rate": 60,
    "monitor_frames": 0,
    "gop_length": 0,
    "quantization": {
      "qp_min": 10,
      "qp_max": 48,
      "qp_hdr": -1,
      "intra_qp_delta": -5,
```

(continues on next page)

(continued from previous page)

```

    "fixed_intra_qp": 0
  },
  "bitrate": {
    "target_bitrate": 10000000,
    "bit_var_range_i": 2000,
    "bit_var_range_p": 2000,
    "bit_var_range_b": 2000,
    "tolerance_moving_bitrate": 15,
    "variation": 100
  }
}

```

- **rate_control**: Rate control settings.
 - **picture_rc**: Whether picture rate control is enabled (e.g., true).
 - **picture_skip**: Whether picture skip is enabled (e.g., false).
 - **ctb_rc**: Whether CTB rate control is enabled (e.g., false).
 - **block_rc_size**: The block rate control size (e.g., 64).
 - **hrd**: Whether HRD is enabled (e.g., false).
 - **padding**: Whether padding on underflow is enabled (e.g., false).
 - **cvbr**: **Flags to modify rate control behavior (can be ORed together):**
 - ✱ **0**: Default behavior.
 - ✱ **1**: Asymetric bit allocations for max and min frame size.
 - ✱ **2**: Limit QP change between frames for smoother quality changes.
 - ✱ **4**: Allow bigger QP changes at the end of the moving window.
 - ✱ **8**: Don't allow scene change detection to increase target bits for next frame.
 - **hrd_cpb_size**: The HRD CPB size (e.g., 1000000).
 - **intra_pic_rate**: The intra picture rate (e.g., 60).
 - **monitor_frames**: The number of frames to monitor (e.g., 30).
 - **gop_length**: The GOP length (e.g., 120).
 - **quantization**: Quantization settings.
 - ✱ **qp_min**: Minimum QP value (e.g., 20).
 - ✱ **qp_max**: Maximum QP value (e.g., 51).
 - ✱ **qp_hdr**: Header QP value (e.g., 26).
 - ✱ **intra_qp_delta**: Intra QP delta (e.g., -5).
 - ✱ **fixed_intra_qp**: Fixed intra QP value (e.g., 0).
 - **bitrate**: Bitrate settings.
 - ✱ **target_bitrate**: Target bitrate (e.g., 10000000).
 - ✱ **bit_var_range_i**: Bit variance range for I-frames (e.g., 10).
 - ✱ **bit_var_range_p**: Bit variance range for P-frames (e.g., 10).
 - ✱ **bit_var_range_b**: Bit variance range for B-frames (e.g., 10).
 - ✱ **tolerance_moving_bitrate**: Tolerance for moving bitrate (e.g., 0).
 - ✱ **variation**: Allowed variation (e.g., 100).

Monitors Control

```
{
  "monitors_control": {
    "bitrate_monitor": {
      "enable": true,
      "period": 3,
      "result_output_path": "bitrate.txt",
      "output_result_to_file": false
    },
    "cycle_monitor": {
      "enable": true,
      "start_delay": 0,
      "deviation_threshold": 5,
      "result_output_path": "cycle.txt",
      "output_result_to_file": false
    }
  }
}
```

- **monitors_control**: Monitors control settings.
 - **bitrate_monitor**: Bitrate monitor settings.
 - * **enable**: Whether the bitrate monitor is enabled (e.g., true).
 - * **period**: The period for the bitrate monitor (e.g., 3).
 - * **result_output_path**: The path to output the results (e.g., bitrate.txt).
 - * **output_result_to_file**: Whether to output the results to a file (e.g., false).
 - **cycle_monitor**: Cycle monitor settings.
 - * **enable**: Whether the cycle monitor is enabled (e.g., true).
 - * **start_delay**: The start delay for the cycle monitor (e.g., 0).
 - * **deviation_threshold**: The deviation threshold for the cycle monitor (e.g., 5).
 - * **result_output_path**: The path to output the results (e.g., cycle.txt).
 - * **output_result_to_file**: Whether to output the results to a file (e.g., false).

Smart Encoder

```
{
  "smart_encoder": {
    "enabled": true,
    "background_qp_delta": 10,
    "rois": [
      {
        "x": 0.1,
        "y": 0.05,
        "width": 0.2,
        "height": 0.15
      },
      {
        "x": 0.3,
        "y": 0.2,
        "width": 0.1,
        "height": 0.1
      }
    ]
  }
}
```

(continues on next page)

(continued from previous page)

```
}
}
```

- **smart_encoder:** Smart encoding settings for Region of Interest (ROI) prioritization. Allows configuring one or more high-priority regions that maintain higher visual quality while background areas are compressed more aggressively to save bitrate.
 - **enabled:** Master switch for the smart encoder feature (boolean, default: `false`). When `true`, background regions are encoded with increased QP (reduced quality) while ROI regions are encoded at full quality. This feature is only supported with H.264 codec and CVBR rate control mode.
 - **background_qp_delta:** QP (Quantization Parameter) delta applied to all frame regions not covered by a defined ROI. Higher values increase compression and reduce quality in background areas, freeing bitrate for ROI regions. Valid range: 0–16 (default: 10).
 - **rois:** Array of Regions of Interest (maximum 10 per frame, default: []). Each ROI specifies a rectangular region with normalized coordinates relative to frame dimensions, making configuration resolution-independent. Each ROI object contains:
 - ✱ **x:** Normalized x coordinate of the ROI top-left corner (float, range [0.0, 1.0]). 0.0 = left edge, 1.0 = right edge.
 - ✱ **y:** Normalized y coordinate of the ROI top-left corner (float, range [0.0, 1.0]). 0.0 = top edge, 1.0 = bottom edge.
 - ✱ **width:** Normalized width of the ROI (float, range [0.0, 1.0]).
 - ✱ **height:** Normalized height of the ROI (float, range [0.0, 1.0]).

Configuration Rules:

- ROIs must not extend beyond frame boundaries ($x + \text{width} \leq 1.0$, $y + \text{height} \leq 1.0$).
- Smart encoder can only be enabled when using H.264 codec and CVBR rate control mode.
- The feature can be configured via the Profile Manager tool and changed at runtime if required.

JPEG Encoder JSON example

```
{
  "jpeg_encoder": {
    "n_threads": 1,
    "quality": 85
  }
}
```

- **jpeg_encoder:** JPEG related configuration.
 - **n_threads:** The number of threads to use for the encoding.
 - **quality:** Quality of encoding.

Restrictions

- Input and output resolutions and format: are constant, and cannot be changed during pipeline running
- Codec type can be changed during pipeline running

API Reference

Encoder.

2.5. Tuning Encoder

2.5.1. General Encoder GOP

GOP is a group of successive pictures within a video stream. GOP usually contains an Intra frame (I-frame) and several Inter frames (Predicted frame). Intra frames allow the video to be more easily searched and editable, however they use more bits which decreases the overall quality of the stream.

User's GOP configuration includes the following parameters:

- intra-pic-rate
Intra-frame (i-frame) interval for the stream.
- gop-size
Number of b-frames (including the following p-frame).
Examples:
 - intra-pic-rate 10, GOP Size of 1: i-p-p-p-p-p-p-p ... i-p-p-p-p-p-p-p
 - intra-pic-rate 10, GOP Size of 3: i-b-b-p-b-b-p-b-b-p ... i-b-b-p-b-b-p-b-b-p
- Bitrate
Target bitrate.
- Profile
Specifies the HEVC(H.265)/AVC(H.264) profile of the generated stream.
- AVC allows the following profiles:
 - Main, used for standard-definition digital TV broadcasts that use the MPEG-4 format.
 - High, the primary profile for broadcast and disc storage applications, particularly for high-definition TV applications.
 - High-10, this profile builds on top of the High Profile, adding support for up to 10-bits per sample
- HEVC allows the following profiles:
 - Main, allows for a bit depth of 8-bits per sample with 4:2:0 chroma sampling.
 - Main-10, Allows for a bit depth of 8-bits to 10-bits per sample with 4:2:0 chroma sampling.
- Level
Specifies the HEVC(H.265)/AVC(H.264) level of the generated stream.
See the API manual for specific level per resolution and target bitrate.

2.5.2. Rate Control

The encoding bitrate and quality are controlled by setting QP value to the hardware.

The rate control is done by the software (on the A53) by adjusting the QP value per frame (in CBR and CVBR modes).

Rate Control (RC)

In general, the software RC algorithm implements the R-Q model to adaptively adjust the QP value per frame to maintain a target bitrate.

The intra-frame QP value is set according to the previous GOP's average QP, and the inter-frame QP value is set according to the previous frame's QP value.

There are several RC modes available, which differ in the way they configure the algorithm's tolerance to changes, thresholds and QP step sizes.

The QP range for RC operations is between configured qp-min and qp-max.

Additionally, several other features and configuration methods are available to adjust the RC for specific behavior.

Hailo supports the following encoding RC modes:

- Constant Bitrate (CBR)

The RC algorithm always aims to maintain the target bitrate.

In CBR mode, the RC adapts and corrects the bitrate and quality to compensate overflows and underflows in previous frames and GOPs. There are restrictions on moving average overflows and underflows to maintain accurate target bitrate.

This is the stricter RC configuration, and it allows minimal tolerance to bitrate variations.

In this mode, the QP range is wide to avoid overflows and underflows. If the qp-min is too high, the encoder may not be able to achieve the minimal target bitrate.

User may set symmetric overflow and underflow percentage to allow some flexibility in the bitrate variations. Minimal variation is 15% and maximal is 2000%. The default is 15% (minimal).

Overflows and underflows are being kept in the user specific boundaries (allowed bitrate variation). Boundaries are maintained within a GOP period.

- Constrained Variable Bitrate (CVBR)

This mode is less strict than CBR, and allows more flexibility in achieving target bitrate and fluctuations in bitrate per scene.

In CVBR mode, the RC may adapt and correct the bitrate and quality to compensate overflows in previous frames and GOPs. There are restrictions on moving average overflows to maintain accurate target bitrate.

In this mode, the user sets the minimal image-quality required (minimal QP value), which is higher than the qp-min value of the CBR to allow saving bitrate bandwidth when possible.

The CVBR presets differ in the qp-min value to allow underflows and maintain the desired quality, and the user can also override it if a higher or lower image-quality is required.

A typical use case is setting the target bitrate to the desired average and allowing additional bandwidth using the tolMovingBitrate parameter for sudden fast motions. For example, setting the target bitrate to 4.8Mbps and allowing additional 25% tolerance will allow the encoder to reach up to 6Mbps for short periods of time, while maintaining the average target bitrate of 4.8Mbps or less when scene is static.

- Variable Bitrate (VBR)

The simplest quality-only configuration is VBR, in which RC is disabled, and the user sets the QP value for the entire stream. Unless changed by a user, the QP value remains constant for the entire stream.

Output bitrate is not controlled while streaming, and is only affected by the QP value and the scene complexity.

This mode might be in use with local networks, where the bitrate is less significant (live stream without storage, etc).

Note that the CTB feature must be disabled in this mode.

Coding Tree Block (CTB) Feature

Standalone RC feature (regardless of actual configured RC mechanism), which allows setting a different QP value per CTB inside a frame.

Enabling `ctb-rc` allows the encoder to decrease QP value on flat surfaces to increase quality on these regions and reduce encoding effects.

As this feature may increase quality on specific regions in the image, it is recommended to use only when target bitrate is high enough.

This feature may be enabled in CBR and CVBR modes, and controlled by the `ctb-rc` configuration parameter.

Smart ROI Encoder

The Smart ROI Encoder allows users to mark high-priority regions in a video frame. ROI areas maintain higher visual quality, while background areas use a higher QP (lower quality) to reduce bitrate without sacrificing important details.

This feature enables efficient bitrate allocation by concentrating quality in user-defined Regions of Interest (ROIs), while allowing the encoder to compress background regions more aggressively. The background QP delta parameter controls the quality tradeoff: higher delta values increase compression in background areas, freeing bitrate for ROI regions to encode at the base QP level.

Up to 10 ROI regions can be defined per frame using normalized coordinates (0.0–1.0), making it easy to specify regions of interest relative to the frame dimensions regardless of input resolution.

Compatibility: The Smart ROI Encoder is supported in CVBR mode only with H.264 codec. ROI regions can be configured through the Camera Viewer web UI (by drawing bounding boxes) or directly in the encoder JSON configuration under the `smart_encoder` section.

Limitation: In heavy motion, complex scenes, or when ROI areas are very large, this feature may not result in significant bitrate savings due to the complexity of allocating quality appropriately across the frame.

2.5.3. Encoder Rate Control Configuration Parameters

- picture-rc

Enable or disable rate control.
- gop-length

Rate control uses the GOP length to match the average bit rate of each GOP to the target bit rate. Therefore the GOP length setting affects how fast the rate control reacts to changes in the video sequence and encoded bits per frame. e.g. a GOP length of 30 frames on a 30 fps stream will match the target bitrate every second, whereas a GOP length of 150 frames will match the target bitrate over five seconds.

Recommended value is intra-pic-rate (to maintain the bitrate per GOP). This variable shall be a multiplication of intra-pic-rate to include a fixed number of intra-frames.
- monitor-frames

Specifies how many frames will be monitored for moving bit rate.

By default, this variable is set to the intra-pic-rate (CBR) or 2*intra-pic-rate (CVBR). This variable shall be a multiplication of intra-pic-rate to include a fixed number of intra-frames.
- hrd

Enables the use of a Hypothetical Reference Decoder (HRD) model to restrict the instantaneous bitrate.

Enabling HRD will force the configured bitrate as much as possible and try to comply with a stricter bitrate policy. It might drop frames in a case where the bitrate exceeds the target bitrate.
- hrd-cpb-size

The size of the theoretical buffer is indicated by the hrd-cpb-size parameter.

Relevant only for HRD RC mode.

Normal value would be the target bitrate (has to be configured by the user).
- tol-moving-bitrate

Percentage tolerance over target bitrate for a moving bit rate.
- bitVarRangeI/bitVarRangeP/bitVarRangeB

Percentage variations over average bits per frame for I/P/B frame.

 - bitVarRangeI

Variation for I-frames, is set to minimum in CBR mode (10) and higher in CVBR mode (2000).
 - bitVarRangeP/bitVarRangeB

Variation for P/B frames, is set to 2000 to allow RC to adjust the bitrate on these frames according to the encoding policy.
- cvbr

Runtime flag for bits allocation and QP variation between frames.

Default value: 0

See presets tables for recommended value per mode/bitrate.
- padding

When the error from the theoretical buffer is smaller than the average frame size (for example, 1M bitrate at 30fps -> avg. frame size = 33.3K), padding is added to the frame to prevent underflow of the theoretical buffer.

This is available for all RC modes.

2.5.4. Encoder Rate Control Quality Parameters

- qp-min

The minimum value QP that can be used by the encoder (best quality)

- qp-max

The maximum value QP that can be set by the encoder (worst quality)

- qp-hdr

When RC is enabled, this is the initial QP value with which the encoder will start.

Default value: -1 (automatic, set by resolution and target bitrate).

In VBR mode, this value is the QP set for the entire stream.

Must be within the range of qp-min and qp-max.

- ctb-rc

Enable or disable the ctb-rc feature, recommended to enable this feature in CBR/CVBR modes when target bitrate is high enough.

- intraQpDelta

The intra frames in AVC/HEVC videos can sometimes introduce noticeable flickering because of different prediction methods. This problem can be overcome by adjusting the quantization of the intra frames compared to the surrounding inter frames.

Valid value range: [-51, 51] (full QP range)

Default value: 0

It is not recommended to use values below -5 or above 0.

- fixed-intra-qp

Use this value for all Intra picture quantization. Value 0 disables the feature. Min/Max range checking still applies. intraQpDelta does not apply when fixedIntraQp is in use.

Valid value range: [0, 51]

Default value: 0

2.5.5. Recommended Settings (General)

There are several use cases for video encoding, systems differ from one another in terms of video quality and bitrate limitations:

- **Local playback** - no storage/bitrate limitations, video quality is the main factor.
- **Internal streaming** - bitrate is restricted (yet some variation is allowed), this means video quality needs to be maintained within the boundaries.
- **Real-time streaming** - bitrate restrictions must be enforced, even at the cost of losing frames.

Recommended Rate Control Settings (General)

The table below describes the recommended ranges for the above parameters per the application requirements and device capabilities.

Table 1. Recommended rate control settings

Parameter	Local Playback	Internet Streaming	Real-time Communication
gopLen	150	30 - 150	5 - 30
pictureRc	1	1	1
ctbRc	0	0	1
hrd	0	0	1 (can skip frames)
qpMin	20 - 30	20 - 25	10
qpMax	35 - 45	45 - 50	51
qpHdr	-1	-1	-1

Recommended Rate Control Settings (Detailed)

- **General Settings**
 - Use ctb-rc for better quality and bitrate control. This feature is recommended to be enabled when target bitrate is high enough to cope with internal qp changes. When enabled in low bitrate domains, large movement will produce more trailing artifacts due to the limited bitrate.
 - Set gop-length and monitor-frames to intra-rate (CBR) or intra-rate*2 (CVBR) to maintain the bitrate in a single GOP period. See maximum allowed values per argument to keep within limits.
 - Set hrd and hrd-cpb-size according to the decoder buffer size (instead of default 10M).
 - tol-moving-bitrate should be set according to the RC mode (see below).
 - bitVarRangeI should be set to a low value in CBR mode (10) and higher in CVBR mode (100-2000).
 - bitVarRangeP/bitVarRangeB should be set to a high value (2000) to allow RC to adjust the bitrate on these frames according to the encoding policy.
 - block-rc-size For AVC use block-rc-size of 0 (16x16). For HEVC use block-rc-size of 2 (64x64).
 - hrd-cpb-size The default hrd-cpb-size is 10M. When HRD is enabled, set it to the decoder bucket size.
 - Padding Padding may be enabled in CBR/HRD mode to disallow underflows.
 - Profile and Level The encoding profile and level are set according to the encoded resolution, see table below.

Table 2. Profile and Level per resolution (AVC/HEVC)

Resolution	Profile	Level
up to 480p	Main	3.0
up to 720p	High	3.1
up to 1080p	High	4.1
up to 4K	High	5.1

• CBR Mode

- Bitrate shall match the configured target in 1 GOP interval (moving average bitrate is less of a concern)
- picture-rc is enabled
- ctb-rc is enabled only when target bitrate is high enough to cope with internal qp changes
- The Limit variable (in the table below) stands for the fluctuation of the bitrate above and below the target bitrate in percentage, minimum limitation is 15%, maximum is 2000%
- User may set a permitted overflow above the target bitrate (default is 0) however it is unlikely it will be used
- HRD and padding may be enabled to work in conjunction with CBR to create a stricter bitrate (dropping frames upon overflow, and padding with zeros when underflow) hrd-cpb-size shall be configured according to the decoder bucket size (instead of default 10M)
- See below tables for specific configuration per resolution/bitrate
- Limit stands for the allowed fluctuation below/above the target bitrate (percentage) for a GOP period
 - ✱ Minimum value is 15%
 - ✱ bitVarRangel is equal to (limitation - 5)

• CVBR Mode

- Bitrate shall not exceed the target bitrate by more than the allowed variation value, but underflows are allowed to save storage/network bandwidth on static/slow scenes
- While similar to CBR mode, bitrate fluctuations are allowed to adapt to scene changes better
- Allowing underflows is done by restricting the qp value from dropping below a certain value (depending on target bitrate), and setting a minimal compression ratio on all frames
- Typical use case is set specific target bitrate, and allow additional overflow to handle scene changes (variation in the table below)
- This feature may need tuning of qp-min per camera, scene type, and image-quality factor
- See below tables for specific configuration per resolution/bitrate
- Variation stands for the limitation that RC can overflow above the target bitrate (percentage)
 - ✱ Range is 0-2000
 - ✱ Variation is inserted "as is" into the tol-moving-bitrate parameter

• VBR Mode

- User needs to set the desired encoding quality, no rate control is enabled in this mode
- picture-rc is disabled
- ctb-rc is enabled for better quality
- qp-min and qp-max allow maximum range (1-51)
- qp-hdr sets the desired quality of the video, recommended values 21-30
- intra-qp-delta shall be no more than -5 (recommended default value)
- High bitrate is expected (at least 20Mbps in 4K)

2.5.6. Recommended Configuration - Static Camera

- Use case is recommended for static camera.
- Intra-pic-rate may be up to 4 seconds (120 when input is 30 fps).
- GOP size may be up to 8 (7 b-frames), the recommended value is 4.

2.5.7. Recommended Configuration - Moving Camera

- Recommended for moving camera and strict BR.
- Intra-pic-rate may be up to 2 seconds (60 when input is 30 fps), depends on the scene change speed.
- GOP size may be up to 4, the recommended value is 4.
- hrd may be enabled - will keep target bitrate more accurately but may drop frames when stream exceeds target bitrate.

2.5.8. Security Camera Configuration

- In general, VBR configuration should be sufficient for a security camera for both static and dynamic scenes.
- When a camera is static the bitrate will usually be equal or below the target bitrate.
- Rate control updates are available via the low-level API, gstreamer elements and media-library.
- User can set different rate control configuration for different camera states (static/moving) during execution.

2.5.9. 'Rules of Thumb'

- Quantization Parameters (qp-min/max)
 - Relevant only when RC is enabled (not in VBR mode).
 - When qp-min is set to low values, underflows are prevented and image quality is better but large scene changes increases the bitrate for a short while.
 - When qp-max set to high values, the bitrate target will be kept to its limits better, but with noticeable reduced quality on fast scene changes.
- i-frames
 - Intra frames make video easier to search and edit, but they use more bits which decreases the overall quality of the stream. Rate control uses the GOP length to match the average bit rate of each GOP to the target bit rate. So that the GOP length setting affects how fast the rate control reacts to changes in the video sequence and encoded bits per frame. The less i-frames the better bitrate/quality ratio, but it depends on if high speed scene changes require more i-frames.
 - intra-pic-rate settings should be aimed to be as high as possible according to the decoder usage and scene changes. Keeping it 120 and above gives the best VMAF/SSIM scores in both static and moving scenes.
 - gop-length and monitor frames must be kept as close to the actual intra-pic-rate to maintain the bitrate in a single GOP period.

Recommended settings are to use gop-length and monitor-frames equal to intra-pic-rate in CBR mode, and 2*intra-pic-rate in CVBR mode. See maximum allowed values per argument to keep within limits.

3. GStreamer Elements

Deprecated since version GStreamer: bin elements are deprecated. Use the higher-level API elements that wrap the MediaLibrary profile system instead. See [GStreamer API Elements](#).

3.1. Hailo Frontendbinsrc

3.1.1. Overview

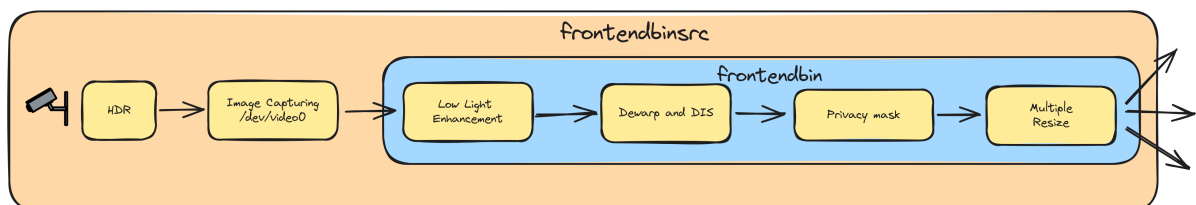
`hailofrontendbinsrc` is responsible for advanced image correction, digital and optical zoom, stream duplication and scaling, low-light enhancement and correction and future capabilities such as neural network-based image enhancement and interface control for IQ and 3A via specific APIs.

Note: Currently only NV12 format are supported by `hailofrontendbinsrc`.

3.1.2. Relation between `hailofrontend` and `hailofrontendbinsrc`

The `hailofrontendbinsrc` is a high-level GStreamer element designed to simplify the video processing pipeline by encapsulating a series of essential functions, including image capturing, HDR, and the logic provided by `hailofrontend`.

Although it is technically possible to use `hailofrontend` or even lower-level elements like `hailodewarp` and `hailomultiresize` individually, this approach is highly discouraged. These elements, when used in isolation, lead to increased complexity and a higher likelihood of errors. Therefore, Hailo strongly recommend customers utilize `hailofrontendbinsrc` as it is optimized for most applications, encapsulating best practices and reducing the need for manual intervention across the pipeline.



Parameters

The `hailofrontend` element requires the user to provide configuration for the expected behavior. Configuration determines what features are enabled/disabled and can be provided either as a JSON file or as a JSON string.

3.1.3. JSON Example

For more explanations on the JSON parameters, see the [Video Frontend Configurations](#).

```

{
  "input_video": {
    "resolution": {
      "width": 3840,
      "height": 2160,
      "framerate": 30
    }
  }
}

```

(continues on next page)

(continued from previous page)

```

},
"application_input_streams": {
  "method": "INTERPOLATION_TYPE_BILINEAR",
  "format": "IMAGE_FORMAT_NV12",
  "grayscale": false,
  "resolutions": [
    {
      "width": 3840,
      "height": 2160,
      "framerate": 30,
      "pool_max_buffers": 20,
      "scaling_mode": "STRETCH"
    },
    {
      "width": 1280,
      "height": 720,
      "framerate": 30,
      "pool_max_buffers": 20,
      "scaling_mode": "STRETCH"
    }
  ]
},
"application_analytics": {},
"dewarp": {
  "enabled": true,
  "color_interpolation": "INTERPOLATION_TYPE_BILINEAR",
  "sensor_calib_path": "/etc/imaging/cfg/hailo15h/imx678/kit/4k/shared/
↪calibration/cam_intrinsics.txt",
  "camera_type": "CAMERA_TYPE_PINHOLE"
},
"dis": {
  "enabled": false,
  "minimum_coefficient_filter": 0.1,
  "decrement_coefficient_threshold": 0.001,
  "increment_coefficient_threshold": 0.01,
  "running_average_coefficient": 0.033,
  "std_multiplier": 3.0,
  "black_corners_correction_enabled": true,
  "black_corners_threshold": 0.5,
  "average_luminance_threshold": 0,
  "camera_fov_factor": 0.85,
  "angular_dis": {
    "enabled": false,
    "vsm": {
      "hoffset": 1856,
      "voffset": 1016,
      "width": 1920,
      "height": 1080,
      "max_displacement": 64
    }
  }
},
"debug": {
  "generate_resize_grid": false,
  "fix_stabilization": false,
  "fix_stabilization_longitude": 0.0,
  "fix_stabilization_latitude": 0.0
},
},
"eis": {
  "enabled": false,
  "stabilize": true,

```

(continues on next page)

(continued from previous page)

```

    "eis_config_path": "/home/root/apps/resources/eis_calibration.json",
    "window_size": 10,
    "rotational_smoothing_coefficient": 0.0,
    "iir_hpf_coefficient": 0.997,
    "camera_fov_factor": 0.85,
    "line_readout_time": 7410,
    "hdr_exposure_ratio": 0.2,
    "min_angle_deg": 0.005
  },
  "gyro": {
    "enabled": false,
    "sensor_name": "lsm6dsr_gyro",
    "sensor_frequency": "833.000000",
    "scale": 0.000152716
  },
  "gmvr": {
    "source": "isp",
    "frequency": 0.0
  },
  "optical_zoom": {
    "enabled": true,
    "magnification": 1.0
  },
  "digital_zoom": {
    "enabled": false,
    "mode": "DIGITAL_ZOOM_MODE_ROI",
    "magnification": 1.0,
    "roi": {
      "x": 200,
      "y": 200,
      "width": 2800,
      "height": 1800
    }
  },
  "motion_detection": {
    "enabled": false,
    "resolution": {
      "width": 640,
      "height": 480,
      "framerate": 30
    },
    "roi": {
      "x": 0,
      "y": 0,
      "width": 640,
      "height": 480
    },
    "sensitivity_level": "MEDIUM",
    "threshold": 0.5,
    "buffer_pool_size": 8
  },
  "rotation": {
    "enabled": false,
    "angle": "ROTATION_ANGLE_180"
  },
  "flip": {
    "enabled": false,
    "direction": "FLIP_DIRECTION_HORIZONTAL"
  },
  "isp": {
    "isp_config_files_path": "/usr/bin"
  }

```

(continues on next page)

(continued from previous page)

```

},
"haulort": {
  "device-id": "device0"
},
"hdr": {
  "enabled": false,
  "dol": 2
},
"denoise": {
  "enabled": false,
  "sensor": "imx678",
  "method": "HIGH_QUALITY",
  "loopback-count": 1,
  "network": {
    "network_path": "/usr/lib/medialib/denoise_config/vd_m_imx678.hef",
    "y_channel": "model/input_layer1",
    "uv_channel": "model/input_layer4",
    "feedback_y_channel": "model/input_layer3",
    "feedback_uv_channel": "model/input_layer2",
    "output_y_channel": "model/conv17",
    "output_uv_channel": "model/conv14"
  }
}
}
}

```

3.1.4. Hierarchy

```

Object
+----GInitiallyUnowned
  +----GstObject
    +----GstElement
      +----GstBin
        +----GstHailoFrontend

Implemented Interfaces:
GstChildProxy

Pad Templates:
SINK template: 'sink'
Availability: Always
Capabilities:
ANY

SRC template: 'src_%u'
Availability: On request
Capabilities:
ANY

Element has no clocking capabilities.
Element has no URI handling capabilities.

Pads:
SINK: 'sink'
  Pad Template: 'sink'

Element Properties:

async-handling    : The bin will handle Asynchronous state changes
                    flags: readable, writable

```

(continues on next page)

(continued from previous page)

```

        Boolean. Default: false
config      : Frontendbinsrc config as frontend_config_t
            flags: readable, writable, changeable in NULL, READY, PAUSED or PLAYING
↪state
        Pointer.
config-file-path : JSON config file path to load
            flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
        String. Default: null
config-string   : JSON config string to load
            flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
        String. Default: null
freeze         : Freeze the image
            flags: readable, writable, changeable in NULL, READY, PAUSED or PLAYING
↪state
        Boolean. Default: false
hailort-config  : HailoRT config as hailort_t
            flags: readable
        Pointer.
hdr-config     : HDR config as hdr_config_t
            flags: readable
        Pointer.
input-video-config : video input config as input_video_config_t
            flags: readable
        Pointer.
isp-config     : isp config as isp_t
            flags: readable
        Pointer.
message-forward : Forwards all children messages
            flags: readable, writable
        Boolean. Default: false
name          : The name of the object
            flags: readable, writable, 0x2000
            String. Default: "hailofrontendbinsrc0"
num-buffers    : Number of buffers to output before sending EOS (-1 = unlimited)
            flags: readable, writable
            Integer. Range: -1 - 2147483647 Default: -1
parent        : The parent of the object
            flags: readable, writable, 0x2000
            Object of type "GstObject"
privacy-mask   : Pointer to privacy mask blender
            flags: readable
        Pointer.

Children:
hailofrontendelement
queue0
frontendcapsfilter
v4l2src0

```

3.2. Hailo Encoder

3.2.1. Overview

hailoencodebin is a bin element that enables the user to encode a video in H265/H264/JPEG coding format, using the **Hailo-15 encoding hardware accelerator**.

It also supports *OSD* (On Screen Display) to blend overlays on incoming frame.

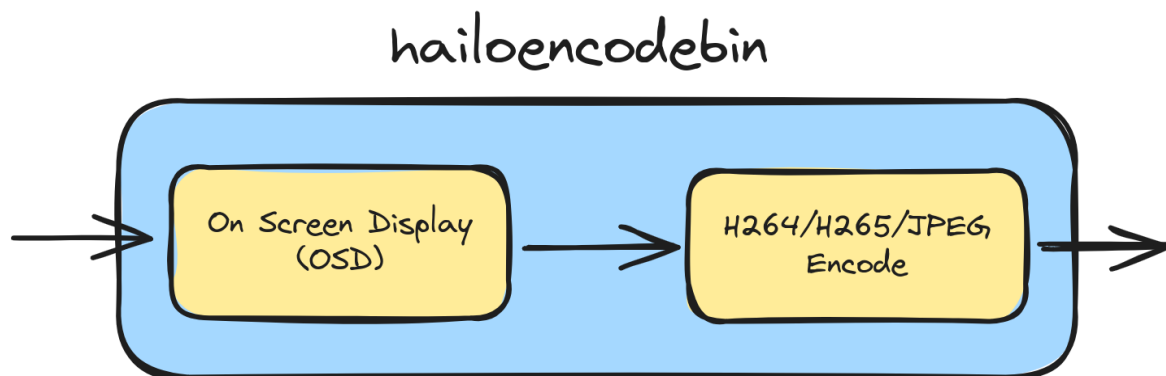
Note: JPEG Encoding is not hardware accelerated.

3.2.2. Relation between hailoencodebin, hailoencoder and hailosd

The hailoencodebin is a GStreamer element designed to handle the final stages of video processing by encapsulating both the OSD and encoding functions.

The hailoencodebin includes an OSD component, which overlays information directly onto the video stream, making it visible during playback. Following the OSD, the hailoencodebin handles the encoding of the video into formats such as H.264, H.265, or JPEG.

While it is technically possible to use the lower-level elements hailoencoder and hailosd individually, this approach is not recommended. By bundling these functionalities into a single element, hailoencodebin simplifies the pipeline and ensures efficient processing.



Parameters

The hailoencodebin element provides a variety of properties that control the encoding performance and quality.

The element requires the user to provide configuration for the expected behavior.

Configuration determines what features are enabled/disabled and can be provided either as a JSON file or as a JSON string.

Changing Configurations at Runtime

The configuration can be changed during the pipeline execution.

Changes to such properties during the pipeline will be updated at the end of the group of images (GOP).

A change in bitrate during the pipeline will be applied at the end of the GOP, for instance.

In the following example, the user get the encoder element and change the bitrate at runtime:

```
GstElementPtr encoder_element = glib_cpp::ptrs::get_bin_by_name(pipeline, "encoder
↪");
```

Get the current user configuration from the encoder element:

```
gpointer value = nullptr;
g_object_get(encoder_element.as_g_object(), "user-config", &value, NULL);
encoder_config_t *config = reinterpret_cast<encoder_config_t *>(value);
```

Set a new target_bitrate value and update the configuration:

```
config->rate_control.bitrate.target_bitrate = 20000000;
g_object_set(encoder_element.as_g_object(), "user-config", config, NULL);
```

Getting the actual configuration at runtime

The actual running configuration (After setting defaults and values from a config preset), can be retrieved at runtime.

In the following example, the user gets the encoder element and reads the actual gop_length at runtime:

```
GstElementPtr encoder_element = glib_cpp::ptrs::get_bin_by_name(pipeline, "encoder
↪");
```

Get the current actual configuration from the encoder element:

```
gpointer value = nullptr;
g_object_get(encoder_element.as_g_object(), "config", &value, NULL);
encoder_config_t *config = reinterpret_cast<encoder_config_t *>(value);
```

Read the actual gop_length value:

```
auto gop_length = config->rate_control.gop_length;
```

3.2.3. Hierarchy

```
GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstBin
+----GstHailoEncodeBin

Implemented Interfaces:
GstChildProxy
```

(continues on next page)

(continued from previous page)

Pad Templates:

SINK template: 'sink'
Availability: Always
Capabilities:
ANY

SRC template: 'src'
Availability: Always
Capabilities:
ANY

Element has no clocking capabilities.
Element has no URI handling capabilities.

Pads:

SINK: 'sink'
Pad Template: 'sink'
SRC: 'src'
Pad Template: 'src'

Element Properties:

```

async-handling : The bin will handle Asynchronous state changes
                flags: readable, writable
                Boolean. Default: false
blender        : Pointer to blender object
                flags: readable, controllable
                Pointer.
config         : Pointer to the actual config object
                flags: readable
                Pointer.
user-config    : Pointer to the user config object
                flags: readable, writable, changeable in NULL, READY, PAUSED or PLAYING
↪state
                Pointer.
config-file-path : JSON config file path to load
                flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
                String. Default: null
config-string   : JSON config string to load
                flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
                String. Default: null
enforce-caps    : Enforce caps on the input/output pad of the bin
                flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
                Boolean. Default: true
message-forward : Forwards all children messages
                flags: readable, writable
                Boolean. Default: false
name           : The name of the object
                flags: readable, writable, 0x2000
                String. Default: "hailoencodebin0"
parent         : The parent of the object
                flags: readable, writable, 0x2000
                Object of type "GstObject"
queue-size     : Size of the queues in the bin, there are 2 queues.
                flags: readable, writable, controllable, changeable in NULL, READY,
↪PAUSED or PLAYING state
                Unsigned Integer. Range: 1 - 4294967295 Default: 2
wait-for-writable-buffer: Enables the element thread to wait until incoming buffer
↪is writable

```

(continues on next page)

(continued from previous page)

```

    flags: readable, writable, controllable, changeable in NULL, READY,
    ↪ PAUSED or PLAYING state
    Boolean. Default: false

```

4. GStreamer API Elements

4.1. Overview

The GStreamer API elements provide an interface for building video capture and encoding pipelines using the MediaLibrary C++ API (`media_library.hpp`). These elements wrap the `MediaLibrary` class and use the profile-based configuration system.

Two elements are provided:

- **gsthailovision** — A source element that creates and owns the shared `MediaLibrary` instance for a pipeline. Manages video capture via the frontend and exposes one dynamically-requested source pad per output stream.
- **gsthailoencoder** — A video encoder element that obtains the shared `MediaLibrary` instance and delegates H.264/H.265 encoding to it, identified by `stream-id`.

4.2. Architecture

A typical pipeline places one `gsthailovision` element at the head and one or more `gsthailoencoder` elements downstream. The `gsthailovision` element initializes the `MediaLibrary` and registers it in a per-pipeline singleton registry (`MediaLibInstanceRegistry`). Each `gsthailoencoder` element looks up the same `MediaLibrary` instance using the pipeline name as the registry key. This ensures all elements in a pipeline share a single `MediaLibrary` instance and its lifecycle.

Each output stream is represented by a dynamically-requested source pad on `gsthailovision` (e.g., `v.sink0`, `v.sink1`), connected through a queue to a corresponding `gsthailoencoder` element identified by its `stream-id` property. The encoder output is then packetized (e.g., via `rtph264pay` or `rtph265pay`) and sent to a sink such as `udpsink` or `filesink`.

For a ready-to-run shell script demonstrating this pipeline layout, see `hailo-analytics/apps/case_studies/gst_vision_encoder/gst_vision_encoder_pipeline.sh`.

4.3. Reference Code Examples

Reference code examples are available at `hailo-media-library/api/examples/gst_vision_encoder/gst_vision_encoder_example.cpp`. It demonstrates pipeline construction, runtime profile switching, and specific parameter overrides.

For an advanced example that combines GStreamer elements with the Hailo AI Analytics tiling detection and overlay application, see the `gst_vision_analytics_encoder` reference pipeline at `hailo-analytics/apps/case_studies/gst_vision_analytics_encoder/`. It demonstrates how to bridge GStreamer-based video I/O with analytics processing in a single application.

4.3.1. Hailo Vision

Overview

`gsthailovision` is a GStreamer source element that creates, owns, and manages the shared `MediaLibrary` instance for a pipeline. It drives video capture via the `MediaLibrary` frontend and exposes one dynamically-requested source pad per output stream defined in the active profile.

Downstream `gsthailoencoder` elements automatically obtain the same `MediaLibrary` instance via the internal `MediaLibInstanceRegistry`, using the pipeline name as the lookup key.

Configuration

The element accepts configuration through two mutually exclusive properties, which must be set before the element transitions to READY:

- `config-string` — An inline MediaLibrary JSON configuration string.
- `config-path` — A file path to a MediaLibrary JSON configuration file.

Only one of the two may be set. Setting a second one after the first is already set will result in an error.

For details on the JSON configuration format, see [Encoder-OSD Configurations](#).

Runtime Configuration

Profile Switching

Switch to a different profile at runtime:

```
g_object_set(vision, "profile-name", "Lowlight", NULL);
```

Reading the Current Profile

Read the active profile configuration. The caller owns the returned pointer and must delete it:

```
config_profile_t *profile = nullptr;
g_object_get(vision, "current-profile", &profile, NULL);
// ... use profile ...
delete profile;
```

Overriding Parameters

Apply partial parameter changes (e.g. resolution) without a full profile switch:

```
config_profile_t modified = *profile;
// modify fields in modified...
g_object_set(vision, "override-profile", &modified, NULL);
```

For a complete usage example, see `hailo-media-library/api/examples/gst_vision_encoder/gst_vision_encoder_example.cpp`.

Dynamic Pads

Source pads are created on request. The pad name must correspond to a stream ID from the active profile's `encoded_output_streams` map. For example, in a pipeline string:

```
gsthailovision name=v config-path=config.json v.sink0 ! ...
```

The pad named `sink0` will be matched to the stream ID `sink0` in the profile. Each stream ID can only be claimed by one pad.

Hierarchy

```
GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstHailoVision
```

Pad Templates

```
SRC template: '%s'
Availability: On request
Capabilities:
  ANY
```

Element Properties

Property	Type	Flags	Description
config-string	String	R/W, READY	Inline MediaLibrary JSON configuration string.
config-path	String	R/W, READY	Path to a MediaLibrary JSON configuration file.
profile-name	String	R/W, PLAYING	Profile name to switch to at runtime.
current-profile	Pointer	Read-only	Returns a newly allocated <code>config_profile_t*</code> of the active profile. Caller owns the pointer and must delete it.
override-profile	Pointer	Write-only, PLAYING	Pointer to <code>config_profile_t</code> to apply via <code>set_override_parameters()</code> without a full profile switch.

4.3.2. Hailo Encoder

Overview

`gsthailoencoder` is a GStreamer video encoder element that delegates encoding to the MediaLibrary backend. It is identified by its `stream-id` property, which must match a key in the active profile's `encoded_output_streams` map. Designed to be used alongside `gsthailovision` in the same pipeline.

MediaLibrary Instance Sharing

During the READY to PAUSED transition, the encoder blocks and waits for the `gsthailovision` element to create and initialize the MediaLibrary instance. The pipeline name is used as the registry key. The `stream-id` is claimed atomically — only one `gsthailoencoder` element per stream ID is allowed in a given pipeline.

Output Capabilities

The encoder supports **H.264** and **H.265 (HEVC)** output in byte-stream format. The codec is selected based on the codec field in the active profile's encoder configuration (CODEC_TYPE_H264 or CODEC_TYPE_HEVC).

Output caps are derived automatically from the active profile's encoder configuration for the given stream ID.

Hierarchy

```
GObject
+----GInitiallyUnowned
+----GstObject
+----GstElement
+----GstVideoEncoder
+----GstHailoApiEncoder
```

Pad Templates

```
SINK template: 'sink'
Availability: Always
Capabilities:
    ANY

SRC template: 'src'
Availability: Always
Capabilities:
    video/x-h264,
        stream-format = (string) byte-stream,
        alignment = (string) au,
        profile = (string) { base, main, high }
    video/x-h265,
        stream-format = (string) byte-stream,
        alignment = (string) au,
        profile = (string) { main, main-10 }
```

Element Properties

Property	Type	Flags	Description
stream-id	String	R/W, READY	Mandatory stream identifier matching a key in the profile's encoded_output_streams map.

5. Changelog

Media Library v1.11.0 (March 2026)

New Features and Enhancements:

- Added dedicated wrapper APIs for frontend and encoder operations
- Added GStreamer elements for profile-based pipeline construction, including hailovision (frontend wrapper) and hailoencodebin (encoder wrapper), enabling seamless Media Library integration in GStreamer pipelines
- Added support for the LAION-2B vision-language model, multi-class classification and search, and full-frame indexing for improved out-of-class embedding capabilities in the CLIP model
- Added new analytics pipeline application for license plate detection and recognition
- Added region of interest (ROI) dynamic encoding, enabling higher quality encoding for selected regions while reducing bandwidth for background areas
- Hailo-15H only: Added user-space control and HDR mode support for Sony IMX662 Camera Sensor
- Integrated Dynamic Privacy Masking (DPM) into the webserver application with a dedicated configuration endpoint. Supports all output resolutions with configurable IoU thresholds and dilation size
- Added runtime control to enable or disable SDR AI-ISP without requiring a pipeline restart
- Deprecated the AI example application

Bug Fixes:

- Fixed "Hailo-15H only: Electronic Image Stabilization (EIS) may produce visual artifacts along frame boundaries under high-amplitude vibrations"
- Fixed "The CLIP model may occasionally fail if the source media was removed by the automatic cleanup mechanism"

Media Library v1.10.1 (February 2026)

Bug Fixes:

- Fixed "Hailo-15L only: The CLIP model is not supported when using the Sony IMX678 camera sensor"
- Fixed "Hailo-15H only: In Hailo AI Analytics applications, where multiple faces, appear bounding boxes and facial landmarks may flicker"

Media Library v1.10.0 (January 2026)

New Features and Enhancements:

- AI-ISP Gen2: Added support for AI-ISP Gen2 with Denoise capabilities. This advanced AI-powered denoising enhance the image quality in low-light conditions while preserving fine details.
- Persistent Configuration: Introduced persistent configuration support for saving and loading runtime configurations across system reboots and application restarts. This feature simplifies profile switching and ensures consistent behavior in production environments.
- Hailo AI Analytics: Fully integrated Hailo AI Analytics into the Media Library, supporting AI-powered video analytics including object detection, tracking, and instance segmentation. Includes a new Analytics Viewer tool for real-time visualization of AI results on the host system.
- Profile Configuration: Enhanced the profile system with comprehensive rule checking and validation mechanisms. Provides improved error detection, consistency checks, and detailed error messages to help catch misconfigurations early.

- **Perfetto Tracing Modes:** Added two tracing modes—‘applications’ for high-level performance analysis and ‘applications_detailed’ for in-depth, granular timing information. Both modes are supported in the Media Library and Hailo AI Analytics.
- **HDR/SDR Switching:** Significantly improved switching time between SDR and HDR modes, minimizing delay and reducing visual artifacts during mode transitions. Only Hailo-15H.
- **Profile Backup and Restore:** Added profile backup and restore functionality for complete profile configurations, including sensor settings, encoding parameters, Image Quality (IQ) tuning, masking, and OSD settings.
- **Dynamic Privacy Mask:** Improved Dynamic Privacy Mask performance with new neural networks offering higher accuracy and better performance, including improved handling of challenging scenarios such as partial occlusions and varying lighting conditions.
- Fixed “system reboot issue presented as ‘encoder error -11”
- Fixed “in some use cases CLIP application failed to open”
- Fixed “memory leak issues in camera viewer”. This issue was revealed in long-run tests
- Fixed “AI Example application stability issues”. Different errors occurred in long-run tests
- Fixed “Motion detection ROI stability issues. When enabling the feature, occasionally error occurred”

Media Library v1.9.1 (December 2025)

Bug Fixes:

- Fixed “error may occur when enabling motion detection mode with invalid x and y values in the JSON file”
- Fixed “VPS header was not generated when using the H265 encoder”

Media Library v1.9.0 (October 2025)

New Features and Enhancements:

- **Pre-ISP 12-bit Denoise:** Introduced advanced pre-ISP 12-bit denoising capabilities for enhanced image quality processing before ISP operations, supporting higher bit depth processing for improved noise reduction.
- **Multi-Sensor Registry:** Enhanced multi-sensor support with improved sensor registry system, providing better sensor detection, configuration management, and seamless switching between different sensor configurations.
- **Enhanced Perfetto Tracing:** Expanded Perfetto tracing capabilities with new denoise-specific traces, enabling improved performance analysis and debugging of denoising operations throughout the media pipeline.
- **Profile Configuration Separation:** Significant improvements to the profile-based configuration system including separation into multiple specialized configuration files for better modularity, maintainability, and customization of sensor, application, stabilizer, IQ, encoding, OSD, and masking settings.

Bug Fixes:

- **APPSRC Support:** Fixed compatibility issues with GStreamer APPSRC elements, improving raw-frame application-based source handling and pipeline stability. This feature is primarily useful for debugging purposes.
- **EIS Artifacts on Sharp Movement:** Resolved Electronic Image Stabilization (EIS) artifacts that occurred during sharp camera movements, improving stabilization quality and reducing visual distortions.

Media Library v1.8.1 (August 2025)

New Features and Enhancements:

- Added support for imx664 sensor.

Media Library v1.8.0 (July 2025)

New Features and Enhancements:

- SoC Profiler: Integrated SoC profiler, performance tracing and profiling system for improved debugging and analysis of media library operations.
- Dynamic Privacy Mask: Enhanced privacy mask functionality with improved dynamic masking capabilities based on analytics database integration for real-time object detection and segmentation.
- Analytics Database: Implemented comprehensive analytics database system for storing and querying detection and instance segmentation data with temporal management, supporting closest, exact, and within-delta query types.
- Snapshot: Added advanced snapshot debugging feature with CLI tool support, enabling capture of pipeline states at multiple stages for debugging and quality analysis.

Bug Fixes:

- Fixed a couple of rare bugs in various locations

Media Library v1.7.1 (May 2025)

Bug Fixes:

- Fixed a rare race condition in the denoise module.
- Fixed aspect ratio bug.
- Fixed rotation by 90 degrees bug.
- Various profile configuration fixes.
- Fixed stability bugs in the EIS gyro device module.

Media Library v1.7.0 (March 2025)

New Features and Enhancements:

- Introduced a new profile-based configuration system for the camera application.
- Media library logger now supports controlling verbosity levels per module.
- Media library now includes thermal monitoring support.
- Denoise module now supports fast model switching at runtime.
- Multi Resize now includes an option to keep the aspect ratio.
- Removed throw statements to improve robustness.

Bug Fixes:

- Fixed various memory leaks, file descriptor leaks, and other bugs.
- Fixed a rare race control conditions in the denoise module.

Media Library v1.6.1 (February 2025)

Bug Fixes:

- Fixed various memory leaks and bugs.

Media Library v1.6.0 (January 2025)

New Features and Enhancements:

- Improved Performance: Enhanced encoder performance under heavy CPU load.
- Introduced Motion Detection in media library frontend

Bug Fixes:

- Resolved various issues to improve stability and reliability.

Media Library v1.5.2 (December 2024)

New Features and Enhancements:

- Added post-denoise filters to enhance video quality.
- Introduced a video freeze feature in the frontend.
- Media library file descriptor duplication is now configurable through an environment variable, and is disabled by default.
- Added a pause option to the EIS module, designed for improved usability with PTZ cameras.

Bug Fixes:

- Resolved an issue causing encoder buffer overflow.
- Fixed ambiguous return values in the frontend's "Get Current FPS" function.
- Addressed a bug in encoder SPS handling.
- Various bug fixes in the EIS module.

Media Library v1.5.1 (November 2024)

New Features and Enhancements:

- HDR 2DOL support in HML
- Improved OSD Text quality

Bug Fixes:

- Fixed various memory leaks and bugs

Media Library v1.5.0 (October 2024)

New Features and Enhancements:

- Media Library API Update - Refactored the Media Library API to utilize smart pointers, enhancing memory management and safety.
- Switched Media library buffer management to be based only on DMA-Buf
- EIS Support
- HDR Configuration in Frontend - Introduced a new configuration option for HDR in the frontend module.

- Enable MJPEG configuration during runtime
- General Improvement Encoder quality

OSD features:

- Added support for alignment properties in overlays
- Added font weight (Bold) support for Text and DateTime overlays
- Added outline support for Text and DateTime overlays
- Added methods to retrieve the displayed overlay dimensions in pixels for Text and DateTime overlays
- Updated *single_stream_osd* example to include a variety of OSD options
- Improved color quality and background alignment, particularly in low-resolution outputs

Bug Fixes:

- Memory Leaks fixes
- Fix encoder gop size large than 1 and gop length larger than 30
- Fix Pipeline when stopping and starting
- Fix AI denoising when stopping and starting
- Fix Dewarp + AI denoising toggle
- Bugfix: Fixed an issue where *background_color* was not set for *TextOverlay* when using the C++ API
- Bugfix: Fixed an issue where shadow properties were not set for *DateTimeOverlay* when using the C++ API

Media Library v1.4.0 (July 2024)

New Features and Enhancements:

- Added configure C++ API to the *hailofrontend* module
- The force keyframe API is now available through our *hailoencoder*.
- OSD: Added shadow support for text and datetime overlays.
- OSD: Added alpha (RGBA) support for foreground, background, and shadow colors in text/datetime overlays.

Bug Fixes:

- Fixed multiple memory leaks.
- Fixed an issue with ARGB blending in our example.

Media Library v1.3.1 (May 2024)

New Features and Enhancements:

- JPEG-Encode C++ API: Introduced a new API for JPEG encoding through C++ API.
- ARGB blending in Hailo-OSD Element: Added the ability to send ARGB buffer for blending through Hailo-OSD element.
- Datetime format in Hailo-OSD: Added the ability to configure the datetime format in OSD.

Bug Fixes:

- Hailo AI-Denoise fixes: Resolved several issues in *hailofrontend*, related to stream AI-Denoise toggle.
- Memory leaks Bug fixes: Fixed memory leaks in various elements including *hailofrontend* and *hailoencoder*.

Media Library v1.3.0 (April 2024)

New Features and Enhancements:

- DIS Performance Improvement: Added new heuristics to enhance DIS .performance across various scenarios, in particular focusing on angular and low-light.
- Front-end C++ API Configuration: Introduced a new configuration option for the frontend C++ API.
- DMA-Buf Support: Implemented DMA-Buf support across the media library.
- Denoising Performance Enhancement: Improved denoising performance.
- Reconfiguration Option for Hailo-OSD Element: Added the ability to reconfigure the Hailo-OSD element.
- Front-end Multi-Resize Output Resolution: Removed the cap on output resolution for Front-end multi-resize, no longer restricted to divisors of 30.

Preview Features:

- Native AI Example: Included a preview of a native AI example.

Bug Fixes:

- Hailo encoder fixes: Resolved several issues in hailo encoder, related to stream restarts and closure
- Privacy Mask Bug Fixes: Addressed various edge-case bugs related to privacy mask functionality.

Media Library v1.2.2 (March 2024)

- Updated webserver features and fixed some stabilization issue
- Changed default parameters of encoder.
- Aligned OSD api for text and datetime (now using the same font size scale).
- New encoder dynamic update is now possible
- Changing framerate explicitly is currently not working in new encoder, will be supported in next release.
- Multi Resize supports more framerates (int 1-30)
- Overall bug fixes

Media Library v1.2.1 (February 2024)

- Camera is automatically detected (imx334/imx678) for dewarp mesh
- New interface for privacy mask in frontend module
- Added background color support in OSD
- Fixed SPS/PPS calculations in encoder module
- Fixed Rotation/Flip

Media Library v1.2.0 (January 2024)

- New front end element enabling the control of video input features (LDC/dewarp/multi resize) in a single element
- Multi-thread JPEG SW encoding (allows more TP when encoding MJPEG by using several CPU cores)
- Refined OSD rotation feature (1 degree granularity)
- Updated hailo upload element to use GP-DMA
- Optical zoom support

Media Library v1.1.2 (December 2023)

- Detached OSD support
- Dynamic fonts supported in OSD element
- Greyscale support

Media Library v1.1.1 (November 2023)

- Performance improvements
- Benchmark TAPPAS application `Basic security camera` runs at 30 fps
- Dynamic overlay support - user application can draw/update its own overlays
- Dynamic OSD - user can update the OSD elements (text, image, timestamp)
- Dynamic digital zoom support (for center ROI) - user can change magnification
- Pre-proc example includes the new features
- Bugs fixes

Media Library v1.1.0 (October 2023)**Media Library C++**

- Lens distortion correction (4K)
- Digital image stabilization (4K, based on VSM from ISP)
- Digital zoom (bilinear, bicubic)
- Multiple rescale kernel
- Encoder C++ API
- OSD C++ API

Media Library GStreamer Plugin

- Plugin created with elements to offer Media Library C++ features

6. Cross-Compiling with Meson

This guide covers cross-compilation for the Media Library components using the Meson build system.

6.1. Prerequisites

1. **Clone the repo** and enter its directory:

```
git clone |repo-url|
cd hailo-media-library
```

2. **Source your Yocto SDK** so that CC, CXX, PKG_CONFIG_PATH, etc. point at the cross toolchain:

```
source /path/to/yocto-sdk/environment-setup-aarch64-poky-linux
```

6.2. Compiling Hailo Media Library

```
cd hailo-media-library

# 1. Configure build directory:
meson setup build \
  --buildtype=debug \
  --prefix=/usr \
  -Dplatform=15h

# 2. Compile:
ninja -C build

# 3. Install to temporary directory:
DESTDIR=/tmp/install ninja -C build install

# 4. Copy files to target device:
scp -r /tmp/install/* root@<target-ip>:/
```

Note: To switch to a **release** build, use `--buildtype=release` instead of `--buildtype=debug`.

Note: For Hailo-15L, use `-Dplatform=15l` instead of `-Dplatform=15h`.

6.2.1. Available Options

- `-Dplatform=[15l|15h]` - Target platform (default: 15h)
- `-Dinclude_unit_tests=[true|false]` - Include unit tests (default: true)
- `-Dbuild_docs=[true|false]` - Build documentation (default: false)
- `--buildtype=[debug|release]` - Build type

6.3. Compiling Hailo AI Analytics

```
cd hailo-analytics

# 1. Configure build directory:
meson setup build \
  --buildtype=debug \
  --prefix=/usr \
  -Dplatform=15h \
  -Dapps_install_dir=/home/root/apps

# 2. Compile (with job limit to prevent memory exhaustion):
ninja -C build -j 10 -l 10

# 3. Install to temporary directory:
DESTDIR=/tmp/install ninja -C build install

# 4. Copy files to target device:
scp -r /tmp/install/* root@<target-ip>:/
```

Note: The `-j 10 -l 10` flags limit both compilation jobs and linker jobs to prevent memory exhaustion during large builds.

6.3.1. Available Options

- `-Dplatform=[15l|15h]` - Target platform (default: 15h)
- `-Dapps_install_dir=<path>` - Installation directory for applications (recommended: `/home/root/apps`)
- `-Dbuild_docs=[true|false]` - Build documentation (default: false)
- `--buildtype=[debug|release]` - Build type

6.4. Compiling Hailo Postprocess

```
cd hailo-postprocess

# 1. Configure build directory:
meson setup build \
  --buildtype=debug \
  --prefix=/usr \
  -Dplatform=15h \
  -Dpost_processes_install_dir=/usr/lib/hailo-post-processes/

# 2. Compile:
ninja -C build

# 3. Install to temporary directory:
DESTDIR=/tmp/install ninja -C build install

# 4. Copy files to target device:
scp -r /tmp/install/* root@<target-ip>:/
```

6.4.1. Available Options

- `-Dplatform=[15l|15h]` - Target platform (default: 15h)
- `-Dpost_processes_install_dir=<path>` - Installation directory for post-process modules (recommended: `/usr/lib/hailo-post-processes/`)
- `--buildtype=[debug|release]` - Build type

6.5. Troubleshooting

6.5.1. Ninja Permission Issues

After install, the `.ninja_log` file may be owned by root. Fix with:

```
sudo chown $(id -u):$(id -g) build/.ninja_log
```

6.5.2. Memory Issues During Build

If you encounter out-of-memory errors during compilation (especially with Hailo AI Analytics):

```
# Limit parallel jobs
ninja -C build -j 4 -l 4

# Or for very constrained systems
ninja -C build -j 1
```

6.5.3. Missing Dependencies

If you see errors about missing packages or libraries:

1. Ensure your Yocto SDK is properly sourced
2. Verify that `PKG_CONFIG_PATH` points to the toolchain's pkg-config directory
3. Check that required dependencies are installed in the toolchain sysroot

```
# Verify environment
echo $CC
echo $CXX
echo $PKG_CONFIG_PATH
```

6.5.4. Build Configuration Errors

If meson reports configuration errors:

1. Check that the platform value is correct (15l or 15h)
2. Verify that all required build dependencies are available
3. Review the meson log:

```
cat build/meson-logs/meson-log.txt
```

7. Using The Encoder

7.1. Introduction

The following manual describes the H-15 AVC/HEVC encoder stack internals, low level API, and functionalities.

Most user applications shall use the Hailo Media Library APIs, which provides further abstraction on top of the low level encoder stack. The main functionalities of the encoder are interfaced in the media-library [encoder block section](#), with much more convenient and easy to use API and also provides pipeline integration with the Media Library frontend.

Moreover, most of media-library demo applications already includes encoding and the reference code how to use it. Hailo recommends to use the media-library for encoding smooth experience.

The underlying low-level API are also available, which allows users to test the encoding capabilities offline, and adopt the encoder stack to their own software architecture if Media Library is not being used.

7.2. Encoder Workflow (Low Level)

The block diagram of a typical encoding process is shown in the figure below. The flowchart below shows the main steps to use the encoder: the initialization, the configuration (optional), the stream production (start, frame encoding and end) and finally the release of the encoder.

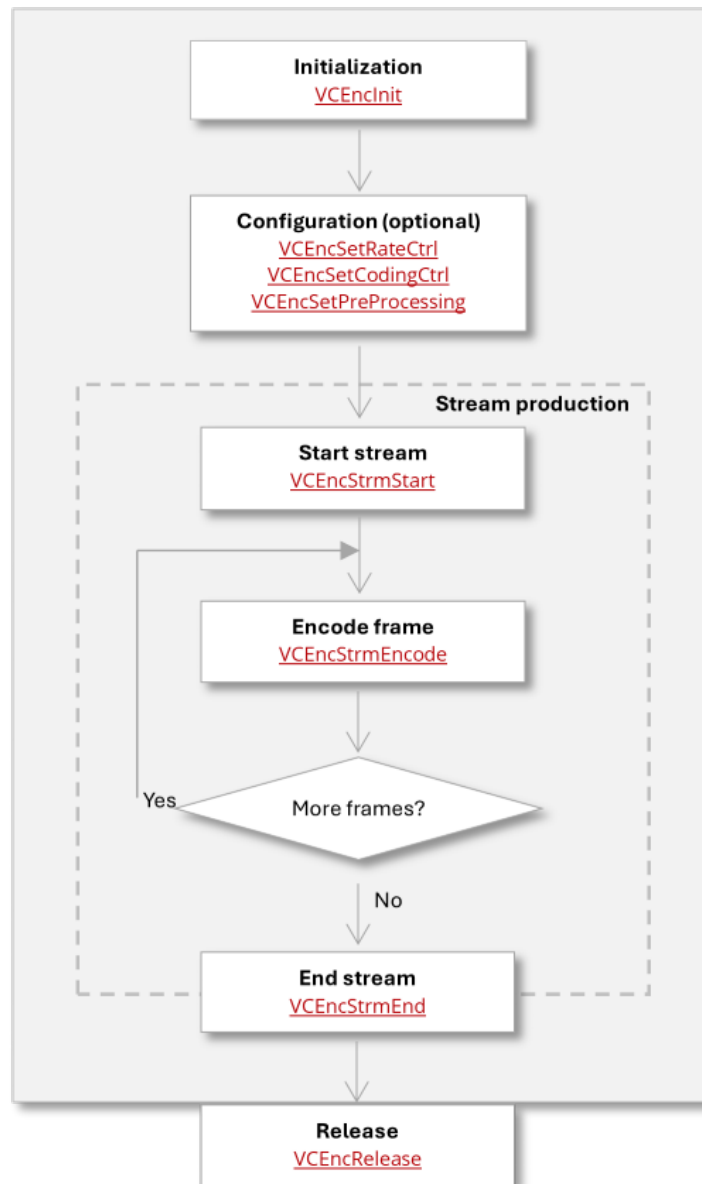


Figure 1. Encoding Process Block Diagram

7.3. Encoder Limitations

The pre-processing block has some limitations on the input picture size and address. The pre-processor will return failure if the application doesn't conform to these limitations. The initialization will also fail if the application tries to use an encoding feature not supported by the hardware.

7.4. Initialization and Release of the Encoder

In order to be able to use the encoder, it must be properly initialized first. The initialization is done by calling `VCEnclnit()`. This call will allocate all the resources needed by the encoder and will execute the necessary setup steps in order to have a fully operational encoder. If successful, the initialization call will return a new instance of the encoder, which will be used as an identifier for all subsequent encoder operations.

The encoder product has support for multiple software instances sharing a single encoder hardware. This means that it is possible to initialize several encoder instances and use them for encoding different streams. The only limitation is that the hardware can only encode one frame at a time so the encoder instances share the hardware with time slicing principle.

The initialization call will return `VCENC_OK` if the initialization is successful. The error code returned in case of a failure will indicate what went wrong during the initialization process.

Certain parameters must be set when initializing the encoder. These parameters cannot be changed during the encoding process.

7.4.1. Stream Profile and Level Indication

The HEVC standard defines Profiles, Levels and Tiers, which sets restrictions to some of the encoder's parameters. The supported level codes are enumerated. By specifying a certain level, the user sets restrictions for many of the encoder parameters. The table below represents the typical encoding values used with each level and the most important restrictions set by each level.

The stream profile will be automatically set by the encoder based on the tools in use. The typical default configuration is Main Profile, Main Tier, but your configuration may vary.

Table 3. HEVC Levels and their Typical Encoding Values

Pro-file&Level	Format Name	Frame Rate (fps)	Max luma picture size (samples)	Max luma sample rate (samples/sec)	Max Bit Rate (kbps)
LEVEL_1_0	QCIF	15	36,864	552,960	128
LEVEL_2_0	CIF	30	122,880	3,686,400	1500
LEVEL_2_1	Q720p	30	245,760	7,372,800	3000
LEVEL_3_0	QHD	30	552,960	16,588,800	6000
LEVEL_3_1	1280x720	30	983,040	33,177,600	10000
LEVEL_4_0	2Kx1080	30	2,228,224	66,846,720	12000
LEVEL_4_1	2Kx1080	60	2,228,224	133,693,440	20000
LEVEL_5_0	4096x2160	30	8,912,896	267,386,880	25000
LEVEL_5_1	4096x2160	60	8,912,896	534,773,760	40000
LEVEL_5_2	4096x2160	120	8,912,896	1,069,547,520	60000
LEVEL_6_0	8192x4320	30	35,651,584	1,069,547,520	60000
LEVEL_6_1	8192x4320	60	35,651,584	2,139,095,040	120000

For more detailed information on the limitations and restrictions for the profiles, refer to the relevant specifications Section [1.3](#) Standard References.

The level selection cannot be changed after the encoder is initialized.

7.4.2. Stream Type

The HEVC standard defines two stream formats: Network Abstraction Layer (NAL) unit stream format and byte stream format. The NAL unit stream format consists of plain NAL units. The byte stream format separates each NAL unit with zero bytes and a start code prefix which makes it easier to separate the NAL units from each other. The NAL unit stream does not carry the size information of the NAL units so this must be communicated some other way. The encoder is able to produce either of these stream formats depending on the application's needs.

When the byte stream mode (VCENC_BYTE_STREAM) is selected, the data produced by the encoder is ready to be stored or delivered as is. This is the default stream format.

If the application requires pure NAL units (VCENC_NAL_UNIT_STREAM), the data produced by the encoder does not contain the 4-byte start code at the beginning of each NAL unit. Without the start code the individual NAL units cannot be separated by simply parsing the stream. To allow the separation, the encoder returns the size of each generated NAL unit in a separate user-allocated buffer. This mode is most commonly used in streaming applications.

7.4.3. Encoded Picture Size

The width and height of the encoded picture in pixels must be specified when initializing an encoder instance. Notice the difference between the input picture size as captured by a camera and the final encoded image size. The input image size can be different from the encoded one if the input image is cropped before the encoding process. The main limitations of the encoded picture size are set by the selected profile and level. Refer to [Levels table](#) for more details. The picture size cannot be altered after the initialization.

Some implementation specific limitations are in force for H.265:

- the encoded picture width must be a multiple of 2
- the height must be a multiple of 2
- the smallest encoded picture size is 130x130 and
- The maximum encoded picture size is 4096 x 4096 pixels.

Limitations apply to the width of the input picture. Even though the encoded picture width can be a multiple of 2, the horizontal scanline must be a multiple of 16. This means that the memory offset from the start of a pixel row to the beginning of the next pixel row is always a multiple of 16. This assumption can be seen in the initial value of the pre-processor parameters, where the input source image width is rounded up to the next multiple of 16 of the encoded width. There is no such limitation on the height of the picture. The figure below shows the described limitation regarding the input picture horizontal scanline.

7.4.4. Frame Rate Descriptor

In order to be able to efficiently control the bit rate the encoder needs a target frame rate. The frame rate is specified with a frame rate numerator and a frame rate denominator. The division of the frame rate numerator and frame rate denominator defines how many frames are being encoded per second and how many bits each of the encoded frames should use on average. The frame rate numerator also defines the stream time scale, i.e. the number of equal sub-intervals, called ticks, within a second. The ticks are used as a base unit for the time increment of each individual frame. More details about the frame timing are given in the [Stream Production](#) section.

NOTE: Keep in mind that this is just a target frame rate. The time increments of the encoded video frames will determine the real frame rate. In other words, even if a 30 fps frame rate (i.e. frame rate numerator 30 and denominator 1) is targeted but the time between two consecutive frames is always 2 ticks, the final stream will have a 15 fps frame rate.

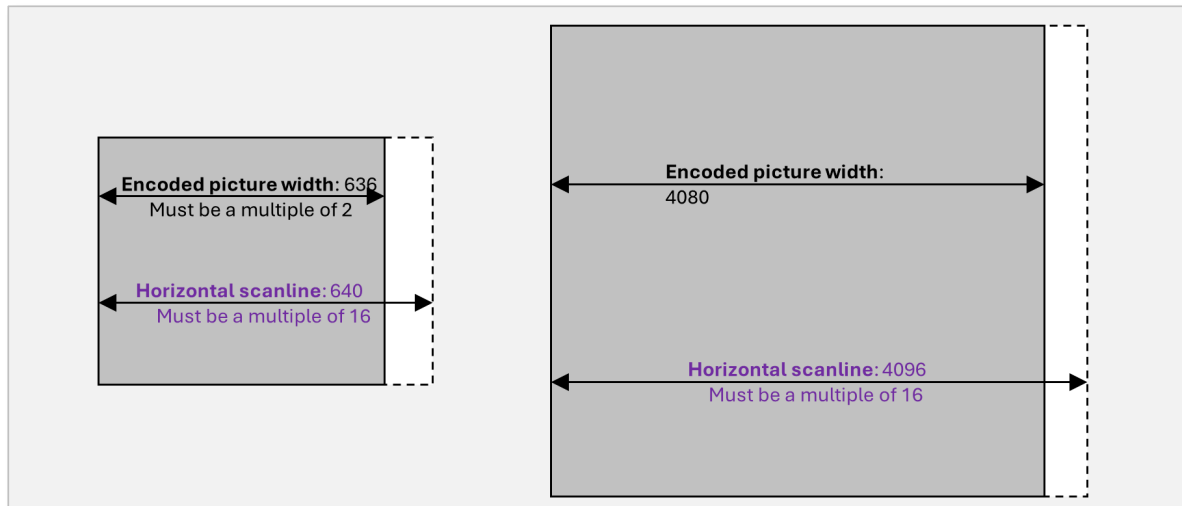


Figure 2. Limitation of Input Picture Width.

7.4.5. Default Encoder Initial Values

- Output bit rate is set to the typical bit rate for the selected profile and level (Refer to [Levels table](#))
- Picture-based rate control: disabled
- Picture skipping: disabled
- HRD: disabled
- Pre-processing: disabled
- Multi-Slices: disabled

7.4.6. Encoder Release

At the end of the encoding process the encoder in use must be released. Use [VCEncRelease\(\)](#) to perform a safe release of all the resources allocated when the encoder was initialized. If a release fails, the encoder instance was most likely corrupted and there is no safe way to assure that all the encoder's resources are freed.

7.5. Configuration of the Encoder

There are several encoding parameters that can be updated after the encoder has been initialized. The configuration API call [VCEncSetCodingCtrl\(\)](#) takes two parameters: an encoder instance and the address of a specific data structure that contains the new values. It is also possible to read the currently used values [VCEncGetCodingCtrl\(\)](#). This provides an easy way to update just some of the values in a set of parameters grouped together in a structure. Some of the encoder parameters can be changed only before a stream is started and others can be altered at any time between two-frame encoding.

7.5.1. Effects of Quantization

In order to compress the video frames the encoder performs quantization. The size of the quantization step is controlled by adjusting the Quantization Parameter (QP). A higher QP means a larger quantization step. This will lower the visual quality and simultaneously raise the compression rate. A lower QP means a smaller quantization step, which makes the visual quality better and simultaneously lowers the compression rate.

7.5.2. Bit Rate Control Configuration

The purpose of Rate Control (RC) is to adjust the QP so that the resulting stream will have the required amount of bits. The rate control is configured initially to produce the maximum bit rate allowed within the selected stream profile and level. The bit rate and some other rate control parameters can be updated after the initialization of the encoder using [VCEncSetRateCtrl\(\)](#). The current parameters can be retrieved using [VCEncGetRateCtrl\(\)](#). The RC controls the output bit rate by changing the quantization parameter (QP) or, in extreme cases, by skipping entire frames from being encoded.

The rate control parameter values can be changed at any time during the encoding sequence if the Hypothetical Reference Decoder (HRD) is disabled. However, the rate control is reset whenever the parameters are updated. The rate control parameters are described as follows:

- **Picture based rate control** - a frame based rate control algorithm, which can be turned ON or OFF. If ON, the RC can adjust the QP between frames.
- **CTB based rate control** - a Coding Tree Block (CTB) based rate control, which can be turned ON or OFF. If ON, the RC can adjust the QP inside a frame.
- **Picture skipping** - When the output rate cannot be adjusted via the QP changes, picture skipping can help control it; picture skipping can be turned ON or OFF. Note that if HRD is enabled it may skip frames even if picture skipping is disabled.
- **Default QP** - This is the default or initial QP used by the encoder. This will be used for the first encoded picture when the rate control is turned ON or for every encoded picture if the rate control is turned OFF. A value of (-1) lets the encoder calculate a typical QP value based on the resolution, frame rate and target bitrate.
- **Min QP** - the minimum value QP that can be used by the encoder and is relevant only when the RC is turned ON.
- **Max QP** - the maximum QP which can be set by the encoder; also in use only if the RC is turned ON.
- **Output bit rate** - the target bit rate (in bits per second) for the output stream and it is used when one or more of the following is turned ON: picture based RC, picture skipping or HRD. It is initially set to the maximum value defined by the profile and level of the stream but it can be changed if needed.
- **HRD** - an algorithm for checking a bit stream with its bit rate to verify that the amount of input stream buffer memory required in a decoder is less than the standard-defined buffer size. This can be turned ON or OFF, but by turning it OFF the output stream might not be 100% standard compatible.
- **HRD CPB size** - the size of the Coded Picture Buffer (CPB) used in the HRD model. By default, it is set to the maximum size allowed for the initialized encoder level. Do not change this parameter if you are unfamiliar with this feature.
- **GOP length** - GOP is a group of successive pictures within a video stream. GOP usually contains an Intra frame (I-frame) and several Inter frames (Predicted frame). Intra frames make video easier to search and edit, but they use more bits which decreases the overall quality of the stream. Rate control uses the GOP length to match the average bit rate of each GOP to the target bit rate. Thus the GOP length setting affects how fast the rate control reacts to changes in the video sequence and encoded bits per frame. E.g., a GOP length of 30 frames on a 30 fps stream will match the target bitrate every second, whereas a GOP length of 150 frames will match the target bitrate over five seconds. Note that it is up to the application to choose whether to insert an I-frame at the beginning of each GOP or not. i.e. every I-frame starts a new GOP but every GOP doesn't need to start with an I-frame.
- **Intra QP delta** - The intra frames in HEVC video can sometimes introduce noticeable flickering because of different prediction methods. This problem can be overcome by adjusting the quantization of the intra frames compared to the surrounding inter frames.

Typically it is sufficient to just set the target bit rate. This can be handled by calling `VCEncGetRateCtrl()` to get the original values, changing the target bit rate and setting the values by `VCEncSetRateCtrl()`.

To achieve accurate control of the output bit rate, the CTB based rate control is enabled during initialization. If QP does not change in the middle of a picture, then the CTB should be disabled.

In cases where the target bit rate is too low, the encoder may not be able to reach the target no matter what QP is used. Therefore it may be useful to enable picture skipping for very low bit rates. If the output frame rate is not required to be the same as the input frame rate, this parameter may be enabled to give the rate control algorithm more freedom to allocate bits for pictures.

Setting minimum and/or maximum QP can be used to restrict the rate control to only use QP values in the range [Min QP, Max QP]. This is feasible in a Variable Bitrate (VBR) operation where constant quality is preferred over the constant bitrate. On the other hand, these limitations also limit the rate control's possibilities to achieve the target bitrate.

The HRD feature of the encoder is disabled by default. The encoder will not be able to guarantee that restrictions placed on the standard¹ will always apply. When the HRD is enabled, the encoder runs a model of the decoder input stream buffer (coded picture buffer) to make sure that at the decoding time of a certain picture, the buffer contains enough data so that the decoder may decode the picture. On the other hand, the HRD model assures that the buffer can accommodate the stream data at any point of time. The model assumes that there is a constant bit rate channel between the encoder and the decoder. The size of the coded picture buffer is specified in the standard² for each level. When the HRD is disabled, the operation of the rate control is alleviated so that the encoder does not have to keep the channel occupied all the time and the encoder may produce images whose size exceeds the buffer occupancy at the decoding time of the picture.

NOTE: When HRD is enabled, the selected bitrate and any custom CPB size must be within the limits set by the initialized encoder level (Refer to values in the HEVC standard³). Once the stream has been started, the rate control parameters (bitrate) cannot be altered anymore. This is due to the fact that the stream start headers contain information about the selected bitrate and CPB size.

7.5.3. VBR and CBR Video

Variable Bit Rate (VBR) is the preferred choice for locally stored video. The visual quality stays constant, more bits are allocated for complex sections and fewer bits are allocated for simple sections. VBR has better quality versus space ratio than CBR video. The simplest solution is to use a constant QP for the entire video. In this encoder, VBR can be constrained so that rate control tries to reach the target bit rate over the time. Refer to the HEVC standard⁴ for maximum bit rate limits.

Constant Bit Rate (CBR) is useful for real-time streaming video over a constant bandwidth channel. CBR is not a good choice for storage, since it doesn't allocate enough bits for complex sections and wastes bits on simple sections. Perfectly constant bit rate is close to impossible to achieve with a real-time one-pass encoder such as this encoder. The best way for a one-pass encoder to achieve CBR is to change the QP of the CTB inside the frame during encoding (CTB-based RC can partially implement this function). This might have the adverse effect of the lower part of the frame becoming blurry.

The figure below illustrates the trade-off between visual quality variance and bitrate variance and how it is mapped to three typical video use cases. The table below shows recommended settings for rate control parameters for each of these example use cases. Keep in mind that the QP limits are heavily dependent on the target bit rate and frame rate.

Table 4. Recommended Rate Control Settings

¹ ITU-T H265

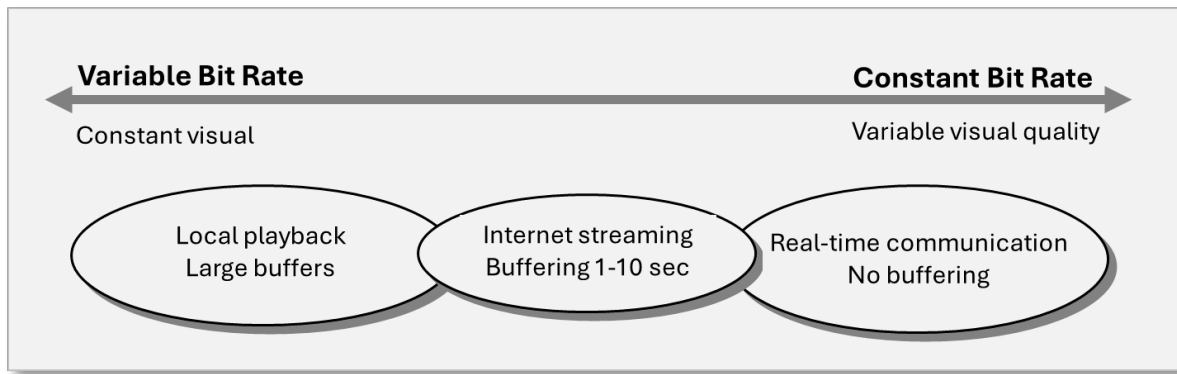


Figure 3. Variable Bit Rate vs. Constant Bit Rate

Parameter	Local Playback	Internet Streaming	Real-time Communication
gopLen	150	30 - 150	30-May
pictureRc	1	1	1
ctbRc	0	0	1
hrd	0	0	1 (can skip frames)
qpMin	20 - 30	20 - 25	10
qpMax	35 - 45	45 - 50	51
qpHdr	-1	-1	-1
pictureSkip	0	0	0

7.5.4. Coding Control Settings

There are several coding control parameters that can be altered using [VCEncSetCodingCtrl\(\)](#). Current settings can be retrieved using [VCEncGetCodingCtrl\(\)](#). The recommended way of changing any of the coding control parameters is to first retrieve the current settings and then change the desired parameters. All the parameters with the exception of the "slice size" can only be set before starting the encoding.

Available options:

- **Slice size** - Each encoded picture can be divided into several slices. Slices help in the error recovery of the erroneous streams and their role is to split the picture data into smaller independent packages. The size is specified in Coding Tree Unit (CTU) rows. This value can be altered at any time during the encoding process.
- **SEI messages** - When enabled, a Supplemental Enhancement Information (SEI) message containing picture timing information will be inserted into the stream before every encoded frame. Buffering period information will be inserted into the stream before every encoded Instantaneous Decoder Refresh (IDR) frame when Hypothetical Reference Decoder (HRD) is enabled. If not using these SEI messages, the stream doesn't contain any timing information so it must be communicated some other way.
- **Video full range** - Defines the input YCbCr data sample range that will be included in the stream headers. If not set correctly, the video range dynamic might suffer degradation on the decoder side.
- **Disable de-blocking filter** - Disables the de-blocking filter, thus lowering the encoding and decoding complexity but also lowering the video quality.
- **Disable SAO filter** - Disables the Sample Adaptive Offset (SAO) filter, thus lowering the encoding and decoding complexity but also lowering the video quality.
- **Sample aspect ratio width and height** - Defines the aspect ratio of the input picture samples that will be included in the stream headers. The default aspect ratio is square.

- **Field order** - When interlaced viewMode is used, the field order can either be bottom-first or top-first. Some applications may assume a certain field order. The default value of 0 specifies a field order of bottom-first.
- **ROI encoding** - In Region-of-Interest (ROI) encoding, two rectangular areas can be set to have higher encoding quality.
- **QP map based ROI** - QP delta for each CU can be defined through the QP map stored in DRAM. This enables more flexible, arbitrary shape ROI areas.
- **Denoise filtering** - Allows filtering of the noise in the input frame caused by for example, low light conditions in the camera sensor. This improves both the input frame quality and the compression rate.
- **GDR** - Instead of I-frames, Gradual Decoder Refresh (GDR) uses I-slices that gradually cover the whole frame area. This enables faster frame loss recovery and spreads the heavier computational load of I-frames over several frames.

Example:

```
VCEncInst encoder;
VCEncCodingCtrl codingCfg;

/* encoder is already initialized and we have an instance */
if(VCEncGetCodingCtrl(encoder, &codingCfg) != VCENC_OK)
{
    /* handle error */
}
else
{
    codingCfg.sliceSize = 5; /* 5 CTU rows in each slice */
    if(VCEncSetCodingCtrl(encoder, &codingCfg) != VCENC_OK)
    {
        /* handle error */
    }
}
```

After the encoding has been started, from the above parameters, only the slice size can be altered.

7.6. Stream Production

After a successful initialization and an optional configuration step, the encoder is ready to produce an HEVC-or H.264-compliant stream. There are three separate phases for producing a stream:

1. **Starting the stream** by calling [VCEncStrmStart\(\)](#)
2. **Encoding video frames** by calling [VCEncStrmEncode\(\)](#)
3. **Ending the stream** by calling [VCEncStrmEnd\(\)](#)

7.6.1. Starting the Stream

A stream must be started by a call to [VCEncStrmStart\(\)](#). Video Parameter Set (VPS), Sequence Parameter Set (SPS) and Picture Parameter Set (PPS) NAL units are produced at this time. The user must provide an output buffer where the stream header data will be written. The output buffer is specified with a pointer to the buffer and the size of the buffer in bytes. If successful, the call will return VCENC_OK and the size of the generated stream in bytes. If NAL units are needed, then the buffer for sizes must be specified when starting the stream.

7.6.2. Encoding Video Frames

A video sequence can be encoded after a stream has been started. A video sequence is encoded frame by frame by calling `VCEncStrmEncode()`. The main inputs for this function are an input buffer, which contains the video frame to be encoded, and an output buffer where the generated stream will be written.

The supported formats of the input video are planar YCbCr 4:2:0, semiplanar YCbCr 4:2:0, interleaved YCbCr 4:2:2, planar I010, semiplanar P010, planar YCbCr 4:2:0 10 bits packed, YCbCr 4:2:0 10 bits (Y0L2) and RGB (565, 555, 444, 888 or 101010). The format in use has been selected during the pre-processor configuration. Regardless of the input format used the input frame buffers must be 128-bit aligned, linear and hardware accessible. In other words, the addresses which are passed as parameters to the `VCEncStrmEncode()` function, are bus addresses and will be given to the hardware unmodified. These addresses will be used by the hardware to read the input video frame.

The output buffer is used by the software to construct the final stream. The pointer to the buffer will be passed to the `VCEncStrmEncode()` encode function. The size of the buffer must be correlated with the target bit rate and frame rate. For example, when the target bit rate is 3 Mbps and the frame rate is 30 fps, the average frame size is 100 000 bits. Keep in mind that this is an average value and single encoded frames can take many times more space. In a typical case, a buffer size equal to the bitrate of the stream is adequate. The maximum size of the encoded frame depends on the input picture. If the encoder reaches the end of the output buffer, it will discard the current frame and return `VCENC_OUTPUT_BUFFER_OVERFLOW`. This is even more likely when the rate control is disabled. Even if the current frame is lost, the encoding process can continue with a new frame which will be INTRA coded to assure that any errors are not propagated.

Another important parameter for every video frame is its time stamp. This must be specified at every call to `VCEncStrmEncode()`. The time stamp is relative to the previous frame and it is specified as a time increment in ticks. The duration of a tick is specified at the initialization according to the frame rate numerator. The first encoded frame normally has a time increment of zero. For fixed frame rate video, i.e. every frame has equal duration, the time increment is equal to frame rate denominator. For example, if the frame rate is set to 1000/33 the time scale is 1000 ticks and the relative time between each frame is 33 ticks. When the frame has been successfully encoded, `VCEncStrmEncode()` returns `VCENC_FRAME_READY`.

Each encoded frame can be divided into several slices. Slices help in the error recovery of the erroneous streams and their role is to split the frame data into smaller independent packages. The size of a slice is specified in CTU rows.

The individual NAL unit sizes can also be known. The user must provide a buffer where the sizes of all NAL units are returned. The encoder will fill this buffer with the size of each unit in bytes. After the last unit, a zero value is written to the size buffer. This information is needed for separating each NAL unit from the byte stream for the case of stream type of `VCENC_NAL_UNIT_STREAM`. A table of 516 bytes ($129 * \text{sizeof}(u32)$) is sufficient to fit sizes for the maximum number of slices that can be generated by the encoder.

7.6.3. Ending the Stream

A stream will be ended by a call to `VCEncStrmEnd()`. If successful, the call will return `VCENC_OK`. Stream end consists of a NAL unit of type 'End-of-Sequence'.

7.7. Interlaced Coding

Interlaced video consists of fields where two fields (top and bottom) comprise one video frame. Interlaced coding mode encodes top and bottom fields of a frame separately. This improves the compression efficiency of interlaced video content compared to normal progressive frame coding.

The input picture for interlaced coding is both fields interlaced into one frame. When the interlaced frame input format is used, the same frame is used as input for two consecutive encode calls and the pre-processor switches between reading either top or bottom field.

7.8. Video Pre-processor Usage

The encoder includes the following pre-processing blocks:

1. 30-, 24-, 16-, 15- or 12-bit RGB to YCbCr 4:2:0 conversion
2. YCbYCr 4:2:2 to YCbCr 4:2:0 conversion
3. I010, P010, planar YCbCr 4:2:0 10 bits packed, Y0L2 to YCbCr 4:2:0 conversion
4. Picture rotation
5. Picture cropping

By default, the encoder input is considered to be in planar YUV 4:2:0 format. Cropping and rotation are disabled. The pre-processor is controlled by the [VCEncSetPreProcessing\(\)](#) and [VCEncGetPreProcessing\(\)](#) API functions. It is recommended to retrieve the current values from the encoder first and then do the necessary alterations.

7.8.1. Color Conversion

The color conversion block reads the input picture in 30-, 24-, 16-, 15- or 12-bit RGB or interleaved YCbCr 4:2:2 format. The picture is converted into planar YCbCr 4:2:0 format by color conversion, sub-sampling the chrominance samples vertically and rearranging the samples. The conversion is done during the configuration phase and it will affect every encoded picture.

7.8.2. Picture Rotation

The rotation block rotates the picture 180 degrees or 90 degrees clockwise or counter-clockwise. The rotation is configured during the initialization phase and it will affect every encoded picture. The figure [below](#) illustrates the resultant encoded picture dimensions after rotating the input picture.

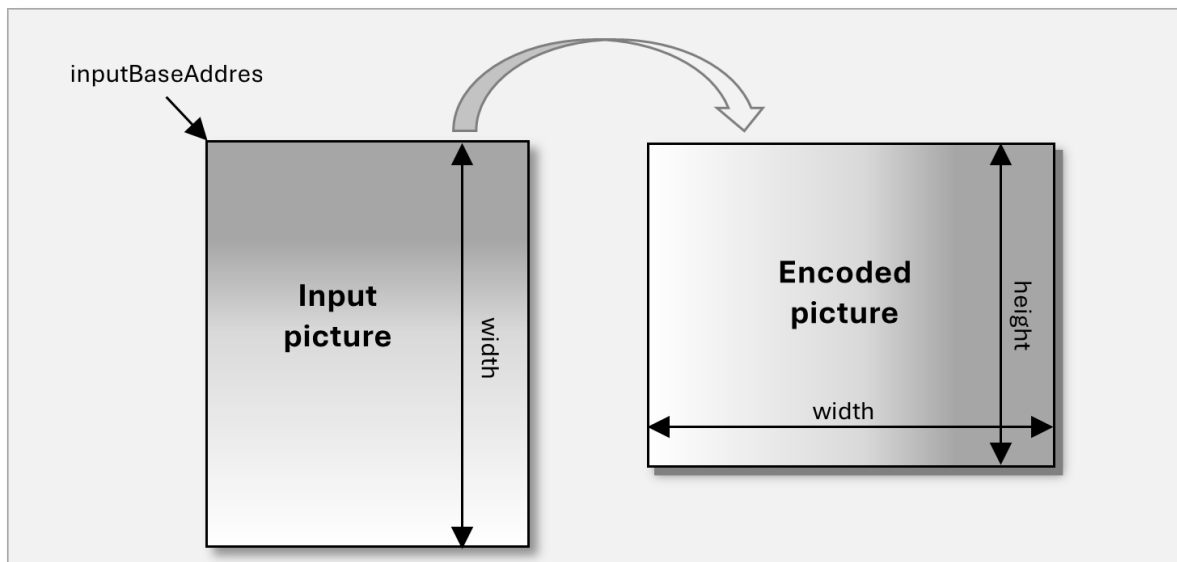


Figure 4. Picture Dimensions after Rotation

7.8.3. Picture Cropping

Picture cropping provides a way to read and encode a smaller selected portion of a larger picture. The cropping is controlled by setting the size of the input source picture and the top-left corner coordinates of the encoded picture relative to the top-left corner of the input picture.

The maximum size of the input source image is 8192x8192 pixels. The maximum size of the output image is 4096x8192 pixels.

The figure [below](#) illustrates the relationship between picture parameters. The conditions that must be satisfied in order to enable the cropping block are as follows:

$$origWidth - xOffset - width \geq 0$$

$$origHeight - yOffset - height \geq 0$$

The input source picture horizontal stride (in pixels) must be multiple of 16.

For example, when the input width is 630 pixels the horizontal stride must be 640 pixels.

Offset of pixel n in the picture buffer:

$$pixel_offset = \left(\frac{n}{width} \right) * ((width + 15) \&(\sim 15)) + (n \% width)$$

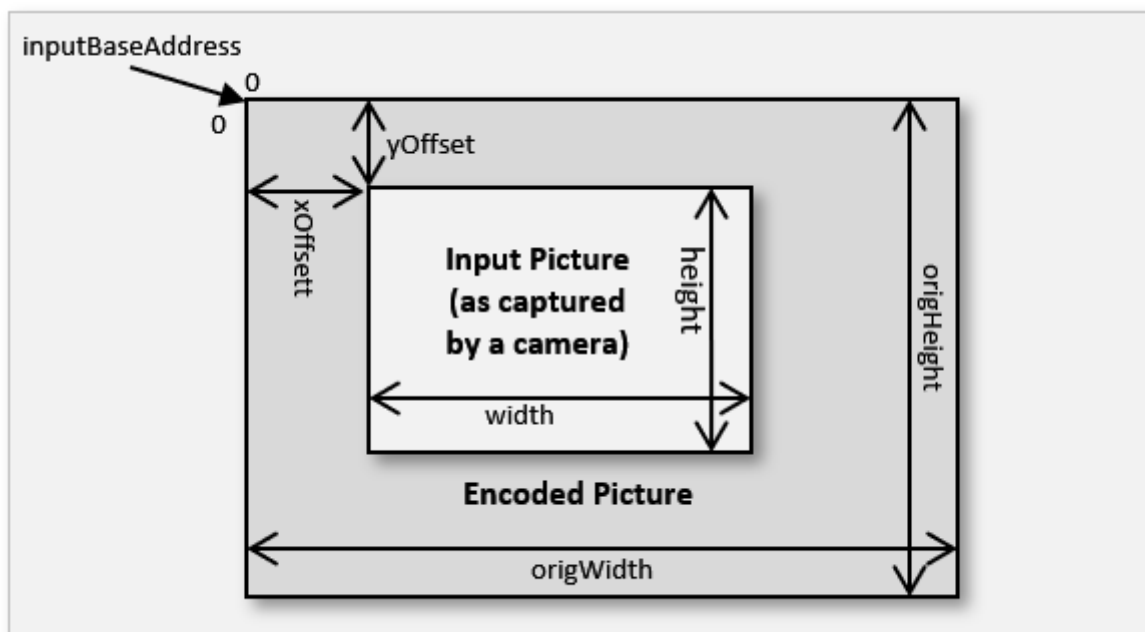


Figure 5. Relationship between Input and Encoded Image Sizes

7.9. Video Frame Storage Format

Input picture formats supported by the encoder are:

1. planar YCbCr 4:2:0,
2. semiplanar YCbCr 4:2:0,
3. interleaved YCbCr 4:2:2,
4. 32-bit RGB,

5. 16-bit RGB,
6. planar YCbCr 4:2:0 10 bits (I010),
7. semiplanar YCbCr 4:2:0 10 bits (P010),
8. YCbCr 4:2:0 10 bits (Y0L2) and
9. planar YCbCr 4:2:0 10 bits packed.

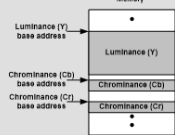
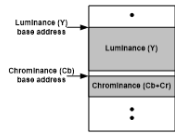
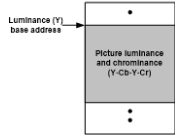
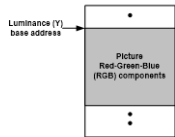
The table on the following page shows how the supported input picture formats are stored in external memory.

A planar YCbCr 4:2:0 10 bits (I010) and a planar YCbCr 4:2:0 10 bits packed picture are stored in external memory like a planar YCbCr 4:2:0 picture.

A semiplanar YCbCr 4:2:0 10 bits (P010) picture is stored in external memory like a semiplanar YCbCr 4:2:0 picture.

YCbCr 4:2:0 10 bits (Y0L2) picture are stored in external memory like a YCbCr 4:2:2 picture. The pixels are stored in 64-bit chunks containing 2x2 luma values and a set of chroma values (and optional punch-through alpha per pixel). An alpha-value of 0 means the corresponding pixel is completely transparent while an alpha value of 1 means the corresponding pixel is completely opaque. The luma values in the 2x2 chunks are numbered from top to bottom, left to right. An example showing the addressing of a 16x4 pixel image is defined in the [table below](#).

Table 5. Storage of Input Picture Formats in External Memory

Planar YCbCr 4:2:0: The luminance and both chrominance components are stored in three buffers and they must be located in a linear and physically contiguous memory block.	
Semiplanar YCbCr 4:2:0: The luminance is equal to planar format but the Cb and Cr chrominance components are interleaved together. Separate formats are specified for different orders of Cb and Cr.	
Interleaved YCbCr 4:2:2: The sample order is selected at initialization to be either Y-Cb-Y-Cr or Cb-Y-Cr-Y.	
RGB: The sample order and bit-depth of color components are selected at initialization to be 16-bit¹ or 32-bit² RGB formats.	

Planar and semiplanar YCbCr 4:2:0 pixel formats are defined in the table below.

¹ 16-bit RGB formats: RGB565, BGR565, RGB555, BGR555, RGB444 or BGR444. For example, RGB565 corresponds to a 16-bit word containing 5 MSB bits for Red-component, 6 bits for Green-component and 5 LSB bits for Blue-component.

² 32-bit RGB formats: RGB888, BGR888, RGB101010, BGR101010

Pixel Format																			
Planar <u>YCbCr 4:2:0</u> 10 bits (I010)		Start address + 1								Start address + 0									
		Bit7								Bit0									
		0	0	0	0	0	0	Y ₉	Y ₈	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀		
Planar <u>YCbCr 4:2:0</u> 10 bits packed 4-pixel unit		Start address + 4								Start address + 3									
		Bit7								Bit0									
		Y ₃₉	Y ₃₈	Y ₃₇	Y ₃₆	Y ₃₅	Y ₃₄	Y ₃₃	Y ₃₂	Y ₃₁	Y ₃₀	Y ₂₉	Y ₂₈	Y ₂₇	Y ₂₆	Y ₂₅	Y ₂₄		
		Start address + 2								Start address + 1									
		Bit7								Bit0									
		Y ₂₃	Y ₂₂	Y ₂₁	Y ₂₀	Y ₁₉	Y ₁₈	Y ₁₇	Y ₁₆	Y ₁₅	Y ₁₄	Y ₁₃	Y ₁₂	Y ₁₁	Y ₁₀	Y ₉	Y ₈		
		Start address + 0								Start address + 0									
		Bit7								Bit0									
		Y ₀₇	Y ₀₆	Y ₀₅	Y ₀₄	Y ₀₃	Y ₀₂	Y ₀₁	Y ₀₀										
<u>Semiplanar</u> <u>YCbCr 4:2:0</u> 10 bits (P010) Y-pixel format		Start address + 1								Start address + 0									
		Bit7								Bit0									
		Y ₉	Y ₈	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀	0	0	0	0	0	0		
<u>YCbCr 4:2:0</u> 10 bits, 64-bit chunk format in Y0L2 format Luma Pixel order of a 2x2 chunk in Y0L2 format:		Start address + 7								Start address + 6									
		Bit7								Bit0									
		A ₃	A ₂	Y ₃₉	Y ₃₈	Y ₃₇	Y ₃₆	Y ₃₅	Y ₃₄	Y ₃₃	Y ₃₂	Y ₃₁	Y ₃₀	V ₉	V ₈	V ₇	V ₆		
		Start address + 5								Start address + 4									
		Bit7								Bit0									
		V ₅	V ₄	V ₃	V ₂	V ₁	V ₀	Y ₂₉	Y ₂₈	Y ₂₇	Y ₂₆	Y ₂₅	Y ₂₄	Y ₂₃	Y ₂₂	Y ₂₁	Y ₂₀		
		Start address + 3								Start address + 2									
		Bit7								Bit0									
		A ₁	A ₀	Y ₁₉	Y ₁₈	Y ₁₇	Y ₁₆	Y ₁₅	Y ₁₄	Y ₁₃	Y ₁₂	Y ₁₁	Y ₁₀	U ₉	U ₈	U ₇	U ₆		
		Y ₀	Y ₁	Start address + 1								Start address + 0							
Y ₂		Y ₃		Bit7								Bit0							
				U ₅	U ₄	U ₃	U ₂	U ₁	U ₀	Y ₀₉	Y ₀₈	Y ₀₇	Y ₀₆	Y ₀₅	Y ₀₄	Y ₀₃	Y ₀₂	Y ₀₁	Y ₀₀

Table 6. Addressing of a 16x4 pixel image in Y0L2 Format

Relative chunk address (bytes)								
	0	8	16	24	32	40	48	56
0	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA
	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA
64	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA	YUYAA
	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA	YVYAA

The input picture data endianness is set when the encoder software is compiled and can't be changed during run-time. The sizes of the different components are based on the picture's dimensions and the input format as defined in the table below.

(where: W = width, H = height)

Table 7. Component Sizes of Input Formats

Input Formats	Y (bytes)	Cb (bytes)	Cr (bytes)	Y+Cb+Cr (bytes)	Cb+Cr (bytes)	R+G+B (bytes)
Planar YCbCr 4:2:0	$W \times H$	$(W/2) \times (H/2)$	$(W/2) \times (H/2)$			
Planar YCbCr 4:2:0 10 bits(I010)	$W \times H \times 2$	$(W/2) \times H$	$(W/2) \times H$			
Semiplanar YCbCr 4:2:0 10 bits(P010)	$W \times H \times 2$				$W \times H$	
Planar YCbCr 4:2:0 10 bits packed	$(W \times H \times 10) / 8$	$((W / 2) \times (H \times 5)) / 8$	$((W / 2) \times (H \times 5)) / 8$			
YCbCr 4:2:0 10 bits (Y0L2)				$W \times H \times 2$		
Semiplanar YCbCr 4:2:0	$W \times H$				$(W \times H) / 2$	
Interleaved YCbCr 4:2:2				$W \times H \times 2$		
16-bit RGB						$W \times H \times 2$
32-bit RGB						$W \times H \times 2$

7.9.1. Frame Size Limitations

Input picture formats have limitations based on the input picture size and the encoded picture size.

Input Format	Limitations ^{Page 90, 3} ^{Page 90, 4}
YCbCr 4:2:0 YCbCr 4:2:2 RGB Planar YCbCr 4:2:0 10 bits (I010) Semiplanar YCbCr 4:2:0 10 bits (P010)	$\text{encoded_width} \% 2 = 0$ $\text{encoded_height} \% 2 = 0$ $\text{input_width} \geq (\text{encoded_width} + 7) \& (\sim 7)$ $\text{input_width} \% 16 = 0$ $\text{input_height} \geq \text{encoded_height}$ $\text{input_height} \% 2 = 0$
Planar YCbCr 4:2:0 10 bits packed	$\text{encoded_width} \% 2 = 0$ $\text{encoded_height} \% 2 = 0$ $\text{input_width} \geq (\text{encoded_width} + 7) \& (\sim 7)$ $\text{input_width} \% 64 = 0$ $\text{input_height} \geq \text{encoded_height}$ $\text{input_height} \% 2 = 0$
YCbCr 4:2:0 10 bits (Y0L2)	$\text{encoded_width} \% 2 = 0$ $\text{encoded_height} \% 2 = 0$ $\text{input_width} \geq (\text{encoded_width} + 3) \& (\sim 3)$ $\text{input_width} \% 4 = 0$ $\text{input_height} \geq \text{encoded_height}$ $\text{input_height} \% 2 = 0$

7.10. GOP Structure Configuration

The GOP structure is represented by a table, which is given as an input parameter to configure the encoder. The following two simple examples show how to define a GOP structure table.

7.10.1. GOP Structure of Size 4

Consider the GOP structure of size 4 shown in the figures below.

The four frames within a GOP are described below in decoding order.

- Frame 1 is a P frame with POC 4, referencing one frame with POC 0. The reference frame is defined by delta POC value relative to current frame. In this case it is -4.
- Frame 2 is a B frame with POC 2, referencing two frames with POC 0 and 4 respectively. So its reference frames are listed as -2 and 2.
- Frame 3 is a B frame with POC 1, referencing two frames with POC 0 and 2 respectively. It also needs to keep the frame with POC 4 as a reference frame to be used in future. So its reference frame list is -1, 1 and 3.

³ % - remainder operator

⁴ & - bitwise AND operator

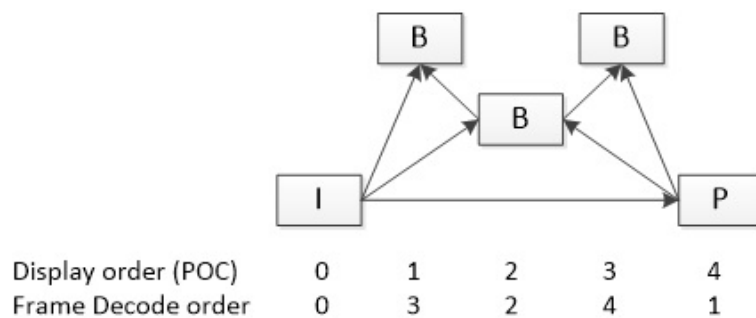


Figure 6. GOP Structure of Size 4

- Frame 4 is a B frame with POC 3, referencing two frames with POC 2 and 4 respectively. Its reference list is -1 and 1.

The corresponding GOP structure table is shown below, where:

- **QP Offset** will be added to the QP parameter to set the final QP;
- **QP Factor** will be used in rate distortion optimization. Higher values mean lower quality and less bits. Typical range is between 0.3 and 1;
- **Used by Current Frame** specifies whether each reference frame in **Reference List** is used in current (1) or future (0).

Table 8. GOP Structure Table of Size 4

Frame	Type	POC	QP Offset	QP Factor	Number of References	Reference List	Used by Current Frame
1	P	4	0	0.5	1	-4	1
2	B	2	0	0.3536	2	-2 2	1 1
3	B	1	0	0.5	3	-1 1 3	1 1 0
4	B	3	0	0.5	2	-1 1	1 1

7.10.2. GOP Structure of Size 1

The GOP structure in the figure below only contains one P frame which references only its previous frame, so the reference list is simply -1.

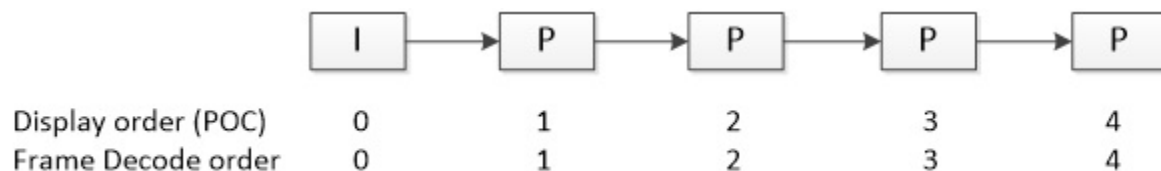


Figure 7. GOP Structure of Size 1

The GOP structure table is shown in the following table.

Table 9. GOP Structure Table of Size 1

Frame	Type	POC	QP Offset	QP Factor	Temporal Id	Number of References	Reference List	Used by Current Frame
1	P	1	0	0.8	0	1	-1	1

All GOP structure tables that will be used in the sequence need to be saved in the input parameter `encIn.gopConfigs` to configure encoder. Use the `VCEncGopConfig` data structure within the `VCEncIn` data structure to configure the parameters in the GOP structure.

7.10.3. GOP Structure of Size 1 with Long Term Reference

The GOP structure only contains one B frame which references only its previous frame, so the reference list is simply -1. In addition, it specifies `longTermRefPicRate=50` to enable long term reference, which updates every 50 frames. So the total number of references is 2, and the type is B frame. The GOP structure table is shown in Table 10.

Table 10. GOP Structure Table of Size 1 with Long Term Reference

Frame	Type	POC	QP Off-set	QP Fac-tor	Tempo-ralId	Number of Refer-ences	Reference List	Used by Current Frame
1	B	1	0	0.8	0	2	-1	1

7.10.4. GOP Structure of Size 4 with 3 Temporal Layers

The GOP structure defines a 3 layer pattern:

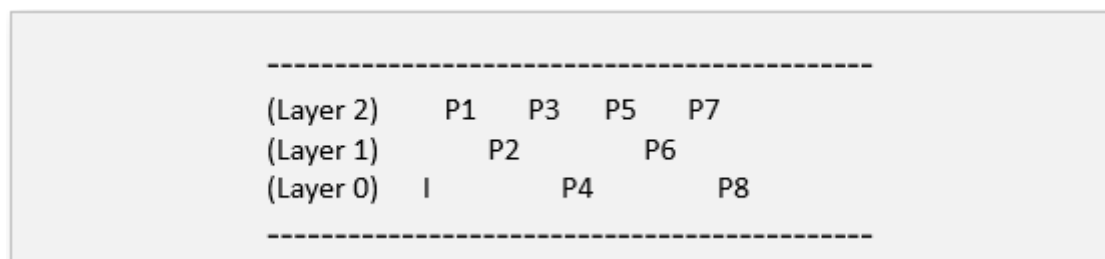


Figure 8. GOP Structure of Size 4 with 3 Temporal Layers

The GOP structure table is shown in the following table.

Table 11. GOP Structure Table of Size 4 with 3 Layers

Frame	Type	POC	QP Off-set	QP Fac-tor	Tempo-ralId	Number of Refer-ences	Reference List	Used by Current Frame
1	B	1	0	0.4624	2	1	-1	1
2	B	2	0	0.4624	1	1	-2	1
3	B	3	0	0.4624	2	2	-4	1 1
4	B	4	0	0.578	0	1	-4	1

7.11. Encode H264

```

#include "hevcencapi.h"
#include <stdlib.h>

VCEncRet ret;
VCEncInst encoder;
VCEncConfig cfg;
VCEncCodingCtrl codingCfg;
VCEncRateCtrl rcCfg;
VCEncIn encIn;

```

(continues on next page)

(continued from previous page)

```

VCEncOut encOut;
i32 next, last;
u32 picture_size;
u32 pict_bus_address = 0;
u32 pict_bus_address_stab = 0;
u32 outbuf_size;
u32 outbuf_bus_address = 0;
u32 *outbuf_virt_address = NULL;

/* Step 1: Initialize an encoder instance */
/* 30 fps frame rate */
cfg.frameRateDenom = 1;
cfg.frameRateNum = 30; /* this is the time resolution also */

/* VGA resolution */
cfg.width = 640;
cfg.height = 480;

/* Stream type */
cfg.streamType = VCENC_BYTE_STREAM;
cfg.profile = VCENC_H264_HIGH_PROFILE;
cfg.level = VCENC_H264_LEVEL_5_1;
cfg.codecH264 = 1;

/* Output buffer size */
outbuf_size = cfg.width * cfg.height;

/* Picture buffer size YUV 420 */
picture_size = (cfg.width * cfg.height * 3) / 2;

if ((ret = VCEncInit(&cfg, &encoder)) != VCENC_OK) {
    /* Handle here the error situation */
    goto end;
}

/* Step 2: Optional extra encoder configuration */
/* Step 3: Allocate linear memory resources. This implementation is OS specific and not
↳ part of this example. */
AllocDMAMemory(&outbuf_virt_address, &outbuf_bus_address, &outbuf_size);
AllocDMAMemory(NULL, &pict_bus_address, &picture_size);

encIn.pOutBuf = outbuf_virt_address;
encIn.busOutBuf = outbuf_bus_address;
encIn.outBufSize = outbuf_size; /* bytes */

/* Step 4: Start the stream */
ret = VCEncStrmStart(encoder, &encIn, &encOut);
if (ret != VCENC_OK) {
    /* Handle here the error situation */
    goto close;
}

/* Save the generated stream. This implementation is OS specific and not part of this
↳ example. */
SaveStream(encIn.pOutBuf, encOut.streamSize);

/* Step 5: Encode the video frames */
ret = VCENC_FRAME_READY;
next = 0;
last = 300; /* encode 300 frames */

```

(continues on next page)

(continued from previous page)

```

while ((next <= last) && (ret == VCENC_FRAME_READY || ret == VCENC_OUTPUT_BUFFER_
→OVERFLOW)) {
    if (next == 0)
        encIn.timeIncrement = 0; /* start time is 0 */
    else
        encIn.timeIncrement = 1; /* time units between frames */

    /* Select frame type */
    encIn.codingType = (next == 0) ? VCENC_INTRA_FRAME : VCENC_PREDICTED_FRAME;

    /* Read next frame and get the bus address. This implementation is OS specific and not
    →part of this example. */
    ReadYuv420PlanarFrame(pict_bus_address);

    /* We assume one linear buffer for the input image in planar YCbCr 4:2:0 format */
    encIn.busLuma = pict_bus_address;
    encIn.busChromaU = encIn.busLuma + cfg.width * cfg.height;
    encIn.busChromaV = encIn.busChromaU + (cfg.width / 2) * (cfg.height / 2);

    ret = VCEncStrmEncode(encoder, &encIn, &encOut, NULL, NULL);

    switch (ret) {
        case VCENC_FRAME_READY:
            SaveStream(encIn.pOutBuf, encOut.streamSize);
            break;
        case VCENC_SYSTEM_ERROR:
            /* Fatal system error, can't continue */
            next = last;
            break;
        default:
            /* All the others are ERROR codes */
            /* The next frame will be forced to INTRA coded by the encoder */
            break;
    }
    next++;
} /* End of main encoding loop */

/* Step 6: End stream */
ret = VCEncStrmEnd(encoder, &encIn, &encOut);
if (ret != VCENC_OK) {
    /* Handle here the error situation */
    goto close;
} else {
    SaveStream(encIn.pOutBuf, encOut.streamSize);
}

/* Free all resources */
FreeDMAMemory(outbuf_virt_address, outbuf_bus_address, outbuf_size);
FreeDMAMemory(NULL, pict_bus_address, picture_size);

close:
/* Last Step: Release the encoder instance */
if ((ret = VCEncRelease(encoder)) != VCENC_OK) {
    /* There is nothing much to do for an error here, just a notification message. */
    /* There must be a fault somewhere else that caused a failure of the encoder release.
    →*/
}

end: /* Nothing else to do */

```

8. Media Library C++ API Reference

8.1. Buffer Pool

8.1.1. Overview

A buffer pool is a technique used to manage memory efficiently by pre-allocating a pool (or a collection) of memory buffers of fixed size. Instead of allocating and deallocating memory dynamically each time a buffer is needed or released, the buffer pool pre-allocates these buffers once and manages them using acquire and release operations.

How Does it work?

1. Initialization: - During initialization or setup phase, a buffer pool is created. - It allocates a fixed number of memory buffers, all of the same size.
2. Acquiring Buffers: - When a component or module needs a buffer, it requests one from the buffer pool using an acquire operation. - The buffer pool keeps track of which buffers are currently in use and which are free.
3. Releasing Buffers: - After a component finishes using a buffer, it releases it back to the buffer pool using a free operation. - The buffer pool marks the buffer as free and makes it available for reuse.
4. Efficiency: - Since buffers are pre-allocated and reused, the overhead of frequent dynamic memory allocation and deallocation is reduced. - This approach can improve performance by reducing memory fragmentation and minimizing the time spent in memory allocation routines.

Benefits

1. Reduced Overhead: Avoids frequent calls to new and delete , which can be costly.
2. Improved Performance: Faster allocation and deallocation times due to preallocation and reuse.
3. Control Over Memory Usage: Allows better control over how memory is allocated and used within an application.

Buffer pools are particularly useful in scenarios where memory allocation and deallocation operations are frequent and need to be efficient, such as in networking applications, databases, or multimedia processing systems.

8.1.2. MediaLibraryBufferPool API Guide

The *MediaLibraryBufferPool* class provides a buffer pool for managing media buffers. This guide explains how to use the main functions of the *MediaLibraryBufferPool* API: *init*, *free*, *acquire_buffer*, and *release_buffer*.

Note: The buffers in the *MediaLibraryBufferPool* are allocated from a physically contiguous memory region. This ensures better performance and compatibility with hardware accelerators.

Initialization

media_library_return **MediaLibraryBufferPool::init()**

Initialization of *MediaLibraryBufferPool* Allocates all the required buffers (according to max_buffers)

Example usage:

```
MediaLibraryBufferPool buffer_pool(width, height, format, max_buffers, memory_
→type);
media_library_return result = buffer_pool.init();
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Returns media_library_return

Deinitialization (Free)

media_library_return **MediaLibraryBufferPool::free**(bool fail_on_used_buffers = true)

Free all the allocated buffers.

Example usage:

```
media_library_return result = buffer_pool.free();
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Parameters fail_on_used_buffers - [in] - bool flag to indicate if the function should fail if there are still used buffers

Returns media_library_return

Acquire Buffer

media_library_return **MediaLibraryBufferPool::acquire_buffer**(HailoMediaLibraryBufferPtr ...)

Acquire a buffer from the pool.

Example usage:

```
HailoMediaLibraryBufferPtr buffer;
media_library_return result = buffer_pool.acquire_buffer(buffer);
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Parameters buffer - [out] - HailoMediaLibraryBufferPtr to acquire

Returns media_library_return

Release Buffer

`media_library_return MediaLibraryBufferPool::release_buffer(HailoMediaLibraryBufferPtr ...)`
Release a buffer back to the pool.

Example usage:

```
media_library_return result = buffer_pool.release_buffer(buffer);
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Parameters `buffer` - [out] - HailoMediaLibraryBufferPtr to release

Returns `media_library_return`

Usage Example

`class MediaLibraryBufferPool: public std::enable_shared_from_this<MediaLibraryBufferPool>`
A class representing a buffer pool in the media library.

The `MediaLibraryBufferPool` class is responsible for managing a pool of buffers used in the media library. It provides methods for acquiring and releasing buffers, as well as initializing and freeing the pool. The buffer pool can be configured with a specific width, height, format, and maximum number of buffers.

Example usage:

```
// Create a buffer pool with a width of 640, height of 480, and maximum of 10 buffers
MediaLibraryBufferPool bufferPool(640, 480, HAILO_FORMAT_RGB, 10);

// Initialize the buffer pool
bufferPool.init();

// Acquire a buffer from the pool
HailoMediaLibraryBufferPtr buffer = std::make_shared<hailo_media_library_
    ↳buffer>();
bufferPool.acquire_buffer(buffer);

// Use the acquired buffer
// ...

// Release the buffer back to the pool
bufferPool.release_buffer(&buffer);

// Free the buffer pool
bufferPool.free();
```

8.2. Electronic Image Stabilization (EIS)

8.2.1. Introduction

Electrical Image Stabilization (EIS) is a digital signal processing technique that compensates for unwanted camera motion to produce smoother, more stable video footage. Unlike optical image stabilization (OIS) which uses physical lens or sensor movement, EIS operates entirely in the digital domain by analyzing motion between consecutive frames and applying corrective transformations to counteract detected shake.

Hailo's EIS solution leverages gyroscope data to detect camera movement and applies rotational transformations to stabilize the output video. The system also includes rolling shutter compensation. When using EIS, the Field of View (FOV) is slightly reduced to allow margin for rotational correction.

Currently, the only supported Gyroscope sensor is the [lsm6dsr](#) by STMICROELECTRONICS®.

8.2.2. Key Principles

The gyroscope measures three-dimensional angular velocity, which is integrated over time to determine the camera's rotational position. To compensate for camera shake and stabilize frames, the camera's 3D orientation must be accurately calculated. Since the gyroscope and camera may not be perfectly aligned, a calibrated rotation matrix is used to transform gyroscope measurements into the camera's coordinate frame.

This transformation is established using a dedicated calibration tool that defines the rotational relationship between the two sensors. Frame rotation compensation relies on the camera's intrinsic parameters, including the camera matrix, focal length, and lens distortion coefficients.

These parameters define the internal geometry and optical properties required for accurate geometric transformations.

To address rolling shutter artifacts, each image row (or a group of consecutive rows) is rotated individually based on its capture timestamp. Since rolling shutter sensors expose rows sequentially rather than simultaneously, each row experiences slightly different motion. By applying row-specific rotation corrections that correspond to each row's exposure timing, geometric distortions typical of rolling shutter capture are effectively minimized.

8.2.3. Gyroscope Calibration

Before utilizing gyroscope measurements, it is essential to calibrate the following parameters using the provided calibration tool:

- **Gyroscope Bias:** Gyroscopes inherently produce biased readings, which must be corrected through calibration. This correction involves three float values, each corresponding to an axis.
- **Gyroscope Relative Orientation:** The axes of the sensor and gyroscope may not align perfectly. Calibration determines the static rotation matrix to align these coordinate systems, represented as three float values (using angle-axis representation).

The calibration process produces a static JSON configuration file, which is used for real-time stabilization with gyroscope measurements. The calibration process is long, it may take up to 30 minutes.

Note: These are the conditions that are recommended for creating the most accurate calibration:

- *Lighting:* Ensure the recorded scene is well-lit; avoid dark or poorly illuminated scenes.
- *Scene Content:* The scene should include distinct key points; avoid recording videos of featureless surfaces like a blank wall. For best results the use of a checkerboard pattern is highly recommended.
- *Movement Control:* During calibration, ensure that no unintended or unrelated movements appear in the frames.

Installing the calibration tool:

1. Download the whl file (hailo15_eis_calibration_tool-X.X.X-py3-none-any.whl) - The .whl is located at the Hailo-SW package under imaging_tools.
2. Create a new virtualenv - python3 -m virtualenv VENV_NAME_HERE
3. Enter the virtualenv (source ./VENV_NAME_HERE/bin/activate)
4. Install the whl (pip install hailo15_eis_calibration_tool-X.X.X-py3-none-any.whl)
5. Copy your cam intrinsics file (e.g.: /etc/imaging/cfg/<platform>/imxXXX/<lens>/4k/shared/calibration/cam_intrinsics.txt) into: ./VENV_NAME_HERE/lib/python3.10/site-packages/hailo15_eis_calibration_tool

The calibration tool usage is as follows (from inside the created virtualenv):

```
hailo15_eis_calibration_tool [record / run / all] -e <path>/cam_intrinsics.txt
```

There are two steps in running the calibration tool:

1. Record (*hailo15_eis_calibration_tool record*) - This step records the gyroscope data, frame timestamps, and records a video, typically three times by default (all three of them). The recording is interactive, guiding the user through camera movements at various angles.
2. Run (*hailo15_eis_calibration_tool run*) - This non-interactive step processes the recorded data and generates the calibration file.

Flags for both *record* and *run*:

- *-e* : Sensor type for cam intrinsics file (the default is *cam_intrinsics.txt*, meaning that the script will look for that file).
- *-c* : Number of recording sessions (the default (and recommended) is 3).
- *-r* : Where to store (for *record*) or find (for *run*) the output calibration files (the default is */tmp/eis-calibration-records*).

Flags for *record*:

- *-i* : IP address of the board (the default is 10.0.0.1).
- *-n* : Gyro device name (the default is *lsm6dsr_gyro*).
- *-f* : Frequency of the gyro (the default and recommended is 833).
- *-p* : Device type - hailo15h / hailo15l (the default is *hailo15h*).
- *-t* : Sensor type - default is *imx678*.
- *-l* : Print a list of the available IIO devices.
- *-v* : Print a verbose list of the available IIO devices.

Flags for *run*:

- *-s* : Gyroscope scale (the default is 0.000152716).
- *-o* : Number of optimization iterations (the default (and recommended) is 300)

The *all* option runs both *record* and *run* sequentially.

For additional information, regarding the flags for each option, run: *python calibration_tool.py [record /run / all] -h*

After running the calibration tool, a JSON file will be created in *./output/final_calibration.json*

Example output:

```
{
  "rot_x": -1.3166675383447541,
  "rot_y": 1.4847855684859292,
  "rot_z": 1.0729443141500428,
  "gbias_x": 0.002363248456949236,
  "gbias_y": -0.009985423836812312,
  "gbias_z": 0.004928467067823413
}
```

this file is the finalized gyroscope calibration file that should be used for EIS stabilization. it is expected to be located on the device under the path specified in the stabilization configuration in the media library profile (default is */home/root/apps/resources/final_calibration.json*).

8.2.4. EIS Online Algorithm

The EIS online algorithm's goal is to find the per-frame rotation matrix to stabilize the video. The algorithm is part of Hailo's media library package. It processes gyroscope measurements along three axes, each with an associated timestamp to produce the stabilization matrix. The general EIS flow is as follows:

1. **Frame Capture:** The camera captures a frame.
2. **Gyroscope Data Collection:** The gyroscope sensor collects rotation data related to the frame (the V-SYNC signal is used to synchronize between the gyro samples and the frame ISP timestamp).
3. **IIR filter:** The gyroscope data is filtered using a high pass filter to reduce bias.
4. **Integrate rotations:** The filtered gyroscope data is integrated to obtain the correction matrix.
5. **Mesh creation:** A dewarp mesh is created based on the correction matrix.
6. **Dewarp:** The frame is dewarped (in the DSP) using the created mesh.

8.2.5. Notes and Limitations

- **EIS and Lens Calibration:** EIS performance is heavily dependent on accurate lens calibration. It is recommended to perform lens calibration for the specific camera module and sensor in use to achieve optimal stabilization results.
- **EIS and HDR Compatibility:** Electronic Image Stabilization (EIS) is currently not supported when High Dynamic Range (HDR) is enabled.
- **EIS State Consistency:** Changing the EIS state (turning it on or off) on a per-frame basis can lead to system instability and may cause a crash.
- **EIS and 3DNR Performance:** EIS relies on precise synchronization between the gyroscope and the camera, assuming each frame represents a discrete point in time. Temporal averaging violates this assumption by blending pixel data from multiple frames, creating a mismatch between:
 - Gyro data: Represents instantaneous motion at frame capture time
 - Averaged frames: Contain blended motion data from multiple time intervals

EIS can operate with temporal averaging enabled, but stabilization performance may degrade. For applications prioritizing stabilization quality, it is recommended to disable or minimize temporal averaging strength.

- **Motion Blur Limitation:** EIS cannot correct motion blur, as blur occurs during exposure, before any digital stabilization can be applied. The relationship between shutter speed and blur is direct: longer exposure times produce more pronounced blur artifacts. This limitation becomes particularly significant under the following conditions:
 - High external vibration: Excessive camera shake increases pixel displacement
 - High zoom levels: Magnification amplifies the apparent movement, causing greater displacement

For conditions involving significant vibration or high zoom, it is recommended to limit shutter speed to 15 milliseconds or less, to minimize motion blur effects that EIS cannot compensate for.

8.3. MediaLibrary

8.3.1. Hailo Camera Configuration Management System

Introduced in version 1.7.0, Hailo has implemented a new profile-based configuration system for its camera application. This system simplifies the management of camera settings by allowing users to define distinct operating profiles, each representing a specific working point of the system.

Each profile fully encapsulates the camera's functionalities including:

- **General attributes:** Resolution, color format, number of output streams from the image processing pipeline, and number of streams going into the encoder.
- **Sensor-specific settings:** Calibration and tuning parameters for the image processing pipeline.
- **Processing configurations:** Codec settings and On-Screen Display (OSD) definitions (if applicable).

The profile file replaces the legacy *frontend_config.json* and *encoder_config.json* files, while expanding their capabilities by adding additional references to ISP tuning files, and several other small differences, which replace the existing functionality of updating 3aconfig files in */usr/bin*.

For more details, refer to [Media Library Configurations](#).

Reference Configuration Files

To facilitate integration, Hailo provides reference configuration files for the Sony IMX678, IMX675, IMX715, IMX662 sensors and Theia SL410M lens. For a complete list of all supported camera sensor modules, please refer to the Hailo-15 Approved Vendor List. These configurations align with common use cases in our reference application, including:

- **Daylight:** System tuned for a well-lit environment, with the sensor working in single exposure and ISP denoising enabled.
- **High Dynamic Range (HDR):** System tuned for a high dynamic range environment, with the HDR 2-DOL feature enabled.
- **Low-Light:** System tuned for a low-light environment, with AI-based denoising enabled.
- **Infrared (IR):** Similar to the daylight profile but outputs grayscale images from the processing pipeline.

Reference Profiles

As part of the release, Hailo also provides a reference profile named *lowlight_r0225c5_profile*, located in the same folder as the *lowlight_r0225c8_profile*. This profile is similar to the lowlight profile but with a different noise/detail trade-off. For users wishing to switch to either of these networks instead of the default one, simply change the JSON path in the *medialib_config.json*.

Profile File Usage

Profile files are designed to be treated as pre-configured points generated offline for the system, which set it in a designated state. On top of this, additional parameter changes can be applied via an API. As such, the profile needs to be treated as a **read-only file** by the runtime application.

Example: Initialize MediaLibrary and Using Profiles

The following example demonstrates how to initialize the MediaLibrary, switch profiles, retrieve the current profile, and override parameters:

```
#include <iostream>
#include <memory>
#include <string>
#include "media_library.h"

MediaLibraryPtr m_media_lib;

auto media_lib_expected = MediaLibrary::create();
if (!media_lib_expected.has_value())
{
    std::cout << "Failed to create media library" << std::endl;
    return 1;
}
m_media_lib = media_lib_expected.value();

std::string medialib_config_path =
    "/etc/imaging/apps/h15/native/configurations/cfg/medialib_configs/webserver_
    ↳medialib_config.json";

std::string medialib_config_string = read_string_from_file(medialib_config_path.c_
    ↳str());

// Initialize the media library with the sample configuration file and start with the
    ↳default profile
if (m_media_lib->initialize(medialib_config_string) != media_library_return::
    ↳MEDIA_LIBRARY_SUCCESS)
{
    std::cout << "Failed to initialize media library" << std::endl;
    return 1;
}

// Switch to lowlight profile
if (m_media_lib->set_profile("Lowlight") != media_library_return::MEDIA_LIBRARY_
    ↳SUCCESS)
{
    std::cout << "Failed to set profile" << std::endl;
    return 1;
}

// Get the current active profile
auto profile_config_expected = m_media_lib->get_current_profile();
if (!profile_config_expected.has_value())
{
    std::cout << "Failed to get current profile" << std::endl;
    return 1;
}

// Update a parameter inside the configuration and set it on top of the profile
auto profile_config = profile_config_expected.value();
profile_config.ldc_config.dewarp_config.enabled = true;

if (m_media_lib->set_override_parameters(profile_config) != media_library_return::
    ↳MEDIA_LIBRARY_SUCCESS)
{
    std::cout << "Failed to set parameters" << std::endl;
    return 1;
}
```

Note: Future releases will include the ability to move common settings between multiple profiles into shared files. Currently, even if information like lens distortion calibration is identical, it will exist explicitly in the profiles, although multiple profiles may point to the same calibration or tuning file.

Profile Backup and Restore

Media Library supports backing up and restoring profile states, allowing to preserve the current configuration state and restore it after system restarts.

Backup Functionality:

The `backup_profiles()` method creates a backup of the current profile state, including the active profile name and all profile configurations. The backup is stored as JSON files in a configured backup folder. The default backup folder path is extracted from the media library configuration file during initialization, using the `backup_folder_path` parameter. The default backup folder path can be overridden by setting it programmatically using `set_default_backup_folder_path()`.

Restore Functionality:

During initialization, Media Library will automatically restore from a backup if the `should_restore_backup` parameter is set to `true`. The library will look for the backup configuration files in the configured backup folder. If found and valid, it will initialize using the backup configuration instead of the provided configuration string. The active profile that was saved during backup is also restored, ensuring the system returns to the same profile state as when the backup was created. If the backup restoration fails, the library falls back to using the original configuration. Note that if a custom backup folder path should be used instead of the default one, it must be set by using `set_default_backup_folder_path` before initialization.

Usage Example:

```
// Set the backup folder path before initialization (optional)
// This will override any backup_folder_path in the config JSON
m_media_lib->set_default_backup_folder_path("/custom/backup/path");

// Initialize with backup restoration enabled
std::string medialib_config_string = read_string_from_file(medialib_config_path.c_
↳str());

// Initialize with restore from backup if available
if (m_media_lib->initialize(medialib_config_string, true) != media_library_return::
↳MEDIA_LIBRARY_SUCCESS)
{
    std::cout << "Failed to initialize media library" << std::endl;
    return 1;
}

// ... perform profile operations ...

// Backup current profile state to the default backup folder
if (m_media_lib->backup_profiles() != media_library_return::MEDIA_LIBRARY_SUCCESS)
{
    std::cout << "Failed to backup profiles" << std::endl;
}
```

Profile Switching Logic

Profiles themselves do not contain switching logic. However, as they include the path to a *3aconfig* file, internal interpolation related to the 3A algorithms based on gains will continue to be handled internally within the profile. As referenced in the *face_landmarks* app, it is expected that the user application switches between profiles based on runtime information (e.g., gain, dynamic range statistics) or via a UI decision. **Important:** Profile switches may cause a reset to the running stream if there are significant differences between the active and new profile being set.

When switching profiles: - The *isp_config_files* section fields (*3aconfig_path* and *sensor_entry*) are copied to the */tmp* directory. - Symlinks are created for these files in */usr/bin/isp_3aconfig_0* and */usr/bin/isp_sensor_entry_0*. - The activated files are the symlinks themselves. If you need to edit the values, you can do so by modifying the symlinked files. The symlinks always point to the currently activated files.

Persistent Settings Override

Media Library supports preserving certain user-configured settings across profile switches through the **override persistent settings** mechanism.

When *set_override_persistent_settings(true)* is called, profile switching works as follows:

- Before applying a new profile, the library will override all configuration structs marked as “persistent” in the new profile with the corresponding values from the currently active profile.
- This allows user-configured settings (such as zoom levels, rotation angles, stabilization parameters, etc.) to persist even when switching to a different profile.
- The default persistence state for each configuration struct can be changed at runtime by directly setting the static *is_persistent* member of the configuration struct.

Note: The *automatic_algorithms_config_t* struct has a special persistence mechanism:

- **is_recursively_persistent:** When set to *true*, the entire *automatic_algorithms_config_t* is overridden as a single unit, regardless of individual struct persistence flags.
- **Individual is_persistent flags:** Each of the structs within *automatic_algorithms_config_t* (e.g., *Aev1_config_t*, *Awbv2_config_t*, *AGamma64_config_t*) has its own *is_persistent* flag, so if *is_recursively_persistent* in *automatic_algorithms_config_t* is *false*, each struct is evaluated individually based on its own *is_persistent* flag.

Version Compatibility:

- **v1.10 (previous release):** The *override_persistent_settings* flag defaulted to **false** for backwards compatibility.
- **v1.11 (current release):** The default is now **true**, enabling persistent settings override by default. Applications that rely on persistent settings NOT being overridden will need to explicitly call *set_override_persistent_settings(false)*.

Usage Example:

```
// Enable persistent settings override, this is the default behavior
media_library->set_override_persistent_settings(true);

// Customize which settings persist across profile switches
dewarp_config_t::is_persistent = false;
config_application_input_streams_t::is_persistent = true;

// Control automatic algorithms persistence at bulk level
automatic_algorithms_config_t::is_recursively_persistent = true; // Override all
↳at once

// OR control at individual struct level
automatic_algorithms_config_t::is_recursively_persistent = false;
Aev1_config_t::is_persistent = true; // Only persist AE v1 settings
```

(continues on next page)

(continued from previous page)

```
Awbv2_config_t::is_persistent = false; // Don't persist AWB v2 settings

// Switch profiles, all persistent settings will be preserved from previous profile
media_library->set_profile("Lowlight");

// Disable persistent settings override
media_library->set_override_persistent_settings(false);

// Switch profiles. Now all settings from the new profile will be loaded, without any
↳ persistent settings being overridden
media_library->set_profile("High_Dynamic_Range");
```

Configuration Hierarchy

The supported profiles and their respective configurations files are defined in a media library configuration file. This file contains a list of profiles, each pointing to its respective configuration files and calibration artifacts.

In each media library configuration file, all supported profiles should be defined, and each profile should point to its respective configuration file. In each profile configuration file, all the required configuration files and calibration artifacts should be defined, including all encoders settings.

The recommended flow is to use the provided reference hierarchy structure located under `/etc/imaging/cfg/<Device Type>/<Sensor Type>/<Lens Type>`, which contains the configuration file, profiles, and all the tuning and calibration artifacts, as a starting point for generating the user environment. The recommended flow for a system to support multiple sensors, lenses, and use cases is to create multiple media library configuration files, each containing a list of profiles relevant to the specific application, and have you application point to a unified location which can be modified using a soft-link.

For users using sensor or lens that differ from the reference ones: - It is recommended to rename the folder hierarchy to avoid confusion or copy it into a new directory structure with the correct sensor and lens.

Example Folder Structure

Below is an example of the folder structure for the configuration hierarchy:

```
<device_type>/
  <sensor_name>/
    <lens_name>/
      <resolution>/
        profiles/
          <profile_name>/
            <application>/
              <application>_profile.json
              application_settings.json
              stabilizer_settings.json
              calib.json
              iq_settings.json
              sensor_config.json
        shared/
          codecs/
            codec_config1.json
            ...
            codec_configN.json
          calibration/
            cam_intrinsics.txt
            eis_calibration.xml
```

This structure organizes the configurations by sensor, lens, resolution, and profiles, making it easier to manage and customize for specific use cases.

It is recommended to stay in a similar hierarchy in the user environment, as the tuning tool assumes that this is the folder structure in use.

Configuration Generation Flow

1. Add or modify profiles:

- The baseline profiles are *daylight*, *lowlight*, *HDR*, and *IR*. If some are not relevant to the user application, or there are more profiles, the user should:
 - Generate the corresponding folders in the same hierarchy.
 - Update the profile name and path in the *medialib_config.json*.

2. For each of the profiles, update the following:

- **"input_video"**: The input video stream to the media library.
- **"application_input_streams"**: The number of streams sent to the application.
- **"encoded_output_streams"**: The number of streams sent to the encoder.
- **additional parameters can be configured as needed**

3. Run the calibration for the profiles, overwriting the reference files:

- *cam_intrinsics.json* - location can be found and updated in the *application_settings.json* file.
- *eis_calibration.xml* - location can be found and updated in the *stabilizer_settings.json* file.
- *sony_imx678.xml*

4. Run the tuning tool for the each of profiles

As mentioned earlier, since the profiles also contain application logic (e.g., resolutions and number of outputs), it is expected that in the case of different applications running on the same device, the user will create multiple *medialib_config.json* files.

Each of these *medialib_config.json* files would contain a list of different profiles generated to match the specific application. However, as applicable, these profiles would point to the same calibration and tuning files.

Note:

- Updating the number of streams sent to the application is **not supported** as a run-time operation by the Hailo media library. If such functionality is required, the profiles should be originally created with the maximal required streams but with the FPS set to zero for the inactive ones. To enable them, use the *set_override_parameters* function to set the actual FPS.
- In future releases, Hailo will provide a **UI-based tool** to dynamically generate configurations and profiles, as well as connect to the calibration and tuning tools.

Reference Application Usage

The following reference applications are going to switch to using the new profile configuration scheme:

- **Tuning tool**: Please see the Hailo Imaging Camera Calibration and Tuning Guide under the Hailo Developer Zone.
- **face_landmarks_app**: See example code in [hailo-camera-apps](#).
- **frontend_example**: See example code in [hailo-media-library](#).

These applications will move to use the new API and work based on the profile support.

All existing GStreamer-based reference applications are expected to continue working with the legacy flow.

8.3.2. API Reference

class `MediaLibrary`

A class to manage media library operations including frontend and encoders.

Public Functions

`MediaLibrary()`

Constructor for the media library module.

Note: This constructor is used internally by the create function.

media_library_return `initialize(std::string frontend_config_json_string, std::map<output_stream_id_t, ...>)`
Initialize the media library with frontend and encoder configurations.

Parameters

- `frontend_config` - Configuration for the frontend.
- `encoders_config` - Vector of encoder configurations.

Returns Status of the initialization.

media_library_return `initialize(std::string medialib_config_string, bool should_restore_backup = false)`
Initialize the media library with a configuration string.

Parameters

- `medialib_config_string` - string containing the media library configuration.
- `should_restore_backup` - Whether to restore the backup configuration, if exists.

Returns Status of the initialization.

media_library_return `subscribe_to_frontend_output(FrontendCallbacksMap fe_callbacks)`
Subscribe to frontend output.

Parameters `fe_callbacks` - Map of frontend callbacks.

Returns Status of the subscription.

media_library_return `subscribe_to_frontend_gst_output(FrontendGstBufferCallbacksMap ...)`
Subscribe to frontend output receiving the original GstBuffer.

Unlike *subscribe_to_frontend_output()*, the callback receives the original GstBuffer produced by the internal frontend pipeline, avoiding a teardown/rebuild cycle. The GstBuffer is refed before the callback; the callee owns that ref.

Parameters `fe_callbacks` - Map of GstBuffer callbacks keyed by stream id.

Returns Status of the subscription.

media_library_return `subscribe_to_encoder_output(output_stream_id_t streamId, ...)`
Subscribe to encoder output.

Parameters

- `streamId` - ID of the output stream.
- `callback` - Application wrapper callback.

Returns Status of the subscription.

media_library_return `start_pipeline()`
Start the media pipeline.

Returns Status of the operation.

media_library_return stop_pipeline()
Stop the media pipeline.

Returns Status of the operation.

~MediaLibrary()
Destructor.

media_library_return set_override_parameters(const *config_profile_t* &profile)
Sets the profile for the media library.

This function sets the profile for the media library using the provided profile object.

- An update to one of the following fields will trigger an internal stream reset:
 - **input_video**
 - **application_input_streams**
 - **rotation**
 - **isp** section
- Changing the HDR or AI-denoise state from enabled to disabled or vice versa is **not allowed** by this API. The function will return an error return code if such a change is attempted.

Parameters profile - The profile to be set.

Returns media_library_return The result of the operation.

media_library_return set_automatic_algorithm_configuration(std::string...)
Set the automatic algorithm configuration as json object.

Parameters automatic_algorithms -

Returns medialibrary_return

media_library_return set_profile(const std::string &profile_name)
Sets the profile for the media library.

This function sets the profile for the media library using the provided profile name. If override persistent settings is enabled, the persistent settings will be overridden by the previous profile.

Parameters profile_name - The name of the profile to be set.

Returns media_library_return The result of the operation.

void set_override_persistent_settings(bool override_persistent_settings)
Sets the override persistent settings flag.

This function sets the override persistent settings flag.

Parameters override_persistent_settings - Whether to override the persistent settings.

media_library_return reset_profiles()
Reset any overrides made to the profiles.

This function resets any overrides made to the profiles, restoring them to their default state.

Returns media_library_return The result of the operation.

media_library_return set_auto_profile_restriction_enabled(bool enabled)
Enable or disable automatic profile restriction switching feature. This feature allows the media library to automatically switch profiles and notify the user when a restriction occurs due to throttling conditions. Automatically switch to the fallback profile upon restriction. Enabled by default.

Parameters enabled - True to enable automatic profile restriction switching, false to disable.

Returns media_library_return The result of the operation.

media_library_return `set_restriction_fallback_profile(const std::string &profile_name)`
Sets the restriction fallback profile.

This function sets the fallback profile to be used when the system is under throttling conditions and the current profile is restricted. Without setting this, the default profile will be used as the fallback profile.

Parameters `profile_name` - The name of the fallback profile to be used on restriction.

Returns `media_library_return` The result of the operation.

`bool get_auto_profile_restriction_enabled()`
Get the automatic profile restriction switching feature enabled state.

Returns `boolean`

`tl::expected<config_profile_t, media_library_return> get_profile(const std::string &profile_name)`
Retrieves the profile with the given name.

Parameters `profile_name` - The name of the profile to be retrieved.

Returns `tl::expected<config_profile_t, media_library_return>` The profile with the given name.

`tl::expected<config_profile_t, media_library_return> get_current_profile()`
Retrieves the current profile.

This function returns the current profile being used.

Returns `tl::expected<config_profile_t, media_library_return>` An expected object containing the current profile configuration if successful, or an error code otherwise.

`tl::expected<std::string, media_library_return> get_current_profile_str()`
Retrieves the current profile as a JSON string.

This function returns the current profile being used in JSON string format.

Returns `tl::expected<std::string, media_library_return>` An expected object containing the current profile as a JSON string if successful, or an error code otherwise.

`bool stream_restart_required(config_profile_t previous_profile, config_profile_t new_profile)`
Checks if a stream restart is required based on the provided profile.

Parameters `previous_profile` - The configuration of the previously active profile, used to determine if a stream restart is necessary.

Returns A boolean value indicating whether a stream restart is required.

`tl::expected<std::vector<frontend_output_stream_t>, media_library_return> get_frontend_output_streams()`
Get the output streams from the frontend.

Returns `tl::expected` containing a vector of `frontend_output_stream_t` or an error code.

media_library_return `unsubscribe_all_from_frontend()`
Unsubscribe all callbacks from the frontend output streams.

Returns Status of the operation.

media_library_return `add_buffer_to_encoder(output_stream_id_t stream_id, ...)`
Add a buffer to a specific encoder for encoding.

Parameters

- `stream_id` - ID of the output stream / encoder.
- `buffer` - Shared pointer to the buffer to be encoded.

Returns Status of the operation.

media_library_return `on_profile_restricted(std::function<void(config_profile_t, config_profile_t)> ...)`
Set the On profile restricted user callback.

When the system is under thermal event that restricts the current profile. Media Library will restrict the current profile automatically and set the default profile as a fallback. In some scenarios, the default profile may include some specific restriction (AI-ISP for example), Media Library will also configure it off.

You can override the restricted fallback profile using `set_restriction_fallback_profile` function.

After automatically switching, the user callback will be called with both profiles (previous and new).

Parameters `callback` – The callback to be set - includes the previous and new restricted profiles.

Returns `media_library_return` The result of the operation.

media_library_return `subscribe_to_profile_restricted(std::function<void(const ...)>)`
Subscribe to profile restricted user callback.

When the system is under thermal event that restricts the current profile. Media Library will restrict the current profile automatically and set the default profile as a fallback. In some scenarios, the default profile may include some specific restriction (AI-ISP for example), Media Library will also configure it off.

You can override the restricted fallback profile using `set_restriction_fallback_profile` function.

After automatically switching, the user callback will be called with both profiles (previous and new).

Parameters `callback` – The callback to be set - includes the previous and new restricted profiles.

Returns `media_library_return` The result of the operation.

media_library_return `on_profile_restriction_done(std::function<void()> callback)`
Subscribe to the profile restriction done user callback.

- When the system has recovered from the event that restricted the current profile, Media Library will only notify the user that the profile restriction is ended, without changing the profile back

Parameters `callback` – The callback to be set.

Returns `media_library_return` The result of the operation.

media_library_return `subscribe_to_profile_restriction_done(std::function<void()> ...)`
Subscribe to the profile restriction done user callback.

- When the system has recovered from the event that restricted the current profile, Media Library will only notify the user that the profile restriction is ended, without changing the profile back

Parameters `callback` – The callback to be set.

Returns `media_library_return` The result of the operation.

media_library_return `on_pipeline_state_change(std::function<void(media_library_pipeline_state_t)> ...)`
Subscribe to pipeline state change user callback.

When the pipeline state changes, this callback will be called.

Parameters `callback` – The callback to be set.

Returns `media_library_return` The result of the operation.

media_library_return `subscribe_to_pipeline_state_change(std::function<void(media_library_pipeline_state_t)> ...)`
Subscribe to pipeline state change user callback.

When the pipeline state changes, this callback will be called.

Parameters `callback` – The callback to be set.

Returns `media_library_return` The result of the operation.

`tl::expected<media_library_throttling_state_t, media_library_return> get_throttling_state()` const
Get the current pipeline throttling state.

Returns `tl::expected<media_library_throttling_state_t, media_library_return>` The current pipeline throttling state or an error code.

media_library_return `subscribe_to_throttling_state_change(std::function<void(media_library_throttling_state_t)> callback)`
Subscribe to throttling state change events.

Parameters `callback` - The callback function to be called on throttling state changes.

Returns `media_library_return` The result of the operation.

media_library_return `unsubscribe_from_profile_restriction_callbacks()`
Unsubscribe from all profile restriction callbacks.

Removes the callbacks registered via `subscribe_to_profile_restricted` and `subscribe_to_profile_restriction_done`. This should be called during cleanup to prevent callbacks being invoked on destroyed objects.

Returns `media_library_return` The result of the operation.

media_library_return `unsubscribe_from_throttling_state_change()`
Unsubscribe from throttling state change events.

Removes the callback registered via `subscribe_to_throttling_state_change`. This should be called during cleanup to prevent callbacks being invoked on destroyed objects.

Returns `media_library_return` The result of the operation.

`media_library_pipeline_state_t get_pipeline_state()` `const`
Get the current pipeline state.

Returns The current pipeline state (STARTED or STOPPED).

AnalyticsDB `&get_analytics_db()`
Get the analytics database.

Returns Reference to the analytics database.

media_library_return `shutdown()`
Gracefully shuts down the media library instance.

Stops all running pipelines, releases allocated resources, and performs any necessary cleanup before exiting. After calling this function, the instance should not be used again unless it is re-initialized.

Returns `MEDIA_LIBRARY_SUCCESS` on success, or an appropriate error code of type `media_library_return` on failure.

media_library_return `backup_profiles()`
Backup the current profiles state to the configured backup folder.

Creates a folder with JSON files for each profile containing the current profile state, including the active profile name and all profile configurations.

Returns `media_library_return` The result of the operation.

`void set_default_backup_folder_path(const std::string &path)`
Set the default backup folder path.

Parameters `path` - Path to the default backup folder

Public Members

MediaLibraryFrontendPtr `m_frontend`
Pointer to the frontend object.

`std::map<output_stream_id_t, MediaLibraryEncoderPtr>` `m_encoders`
Map of output stream IDs to encoder pointers.

Public Static Functions

static `tl::expected<MediaLibraryPtr, media_library_return> create( )`
Constructs a [MediaLibrary](#) object.

Returns `tl::expected<MediaLibraryPtr, media_library_return>` - An expected object that holds either a shared pointer to a [MediaLibrary](#) object, or a error code.

```
using MediaLibraryPtr = std::shared_ptr<MediaLibrary>
```

8.4. Frontend

8.4.1. C++ Configuration API

The Frontend can be configured using the following C++ API:

example:

```
auto config_expected = frontend->get_config();
if (!config_expected)
{
    return media_library_return::MEDIA_LIBRARY_ERROR;
}

frontend_config_t config = config_expected.value();

config.frontend_element_config.ldc_config.dewarp_config.enabled = false;
if (m_media_lib->frontend->set_config(config) != media_library_return::MEDIA_
    ↳LIBRARY_SUCCESS)
{
    return media_library_return::MEDIA_LIBRARY_ERROR;
}
```

For more information about the configuration structs see the `frontend_config_t` struct in [Media Library Types](#) documentation.

8.4.2. API Reference

`typedef std::function<void(HailoMediaLibraryBufferPtr, uint32_t)> FrontendWrapperCallback`
type for user callback function to receive output frames

The provided user function receives the output frames and their sizes as `uint32_t` argument. [MediaLibraryFrontend::subscribe](#)

`class MediaLibraryFrontend`
Frontend object with Dewarp and Multi Resize features.

Each instance represents a Frontend Bin. Each Frontend bin have one input stream and may have several output streams, depending on configuration.

Public Functions

`MediaLibraryFrontend(std::shared_ptr<Impl> impl)`
 Constructor for the frontend module.

Note: This constructor is used internally by the create function.

Parameters `impl` - [in] - shared pointer to the implementation object

`tl::expected<frontend_config_t, media_library_return> get_config()`
 Get the configuration of the *MediaLibraryFrontend* module

Returns `tl::expected<frontend_config_t, media_library_return>`

`media_library_return set_config(const frontend_config_t &config)`
 Set the config object for the *MediaLibraryFrontend* module.

Parameters `config` - [in] - [*frontend_config_t*] configuration object, obtained from the `get_config` function

Returns `media_library_return`

`media_library_return start()`
 Start the *MediaLibraryFrontend* module, the *MediaLibraryFrontend* module will be ready to receive buffers. `set_config(const string&)` must be called before `start()`

Returns `media_library_return` - status of the start operation

`media_library_return stop()`
 Stop the *MediaLibraryFrontend* module, the *MediaLibraryFrontend* module will stop receiving buffers.

Returns `media_library_return` - status of the stop operation

`media_library_return pause_pipeline()`
 Pause the *MediaLibraryFrontend* module, the *MediaLibraryFrontend* module will pause processing and wait for the pipeline to reach paused state.

Returns `media_library_return` - status of the pause operation

`media_library_return unpause_pipeline()`
 Unpause the *MediaLibraryFrontend* module, the *MediaLibraryFrontend* module will resume processing and wait for the pipeline to reach playing state.

Returns `media_library_return` - status of the unpause operation

`media_library_return set_config(const std::string &json_config)`
 Configure the *MediaLibraryFrontend* module with the given configuration.

Parameters `json_config` - [in] - a json string containing the configuration

Returns `media_library_return` - status of the configuration operation

`media_library_return subscribe(FrontendCallbacksMap callbacks)`
 Subscribe to the *MediaLibraryFrontend* module to receive the output buffers.

Note: if the user wishes to add additional arguments for the callback execution a lambda wrapper and pass as single argument to the subscribe function. [AppWrapperCallback](#)

Parameters `callbacks` - [in] - a map of callbacks functions to be called when a buffer is ready. The callback function `AppWrapperCallback` object. The callback function should be thread-safe. number of callback functions should be equal to number of outputs. The callback function should not:

- Block.
- Throw exceptions.

- Call the [MediaLibraryFrontend](#) module.

Returns media_library_return - status of the subscription operation

[media_library_return](#) subscribe_gst (FrontendGstBufferCallbacksMap callbacks)
Subscribe to receive the original GstBuffer from the frontend pipeline.

Unlike [subscribe\(\)](#), the callback receives the original GstBuffer produced by the internal pipeline — avoiding the teardown/rebuild cycle. The GstBuffer is refed before the callback is invoked; the callee owns that ref and must either unref it or transfer ownership (e.g. gst_pad_push).

For a given stream_id, only ONE callback type fires (gst takes priority over the plain buffer callback registered via [subscribe\(\)](#)).

Parameters callbacks - [in] - a map of GstBuffer callback functions keyed by stream id

Returns media_library_return - status of the subscription operation

tl::expected<std::vector<std::string>, [media_library_return](#)> get_all_subscribers_ids ()
Get all subscriber IDs currently registered with the [MediaLibraryFrontend](#) module.

This function retrieves a list of all subscriber IDs that are currently registered to receive output buffers from the [MediaLibraryFrontend](#) module.

Returns tl::expected<std::vector<std::string>, media_library_return> - An expected object that holds either a vector of subscriber ID strings, or an error code indicating the reason for failure.

[media_library_return](#) unsubscribe_all ()
Unsubscribe all currently registered subscribers from the [MediaLibraryFrontend](#) module.

This function removes all active subscriptions, effectively stopping all callback functions from receiving output buffers. After calling this function, no callbacks will be invoked until new subscriptions are made.

Returns media_library_return - status of the unsubscribe all operation

[media_library_return](#) unsubscribe (const std::string &id)
Unsubscribe a specific subscriber from the [MediaLibraryFrontend](#) module.

This function removes the subscription for a specific subscriber ID, stopping the associated callback function from receiving output buffers. Other active subscriptions remain unaffected.

Parameters id - [in] - the subscriber ID to unsubscribe. This should match an ID that was previously registered through the subscribe function.

Returns media_library_return - status of the unsubscribe operation

[media_library_return](#) add_buffer (HailoMediaLibraryBufferPtr ptr)
Add a buffer to the [MediaLibraryFrontend](#) module, to be processed. The add_buffer function receives raw video frame and applies Dewarping and MultiResize on the raw frames.

Note: The [MediaLibraryFrontend](#) module will take ownership of the buffer.

Warning: Can be called only when the [MediaLibraryFrontend](#) module is configured with FRONTEND_SRC_ELEMENT_APPSRC.

Parameters ptr - [in] - a shared pointer to hailo_media_library_buffer to be processed.

Returns media_library_return - status of the add buffer operation.

tl::expected<std::vector<frontend_output_stream_t>, [media_library_return](#)> get_outputs_streams ()
Retrieves the IDs of the output streams.

This function returns a `tl::expected` object containing a vector of `output_stream_id_t`. If the operation is successful, the expected object will hold the vector of output stream IDs. Otherwise, it will hold a `media_library_return` error code indicating the reason for failure.

Returns A `tl::expected` object containing either the vector of output stream IDs or an error code.

```
std::unordered_map<output_stream_id_t, float> get_output_streams_current_fps( )
```

Get the current fps of the frontend outputs by the output ids.

Returns `std::unordered_map<output_stream_id_t, float>` - An unordered map of frontend output ids and their corresponding current fps

```
~MediaLibraryFrontend( ) = default
```

Destructor for the frontend module.

```
media_library_return set_freeze( bool freeze )
```

Set the freeze state of the frontend.

This function sets the freeze state of the frontend. When the frontend is frozen, the output buffer's image will be constant and will not change. The first buffer received after the freeze state is set will be the buffer that will be pushed.

Parameters `freeze` - - the freeze state to set

Returns `media_library_return` - status of the set freeze operation

```
bool wait_for_pipeline_state( PipelineState target_state, std::chrono::milliseconds timeout )
```

Wait for the pipeline to reach a target state.

This function waits for the frontend pipeline to transition to the specified target state within the specified timeout period.

Parameters

- `target_state` - - the `PipelineState` to wait for (e.g., `PipelineState::PLAYING`, `PipelineState::PAUSED`)
- `timeout` - - the maximum time to wait for the pipeline to reach the target state

Returns `bool` - true if the pipeline reached the target state within the timeout, false otherwise

```
void set_config_manager_interactor( const ConfigManagerInteractor ... )
```

Set the `ConfigManagerInteractor` for the `MediaLibraryFrontend` module.

This function sets the `ConfigManagerInteractor` object for the `MediaLibraryFrontend` module. The `ConfigManagerInteractor` is used to manage configuration changes and interactions within the module.

Parameters `config_manager_interactor` - - The `ConfigManagerInteractor` object to be set

Public Static Functions

```
static tl::expected<MediaLibraryFrontendPtr, media_library_return> create( )
```

Constructs an `MediaLibraryFrontend` object.

Returns `tl::expected<MediaLibraryFrontendPtr, media_library_return>` - An expected object that holds either a shared pointer to an `MediaLibraryFrontend` object, or a error code.

```
class Impl
```

```
struct output_view_t
```

```
using MediaLibraryFrontendPtr = std::shared_ptr<MediaLibraryFrontend>
```

8.5. Encoder

8.5.1. Overview

HailoEncoder API enables the user to encode a video in h265/h264/MJPEG coding format, using the **Hailo-15 encoding hardware accelerator**. It also supports [OSD](#) (On Screen Display) to blend overlays on incoming frame.

8.5.2. Usage

Create an instance of the Encoder with configuration string and stream id.

```
tl::expected<MediaLibraryEncoderPtr, media_library_return> encoder_expected =
    ↪MediaLibraryEncoder::create(encoderosd_config_string, id);
```

Subscribe to hailofrontend callbacks: when a new buffer is available - add the buffer to the Encoder.

```
auto streams = media_lib->frontend->get_outputs_streams();
FrontendCallbacksMap fe_callbacks;
frontend_output_stream_t s = streams[0];
fe_callbacks[id] = [s, media_lib](HailoMediaLibraryBufferPtr buffer, size_t size)
{
    encoder->add_buffer(buffer);
};
media_lib->frontend->subscribe(fe_callbacks);
```

Start the Encoder

```
encoder->start();
```

encoder_config_t can hold 2 types of configurations: hailo_encoder_config_t for h264/h265 and jpeg_encoder_config_t for mjpeg.

Examples for reading and updating a configuration of type hailo_encoder_config_t: Update the user encoder configuration while the pipeline is running

```
encoder_config_t encoder_config = encoder->get_user_config();
hailo_encoder_config_t &hailo_encoder_config = std::get<hailo_encoder_config_t>
    ↪(encoder_config);
std::cout << " Current bitrate: " << hailo_encoder_config.rate_control.bitrate.
    ↪target_bitrate << std::endl;
hailo_encoder_config.rate_control.bitrate.target_bitrate = 20000000;
if (encoder->configure(encoder_config) != media_library_return::MEDIA_LIBRARY_
    ↪SUCCESS)
{
    std::cout << "Failed to configure Encoder " << std::endl;
}
```

Read the actual Encoder configuration (after a preset is set according to the user config) while the pipeline is running

```
encoder_config_t encoder_config = encoder->get_config();
hailo_encoder_config_t &hailo_encoder_config = std::get<hailo_encoder_config_t>
    ↪(encoder_config);
std::cout << " Current gop_length: " << hailo_encoder_config.rate_control.gop_
    ↪length << std::endl;
```

In this example we receive the actual encoder_config struct from the encoder get_config() method, and read the gop_length value that was either set by the system or overridden by a value from the user.

Examples for reading and updating a configuration of type hailo_encoder_config_t: Read and update the user encoder configuration while the pipeline is running:

```
encoder_config_t encoder_config = encoder->get_user_config();
jpeg_encoder_config_t &jpeg_encoder_config = std::get<jpeg_encoder_config_t>
    (encoder_config);
std::cout << "Current quality: " << jpeg_encoder_config.quality << std::endl;
jpeg_encoder_config.quality = 80;
if (encoder->configure(encoder_config) != media_library_return::MEDIA_LIBRARY_
    SUCCESS)
{
    std::cout << "Failed to configure Encoder " << std::endl;
}
```

Note that no preset is set for the MJPEG encoder, so the user configuration is the actual configuration.

Stop the Encoder

```
encoder->stop();
```

8.5.3. API

typedef std::function<void(HailoMediaLibraryBufferPtr, uint32_t)> AppWrapperCallback
type for user callback function to receive encoded frames

The provided user function receives the encoded frames and their sizes as uint32_t argument. EncoderOSD::subscribe

class MediaLibraryEncoder

Encode stream object with OSD features.

Each instance represents an encoded output stream. Per each stream user can configure several static and dynamic OSD overlays.

Encoding and static OSD configurations are applied during instance construction.

Encoder static configuration includes the resolution, framerate, encoding format. Encoder dynamic configuration includes rate, Qp, ROI and other arguments. TBD json configuration.

OSD static overlays support images, static text and timestamp blocks. TBD json configuration.

Note: OSD is mostly executed by the DSP co-processor, and encoding is executed by the A53 or the HEVC/H264 encoder co-processor.

Public Functions

media_library_return start()

Start the *MediaLibraryEncoder* module, the *MediaLibraryEncoder* module will be ready to encode buffers. set_config(const string&) must be called before start().

Returns media_library_return - status of the start operation

media_library_return stop()

Stop the *MediaLibraryEncoder* module, the *MediaLibraryEncoder* module will stop encoding buffers.

Returns media_library_return - status of the stop operation

media_library_return subscribe(AppWrapperCallback callback)

Subscribe to the *MediaLibraryEncoder* module to receive the output buffers.

Note: if the user wishes to add additional arguments for the callback execution it may use lambda wrapper can be used and passed on as a single argument to the subscribe function. [AppWrapperCallback](#)

Parameters `callback` - **[in]** - callback function to be called when a buffer is ready. The callback function AppWrapperCallback object. The callback function should be thread-safe. The callback function should not block. The callback function should not throw exceptions. The callback function should not call the [MediaLibraryEncoder](#) module.

Returns `media_library_return` - status of the subscription operation

[media_library_return](#) `unsubscribe()`

Unsubscribe all currently registered subscribers from the [MediaLibraryEncoder](#) module.

Returns `media_library_return` - status of the unsubscription operation

[media_library_return](#) `add_buffer(HailoMediaLibraryBufferPtr ptr)`

Add a buffer to the [MediaLibraryEncoder](#) module, to be encoded. The `add_buffer` function receives raw video frame, and applies the stream's overlays on the raw frames, before encoding according to the streams configuration. The OSD and encoding are executed in a separate context, allowing pipeline smooth flow.

Note: The [MediaLibraryEncoder](#) module will take ownership of the buffer.

Parameters `ptr` - **[in]** - a shared pointer to `hailo_media_library_buffer` to be encoded

Returns `media_library_return` - status of the add buffer operation.

`std::shared_ptr<osd::Blender> get_osd_blender()`

Get an overlay manager object.

Returns `shared_ptr` containing the object `osd::Blender`

`std::shared_ptr<PrivacyMaskBlender> get_privacy_mask_blender()`

Get a privacy mask manager object.

Returns `shared_ptr` containing the object `PrivacyMaskBlender`

[media_library_return](#) `set_config(encoder_config_t &config)`

Configure the encoder module with a new configuration object.

Parameters `config` - **[in]** - encoder configuration object

Returns `media_library_return` - status of the configuration operation

[media_library_return](#) `set_config(const std::string &json_config)`

Configure the encoder module with a json config.

Parameters `config` - **[in]** - encoder configuration in a json format

Returns `media_library_return` - status of the configuration operation

`encoder_config_t get_config()`

Get a copy of the current actual configuration of the encoder module.

Returns `encoder_config_t` - the current actual configuration of the encoder module

`encoder_config_t get_user_config()`

Get a copy of the configuration of the encoder module, as given by the user.

Returns `encoder_config_t` - the user configuration of the encoder module

[EncoderType](#) `get_type()`

Get the encoder type.

Returns `EncoderType` - the type of the encoder

media_library_return `set_force_videorate (bool force = true)`
 Forces videorate to reuse frames if needed.

Parameters `force` - [in] - true to force videorate, false to disable

Returns `media_library_return` - status of the `set_force_videorate` operation

`MediaLibraryEncoder (std::shared_ptr<Impl> impl)`
 Constructor for the encoder module.

Note: This constructor is used internally by the `create` function.

Parameters `impl` - [in] - shared pointer to the implementation object

`float get_current_fps ()`
 Get the current fps of the encoder.

Returns `float` - the current fps of the encoder

encoder_monitors `get_encoder_monitors ()`
 Get the *encoder_monitors* of the encoder.

Returns *encoder_monitors* - the *encoder_monitors* of the encoder

`void set_sensor_index (size_t sensor_index)`
 Set the sensor index for this encoder.

Parameters `sensor_index` - - the sensor index

`~MediaLibraryEncoder ()` = default
 Destructor for the encoder module.

`void set_config_manager_interactor (const ConfigManagerInteractor ...)`
 Set the `ConfigManagerInteractor` for the *MediaLibraryEncoder* module.

This function sets the `ConfigManagerInteractor` object for the *MediaLibraryEncoder* module. The `ConfigManagerInteractor` is used to manage configuration changes and interactions within the module.

Parameters `config_manager_interactor` - - The `ConfigManagerInteractor` object to be set

`void add_config_attacher (bool enable)`
 Enable or disable the config attacher in the encoder pipeline.

When running the encoder standalone (without a full *MediaLibrary* pipeline), enable this to ensure buffers have an attached profile for OSD/privacy mask. Must be called before *set_config()*.

Parameters `enable` - - true to add a config attacher, false to disable (default)

Public Static Functions

`static tl::expected<MediaLibraryEncoderPtr, media_library_return> create (const output_stream_id_t ...)`
 Constructs an *MediaLibraryEncoder* object.

Returns `tl::expected<MediaLibraryEncoderPtr, media_library_return>` - An expected object that holds either a shared pointer to an *MediaLibraryEncoder* object, or a error code.

`class Impl`

Public Functions

void on_fps_measurement (GstElement *fpssink, gdouble fps, gdouble droprate, gdouble avgfps)
Below are public functions that are not part of the public API but are public for GStreamer callbacks.

using MediaLibraryEncoderPtr = std::shared_ptr<MediaLibraryEncoder>

8.5.4. Encoder Configurations

Configuring the encoder's parameters is done by providing a JSON file with *config-file-path* parameter. Some of the parameters are optional. These parameters are set according to a preset configuration, when the user does not provide them. The preset is chosen according to resolution, codec, bitrate and rc_mode and is relevant only for h264/h265 coding formats. The presets table is located at: /etc/medialib/encoder_presets.csv Below is a list of the supported parameters:

Table 12. Stream Control

Name	Description	Supported	Can update at runtime	Mandatory
codec	(H264 / H265)	Yes	Yes	Yes
width	Requires encoder restart to update at runtime	Yes	No	Yes
height	Requires encoder restart to update at runtime	Yes	No	Yes
framerate	Requires setting <i>enforce-caps=false</i> to update at runtime	Yes	Yes	Yes
stream_type	Stream type - bytestream / plain NAL units	No	No	Yes
bit_depth_luma	Luma sample bit depth of encoded bit stream (8/9/10)	No	No	Yes
bit_depth_chroma	Chroma sample bit depth of encoded bit stream (8/9/10)	No	No	Yes
format	Input video format	No	No	Yes
profile	(base/main/high/high-10)	Yes	Yes	No
level	Encoding level	Yes	Yes	No

Table 13. Gop

Name	Description	Supported	Can update at runtime	Mandatory
gop_size	(1-7) (1 - no B frames)	Yes	Yes	Yes
b_frame_qp_delta	QP difference between BFrame QP and target QP, -1 = Disabled (1 - 51)	Yes	Yes	Yes

Table 14. Coding Control

Name	Description	Supported	Can update at runtime	Mandatory
type	deblock filter - enable / disable	No	No	Yes
sei_messages	SEI messages configuration object with encoder_timing_sei (encoder's timing SEI) and user_metadata_sei (custom metadata SEI)	Yes	Yes	Yes
tc_offset	deblock parameter - tc_offset	No	No	Yes
beta_offset	deblock parameter - beta_offset	No	No	Yes
deblock_override	Enable deblock override between slice	No	No	Yes
roi_area	Specifying rectangular area of CTBs as Region Of Interest with lower QP	No	No	Yes
ipcm_area	Area for forcing IPCM macroblocks	No	No	Yes

Table 15. Rate Control

Name	Description	Supported	Can update at runtime	Mandatory
rc_mode	Rate control mode (VBR / CVBR / CBR)	Yes	Yes	Yes
picture_rc	Adjust QP between pictures	Yes	Yes	Yes
target_bitrate	Target bitrate for rate control in bits/second (10000 - 40000000)	Yes	Yes	Yes
qp_hdr	Initial target QP, -1 = Encoder calculates initial QP (1-51)	Yes	Yes	Yes
picture_skip	Enable Frame Skip	No	No	Yes
intra_pic_rate	I frames interval (0 - Dynamic IDR Interval) (0 - 300)	No	No	Yes
ctb_rc	Adaptive adjustment of QP inside frame	Yes	Yes	No
block_rc_size	Size of block rate control	Yes	Yes	No
hrd	Restricts the instantaneous bitrate and total bit amount of every coded picture.	Yes	Yes	No
padding	Whether padding on underflow is enabled. Mandatory only for CVBR.	Yes	Yes	No
cvbr	Flags to modify rate control behavior	Yes	Yes	No
hrd_cpb_size	Buffer size used by the HRD model in bits	Yes	Yes	No
gop_length	Rate control uses this to match the average bit rate of each GOP to the target bit rate. affects how fast the rate control reacts to changes in the video sequence. (1 - 300)	Yes	Yes	No
monitor_frames	How many frames will be monitored for moving bit rate. Default is using framerate	Yes	Yes	no
bit_var_range_i	Percent variations over average bits per frame for I frame	Yes	Yes	No
bit_var_range_p	Percent variations over average bits per frame for P frame	Yes	Yes	No
bit_var_range_b	Percent variations over average bits per frame for B frame	Yes	Yes	No
tolerance_moving_bitrate	Percent tolerance over target bitrate of moving bit rate (0 - 2000)	Yes	Yes	No
variation	Allowed variation (%). Relevant for VBR and CVBR. VBR Default: 100. CVBR Default: 15.	Yes	Yes	No
qp_min	Minimum frame header QP (0 - 51)	Yes	Yes	No
qp_max	Maximum frame header QP (0 - 51)	Yes	Yes	No
intra_qp_delta	QP difference between target QP and intra frame QP (-51 - 51)	Yes	Yes	No
fixed_intra_qp	Use fixed QP value for every intra frame in stream, 0 = disabled (0 - 51)	Yes	Yes	No

Table 16. MJPEG Encoder

Name	Description	Supported	Can update at runtime	Mandatory
n_threads	The number of threads to use for the encoding	Yes	No	Yes
quality	Quality of encoding	Yes	Yes	Yes

8.5.5. Encoding Level Limitations

In H265 (HEVC) codec - Each level has resolution and bitrate limitations:

Level	Resolution	Bitrate	High Tier Bitrate
1.0	QCIF (176x144)	128 kbits	
2.0	CIF (352x288)	1.5 Mbits	
2.1	Q720p (640x360)	3.0 Mbits	
3.0	QHD (960x540)	6.0 Mbits	
3.1	720p HD (1280x720)	10.0 Mbits	
4.0	2Kx1080 (2048x1080)	12.0 Mbits	30 Mbits
4.1	2Kx1080 (2048x1080)	20.0 Mbits	50 Mbits
5.0	4096x2160 (4096x2160)	25.0 Mbits	100 Mbits
5.1	4096x2160 (4096x2160)	40.0 Mbits	160 Mbits

For example:

- Level 4.1 only supports resolution up to 2040X1080 means 3840X2160 is not supported.
- For Level 5.0, only support resolution up to 4096x2048. And max bitrate of 25Mbits means 30 fps + 3840X2160 resolution is supported.

8.6. OSD - On Screen Display

The OSD is an element that empowers users to draw dynamic text, images, and timestamps by leveraging the **DSP capabilities provided in Hailo-15**. Offloading the task of image blending to the DSP results in high performance overlays.

It uses `osd : :Blender` class to blend overlays on incoming frames.

A initial configuration can be specified via JSON. A example of such a JSON can be found at the end of this page. After the initial configuration, use the API to dynamically add and remove overlays in runtime.

8.6.1. Supported Overlay Types:

The OSD system supports four main types of overlays:

1. **Text Overlays** - Static text with customizable styling
2. **DateTime Overlays** - Auto-updating timestamp displays
3. **Image Overlays** - Static images from files
4. **Custom Overlays** - User-provided buffers with custom content

Each overlay type has specific parameters and use cases, as detailed in the sections below.

8.6.2. Text Overlays

Text overlays display static text content with extensive customization options.

Code Example:

```
osd::TextOverlay text_overlay("text_id", 0.1, 0.3, "Camera Stream",
    osd::rgba_color_t{255, 255, 255, 255}, // white text
    osd::rgba_color_t{0, 0, 0, 128}, // semi-transparent black background
    24.0, // font size
    1, // line thickness
    1, // z-index
    "/usr/share/fonts/ttf/LiberationMono-Regular.ttf", // font path
    0, // angle
    osd::rotation_alignment_policy_t::CENTER, // rotation policy
    osd::rgba_color_t{-1, -1, -1, -1}, // shadow disabled
    0.0, 0.0, // shadow offsets
    osd::font_weight_t::NORMAL, // font weight
    2, // outline size
    osd::rgba_color_t{0, 0, 0, 255}, // black outline
    osd::HorizontalAlignment::LEFT,
    osd::VerticalAlignment::TOP);

blender->add_overlay(text_overlay);
```

Text Overlay Parameters:

- **label** - Text content to display
- **text_color** - RGBA color for text foreground
- **background_color** - RGBA color for text background
- **font_path** - Path to TTF font file (default: LiberationMono-Regular.ttf)
- **font_size** - Font size in points
- **font_weight** - Either NORMAL or BOLD
- **line_thickness** - Thickness of text strokes
- **outline_size** - Size of text outline (0 to disable)
- **outline_color** - RGBA color for text outline
- **shadow_color** - RGBA color for text shadow (negative values disable shadow)
- **shadow_offset_x/y** - Shadow offset relative to frame size

8.6.3. DateTime Overlays

DateTime overlays automatically update to show the current timestamp.

Code Example:

```
osd::DateTimeOverlay datetime_overlay("timestamp_id", 0.8, 0.05,
    "%d-%m-%Y %H:%M:%S", // datetime format (strftime style)
    osd::rgba_color_t{255, 255, 0, 255}, // yellow text
    osd::rgba_color_t{0, 0, 0, 180}, // dark background
    "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
    18.0, // font size
    1, // line thickness
    2, // z-index
    0, // angle
    osd::rotation_alignment_policy_t::CENTER); // rotation policy
```

(continues on next page)

(continued from previous page)

```
blender->add_overlay(datetime_overlay);
```

DateTime Overlay Parameters:

- **datetime_format** - strftime format string (default: "%d-%m-%Y %H:%M:%S")
- All text overlay parameters (color, font, outline, etc.)

Note: DateTime overlays update automatically once per second.

8.6.4. Image Overlays

Image overlays display static images loaded from files.

Code Example:

```
osd::ImageOverlay image_overlay("logo_id", 0.85, 0.85, // position (bottom-right)
    0.1, 0.1, // size (10% of frame width/height)
    "/path/to/logo.png", // image file path
    3, // z-index (on top)
    0, // no rotation
    osd::rotation_alignment_policy_t::CENTER, // rotation policy
    osd::HorizontalAlignment::RIGHT,
    osd::VerticalAlignment::BOTTOM);

blender->add_overlay(image_overlay);
```

Image Overlay Parameters:

- **image_path** - Path to image file (PNG, JPEG, etc.)
- **width** - Image width relative to frame (0.0-1.0)
- **height** - Image height relative to frame (0.0-1.0)

8.6.5. Custom Overlays

Custom overlays allow users to provide their own buffers with pre-rendered content.

Code Example:

```
osd::CustomOverlay custom_overlay("custom_argb", 0.3, 0.5, 0.1, 0.1, 1,
    osd::custom_overlay_format::ARGB);
blender->add_overlay(custom_overlay);

// Get the buffer and fill it with custom content
auto custom_expected = blender->get_overlay("custom_argb");
auto existing_custom_overlay = std::static_pointer_cast<osd::CustomOverlay>
    (custom_expected.value());
HailoMediaLibraryBufferPtr dspbuffer = existing_custom_overlay->get_buffer();

uint32_t blue = 0xFF0000FF;
// Fill buffer with blue pixels (ARGB format)
for (size_t i = 0; i < dspbuffer->get_plane_size(0); i += 4)
{
    memcpy((int8_t*)(dspbuffer->get_plane_ptr(0) + i), &blue, sizeof(uint32_t));
}
```

(continues on next page)

(continued from previous page)

```
// Enable the overlay to make it visible
blender->set_overlay_enabled("custom_argb", true);
```

Custom Overlay Parameters:

- **format** - Buffer format (ARGB or A420)
- **width/height** - Overlay dimensions relative to frame

Note: Custom overlays are not visible until explicitly enabled via `set_overlay_enabled()`.

8.6.6. Common Overlay Parameters

All overlay types share these common parameters:

- **id** - Unique string identifier for the overlay
- **x** - Horizontal position relative to frame (0.0-1.0, left to right)
- **y** - Vertical position relative to frame (0.0-1.0, top to bottom)
- **z_index** - Layering order (higher values appear on top)
- **angle** - Rotation angle in degrees
- **rotation_policy** - Rotation center (CENTER or TOP_LEFT)
- **horizontal_alignment** - LEFT, CENTER, or RIGHT alignment
- **vertical_alignment** - TOP, CENTER, or BOTTOM alignment

Warning: Setting rotation on DateTime overlays may cause performance issues.

8.6.7. Blender API Reference

The `osd::Blender` class provides the main interface for managing overlays:

Creation:

```
// Create a blender instance
auto blender_result = osd::Blender::create();
if (blender_result.has_value()) {
    auto blender = blender_result.value();
}

// Create with JSON configuration
auto blender_with_config = osd::Blender::create(json_config_string);
```

Adding Overlays:

```
// Synchronous
media_library_return result = blender->add_overlay(overlay);

// Asynchronous
std::shared_future<media_library_return> future_result = blender->add_overlay_
    ->async(overlay);
```

Managing Overlays:

```
// Get overlay information
auto overlay_result = blender->get_overlay("overlay_id");
if (overlay_result.has_value()) {
    auto overlay = overlay_result.value();
    // Cast to specific type if needed
    auto text_overlay = std::static_pointer_cast<osd::TextOverlay>(overlay);
}

// Update overlay
blender->set_overlay(modified_overlay);

// Enable/disable overlay
blender->set_overlay_enabled("overlay_id", true);

// Remove overlay
blender->remove_overlay("overlay_id");
```

Frame Processing:

```
// Set frame dimensions (call once when frame size is known)
blender->set_frame_size(frame_width, frame_height);

// Blend overlays onto frame
media_library_return result = blender->blend(buffer);
```

8.6.8. Alignment and Positioning

Overlay positioning uses a relative coordinate system where (0,0) is the top-left corner and (1,1) is the bottom-right corner.

Horizontal Alignment:

- `HorizontalAlignment::LEFT` - Align to left edge of position
- `HorizontalAlignment::CENTER` - Center horizontally on position
- `HorizontalAlignment::RIGHT` - Align to right edge of position

Vertical Alignment:

- `VerticalAlignment::TOP` - Align to top edge of position
- `VerticalAlignment::CENTER` - Center vertically on position
- `VerticalAlignment::BOTTOM` - Align to bottom edge of position

Example:

```
// Position overlay at bottom-right corner of frame
osd::TextOverlay corner_text("corner", 1.0, 1.0, "Bottom Right",
    text_color, bg_color, font_size, thickness, z_index, font_path, 0,
    osd::rotation_alignment_policy_t::CENTER, shadow_color, 0, 0, osd::font_weight_t:
↪:NORMAL, 0, outline_color,
    osd::HorizontalAlignment::RIGHT, // Align right edge to x position
    osd::VerticalAlignment::BOTTOM); // Align bottom edge to y position
```

8.6.9. Color Formats

Colors use RGBA format with values 0-255:

```
// Fully opaque colors
osd::rgba_color_t red = {255, 0, 0, 255};
osd::rgba_color_t green = {0, 255, 0, 255};
osd::rgba_color_t blue = {0, 0, 255, 255};

// Semi-transparent colors
osd::rgba_color_t semi_black = {0, 0, 0, 128}; // 50% transparency

// Disabled features (negative alpha)
osd::rgba_color_t disabled = {-1, -1, -1, -1}; // Disables shadow/outline
```

8.6.10. Performance Considerations

- **DateTime Rotation:** Avoid rotating DateTime overlays for better performance
- **Z-Index:** Use minimal z-index values; higher values require more blending operations
- **Custom Overlays:** Pre-render content when possible rather than updating every frame
- **Overlay Count:** Limit the number of simultaneous overlays for optimal performance
- **Font Caching:** Reuse font paths across overlays to benefit from internal caching

More examples can be found in:

```
hailo-media-library/api/examples/frontend_example.cpp
```

8.6.11. Error Handling

The OSD API uses the `tl::expected` pattern for error handling:

```
auto blender_result = osd::Blender::create();
if (!blender_result.has_value()) {
    media_library_return error = blender_result.error();
    // Handle error
    return;
}
auto blender = blender_result.value();

// For operations returning media_library_return
media_library_return result = blender->add_overlay(overlay);
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Common Error Scenarios:

- **Invalid overlay ID:** Using duplicate IDs or invalid characters
- **Invalid coordinates:** Position/size values outside [0,1] range
- **Missing files:** Font or image files not found
- **Invalid formats:** Unsupported image formats or invalid color values
- **Memory allocation:** Insufficient memory for overlay buffers

OSD (On Screen Display) CPP API module.

namespace osd

Typedefs

```
using OverlayRepositoryPtr = std::shared_ptr<OverlayRepository>
```

```
using Blender = class Configurer
    Overlay manager.
```

Support the addition, removal and modification of overlays Overlay traits are defined using the structs above.

```
using BlenderPtr = std::shared_ptr<Blender>
```

```
typedef std::shared_ptr<Configurer> ConfigurerPtr
```

```
typedef std::shared_ptr<OsdBlender> OsdBlenderPtr
```

```
class Configurer : public std::enable_shared_from_this<Configurer>
```

Public Functions

```
void set_stream_id(const std::string &stream_id)
```

```
void set_config_manager_interactor(ConfigManagerInteractor ...)
```

```
media_library_return add_overlay(const ImageOverlay &overlay)
    Add a new overlay.
```

The new overlay will be blended upon each subsequent call to :blend

Parameters

- **id** - **[in]** Unique string identifier for the overlay. This id is used for all future operations on the overlay.
- **overlay** - **[in]** Overlay to add

Returns :MEDIA_LIBRARY_SUCCESS if successful, otherwise a :media_library_return error

```
media_library_return add_overlay(const TextOverlay &overlay)
```

```
media_library_return add_overlay(const DateTimeOverlay &overlay)
```

```
media_library_return add_overlay(const CustomOverlay &overlay)
```

```
media_library_return set_overlay_enabled(const std::string &id, bool enabled)
```

```
std::shared_future<media_library_return> add_overlay_async(const ImageOverlay &overlay)
```

```
std::shared_future<media_library_return> add_overlay_async(const TextOverlay &overlay)
```

```
std::shared_future<media_library_return> add_overlay_async ( const DateTimeOverlay &overlay )
```

```
std::shared_future<media_library_return> add_overlay_async ( const CustomOverlay &overlay )
```

```
tl::expected<std::shared_ptr<Overlay>, media_library_return> get_overlay ( const std::string &id )
```

Retrieve info of an existing overlay.

Note: The return overlay should be downcast to the correct overlay type using :std::static_pointer_cast The caller of the function is responsible to know which overlay type is denoted by each id Casting to an incorrect type might result in undefined behavior

Parameters id - [in] String identifier of the overlay to get.

Returns :shared_ptr holding the overlay info if successfull otherwise a :media_library_return error

```
media_library_return set_overlay ( const ImageOverlay &overlay )
```

Modify an existing overlay.

Note: All the traits in the :overlay param must be specified - that this fields which agree to remain unchanged must also be filled Use :get_overlay function to the current overlay state and perform changes

Parameters

- id - [in] String identifier of the overlay to modify.
- overlay - [in] Overlay traits to modify

Returns :MEDIA_LIBRARY_SUCCESS if successful, otherwise a :media_library_return error

```
media_library_return set_overlay ( const TextOverlay &overlay )
```

```
media_library_return set_overlay ( const DateTimeOverlay &overlay )
```

```
media_library_return set_overlay ( const CustomOverlay &overlay )
```

```
std::shared_future<media_library_return> set_overlay_async ( const ImageOverlay &overlay )
```

```
std::shared_future<media_library_return> set_overlay_async ( const TextOverlay &overlay )
```

```
std::shared_future<media_library_return> set_overlay_async ( const DateTimeOverlay &overlay )
```

```
std::shared_future<media_library_return> set_overlay_async ( const CustomOverlay &overlay )
```

```
media_library_return configure ( const std::string &config )
```

```
media_library_return remove_overlay ( const std::string &id )
```

Remove an exiting overlay.

Subsequent calls to :blend will no longer blend the overlay

Parameters id - [in] String identifier of the overlay to remove

Returns :MEDIA_LIBRARY_SUCCESS if successful, otherwise a :media_library_return error

```
std::shared_future<media_library_return> remove_overlay_async ( const std::string &id )
```

Public Static Functions

```
static tl::expected<std::shared_ptr<Configurer>, media_library_return> create ( const std::string ... )
```

Private Functions

```
media_library_return add_overlay_to_profile (const ImageOverlay &overlay)
```

```
media_library_return add_overlay_to_profile (const TextOverlay &overlay)
```

```
media_library_return add_overlay_to_profile (const DateTimeOverlay &overlay)
```

```
media_library_return add_overlay_to_profile (const CustomOverlay &overlay)
```

```
media_library_return remove_overlay_from_profile (const std::string &id)
```

```
media_library_return set_overlay_in_profile (const ImageOverlay &overlay)
```

```
media_library_return set_overlay_in_profile (const TextOverlay &overlay)
```

```
media_library_return set_overlay_in_profile (const DateTimeOverlay &overlay)
```

```
media_library_return set_overlay_in_profile (const CustomOverlay &overlay)
```

Private Members

```
std::string m_stream_id
```

```
ConfigManagerInteractor *m_config_manager_interactor
```

```
OverlayRepositoryPtr m_osd_repository
```

```
class OsdBlender
```

Public Functions

```
OsdBlender ( )
```

```
~OsdBlender ( )
```

```
void set_stream_id (const std::string &stream_id)
```

```
media_library_return set_frame_size (int frame_width, int frame_height)
```

```
media_library_return blend (HailoMediaLibraryBufferPtr &input_buffer)
```

Public Static Functions

```
static tl::expected<std::shared_ptr<OsdBlender>, media_library_return> create (const std::string ...)
```

Private Members

```
std::string m_stream_id
int m_frame_width
int m_frame_height
bool m_frame_size_set
OverlayRepositoryPtr m_osd_repository
```

JSON Examples

For more explanations on the JSON parameters, see the [Encoder + OSD Configurations](#).

Hailo Encoder:

```
{
  "version": "1.1.0",
  "metadata": {
    "architecture": "hailo15h",
    "content_hash":
    ↪ "59360a443a5d1fac85843ef9ee69ffdfcde2e54e9711389b439a5f6b70889109",
    "description": "",
    "generation_timestamp": "2025-11-24T12:11:03.809222+00:00"
  },
  "encoding": {
    "hailo_encoder": {
      "coding_control": {
        "deblocking_filter": {
          "beta_offset": 5,
          "deblock_override": false,
          "tc_offset": -2,
          "type": "DEBLOCKING_FILTER_ENABLED"
        },
        "intra_area": {
          "bottom": 0,
          "enable": false,
          "left": 0,
          "right": 0,
          "top": 0
        },
        "ipcm_area1": {
          "bottom": 0,
          "enable": false,
          "left": 0,
          "right": 0,
          "top": 0
        },
        "ipcm_area2": {
          "bottom": 0,
          "enable": false,
          "left": 0,
          "right": 0,
          "top": 0
        }
      },
      "sei_messages": {
```

(continues on next page)

(continued from previous page)

```

        "encoder_timing_sei": true,
        "user_metadata_sei": true
    },
    },
    "config": {
        "output_stream": {
            "codec": "CODEC_TYPE_H264"
        }
    },
    "gop_config": {
        "b_frame_qp_delta": 0,
        "gop_size": 1
    },
    "monitors_control": {
        "bitrate_monitor": {
            "enable": true,
            "output_result_to_file": false,
            "period": 3,
            "result_output_path": "bitrate.txt"
        },
        "cycle_monitor": {
            "deviation_threshold": 5,
            "enable": true,
            "output_result_to_file": false,
            "result_output_path": "cycle.txt",
            "start_delay": 0
        }
    },
    "rate_control": {
        "bitrate": {
            "target_bitrate": 10000000
        },
        "gop_anomaly_bitrate_adjuster": {
            "enable": false
        },
        "intra_pic_rate": 60,
        "padding": false,
        "picture_rc": true,
        "picture_skip": false,
        "qp_smooth_settings": {
            "alpha": 0,
            "q_step_divisor": 2,
            "qp_delta": 128,
            "qp_delta_limit": 1536,
            "qp_delta_limit_step": 384,
            "qp_delta_step": 128
        },
        "quantization": {
            "qp_hdr": -1
        },
        "rc_mode": "CVBR",
        "zoom_bitrate_adjuster": {
            "mode": "DISABLED"
        }
    },
    "smart_encoder": {
        "enabled": false,
        "background_qp_delta": 10,
        "rois": []
    }
},

```

(continues on next page)

(continued from previous page)

```

    "input_stream": {
        "format": "NV12",
        "framerate": 30,
        "height": 1080,
        "width": 1920
    }
},
"osd": {
    "dateTime": [
        {
            "angle": 0,
            "datetime_format": "%d-%m-%Y %H:%M:%S",
            "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
            "font_size": 100,
            "id": "example_datetime",
            "rotation_policy": "CENTER",
            "text_color": [255, 0, 0],
            "x": 0.1,
            "y": 0.7,
            "z-index": 3
        }
    ],
    "image": [
        {
            "angle": 0,
            "height": 0.13,
            "id": "example_image",
            "image_path": "/home/root/apps/webserver/resources/configs/osd_hailo_
↪static_image.png",
            "rotation_policy": "CENTER",
            "width": 0.2,
            "x": 0.76,
            "y": 0.3,
            "z-index": 1
        }
    ],
    "text": [
        {
            "angle": 0,
            "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
            "font_size": 100,
            "id": "example_text1",
            "label": "HailoAI",
            "rotation_policy": "CENTER",
            "text_color": [255, 255, 255],
            "x": 0.7,
            "y": 0.05,
            "z-index": 2
        },
        {
            "angle": 0,
            "background_color": [0, 0, 0],
            "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
            "font_size": 100,
            "id": "example_text2",
            "label": "camera name",
            "rotation_policy": "CENTER",
            "text_color": [102, 0, 51],
            "x": 0.05,
            "y": 0.1,
            "z-index": 1
        }
    ]
}

```

(continues on next page)

(continued from previous page)

```

    }
  ]
},
"privacy_mask": {
  "color_value": [0, 0, 0],
  "mask_type": "COLOR",
  "pixelization_size": 60
}
}

```

JPEG Encoder:

```

{
  "privacy_mask": {
    "mask_type": "COLOR",
    "pixelization_size": 60,
    "color_value": [0, 0, 0]
  },
  "osd": {
    "image": [
      {
        "id": "example_image",
        "image_path": "/home/root/apps/webserver/resources/configs/osd_hailo_
↪static_image.png",
        "width": 0.2,
        "height": 0.13,
        "x": 0.76,
        "y": 0.3,
        "z-index": 1,
        "angle": 0,
        "rotation_policy": "CENTER"
      }
    ],
    "dateTime": [
      {
        "id": "example_datetime",
        "font_size": 100,
        "datetime_format": "%d-%m-%Y %H:%M:%S",
        "text_color": [255, 0, 0],
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "x": 0.1,
        "y": 0.7,
        "z-index": 3,
        "angle": 0,
        "rotation_policy": "CENTER"
      }
    ],
    "text": [
      {
        "id": "example_text1",
        "label": "HailoAI",
        "font_size": 100,
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "text_color": [255, 255, 255],
        "x": 0.7,
        "y": 0.05,
        "z-index": 2,
        "angle": 0,
        "rotation_policy": "CENTER"
      }
    ],
  }
}

```

(continues on next page)

(continued from previous page)

```

        "id": "example_text2",
        "label": "camera name",
        "font_size": 100,
        "font_path": "/usr/share/fonts/ttf/LiberationMono-Regular.ttf",
        "text_color": [102, 0, 51],
        "background_color": [0, 0, 0],
        "x": 0.05,
        "y": 0.1,
        "z-index": 1,
        "angle": 0,
        "rotation_policy": "CENTER"
    }
]
},
"encoding": {
    "input_stream": {
        "width": 1280,
        "height": 720,
        "framerate": 30,
        "format": "NV12"
    },
    "jpeg_encoder": {
        "n_threads": 1,
        "quality": 85
    }
}
}
}

```

8.7. Media Library Types

MediaLibrary type definitions.

Typedefs

```

using output_stream_id_t = std::string
using PolygonPtr = std::shared_ptr<polygon>

```

Enums

```

enum media_library_return
    Values:

```

```

    enumerator MEDIA_LIBRARY_SUCCESS
    enumerator MEDIA_LIBRARY_ERROR
    enumerator MEDIA_LIBRARY_INVALID_ARGUMENT
    enumerator MEDIA_LIBRARY_CONFIGURATION_ERROR
    enumerator MEDIA_LIBRARY_BUFFER_ALLOCATION_ERROR
    enumerator MEDIA_LIBRARY_DSP_OPERATION_ERROR
    enumerator MEDIA_LIBRARY_UNINITIALIZED
    enumerator MEDIA_LIBRARY_OUT_OF_RESOURCES

```

```

enumerator MEDIA_LIBRARY_ENCODER_ENCODE_ERROR
enumerator MEDIA_LIBRARY_ENCODER_COULD_NOT_GET_PHYSICAL_ADDRESS
enumerator MEDIA_LIBRARY_BUFFER_NOT_FOUND
enumerator MEDIA_LIBRARY_FREETYPE_ERROR
enumerator MEDIA_LIBRARY_PROFILE_IS_RESTRICTED
enumerator MEDIA_LIBRARY_PROFILE_VALIDATION_FAILED
enumerator MEDIA_LIBRARY_MAX
    Max enum value to maintain ABI Integrity

```

```

enum pre_proc_operation
    Values:

```

```

enumerator PRE_PROC_OPERATION_DEWARP
enumerator PRE_PROC_OPERATION_DIS
enumerator PRE_PROC_OPERATION_DIGITAL_ZOOM
enumerator PRE_PROC_OPERATION_ROTATION
enumerator PRE_PROC_OPERATION_FLIP
enumerator PRE_PROC_OPERATION_GMV
enumerator PRE_PROC_OPERATION_MAX
    Max enum value to maintain ABI Integrity

```

```

enum flip_direction_t
    Values:

```

```

enumerator FLIP_DIRECTION_NONE
enumerator FLIP_DIRECTION_HORIZONTAL
enumerator FLIP_DIRECTION_VERTICAL
enumerator FLIP_DIRECTION_BOTH
enumerator FLIP_DIRECTION_MAX
    Max enum value to maintain ABI Integrity

```

```

enum digital_zoom_mode_t
    Values:

```

```

enumerator DIGITAL_ZOOM_MODE_ROI
enumerator DIGITAL_ZOOM_MODE_MAGNIFICATION
enumerator DIGITAL_ZOOM_MODE_MAX
    Max enum value to maintain ABI Integrity

```

```

enum rotation_angle_t
    Values:

```

```

enumerator ROTATION_ANGLE_0
enumerator ROTATION_ANGLE_90
enumerator ROTATION_ANGLE_180
enumerator ROTATION_ANGLE_270

```

enumerator ROTATION_ANGLE_MAX
Max enum value to maintain ABI Integrity

enum frontend_src_element_t
Values:

enumerator UNKNOWN
enumerator V4L2SRC
enumerator APPSRC
enumerator MAX
Max enum value to maintain ABI Integrity

enum denoise_method_t
Values:

enumerator DENOISE_METHOD_NONE
enumerator DENOISE_METHOD_VD1
enumerator DENOISE_METHOD_VD2
enumerator DENOISE_METHOD_VD3
enumerator DENOISE_METHOD_MAX
Max enum value to maintain ABI Integrity

enum EncoderType
Values:

enumerator None
enumerator Hailo
enumerator Jpeg

enum PrivacyMaskType
Values:

enumerator COLOR
enumerator PIXELIZATION

enum motion_detection_sensitivity_levels_t
Values:

enumerator LOWEST
enumerator LOW
enumerator MEDIUM
enumerator HIGH
enumerator HIGHEST
enumerator MOTION_DETECTION_SENSITIVITY_LEVELS_MAX
Max enum value to maintain ABI Integrity

enum sensor_id_t
Values:

enumerator SENSOR_0

enumerator SENSOR_1

enumerator SENSOR_INDEX_MAX

Max enum value to maintain ABI Integrity

enum PipelineState

Values:

enumerator VOID_PENDING

enumerator NULL_STATE

enumerator READY

enumerator PAUSED

enumerator PLAYING

enum hdr_dol_t

Values:

enumerator HDR_DOL_2

enumerator HDR_DOL_3

enum AnalyticsType

Values:

enumerator DETECTION

enumerator INSTANCE_SEGMENTATION

enumerator SEMANTIC_SEGMENTATION

enumerator ANALYTICS_TYPE_MAX

Max enum value to maintain ABI Integrity

enum ScalingMode

Values:

enumerator STRETCH

enumerator LETTERBOX_MIDDLE

enumerator LETTERBOX_UP_LEFT

enumerator SCALING_MODE_MAX

Max enum value to maintain ABI Integrity

enum AnalyticsQueryType

Values:

enumerator Closest

enumerator Exact

enumerator WithinDelta

enum rotation_alignment_policy_t

Values:

enumerator CENTER

enumerator TOP_LEFT

```
enum font_weight_t
```

Values:

enumerator NORMAL

enumerator BOLD

```
enum custom_overlay_format
```

Values:

enumerator A420

A420 Format - planar 4:4:2:0 AYUV. Each component is 8-bit.

For A420 format, the dimensions of the image, both width and height, need to be even numbers

Four planes in the following order: Y plane, U plane, V plane, Alpha plane.

enumerator ARGB

ARGB - RGB with alpha channel first (packed) format. One plane, each color component is 8-bit.

```
+--+--+--+--+--+ +--+--+--+--+--+
|A0|R0|G0|B0| |A1|R1|G1|B1| ...
+--+--+--+--+--+ +--+--+--+--+--+
```

enumerator COUNT

enumerator ENUM

Max enum value to maintain ABI Integrity

Functions

```
mat_dims calculate_text_size(const std::string &label, const std::string &font_path, int font_size, int ...)
```

```
struct mat_dims
```

Public Members

```
int width
```

```
int height
```

```
int baseline
```

```
struct roi_t
```

Public Functions

```
bool operator==(const roi_t &other) const
```

Public Members

```
uint32_t x
uint32_t y
uint32_t width
uint32_t height
struct vsm_config_t
```

Public Members

```
uint32_t vsm_h_size
uint32_t vsm_h_offset
uint32_t vsm_v_size
uint32_t vsm_v_offset
struct isp_t
```

Public Members

```
std::string isp_config_files_path
struct hailort_t
```

Public Members

```
std::string device_id
struct hdr_config_t
```

Public Functions

```
inline void override(const hdr_config_t &other)
```

Public Members

```
bool enabled
float ls_ratio
float vs_ratio
hdr_dol_t dol
```

Public Static Attributes

```
static bool is_persistent = false
struct network_config_t
```


Public Members

```
std::string network_path
std::string y_channel
std::string uv_channel
std::string output_y_channel
std::string output_uv_channel
struct feedback_network_config_t
```

Public Functions

```
inline bool operator==(const feedback\_network\_config\_t &other) const
```

Public Members

```
std::string network_path
std::string y_channel
std::string uv_channel
std::string feedback_y_channel
std::string feedback_uv_channel
std::string output_y_channel
std::string output_uv_channel
struct bayer_network_config_t
```

Public Functions

```
inline bool operator==(const bayer\_network\_config\_t &other) const
```

Public Members

```
std::string network_path
std::string bayer_channel
std::string feedback_bayer_channel
std::string dgain_channel
std::string bls_channel
std::string output_bayer_channel
std::string input_fusion_feedback
std::string input_gamma_feedback
std::string output_fusion_feedback
std::string output_gamma_feedback
std::string skip0_fusion_channel
std::string skip1_fusion_channel
struct dewarp_config_t
```

Public Functions

```
inline bool operator==(const dewarp_config_t &other) const
```

```
inline bool operator!=(const dewarp_config_t &other) const
```

```
inline void override(const dewarp_config_t &other)
```

Public Members

```
bool enabled
```

```
std::string sensor_calib_path
```

```
dsp_interpolation_type_t interpolation_type
```

```
camera_type_t camera_type
```

Public Static Attributes

```
static bool is_persistent = true
```

```
struct digital_zoom_config_t
```

Public Functions

```
inline void override(const digital_zoom_config_t &other)
```

Public Members

```
bool enabled
```

```
digital_zoom_mode_t mode
```

```
float magnification
```

```
roi_t roi
```

Public Static Attributes

```
static bool is_persistent = true
```

```
struct optical_zoom_config_t
```

Public Functions

```
inline void override(const optical_zoom_config_t &other)
```

```
bool operator==(const optical_zoom_config_t &other) const
```

Public Members

```
bool enabled
```

```
float magnification
```

```
float max_dewarping_magnification
```

Public Static Attributes

```
static bool is_persistent = true
struct flip_config_t
```

Public Functions

```
inline void override(const flip_config_t &other)

inline bool operator==(const flip_config_t &other) const

inline bool operator!=(const flip_config_t &other) const

inline flip_direction_t effective_value() const
```

Public Members

```
bool enabled
flip_direction_t direction
```

Public Static Attributes

```
static bool is_persistent = true
struct rotation_config_t
```

Public Functions

```
inline void override(const rotation_config_t &other)

inline bool operator==(const rotation_config_t &other) const

inline bool operator!=(const rotation_config_t &other) const

inline rotation_angle_t effective_value() const
```

Public Members

```
bool enabled
rotation_angle_t angle
```

Public Static Attributes

```
static bool is_persistent = true
struct output_resolution_t
```

Public Functions

```
inline void override(const output_resolution_t &other)
```

```
inline bool operator==(const output_resolution_t &other) const
```

```
inline bool operator!=(const output_resolution_t &other) const
```

```
inline bool operator==(const hailo_buffer_data_t &hailo_buffer_data) const
```

```
inline bool operator!=(const hailo_buffer_data_t &hailo_buffer_data) const
```

```
inline bool dimensions_equal(const output_resolution_t &other, bool rotated = false) const
```

```
inline bool dimensions_and_aspect_ratio_equal(const output_resolution_t &other, bool ...)
```

Public Members

```
uint32_t framerate
```

```
uint32_t pool_max_buffers
```

```
dsp_utils::crop_resize_dims_t dimensions
```

```
std::string stream_id
```

```
dsp_scaling_mode_t scaling_mode
```

Public Static Attributes

```
static override_policy persistent_fields
```

```
struct override_policy
```

Public Members

```
bool framerate
```

```
bool pool_max_buffers
```

```
bool dimensions
```

```
bool scaling_mode
```

```
struct motion_detection_config_t
```

Public Functions

```
inline void override(const motion_detection_config_t &other)
```

Public Members

```
bool enabled
```

```
output_resolution_t resolution
```

```
roi_t roi
```

```
motion_detection_sensitivity_levels_t sensitivity_level
```

```
float threshold
```

```
uint32_t buffer_pool_size
```

Public Static Attributes

```
static bool is_persistent = true
struct config_application_input_streams_t
```

Public Functions

```
inline void override(const config\_application\_input\_streams\_t &other)
```

Public Members

```
dsp_interpolation_type_t interpolation_type
HailoFormat format
std::vector<output\_resolution\_t> resolutions
```

Public Static Attributes

```
static bool is_persistent = true
struct application_input_streams_config_t
```

Public Functions

```
inline application_input_streams_config_t(const config\_application\_input\_streams\_t ...)
inline application_input_streams_config_t(const application\_input\_streams\_config\_t &other)
inline void update(const application\_input\_streams\_config\_t &other)
inline application\_input\_streams\_config\_t &operator=(const application\_input\_streams\_config\_t &other)
inline std::optional<output\_resolution\_t> get_resolution_by_stream_id(const ...)
inline bool has_stream_id(const output\_stream\_id\_t &stream_id)
application_input_streams_config_t() = default
```

Public Members

```
dsp_interpolation_type_t interpolation_type
HailoFormat format
bool grayscale
std::vector<output\_resolution\_t> resolutions
struct input_video_config_t
```

Public Functions

```
inline bool operator==(const input_video_config_t &other) const
```

```
inline bool operator!=(const input_video_config_t &other) const
```

Public Members

```
frontend_src_element_t source_type
```

```
HailoFormat format
```

```
output_resolution_t resolution
```

```
std::string source
```

```
size_t sensor_index
```

```
struct multi_resize_config_t
```

Public Functions

```
inline multi_resize_config_t()
```

```
inline media_library_return update(multi_resize_config_t &mresize_config)
```

```
inline media_library_return set_output_dimensions_rotation(const rotation_config_t ...)
```

```
inline bool is_portrait(rotation_angle_t rotation_angle)
```

```
inline bool is_portrait(dsp_utils::crop_resize_dims_t &dimensions)
```

```
inline bool is_portrait(size_t width, size_t height)
```

Public Members

```
output_resolution_t input_video_config
```

```
application_input_streams_config_t application_input_streams_config
```

```
digital_zoom_config_t digital_zoom_config
```

```
rotation_config_t rotation_config
```

```
motion_detection_config_t motion_detection_config
```

```
struct eis_config_t
```

Public Functions

```
inline void override(const eis_config_t &other)
```

```
bool operator==(const eis_config_t &other) const
```

Public Members

```
bool enabled
bool stabilize
std::string eis_config_path
uint32_t window_size
double rotational_smoothing_coefficient
double iir_hpf_coefficient
float camera_fov_factor
uint64_t line_readout_time
uint8_t num_exposures
float hdr_exposure_ratio
float min_angle_deg
float max_angle_deg
uint32_t shakes_type_buff_size
bool force_clamp_correction_angles
```

Public Static Attributes

```
static bool is_persistent = true
struct gyro_config_t
```

Public Functions

```
inline void override(const gyro_config_t &other)

bool operator==(const gyro_config_t &other) const
```

Public Members

```
bool enabled
std::string sensor_name
std::string sensor_frequency
double gyro_scale
```

Public Static Attributes

```
static bool is_persistent = true
struct ldc_config_t
```

Public Functions

```
bool operator==(const ldc_config_t &other) const

inline ldc_config_t()

media_library_return update(ldc_config_t &ldc_configs)

void update_flip_rotate(ldc_config_t &ldc_configs)

inline bool check_ops_enabled(bool dewarp_actions_only = false) const

inline bool check_ops_enabled_changed(ldc_config_t &other) const
```

Public Members

```
rotation_config_t rotation_config
flip_config_t flip_config
dewarp_config_t dewarp_config
dis_config_t dis_config
optical_zoom_config_t optical_zoom_config
input_video_config_t input_video_config
output_resolution_t application_input_streams_config
eis_config_t eis_config
gyro_config_t gyro_config
```

Private Functions

```
inline void rotate_output_dimensions()
```

```
struct denoise_config_t
```

Public Functions

```
inline denoise_config_t()

inline media_library_return update(denoise_config_t &denoise_configs)

inline void override(const denoise_config_t &other)
```

Public Members

```
bool enabled
bool bayer
std::string sensor
denoise_method_t denoising_quality
uint32_t loopback_count
uint32_t pool_max_buffers
```



```
feedback_network_config_t network_config
```

```
bayer_network_config_t bayer_network_config
```

Public Static Attributes

```
static bool is_persistent = false
```

```
struct frontend_element_config_t
```

Public Functions

```
inline frontend_element_config_t()
```

```
inline frontend_element_config_t(ldc_config_t &ldc, denoise_config_t &denoise, ...)
```

```
inline media_library_return update(frontend_element_config_t &frontend_element_configs)
```

Public Members

```
ldc_config_t ldc_config
```

```
denoise_config_t denoise_config
```

```
multi_resize_config_t multi_resize_config
```

```
struct codec_config_t
```

Public Members

```
std::string stream_id
```

```
std::string config_path
```

```
struct label_t
```

Public Members

```
std::string label
```

```
uint32_t id
```

```
struct hailo_semantic_segmentation_mask_t
```

Public Members

```
uint16_t class_id  
Class ID of the semantic mask
```

```
uint32_t width  
Mask width in pixels
```

```
uint32_t height  
Mask height in pixels
```

```
float transparency  
Transparency value for overlay (0.0-1.0)
```

```
size_t mask_size  
Mask size in bytes
```

```
uint8_t *mask
```

Float Confidence Mask: The mask contains per-pixel confidence values as floats (0.0-1.0). Each pixel represents the confidence that the pixel belongs to the class_id.

The size of the mask is width * height * sizeof(float).

```
float detection_x
```

Detection coordinates in the full frame (normalized 0.0-1.0)

```
float detection_y
```

```
float detection_width
```

```
float detection_height
```

```
struct detection_analytics_config_t
```

Public Members

```
std::string analytics_data_id
```

```
ScalingMode scaling_mode
```

```
uint32_t width
```

```
uint32_t height
```

```
uint32_t original_width_ratio
```

```
uint32_t original_height_ratio
```

```
std::vector<label_t> labels
```

```
size_t max_entries
```

```
struct instance_segmentation_analytics_config_t
```

Public Members

```
std::string analytics_data_id
```

```
ScalingMode scaling_mode
```

```
uint32_t width
```

```
uint32_t height
```

```
uint32_t original_width_ratio
```

```
uint32_t original_height_ratio
```

```
std::vector<label_t> labels
```

```
size_t max_entries
```

```
struct semantic_segmentation_analytics_config_t
```

Public Members

```
std::string analytics_data_id
ScalingMode scaling_mode
uint32_t width
uint32_t height
uint32_t original_width_ratio
uint32_t original_height_ratio
std::vector<label_t> labels
size_t max_entries
size_t mask_size
struct application_analytics_config_t
```

Public Functions

```
inline void override(const application_analytics_config_t &other)
```

Public Members

```
std::unordered_map<std::string, detection_analytics_config_t> detection_analytics_config
std::unordered_map<std::string, instance_segmentation_analytics_config_t>
instance_segmentation_analytics_config
std::unordered_map<std::string, semantic_segmentation_analytics_config_t>
semantic_segmentation_analytics_config
```

Public Static Attributes

```
static bool is_persistent = true
struct frontend_config_t
```

Public Functions

```
inline frontend_config_t()

inline media_library_return update(frontend_config_t &frontend_configs)
```

Public Members

```
input_video_config_t input_config
ldc_config_t ldc_config
denoise_config_t denoise_config
multi_resize_config_t multi_resize_config
hdr_config_t hdr_config
hailort_t hailort_config
isp_t isp_config
application_analytics_config_t application_analytics_config
struct encoder_bitrate_monitor
```

Public Members

```
bool enabled
uint32_t fps
uint32_t period
uint32_t sum_period
uint32_t ma_bitrate
struct encoder_cycle_monitor
```

Public Members

```
bool enabled
uint32_t deviation_threshold
uint32_t monitor_frames
uint32_t start_delay
uint32_t frame_count
uint32_t sum
std::time_t start_time
struct encoder_monitors
```

Public Members

```
encoder_bitrate_monitor bitrate_monitor
encoder_cycle_monitor cycle_monitor
struct OverrideParameters
```

Public Members

```
std::string override_file
bool discard_on_profile_change
struct profile_t
```

Public Functions

```
media_library_return flatten_n_validate_config()
```

Public Members

```
std::string name
std::string config_file
nlohmann::json flattened_config_file_content
struct medialib_config_t
```

Public Functions

```
inline tl::expected<profile_t, media_library_return> get_profile(const std::string &name) const
```

Public Members

```
std::string version
std::string default_profile
std::vector<profile_t> profiles
std::string backup_folder_path
struct rgb_color_t
```

Public Functions

```
bool operator==(const rgb_color_t &other) const
```

Public Members

```
uint r
uint g
uint b
struct vertex
```

Public Functions

```
inline vertex()

inline vertex(int x, int y)

bool operator==(const vertex &other) const
```

Public Members

```
int x
int y
struct polygon
```

Public Functions

```
bool operator==(const polygon &other) const
```

Public Members

```
std::string id
std::vector<vertex> vertices
struct static_privacy_mask_config_t
```

Public Functions

```
bool operator==(const static_privacy_mask_config_t &other) const
```

Public Members

```
bool enabled
```

```
std::vector<polygon> masks
```

```
struct analytics_entry_t
```

Public Functions

```
inline bool operator==(const analytics_entry_t &other) const
```

Public Members

```
std::string analytics_data_id
```

```
std::vector<std::string> masked_labels
```

```
struct dynamic_privacy_mask_config_t
```

Public Functions

```
inline bool operator==(const dynamic_privacy_mask_config_t &other) const
```

Public Members

```
bool enabled
```

```
std::vector<analytics_entry_t> analytics
```

```
size_t dilation_size
```

```
uint32_t timeout_ms
```

```
uint32_t delta_ms
```

```
AnalyticsQueryType query_type
```

Public Static Attributes

```
static bool is_persistent = false
```

```
struct privacy_mask_config_t
```

Public Functions

```
inline void override(const privacy_mask_config_t &other)
```

```
bool operator==(const privacy_mask_config_t &other) const
```

Public Members

```
PrivacyMaskType mask_type
uint32_t pixelization_size
rgb_color_t color_value
std::optional<dynamic_privacy_mask_config_t> dynamic_privacy_mask_config
std::optional<static_privacy_mask_config_t> static_privacy_mask_config
```

Public Static Attributes

```
static bool is_persistent = true
struct calibration_header_t
```

Public Members

```
std::string creation_date
std::string creator
std::string sensor_name
std::string sample_name
std::string generator_version
std::vector<uint32_t> resolution
struct config_input_video_t
```

Public Members

```
struct config_input_video_t::resolution_t resolution
std::string source
frontend_src_element_t source_type
sensor_id_t sensor_id
struct resolution_t
```

Public Members

```
uint32_t width
uint32_t height
uint32_t framerate
struct config_sensor_configuration_t
Subclassed by isp_format_config_sensor_configuration_t
```

Public Functions

```
inline bool operator==(const config\_sensor\_configuration\_t &other) const
```

```
inline bool operator!=(const config\_sensor\_configuration\_t &other) const
```

Public Members

```
std::string name
```

```
std::string drv
```

```
uint32_t mode
```

```
uint32_t pixel_mode
```

```
uint32_t sensor_only
```

```
int32_t af_i2c_bus
```

```
std::string af_i2c_addr
```

```
int32_t custom_readout_timing_short
```

```
struct isp_format_config_sensor_configuration_t : public config\_sensor\_configuration\_t
```

Public Functions

```
inline isp_format_config_sensor_configuration_t (bool hdr_enable, std::string ...)
```

```
inline bool operator==(const isp\_format\_config\_sensor\_configuration\_t &other) const
```

```
inline bool operator!=(const isp\_format\_config\_sensor\_configuration\_t &other) const
```

Public Members

```
bool hdr_enable
```

```
std::string sensor_calibration_file
```

```
uint32_t sensor_i2c_bus
```

```
std::string sensor_i2c_addr
```

```
struct config_framerate_t
```

Public Members

```
std::string name
```

```
double fps
```

```
struct config_resolution_entry_t
```


Public Members

```
std::string name
std::string id
double width
double height
std::vector<config_framerate_t> framerate
```

```
struct config_calibration_header_t
```

Public Members

```
std::string creation_date
std::string creator
std::string sensor_name
std::string sample_name
std::string generator_version
std::vector<config_resolution_entry_t> resolution
```

```
struct config_sensor_config_t
```

Public Functions

```
inline void override(const config_sensor_config_t &other)
```

Public Members

```
std::string version
config_input_video_t input_video
config_sensor_configuration_t sensor_configuration
std::string sensor_calibration_file_path
```

Public Static Attributes

```
static bool is_persistent = false
```

```
struct config_application_settings_t
```

Public Functions

```
inline void override(const config_application_settings_t &other)
```

Public Members

```
std::string version
config_application_input_streams_t application_input_streams
optical_zoom_config_t optical_zoom
digital_zoom_config_t digital_zoom
motion_detection_config_t motion_detection
rotation_config_t rotation
flip_config_t flip
```

hailort_t hailort

application_analytics_config_t application_analytics

Public Static Attributes

static bool is_persistent = true

struct config_dis_angular_t

Public Members

bool enabled

nlohmann::json vsm

struct config_dis_debug_t

Public Members

bool generate_resize_grid

bool fix_stabilization

double fix_stabilization_longitude

double fix_stabilization_latitude

struct config_dis_t

Public Members

bool enabled

double minimun_coefficient_filter

double decrement_coefficient_threshold

double increment_coefficient_threshold

double running_average_coefficient

double std_multiplier

bool black_corners_correction_enabled

double black_corners_threshold

uint32_t average_luminance_threshold

double camera_fov_factor

config_dis_angular_t angular_dis

config_dis_debug_t debug

struct config_eis_t

Public Members

```
bool enabled
bool stabilize
std::string eis_config_path
uint32_t window_size
double rotational_smoothing_coefficient
double iir_hpf_coefficient
double camera_fov_factor
uint64_t line_readout_time
double hdr_exposure_ratio
struct config_gyro_t
```

Public Members

```
bool enabled
std::string sensor_name
std::string sensor_frequency
double scale
struct config_stabilizer_settings_t
```

Public Functions

```
inline void override (const config_stabilizer_settings_t &other)
```

Public Members

```
std::string version
dis_config_t dis
eis_config_t eis
gyro_config_t gyro
```

Public Static Attributes

```
static bool is_persistent = true
struct config_denoise_network_t
```

Public Members

```
std::string network_path
std::string y_channel
std::string uv_channel
std::string feedback_y_channel
std::string feedback_uv_channel
std::string output_y_channel
std::string output_uv_channel
```

```
std::string bayer_channel
std::string feedback_bayer_channel
std::string dgain_channel
std::string bls_channel
std::string output_bayer_channel
std::string input_fusion_feedback
std::string input_gamma_feedback
std::string output_fusion_feedback
std::string output_gamma_feedback
std::string skip0_fusion_channel
std::string skip1_fusion_channel
struct config_denoise_t
```

Public Members

```
bool enabled
std::string sensor
std::string method
uint32_t loopback_count
config\_denoise\_network\_t network
bool bayer
struct config_hdr_t
```

Public Members

```
bool enabled
uint32_t dol
uint32_t lsRatio
uint32_t vsRatio
struct config_gray_scale_t
```

Public Members

```
bool enabled
struct config_iq_settings_t
```

Public Functions

```
inline void override(const config_iq_settings_t &other)
```

Public Members

```
std::string version
```

```
config_gray_scale_t grayscale
```

```
denoise_config_t denoise
```

```
hdr_config_t hdr
```

```
dewarp_config_t dewarp
```

```
automatic_algorithms_config_t automatic_algorithms_config
```

Public Static Attributes

```
static bool is_persistent = true
```

```
class HorizontalAlignment
```

Public Functions

```
HorizontalAlignment() = default
```

```
HorizontalAlignment(HorizontalAlignment const &other) = default
```

```
HorizontalAlignment &operator=(const HorizontalAlignment &other) = default
```

```
inline float as_float() const
```

Public Static Attributes

```
static const HorizontalAlignment LEFT = HorizontalAlignment(0.0)
```

```
static const HorizontalAlignment CENTER = HorizontalAlignment(0.5)
```

```
static const HorizontalAlignment RIGHT = HorizontalAlignment(1.0)
```

Private Functions

```
inline HorizontalAlignment(float alignment)
```

Private Members

```
float m_alignment
```

```
class VerticalAlignment
```

Public Functions

`VerticalAlignment()` = default

`VerticalAlignment(VerticalAlignment const &other)` = default

`VerticalAlignment &operator=(const VerticalAlignment &other)` = default

`inline float as_float()` const

Public Static Attributes

`static const VerticalAlignment TOP = VerticalAlignment(0.0)`

`static const VerticalAlignment CENTER = VerticalAlignment(0.5)`

`static const VerticalAlignment BOTTOM = VerticalAlignment(1.0)`

Private Functions

`inline VerticalAlignment(float alignment)`

Private Members

`float m_alignment`

`struct rgba_color_t`

Public Members

`int red`

`int green`

`int blue`

`int alpha`

`struct Overlay`

`#include <media_library_types.hpp> Overlay base struct`

Subclassed by *BaseTextOverlay*, *ImageOverlay*

Public Functions

`Overlay()` = default

`Overlay(std::string_id, float_x, float_y, unsigned int_z_index, unsigned int_angle, ...)`

Public Members

`std::string_id`

Unique string identifier for the overlay. This id is used for all future operations on the overlay.

`float x`

Horizontal position in frame. Position is relative and denoted with a decimal number between [0, 1].

`float y`

Vertical position in frame. Position is relative and denoted with a decimal number between [0, 1].

unsigned int `z_index`

Blend order when overlays overlay. Overlays with higher `:z_index` value are blender on top of overlays with lower `:z_index` value.

unsigned int `angle`

Blend angle to rotate the overlay.

[`rotation\_alignment\_policy\_t`](#) `rotation_alignment_policy`

Rotation alignment policy. Defines the rotation center of the overlay.

std::optional<[`HorizontalAlignment`](#)> `horizontal_alignment`

Horizontal alignment of the overlay. Defines the horizontal alignment of the overlay relative to the x position.

std::optional<[`VerticalAlignment`](#)> `vertical_alignment`

Vertical alignment of the overlay. Defines the vertical alignment of the overlay relative to the y position.

struct `ImageOverlay` : public [`Overlay`](#)

`#include <media_library_types.hpp>` [`Overlay`](#) containing an image (from a file)

Public Functions

`ImageOverlay()` = default

`ImageOverlay(std::string_id, float_x, float_y, float_width, float_height, std::string_image_path, ...)`

Public Members

float `width`

Width is relative and denoted with a decimal number between [0, 1].

float `height`

Height is relative and denoted with a decimal number between [0, 1].

std::string `image_path`

Path to load image from. The image will be scaled to match the given `:width` and `:height`.

struct `BaseTextOverlay` : public [`Overlay`](#)

`#include <media_library_types.hpp>` [`Overlay`](#) containing text

Subclassed by [`DateTimeOverlay`](#), [`TextOverlay`](#)

Public Functions

`BaseTextOverlay()` = default

`BaseTextOverlay(std::string_id, float_x, float_y, std::string_label, rgba\_color\_t _text_color, ...)`

Public Members

std::string label
Text content.

rgba_color_t text_color
Foreground Text color.

std::optional<*rgba_color_t*> background_color
Background color.

std::string font_path
Path to load font from. The font will be scaled to match the given font_size.

int font_size
Font size.

std::optional<int> line_thickness
Line thickness.

std::optional<*rgba_color_t*> shadow_color
Color of the text shadow. Shadow is disabled if any of the color components are negative.

std::optional<float> shadow_offset_x
Horizontal offset of the shadow relative to the text, in frame width ratio.

std::optional<float> shadow_offset_y
Vertical offset of the shadow relative to the text, in frame height ratio.

std::optional<*font_weight_t*> font_weight
Either normal or bold.

std::optional<int> outline_size
Outline size.

std::optional<*rgba_color_t*> outline_color
Outline Text color.

size_t m_width
Width in pixels.

size_t m_height
Height in pixels.

struct TextOverlay : public *BaseTextOverlay*

Public Functions

TextOverlay() = default

TextOverlay(std::string_id, float_x, float_y, std::string_label, [rgba_color_t](#) _text_color, [rgba_color_t](#) ...)

struct DateTimeOverlay : public [BaseTextOverlay](#)
[#include <media_library_types.hpp>](#) [Overlay](#) containing an auto-updating timestamp

Note: The timestamp is updated once per second

Public Functions

DateTimeOverlay() = default

DateTimeOverlay(std::string_id, float_x, float_y, std::string_datetime_format, [rgba_color_t](#) ...)

Public Members

std::optional<std::string> datetime_format
 format string for the datetime overlay. default is "d-m-Y H:M:S", meaning "day-month-year
 hour:minute:second"

struct config_stream_osd_t

Public Functions

inline void override(const [config_stream_osd_t](#) &other)

Public Members

std::vector<std::shared_ptr<[ImageOverlay](#)>> image_overlays
 std::vector<std::shared_ptr<[TextOverlay](#)>> text_overlays
 std::vector<std::shared_ptr<[DateTimeOverlay](#)>> datetime_overlays

Public Static Attributes

static bool is_persistent = true

struct config_encoded_output_stream_t

Public Functions

inline void override(const [config_encoded_output_stream_t](#) &other)

inline void set_input_stream_dimensions_rotation(const [rotation_config_t](#) ...)

inline bool is_portrait([rotation_angle_t](#) rotation_angle)

inline bool is_portrait(size_t width, size_t height)

Public Members

```
encoder_config_t encoding
config_stream_osd_t osd
privacy_mask_config_t masking
```

Public Static Attributes

```
static bool is_persistent = true
struct config_profile_t
```

Public Functions

```
inline void override(const config_profile_t &other)

inline frontend_config_t to_frontend_config() const

inline std::map<output_stream_id_t, encoder_config_t> to_encoder_config_map() const

inline EncoderType get_encoder_type(const output_stream_id_t &output_stream_id) const

inline void update_encoded_output_streams_rotation()

inline void from_frontend_config(const frontend_config_t &frontend_config)
```

Public Members

```
std::string version
std::string name
config_sensor_config_t sensor_config
config_application_settings_t application_settings
config_stabilizer_settings_t stabilizer_settings
config_iq_settings_t iq_settings
std::map<output_stream_id_t, config_encoded_output_stream_t> encoded_output_streams
```

8.8. Media Library Logger

The Media Library Logger is a simple logger designed to log messages to various destinations using the [spdlog](#) library. It offers two distinct logging outputs:

8.8.1. Console Logging

Messages are logged to the console. By default, the console log level is set to “error.” The console log level can be modified using the environment variable `MEDIALIB_CONSOLE_LOG_LEVEL`. Example:

```
export MEDIALIB_CONSOLE_LOG_LEVEL="debug"
```

8.8.2. File Logging

Messages are logged to a local log file named `medialib.log`. This log file is created in the current working directory. The default log level for the local log file is “info”. The log level for this file can be changed using the environment variable `MEDIALIB_LOG_LEVEL`. Location of this file can be set by using environment variable `MEDIALIB_LOGGER_PATH`. By default, this is a rotating log file. To make it non-rotating, set `MEDIALIB_LOGGER_ROTATE` environment variable to “false”. When used in rotating mode, the maximum file size can be set by using `MEDIALIB_LOGGER_FILE_SIZE`. Default file size is set to 1MB (1048576). *Warning:* When `MEDIALIB_LOGGER_ROTATE` is set to “false”, there is no size limit to the log file. Example:

```
export MEDIALIB_LOG_LEVEL="error"
export MEDIALIB_LOGGER_FILE_SIZE="4194304" # 4MB
export MEDIALIB_LOGGER_ROTATE="false"
```

Log Levels

- trace
- debug
- info
- warn
- error
- critical
- off

Module Specific Logs

Both console and file loggings can receive log levels for specific modules in their environment variables. Examples:

```
export MEDIALIB_CONSOLE_LOG_LEVEL="warn,denoise=info"
```

```
export MEDIALIB_LOG_LEVEL="dewarp=off,buffer_pool=debug"
```

Disable Logs

In order to disable all media-library logs:

```
export MEDIALIB_LOGGER_PATH=NONE
export MEDIALIB_LOG_LEVEL=off
export MEDIALIB_CONSOLE_LOG_LEVEL=off
```

8.9. Digital Image Stabilization

Digital Image Stabilization (DIS) aims to stabilize the vibrations in input frames, due to camera shakes. It does so by cropping from each frame a part which it views from the same part of the scene. DIS stabilizes the frame based on the instantaneous measurement of the Frame Motion Vector (FMV). The crop area is determined by the output (stabilized) Field of View (FOV), which must be smaller than the actual FOV of the camera. The difference forms the room for stabilization. If the shake of the camera is too strong (larger than the room for stabilization), some frames will not see the whole regions in the previous stabilized frames. If DIS discards this fact, then some parts of the output frame will remain not filled, as they are not visible in the input frame (technically such areas are extrapolated from the input image borders). This effect is referred to as “black corners” and DIS tries to avoid it, even if stabilization is compromised.

```
struct dis_config_t
    Configuration for DIS library.
```

Public Members

```
bool enabled
    Enable Digital Image Stabilization.
```

```
float minimun_coefficient_filter
    Minimal value of the coefficient 'k' used to filter the motion vectors (MVs). k takes values in the range [0, 1] and determines how fast changes are observed in output from a given MV, i.e. the result of the current frame's MV will be seen after 1/k frames. Example: k = 0 results in complete filtering and lack of consideration of the current MV, k = 1 means immediate impact (on the following frame).
```

```
float decrement_coefficient_threshold
    Value by which to decrement k whenever the difference of succeeding motion vectors is not too large. Roughly, 3* 1/this is the time (number of frames) that will take to adapt the filter to a new, weaker shaking amplitude and retrieve the filter strength. Values : 0-1, recommended 1/100-1/10, dimensionless.
```

```
float increment_coefficient_threshold
    Value by which to increment k when large motion occurs to prevent black corners. Roughly, 3* 1/this is the time (number of frames) that will take to adapt the filter to a new, higher shaking amplitude and minimize limitations for avoiding black corners. Values : 0-1, recommended 1/100-1/10, dimensionless.
```

```
float running_average_coefficient
    The frame motion vector (MV) is calculated by a HW on each frame and fed into the DIS library. Sometimes this MV is very wrong (gross error) - e.g. when the frame brightness changes rapidly, or when the scene changes rapidly (e.g. put a finger on the lens, or frame MV is suddenly very high - e.g. hit the camera). Such a gross error affects the stabilization at and after the moment of this error. So, detect gross errors and replace the erroneous MV with the MV of the previous frame. The detection works like this: On each frame, calculate the runtime average of MV and its standard deviation (STD). "1/running_average_coefficient" is roughly the number of frames being averaged. If  $\text{abs}(\text{current\_MV} - \text{mean\_MV}) > \text{STD\_MULTIPLIER} * \text{STD}$ , then this sample is a gross error-discard it. (0..1], typically "1 / number-of-frames-to-average". 1 to disable.
```

```
float std_multiplier
    Acceptable deviation, >0, normally 2.5-3.5. Set to a very large value to disable.
```

```
bool black_corners_correction_enabled
    If the shake is too strong, some frames may be impossible to stabilize without black corners appearing. Normally, the stabilized position (and output video) jumps in such cases, violating the stabilization, but avoiding black corners. If desired, The black corners could be left in order to maintain smooth output - set to 1. true: enable, false: disable (smooth stab with black corners)
```

```
float black_corners_threshold
    Filter strength is decreased if the stabilizing rotation is bigger than "BLKCRN_TO_K_THR * room-for-
```

stabilization". The lower this coefficient is, the less chance for limitations, but the more often the stabilization will be weakened without a real need for this. Also, if a panning starts, and k adaptation is disabled (`STAB_K_INC_BLKCRN = 0` or `BLKCRN_TO_K_THR` is much more than 1), the filter will follow the panning with too large delay and limitations will appear on each frame. Hence, the stabilized video will follow the input one, repeating its shakes along the panning, shifted (looks like delayed) by the room for stabilization. If `BLKCRN_TO_K_THR` is between 0 and 1 this panning delay is $(1 - \text{BLKCRN_TO_K_THR}) * \text{room_for_stabilization}$. Values: 0-1, recommended 0.2-0.5, default 0.2, dimensionless.

uint8_t average_luminance_threshold

For low light conditions, the stabilizer causes some noise in the output video. To avoid this, the stabilizer can be disabled when the average luminance of the frame is below a certain threshold. The threshold is set in the range [0, 255]. If the average luminance of the frame is below the threshold, the stabilizer is disabled. If the average luminance of the frame is above the threshold, the stabilizer is enabled. If the threshold is set to 0, the stabilizer is always enabled. If the threshold is set to 255, the stabilizer is always disabled.

float camera_fov_factor

Diagonal FoV factor of output camera. The difference between input and output FOV, (horizontal, vertical and diagonal) is the room for stabilization. Note the relation between aspect ratio and H,V,DFOV ratios:

- for fisheye camera: $\text{HFOV} / \text{VFOV} / \text{DFOV} = \text{width} / \text{height} / \text{diagonal}$
- for pinhole camera: $\tan(\text{HFOV}/2) / \tan(\text{VFOV}/2) / \tan(\text{DFOV}/2) = \text{width} / \text{height} / \text{diagonal}$ Set to 1 to let DIS calculate and use the maximum possible FOV at the given input camera model and output aspect ratio. FoV factor here is multiplied by the maximum possible FoV, allowing more stabilization by decreasing it. values: $0.1 \leq \text{camera_fov_factor} \leq 1$

8.10. Privacy Mask

8.10.1. Overview

The Privacy Mask feature enables the dynamic concealment of sensitive information in a video stream using the Hailo-15 DSP.

There are two types of privacy masks:

- **Static Privacy Mask:** A set of polygons drawn over the video stream in real-time.
- **Dynamic Privacy Mask:** A mask generated based on detected objects in the video stream, such as people.

Note: For an example of Dynamic Privacy Masking, you can refer to the *dynamic_privacy_mask* reference application.

The *PrivacyMaskBlender* object manages the vector of static privacy masks and the settings for both static and dynamic privacy masks. This blender is controlled by the *HailoEncoder* and can be accessed by both the GStreamer and C++ APIs.

To access the blender in the C++ API, use the following code:

```
PrivacyMaskBlenderPtr privacy_mask_blender = media_lib->encoder[id]->get_privacy_
    mask_blender();
```

Alternatively, access it from the GStreamer *hailoencodebin* element using:

```
GstElementPtr encoder = glib_cpp::ptrs::get_bin_by_name(m_pipeline, "hailoencodebin
    ");
PrivacyMaskBlenderPtr privacy_mask_blender;
// Retrieve the privacy mask blender from the frontend bin
GValue val = G_VALUE_INIT;
```

(continues on next page)

(continued from previous page)

```
g_object_get_property(frontend.as_g_object(), "privacy-mask-blender", &val);
void *value_ptr = g_value_get_pointer(&val);
privacy_mask_blender = reinterpret_cast<PrivacyMaskBlender *>(value_ptr)->shared_
    from_this();
g_value_unset(&val);
```

The resulting privacy mask, which combines the static and dynamic masks, supports two operational modes:

1. **Color Mode:** The privacy mask is drawn with a specific color, configurable via the `set_color` API.
2. **Pixelization Mode:** The privacy mask is drawn with a pixelization effect, with intensity adjustable via the `set_pixelization_size` API.

The mode can be changed both before and during pipeline execution. For example:

```
// Set the mode to color and configure the color to black
privacy_mask_blender->set_color({0, 0, 0});

// Set the mode to pixelization and configure the pixelization size to 40
privacy_mask_blender->set_pixelization_size(40);
```

8.10.2. Static Privacy Mask

Static privacy masks can be added, removed, or updated both before and during pipeline execution. For example:

```
polygon example_polygon;
example_polygon.id = "privacy_mask1";
example_polygon.vertices.push_back(vertex(125, 40));
example_polygon.vertices.push_back(vertex(980, 40));
example_polygon.vertices.push_back(vertex(980, 600));
example_polygon.vertices.push_back(vertex(125, 600));
privacy_mask_blender->add_static_privacy_mask(example_polygon);

auto polygon_exp = privacy_mask_blender->get_static_privacy_mask("privacy_mask1");
polygon polygon1 = polygon_exp.value();
polygon1.vertices[0].x = 600;
polygon1.vertices[0].y = 120;
privacy_mask_blender->set_static_privacy_mask(polygon1);

privacy_mask_blender->remove_static_privacy_mask("privacy_mask1");
```

When static privacy masks are manipulated via the API, a quantized bitmask is generated to efficiently represent the polygons. This bitmask is applied to each incoming frame to render the polygons using the DSP.

Restrictions for static privacy masks:

- A maximum of 8 static privacy masks can be included in the blender.
- Each polygon can have a maximum of 8 vertices.

8.10.3. Dynamic Privacy Mask

Dynamic privacy masks enable the masking of objects detected in the video stream based on segmentation data. This feature requires the Analytics Database to be properly configured.

To enable and configure the dynamic privacy mask, follow these steps:

1. **Enable Dynamic Privacy Mask:** Use the `set_dynamic_mask_enabled` API to enable or disable the dynamic privacy mask.

```
privacy_mask_blender->set_dynamic_mask_enabled(true);
```
2. **Configure Analytics Data:** The dynamic privacy mask relies on segmentation data from the Analytics Database. Configure the `analytics` array in the JSON configuration, specifying one or more analytics sources, each with an `analytics_data_id` and the `masked_labels` to be masked.
3. **Masking Behavior:** - The mask is dynamically applied based on detected objects in the video stream using the DSP. - The same color and pixelization settings as the static privacy mask are applied. - The `dilation_size` parameter can be configured to apply a dilation effect, improving mask coverage.
4. **Integration with the Analytics Database:** The dynamic mask queries the Analytics Database for the closest instance segmentation data and uses it to generate masks for the specified labels.

Note: Ensure the Analytics Database is properly initialized and configured with the required analytics data. For the dynamic privacy mask to function correctly, an additional element must process the video stream and add segmentation data to the Analytics Database. The frontend element must include an output stream dedicated to this purpose.

Restrictions:

- The `analytics` field is an array, allowing multiple analytics sources to contribute masks simultaneously.
- Each `analytics_data_id` must match the ID of the analytics data in the Analytics Database.
- The `masked_labels` must correspond to valid labels in the analytics configuration.
- The `dilation_size` must be within the range of 0 to 15.
- `timeout_ms` and `delta_ms` must be positive integers (minimum: 1).
- `query_type` must be one of: `Closest`, `Exact`, `WithinDelta`.

8.10.4. Configuration

The privacy mask can be configured using the `privacy_mask` section in the encoder configuration file.

The configuration must include the following parameters:

- **mask_type:** Specifies the mode of the privacy mask (`COLOR` or `PIXELIZATION`).
- **color_value:** Defines the RGB color of the privacy mask.
- **pixelization_size:** Specifies the pixelization effect size in pixels.

Additional parameters for static and dynamic privacy mask configurations:

- **static_privacy_mask:**
 - **enabled:** Boolean indicating whether static privacy masks are enabled (default: `true`).
 - **masks:** A list of static privacy masks to be added to the blender. Each mask is defined as a polygon with:
 - * **name:** The ID of the privacy mask.

★ **polygon_points**: A list of vertices defining the polygon, with each vertex specified by its x and y coordinates.

• **dynamic_privacy_mask**:

- **enabled**: Boolean indicating whether the dynamic privacy mask is enabled (default: `false`).
- **analytics**: An array of analytics sources, each containing:
 - ★ **analytics_data_id**: The ID of the analytics data in the Analytics Database.
 - ★ **masked_labels**: A list of labels to be masked from this analytics source.
- **dilation_size**: Specifies the dilation effect size in pixels (default: 0, range: 0 to 15).
- **timeout_ms**: Timeout in milliseconds for querying analytics data. If no data is available within this window, the mask is cleared (minimum: 1).
- **delta_ms**: Delta in milliseconds for the WithinDelta query type. Defines the acceptable time window for analytics data matching (minimum: 1).
- **query_type**: Specifies how the encoder queries the Analytics Database for mask data. One of:
 - ★ Closest — Use the analytics data closest in time to the current frame.
 - ★ Exact — Use only analytics data with an exact timestamp match.
 - ★ WithinDelta — Use analytics data within `delta_ms` of the current frame timestamp.

Example dynamic privacy mask configuration:

```
{
  "masking": {
    "mask_type": "PIXELIZATION",
    "pixelization_size": 32,
    "color_value": [0, 0, 0],
    "dynamic_privacy_mask": {
      "enabled": true,
      "analytics": [
        {
          "analytics_data_id": "semantic_segmentation",
          "masked_labels": ["person_face", "vehicle"]
        },
        {
          "analytics_data_id": "dpm_overflow_detection",
          "masked_labels": ["person", "face", "vehicle"]
        }
      ],
      "dilation_size": 7,
      "timeout_ms": 10000,
      "delta_ms": 34,
      "query_type": "Exact"
    }
  }
}
```

Note: When static privacy masks are configured, any existing static privacy masks are removed and replaced with the new configuration.

8.10.5. API

MediaLibrary Privacy Mask API module.

Defines

`MASK_SIZE` 128

Typedefs

```
using PrivacyMaskBlender = class PrivacyMaskConfigurer
using PrivacyMaskBlenderPtr = std::shared_ptr<PrivacyMaskBlender>
using PrivacyMaskPtr = std::shared_ptr<PrivacyMask>
class PrivacyMaskConfigurer : public std::enable_shared_from_this<PrivacyMaskConfigurer>
```

Public Functions

```
PrivacyMaskConfigurer()
    Construct a new Privacy Mask Configurer object.

~PrivacyMaskConfigurer()
    Destructor for Privacy Mask Configurer object.

media_library_return add_static_privacy_mask(const polygon &privacy_mask)
    Add a new static privacy mask.

    Parameters privacy_mask -- privacy mask to add

    Returns media_library_return - error code

tl::expected<polygon, media_library_return> get_static_privacy_mask(const std::string &id)
    Get static privacy mask.

    Parameters id -- privacy mask id

    Returns tl::expected<polygon, media_library_return> - polygon

media_library_return set_static_privacy_mask(const polygon &privacy_mask)
    Update existing static privacy mask.

    Parameters privacy_mask -- privacy mask to update

    Returns media_library_return - error code

media_library_return remove_static_privacy_mask(const std::string &id)
    Remove static privacy mask.

    Parameters id -- privacy mask id

    Returns media_library_return - error code

media_library_return set_color(const rgb_color_t &color)
    Change privacy mask to color mode and set color.

    Parameters color -- color to set

    Returns media_library_return - error code

media_library_return set_pixelization_size(PixelizationSize size)
    Change privacy mask to pixelization mode and set pixelization size.

    Parameters size -- pixelization size affecting the intensity of the pixelization. Must be an
    even number between 2 and 64

    Returns media_library_return - error code
```

media_library_return `set_rotation(const rotation_angle_t &rotation)`
Set rotation.

Parameters `rotation` -- rotation angle

Returns `media_library_return` - error code

tl::expected<rgb_color_t, media_library_return> `get_color()`
Get color.

Returns *tl::expected<rgb_color_t, media_library_return>* - the color if the privacy mask is in color mode, else `MEDIA_LIBRARY_ERROR`

tl::expected<PixelizationSize, media_library_return> `get_pixelization_size()`
Get pixelization size.

Returns *tl::expected<PixelizationSize, media_library_return>* - the pixelization size if the privacy mask is in pixelization mode, else `MEDIA_LIBRARY_ERROR`

tl::expected<std::pair<uint, uint>, media_library_return> `get_frame_size()`
Get frame size (width, height)

Returns *tl::expected<std::pair<uint, uint>, media_library_return>* - frame size width and height

media_library_return `set_frame_size(const uint width, const uint height)`
`set_frame_size`

Parameters

- `width` -- frame width
- `height` -- frame height

Returns `media_library_return` - error code

tl::expected<std::vector<polygon>, media_library_return> `get_all_static_privacy_masks()`
Get all static privacy masks.

Returns *tl::expected<std::vector<polygon>, media_library_return>* - vector of polygons

media_library_return `clear_all_static_privacy_masks()`
Clear all static privacy masks.

Returns `media_library_return` - error code

media_library_return `set_static_mask_enabled(bool enable)`
Enable or disable static mask.

Parameters `enable` -- true to enable static mask, false to disable

Returns `media_library_return` - error code

`bool is_static_mask_enabled()`
Check if static mask is enabled.

Returns `bool` - true if static mask is enabled, false otherwise

media_library_return `set_dynamic_mask_enabled(bool enable)`
Enable or disable dynamic mask.

Parameters `enable` -- true to enable dynamic mask, false to disable

`bool is_dynamic_mask_enabled()`
Check if dynamic mask is enabled.

Returns `bool` - true if dynamic mask is enabled, false otherwise

media_library_return `configure(const std::string &config)`
Configure the *PrivacyMaskConfigurer* object using a JSON configuration string.

Parameters `config` -- JSON configuration string.

Returns `media_library_return` - error code.

media_library_return configure(const std::unique_ptr<privacy_mask_config_t> &config)
Configure the *PrivacyMaskConfigurer* object using a *privacy_mask_config_t* structure.

Parameters config -- *privacy_mask_config_t* structure.

Returns media_library_return - error code.

void set_stream_id(const std::string &stream_id)

void set_config_manager_interactor(ConfigManagerInteractor *config_manager_interactor)

Private Members

std::string m_stream_id

ConfigManagerInteractor *m_config_manager_interactor = nullptr

class PrivacyMask

Public Functions

PrivacyMask()

~PrivacyMask()

media_library_return blend(HailoMediaLibraryBufferPtr input_buffer)
Blend privacy masks Update the current privacy masks, and blend them with the input buffer.

Parameters input_buffer -- input buffer to blend

Returns media_library_return - error code

void set_stream_id(const std::string &stream_id)

Public Static Functions

static tl::expected<PrivacyMaskPtr, *media_library_return*> create()

Private Functions

tl::expected<PrivacyMasksPtr, *media_library_return*> get_updated_privacy_masks(const ...)

size_t get_adjusted_frame_width(size_t width)

size_t get_adjusted_frame_height(size_t height)

bool should_adjust_buffer_pool(HailoMediaLibraryBufferPtr input_buffer)

media_library_return adjust_buffer_pool(HailoMediaLibraryBufferPtr input_buffer)

media_library_return update_info(const *config_encoded_output_stream_t* &masking_config)

media_library_return update_static_mask(const *config_encoded_output_stream_t* &masking_config)

media_library_return update_dynamic_mask(const *config_encoded_output_stream_t* ...)

media_library_return process_instance_segmentation_masks(const *AnalyticsQueryOptions* ...)

```
media_library_return process_semantic_segmentation_masks (const AnalyticsQueryOptions ...)
```

```
media_library_return process_detection_masks (const AnalyticsQueryOptions &opts, const ...)
```

Private Members

```
MediaLibraryBufferPoolPtr m_buffer_pool
std::string m_stream_id
PrivacyMasksPtr m_latest_privacy_masks
privacy_mask_config_t m_masking_config
std::vector<dsp_dynamic_privacy_mask_roi_t> m_dynamic_masks_rois
```

MediaLibrary Privacy Mask type definitions.

Defines

```
MAX_NUM_OF_STATIC_PRIVACY_MASKS 8
MAX_NUM_OF_DYNAMIC_PRIVACY_MASKS 100
MAX_NUM_OF_VERTICES_IN_POLYGON 8
namespace privacy_mask_types
```

Typedefs

```
using PixelizationSize = size_t
using StaticPrivacyMaskDataPtr = std::shared_ptr<static_privacy_mask_data_t>
using DynamicPrivacyMaskDataPtr = std::shared_ptr<dynamic_privacy_mask_data_t>
using PrivacyMasksPtr = std::shared_ptr<privacy_masks_t>
struct yuv_color_t
```

Public Members

```
uint8_t y
uint8_t u
uint8_t v
struct privacy_mask_info_t
```

Public Members

```
PrivacyMaskType type
yuv_color_t color
PixelizationSize pixelization_size
union privacy_mask_types::privacy_mask_info_t::[anonymous] [ anonymous ]
struct static_privacy_mask_data_t
```

Public Functions

```
inline static_privacy_mask_data_t()
```

Public Members

```
HailoMediaLibraryBufferPtr bitmask
std::vector<roi_t> rois
uint rois_count = 0
struct dynamic_privacy_mask_data_t
```

Public Members

```
dsp_dynamic_privacy_mask_t dynamic_mask_group
struct privacy_masks_t
```

Public Members

```
static_privacy_mask_data_t static_data
dynamic_privacy_mask_data_t dynamic_data
privacy_mask_info_t info
```

8.11. Motion Detection

8.11.1. Overview

Motion Detection is a feature that detects motion in a video stream. It is based on the difference between two consecutive frames. The motion detection algorithm is based on the OpenCV library, executed on the CPU.

8.11.2. Configuration

Configuration is done by the motion detection configuration section in the frontend configuration file. There are several parameters that can be configured:

1. **roi** (Region of Interest) - Defines the rectangular area in the frame where motion detection will be applied. Specified with x, y coordinates and width, height dimensions. This allows you to focus motion detection on specific areas (e.g., a doorway, window, or specific zone) rather than the entire frame.
2. **sensitivity_level** - Controls how much pixel brightness change is required between frames to register as motion. Available levels range from LOWEST (least sensitive, ignores minor changes) to HIGHEST (most sensitive, detects subtle changes). Start with MEDIUM and adjust based on your environment and motion characteristics.
3. **threshold** - A fractional value (0.0 to 1.0) representing the percentage of pixels in the ROI that must change to trigger a motion detection event. For example, 0.01 (1%) means motion is detected when 1% or more of the ROI pixels exceed the sensitivity threshold. Lower values make detection more sensitive to small movements.
4. **resolution** - The resolution of the internal processing frame used for motion detection. Lower resolutions (e.g., 320x240) process faster but may miss fine details, while higher resolutions provide more accuracy at the cost of CPU usage.
5. **buffer_pool_size** - The number of buffers allocated for motion detection bitmaps. Set to 0 to use the default pool size. Increase this if you experience buffer allocation failures during high-throughput operation.

8.11.3. How Does it Work?

When motion detection is enabled, it adds an extra internal output to the video stream with the configured resolution. This video stream is for internal processing only and does not go out from the frontend outputs.

Algorithm Overview:

Each frame is processed by comparing it to the previous frame to identify pixel-level changes. The algorithm uses OpenCV operations running on the CPU to generate a motion bitmask. The processing flow is:

1. **Frame Conversion:** An NV12 frame at the configured resolution is converted to a GRAYSCALE (GRAY8) format. This single-channel representation simplifies motion analysis by focusing on brightness changes rather than color.
2. **Frame Differencing:** The absolute difference between the current frame and the previous frame is calculated using `cv::absdiff()`. This produces a difference image where higher pixel values indicate greater change.
3. **Noise Reduction:** The difference image undergoes morphological filtering using `cv::morphologyEx()` with a `MORPH_OPEN` operation and a 5x5 elliptical kernel. This removes small artifacts and random noise that could cause false motion detection.
4. **Thresholding:** The filtered difference image is converted to a binary bitmask using `cv::threshold()` with the configured `sensitivity_level`. Pixels with changes below the sensitivity threshold become 0 (no motion), while pixels exceeding it become 255 (motion detected).
5. **Motion Decision:** The algorithm counts the number of white pixels (value 255) within the configured ROI. If the count exceeds the threshold percentage of total ROI pixels, motion is considered detected for that frame.

Motion Bitmask Structure:

The motion bitmask is a binary image (GRAY8 format) with the same dimensions as the configured motion detection resolution:

- **Pixel Value 0 (Black):** No motion detected at this location
- **Pixel Value 255 (White):** Motion detected at this location
- The bitmask provides spatial information showing exactly where motion occurred in the frame

8.11.4. Usage

Subscribe to `hailofrontend` callbacks: When a new buffer is available, the user receives metadata with the motion detection buffer.

```
fe_callbacks[s.id] = [s, media_lib](HailoMediaLibraryBufferPtr buffer, size_t size)
{
    if (buffer->motion_detection_buffer != nullptr)
    {
        // Check the boolean flag for quick motion detection status
        if (buffer->motion_detected)
        {
            std::cout << "Motion detected in frame" << std::endl;
        }

        // Access the motion detection bitmask for detailed analysis
        auto motion_buffer = buffer->motion_detection_buffer;
        cv::Mat bitmask(motion_buffer->buffer_data->height,
            motion_buffer->buffer_data->width,
            CV_8UC1, // Single-channel 8-bit grayscale
            motion_buffer->get_plane_ptr(0));

        // Example: Count motion pixels in the bitmask
        int motion_pixel_count = cv::countNonZero(bitmask);
        int total_pixels = bitmask.rows * bitmask.cols;
```

(continues on next page)

(continued from previous page)

```
float motion_percentage = (float)motion_pixel_count / total_pixels * 100.0f;
std::cout << "Motion coverage: " << motion_percentage << "%" << std::endl;

// Optional: Save the bitmask for visualization or debugging
cv::imwrite("motion_bitmask.png", bitmask);
}

media_lib->encoders[s.id]->add_buffer(buffer);
};
```

Understanding the Output:

Each video buffer from the frontend includes two motion detection outputs:

1. **motion_detected** (boolean): A simple flag indicating whether motion was detected in the current frame based on your configured ROI and threshold settings. Use this for quick decision-making (e.g., triggering recording, sending alerts).
2. **motion_detection_buffer**: A pointer to the full motion bitmask buffer. This provides pixel-level detail about where motion occurred:
 - **Format**: GRAY8 (single-channel 8-bit grayscale)
 - **Dimensions**: Match the configured motion detection resolution
 - **Content**: Binary image where white pixels (255) indicate motion and black pixels (0) indicate no motion

Performance Considerations:

- Motion detection runs on the CPU using OpenCV operations
- Lower resolution settings reduce processing time but may miss fine details
- The algorithm maintains frame-to-frame state (previous frame buffer)
- Processing time is typically 10-30ms depending on resolution and hardware
- First frame after enabling motion detection has no previous frame for comparison, so *motion_detected* will be false

Tuning Tips:

- Start with sensitivity_level=MEDIUM and threshold=0.01 for general use cases
- Use LOWEST or LOW sensitivity levels to reduce false positives from lighting changes, shadows, or environmental noise
- Use HIGH or HIGHEST sensitivity levels to detect subtle movements or small objects
- Adjust threshold based on expected motion size: smaller values (0.005-0.01) for detecting small objects, larger values (0.02-0.05) for significant scene changes
- Combine LOW sensitivity with low threshold (0.005) to detect only significant movements across small areas
- Combine HIGH sensitivity with higher threshold (0.03-0.05) to detect subtle movements but only when they affect a larger portion of the ROI
- Use a smaller ROI to focus on critical areas and improve performance
- Visualize the motion bitmask output to understand what the algorithm is detecting and adjust parameters accordingly

8.12. Analytics Database

8.12.1. Overview

The Analytics Database is a centralized storage system for AI inference results within the Media Library. It provides a thread-safe, time-indexed archive, enabling other components to query and retrieve analytic results based on type and timestamps.

The Analytics Database supports the following types of analytics data:

- **Detection Analytics:** Stores object detection results with bounding boxes
- **Instance Segmentation Analytics:** Stores instance segmentation results with masks and detection data

Note: For an example of the Analytics Database in use, refer to Dynamic Privacy Masks in [Privacy Mask](#), and to the *dynamic_privacy_mask* reference application, which demonstrate how to integrate instance segmentation results for dynamic privacy masking.

8.12.2. Usage

Configuration

The Analytics Database can be configured using the *application_analytics* section in the profile configuration file, and is used to initialize the Database during the Media Library initialization. The configuration supports the two types of analytics data: *instance_segmentation* and *detection*. Both are optional.

Note that the analytics data represents results derived from an original image that may have been resized and scaled before processing. Ensure that the configuration specifies the final dimensions and scaling details accurately.

The configuration must include the following parameters for each type:

- **analytics_data_id:** A unique identifier for the analytics data.
- **width** and **height:** The final dimensions of the resized image.
- **original_width_ratio** and **original_height_ratio:** The aspect ratio of the original image before resizing. These values help identify the actual image area versus any padding added during letterboxing.
- **scaling_mode:** Indicates how the original image was resized. Options include:
 - *STRETCH:* The image was stretched to fit the new dimensions.
 - *LETTERBOX_MIDDLE:* The image was framed with padding added evenly around it.
 - *LETTERBOX_UP_LEFT:* The image was framed with padding added to the bottom or to the right.
- **max_entries:** The maximum number of entries to store.
- **labels:** A list of labels associated with the analytics data, where each label is defined by:
 - **label:** The name of the label.
 - **id:** The ID of the label.

Example JSON Configuration:

```
{
  "application_analytics": {
    "instance_segmentation": [
      {
        "analytics_data_id": "yolo_segmentation",
        "scaling_mode": "STRETCH",
        "width": 640,
```

(continues on next page)

(continued from previous page)

```

        "height": 480,
        "original_width_ratio": 4,
        "original_height_ratio": 3,
        "max_entries": 10,
        "labels": [{ "label": "person", "id": 0 }, { "label": "car", "id": 1 }]
    }
],
"detection": [
    {
        "analytics_data_id": "yolo_detection",
        "scaling_mode": "LETTERBOX_MIDDLE",
        "width": 640,
        "height": 640,
        "original_width_ratio": 3,
        "original_height_ratio": 2,
        "max_entries": 7,
        "labels": [{ "label": "person", "id": 0 }]
    }
]
}
}

```

Important: Dual Camera Sensor Configuration: When using two input streams (dual sensor), ensure that each Media Library instance uses different `analytics_data_id` values. This prevents conflicts between analytics data from different sensors and ensures proper data isolation in the shared Analytics Database object.

The Analytics Database can also be initialized directly through the C++ API:

```

// Get the singleton instance
auto &analytics_db = AnalyticsDB::instance();

application_analytics_config_t config;

detection_analytics_config_t detection_config;
detection_config.analytics_data_id = "detection_results";
detection_config.scaling_mode = ScalingMode::LETTERBOX_MIDDLE;
detection_config.width = 640;
detection_config.height = 640;
detection_config.original_width_ratio = 4;
detection_config.original_height_ratio = 3;
detection_config.max_entries = 7;
detection_config.labels = {{ "person", 0 }, { "car", 1 }};

config.detection_analytics_config["detection_results"] = detection_config;

instance_segmentation_analytics_config_t seg_config;
seg_config.analytics_data_id = "segmentation_results";
seg_config.scaling_mode = ScalingMode::STRETCH;
seg_config.width = 640;
seg_config.height = 480;
seg_config.original_width_ratio = 4;
seg_config.original_height_ratio = 3;
seg_config.max_entries = 5;
seg_config.labels = {{ "person", 0 }};

config.instance_segmentation_analytics_config["segmentation_results"] = seg_
↪ config;

analytics_db.add_configuration(config);

```

Adding Analytics Data

Detection:

```
DetectionAnalyticsData detection_data;
detection_data.ts = frame_media_lib_buffer.isp_timestamp_ns;
detection_data.analytics_buffer = { /* detection results */};

auto result = analytics_db.add_detection_entry("detection_results", detection_
    ↪data);
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Instance Segmentation:

```
InstanceSegmentationAnalyticsData seg_data;
seg_data.ts = frame_media_lib_buffer.isp_timestamp_ns;
seg_data.analytics_buffer = { /* segmentation results with masks */};
seg_data.medialib_buffer_ptr = mask_media_lib_buffer;

auto result = analytics_db.add_instance_segmentation_entry("segmentation_results",
    ↪seg_data);
if (result != MEDIA_LIBRARY_SUCCESS) {
    // Handle error
}
```

Note: The *analytics_buffer* in both cases should contain the serialized data of the detection or segmentation results, which is a vector of hailort structs. For detection, those structs are of type *hailo_detection_t* and for segmentation, they are of type *hailo_instance_segmentation_t*. The buffer used for the timestamp is the original buffer holding the frame, as opposed to *medialib_buffer_ptr* in the instance segmentation data, which is used to reference the Media Library buffer that contains the mask memory, allocated for the AI inference results.

Querying Analytics Data

The Analytics Database supports three types of queries for retrieving analytics data:

1. **Exact Timestamp Query:** Retrieves analytics data with the exact specified timestamp.

```
AnalyticsQueryOptions options{
    .m_type = AnalyticsQueryType::Exact,
    .m_ts = target_timestamp,
    .m_timeout = std::chrono::milliseconds(100)
};

auto result = analytics_db.query_detection_entry("detection_results",
    ↪options);
if (result.has_value()) {
    auto detection_data = result.value();
    // Process detection data
}
```

2. **Closest Timestamp Query:** Finds the analytics data with the closest timestamp (equal or earlier).

```
AnalyticsQueryOptions options{
    .m_type = AnalyticsQueryType::Closest,
    .m_ts = target_timestamp,
    .m_timeout = std::chrono::milliseconds(500)
};
```

(continues on next page)

(continued from previous page)

```
auto result = analytics_db.query_instance_segmentation_entry("segmentation_
↳results", options);
```

3. **Within Delta Query:** Searches for analytics data within a specified time frame around the target timestamp. This query also requires a delta range parameter.

```
AnalyticsQueryOptions options{
    .m_type = AnalyticsQueryType::WithinDelta,
    .m_ts = target_timestamp,
    .m_delta = std::chrono::milliseconds(40),
    .m_timeout = std::chrono::milliseconds(1000)
};

auto result = analytics_db.query_detection_entry("detection_results",
↳options);
```

Accessing from Media Library

The Analytics Database is easily accessible through the Media Library API:

```
auto &analytics_db = media_library->get_analytics_db();

// Use normally
auto config = analytics_db.get_application_analytics_config();
```

8.12.3. API Reference

Typedefs

```
using Timestamp = std::chrono::time_point<std::chrono::steady_clock>
```

```
struct InstanceSegmentationAnalyticsData
```

Public Members

Timestamp ts

std::vector<hailo_detection_with_byte_mask_t> analytics_buffer

HailoMediaLibraryBufferPtr medialib_buffer_ptr

```
struct SemanticSegmentationAnalyticsData
```

Public Members

Timestamp ts

std::vector<hailo_semantic_segmentation_mask_t> analytics_buffer

std::vector<HailoMediaLibraryBufferPtr> medialib_buffer_ptrs

```
struct DetectionAnalyticsData
```

Public Members

Timestamp ts

std::vector<hailo_detection_t> analytics_buffer

struct AnalyticsQueryOptions

Public Members

AnalyticsQueryType m_type = *AnalyticsQueryType::Closest*

Timestamp m_ts

std::chrono::milliseconds m_delta = {0}

std::chrono::milliseconds m_timeout = {0}

class AnalyticsDB

Public Functions

void clear_db()

void reset_db()

void add_configuration(*application_analytics_config_t* application_analytics_config)

media_library_return add_detection_entry(const std::string &analytics_id, const ...)

media_library_return add_instance_segmentation_entry(const std::string &analytics_id, ...)

media_library_return add_semantic_segmentation_entry(const std::string &analytics_id, ...)

tl::expected<*DetectionAnalyticsData*, *media_library_return*> query_detection_entry(const ...)

tl::expected<*InstanceSegmentationAnalyticsData*, *media_library_return*> query_instance_segmentation_entry(const ...)

tl::expected<*SemanticSegmentationAnalyticsData*, *media_library_return*> query_semantic_segmentation_entry(const ...)

template<typename DataT, typename MapT>

tl::expected<*DataT*, *media_library_return*> query_entry(const *MapT* &db, const std::string ...)

application_analytics_config_t get_application_analytics_config()

Public Static Functions

static *AnalyticsDB* &instance()

Private Functions

AnalyticsDB()

```
template<typename DataT, typename MapT, typename ConfigMapT>
media_library_return add_entry( MapT &db, const std::string &analytics_id, DataT data, const ... )
```

Private Members

```
application_analytics_config_t m_application_analytics_config
std::map<std::string, std::map<Timestamp, DetectionAnalyticsData>> m_detection_entries_db
std::map<std::string, std::map<Timestamp, InstanceSegmentationAnalyticsData>>
m_instance_segmentation_entries_db
std::map<std::string, std::map<Timestamp, SemanticSegmentationAnalyticsData>>
m_semantic_segmentation_entries_db

std::mutex m_mutex
std::condition_variable m_cv
```

Private Static Functions

```
template<typename DataT, typename InnerMapT>
static tl::expected<DataT, media_library_return> find_closest( const InnerMapT &inner_map, ... )

template<typename DataT, typename InnerMapT>
static tl::expected<DataT, media_library_return> find_exact( const InnerMapT &inner_map, Timestamp ts )

template<typename DataT, typename InnerMapT>
static tl::expected<DataT, media_library_return> find_within_delta( const InnerMapT &inner_map, ... )
```

8.13. Snapshot Feature

8.13.1. Overview

The Snapshot feature in the Media Library provides a mechanism to capture the current state of media buffers at specific stages of the media processing pipeline. This feature is particularly useful for debugging, quality analysis, and development purposes.

Snapshots allow developers to:

- Visualize the intermediate processing results at different pipeline stages
- Debug visual artifacts or issues in media processing
- Validate algorithm behavior by examining input/output at specific points
- Capture reference frames for quality comparison and regression testing

8.13.2. How It Works

The Snapshot feature is implemented as a singleton manager that can:

1. Listen for snapshot requests through a command interface
2. Capture NV12 format frames at designated points in the pipeline
3. Save the snapshot frames to a timestamped directory for later analysis

When a snapshot is requested, all registered stages in the pipeline will capture their next frame, allowing you to see a synchronized view of the media processing pipeline.

8.13.3. Enabling Snapshots

Snapshots can be enabled in two ways:

1. **Environment Variable** (recommended for development):

Set the environment variable `MEDIALIB_SNAPSHOT_ENABLE` before running your application:

```
export MEDIALIB_SNAPSHOT_ENABLE=1
# Then run your application
```

2. **Programmatic API** (for runtime control):

```
// Get the singleton instance
SnapshotManager& snapshot_manager = SnapshotManager::get_instance();

// Enable snapshots (true to enable, false to disable)
snapshot_manager.enable_snapshot(true);
```

8.13.4. Using the Snapshot CLI Tool

The Media Library provides a dedicated command-line tool called `snapshot_cli` for controlling the snapshot functionality. This is the recommended way to interact with the snapshot feature.

Running the Tool

Simply execute the `snapshot_cli` tool while your Media Library application is running:

```
$ snapshot_cli
```

CLI Commands

The CLI supports the following commands:

1. **Show help information:**

```
# help
```

2. **Capture a single snapshot from all stages:**

```
# snapshot
Snapshot requested for 1 frame
```

3. **Capture multiple frames:**

```
# snapshot 5
Snapshot requested for 5 frames
```

4. Capture from specific stages only:

```
# snapshot 3 post_isp,dewarp
Snapshot requested for 3 frames
```

5. List available pipeline stages:

```
# list_stages
Available stages for snapshot:
- post_isp
- dewarp
- ...
```

6. Exit the tool:

```
# exit
Snapshot tool exiting.
```

8.13.5. Programmatic API

While the CLI tool is recommended, you can also use the C++ API to control snapshots programmatically:

```
// Get the singleton instance
SnapshotManager& snapshot_manager = SnapshotManager::get_instance();

// Request a snapshot from all stages (1 frame)
snapshot_manager.request_snapshot();

// Request multiple frames (5 frames from all stages)
snapshot_manager.request_snapshot(5);

// Request frames from specific stages
std::set<std::string> stages = {"post_isp", "dewarp"};
snapshot_manager.request_snapshot(3, stages);

// Get list of available stages
std::string stages_list = snapshot_manager.list_available_stages();
```

8.13.6. Integration in Your Pipeline

To add snapshot capability to a custom stage in your media pipeline:

1. Get the singleton instance of the SnapshotManager
2. At appropriate points in your processing, check if a snapshot is requested
3. If requested, pass your buffer to the SnapshotManager with a unique stage name

Example:

```
void MyStage::process_frame(const HailoMediaLibraryBufferPtr& buffer)
{
    // Normal frame processing...

    // Capture snapshot if requested
    SnapshotManager::get_instance().take_snapshot("my_custom_stage", buffer);
}
```

(continues on next page)

(continued from previous page)

```
// Continue processing...
}
```

8.13.7. Snapshot Storage and File Format

Snapshots Location

By default, snapshots are stored in directories under `/tmp/medialib_snapshots/`.

Directory structure:

- Base directory: `/tmp/medialib_snapshots/`
- Each snapshot session creates a timestamped folder: `YYYY-MM-DD_HH-MM-SS_mmm/`
- Full path example: `/tmp/medialib_snapshots/2023-11-15_14-30-22_456/`

File naming:

Each captured snapshot follows this naming pattern:

```
<stage_name>_<width>x<height>.nv12
```

For example:

```
dewarp_output_3840x2160.nv12 post_isp_1920x1080.nv12
```

File Format

Snapshots are saved in raw NV12 format, which is a YUV 4:2:0 planar format with 8 bits per sample. The Y plane comes first, followed by an interleaved U/V plane at half resolution both horizontally and vertically.

Viewing Snapshots

To view NV12 files, you can use tools like FFmpeg or GStreamer:

With FFmpeg:

```
ffmpeg -f rawvideo -pix_fmt nv12 -s 3840x2160 -i <stage_name>_3840x2160.nv12 output.
→png
```

With GStreamer:

```
gst-launch-1.0 filesrc location=<stage_name>_3840x2160.nv12 ! \
rawvideoparse format=nv12 width=3840 height=2160 ! \
videoconvert ! autovideosink
```

8.13.8. Best Practices

- Enable snapshots only when needed as they may impact performance
- Use the `list_stages` command to identify available pipeline stages before capturing
- For multi-frame captures, use a reasonable frame count to avoid excessive disk usage
- Target specific stages when debugging isolated issues rather than capturing all stages
- For production systems, keep snapshots disabled
- Regularly clean up older snapshot directories to avoid filling up disk space

8.13.9. Limitations

- Snapshots are currently limited to NV12 format buffers
- Snapshots are stored in the local filesystem and not automatically cleaned up
- Large or frequent snapshots may impact system performance

8.14. Sensor Registry

8.14.1. Overview

The Sensor Registry is a centralized system for managing sensor types and their configurations within the Media Library. It provides a thread-safe singleton interface for:

- **Sensor Detection:** Automatically detect connected sensors via I2C and subdevice enumeration
- **Resolution Management:** Define and query supported resolutions for different sensors
- **Mode Mapping:** Configure sensor and CSI modes for SDR and HDR capture
- **Capabilities Query:** Retrieve sensor-specific information such as pixel formats, supported resolutions, and device paths

8.14.2. Usage

Basic Usage

The Sensor Registry is accessible as a singleton throughout the Media Library:

```
// Get the singleton instance
auto &registry = SensorRegistry::get_instance();

// Detect connected sensor
auto sensor_type = registry.detect_sensor_type();

// Get sensor capabilities
auto capabilities = registry.get_sensor_capabilities(sensor_type.value());
if (capabilities.has_value()) {
    std::cout << "Sensor name: " << capabilities->sensor_name << std::endl;
    std::cout << "Pixel format: " << capabilities->pixel_format << std::endl;
}
```

Querying Sensor Information

The registry provides various methods to query sensor information:

```
auto &registry = SensorRegistry::get_instance();

// Get I2C bus and address for a sensor
auto i2c_info = registry.get_i2c_bus_and_address(0);
if (i2c_info.has_value()) {
    int bus = i2c_info->first;
    std::string address = i2c_info->second;
}

// Get video device path
auto video_path = registry.get_video_device_path(0);
```

(continues on next page)

(continued from previous page)

```
// Get pixel format for detected sensor
auto pixel_format = registry.get_pixel_format();

// Get sensor name
auto sensor_name = registry.get_sensor_name(SensorType::IMX678);
```

Resolution and Mode Information

Query resolution information and sensor modes:

```
auto &registry = SensorRegistry::get_instance();

// Detect resolution from output configuration
output_resolution_t output_res;
output_res.dimensions.destination_width = 1920;
output_res.dimensions.destination_height = 1080;

auto resolution = registry.detect_resolution(output_res);
if (!resolution.has_value() || resolution.value() != Resolution::FHD) {
    return media_library_return::MEDIA_LIBRARY_ERROR;
}
```

8.14.3. Adding New Sensors

..Note:: Before adding a new sensor to the media library, it would be useful to refer to a process in the ISP guide which is similar to adding a new sensor, such as adding a driver, etc.

Follow these steps to add support for a new sensor to the registry:

Step 1: Update the SensorType Enum

In `sensor_types.hpp` add the new sensor to the `SensorType` enum:

```
enum class SensorType
{
    IMX334,
    ...
    IMX999 // Add the new sensor here
};
```

Step 2: Create a Sensor Capabilities File

Navigate to the `media-library/hailo-media-library/media_library/src/isp/sensor_registry/` directory and create a new header file for the sensor, e.g., `imx999_capabilities.hpp`.

Define the sensor's capabilities by implementing the inline capabilities structure. Tip: use existing files (e.g., `imx678_capabilities.hpp`) as a reference.

Important: Check the sensor and CSI modes and the pixel format before finalizing the configuration.

Step 3: Register the Sensor

Edit `sensor_capabilities.hpp` by doing the following:

1. Include the new sensor's header file:

```
#include "imx999_capabilities.hpp"
```

2. Add the sensor to the `all_sensor_capabilities` map:

```
inline const std::unordered_map<SensorType, SensorCapabilities> all_sensor_
capabilities = {
    {SensorType::IMX334, sensor_config::imx334::capabilities},
    ...
    {SensorType::IMX999, sensor_config::imx999::capabilities}, // Add this line
};
```

8.14.4. Adding New Resolutions

Follow these steps to add support for a new resolution:

Step 1: Define the Resolution

Open `sensor_types.hpp` and add the new resolution to the `Resolution` enum:

```
enum class Resolution
{
    FHD,    // 1920x1080
    ...
    SIX_MP  // Add the new resolution here
};
```

Step 2: Add Resolution Info

Open `sensor_capabilities.hpp` and add the resolution to the `all_resolution_info` map:

```
inline const std::unordered_map<Resolution, ResolutionInfo> all_resolution_info = {
    {Resolution::FHD,
     {.width = 1920, .height = 1080, .name = "fhd",
      .vsm_offsets = {.h_offset = 0, .v_offset = 0}}},
    ...
    {Resolution::SIX_MP,
     {.width = 3072, .height = 2048, .name = "6mp",
      .vsm_offsets = {.h_offset = 576, .v_offset = 64}}}, // Add these lines
};
```

Step 3: Update Sensor Capabilities

Update the `supported_resolutions` and `mode_mappings` in the relevant sensor's capabilities file (e.g., `imx678_capabilities.hpp`), by adding it to the `.supported_resolutions` and `.mode_mappings` maps.

8.14.5. API Reference

MediaLibrary sensor type registry system.

class SensorRegistry

Public Functions

```
std::optional<SensorCapabilities> get_sensor_capabilities(SensorType sensor) const

std::optional<SensorType> detect_sensor_type(size_t sensor_index = 0) const

std::optional<std::string> detect_sensor_type_str(size_t sensor_index = 0) const

std::optional<std::pair<int, std::string>> get_i2c_bus_and_address(size_t sensor_index)

std::optional<Resolution> detect_resolution(const output_resolution_t &resolution) const

std::optional<ResolutionInfo> get_resolution_info(Resolution res) const

std::optional<SensorModelInfo> get_sensor_mode_info(SensorType sensor, const SensorModeKey ...)

std::optional<SensorModelInfo> get_sensor_mode_info_hdr(const output_resolution_t ...)

std::optional<SensorModelInfo> get_sensor_mode_info_sdr(const output_resolution_t ...)

bool is_supported(SensorCapabilities capabilities, Resolution resolution) const

std::optional<int> get_pixel_format()

std::optional<std::string> get_video_device_path(size_t sensor_index)

std::optional<std::string> get_raw_capture_path(size_t sensor_index)

std::optional<std::string> get_sensor_name(SensorType sensor) const

std::optional<std::string> get_imx_subdevice_path(size_t sensor_index) const

std::optional<std::string> get_hdr_capture_path(size_t sensor_index)
```

Public Static Functions

```
static SensorRegistry &get_instance()
```

Private Functions

```
SensorRegistry()
```

```
SensorRegistry(const SensorRegistry&) = delete
```

```
SensorRegistry &operator=(const SensorRegistry&) = delete
```

```
void initialize_resolutions()
```

```
void initialize_sensors()
```

```
std::optional<SensorDeviceInfo> get_sensor_device_info(size_t sensor_index) const
```

Private Members

```
std::unordered_map<SensorType, SensorCapabilities> m_sensor_capabilities
```

```
std::unordered_map<Resolution, ResolutionInfo> m_resolution_info
```

```
struct SensorDeviceInfo
```

Public Members

```
SensorType sensor_type
```

```
int bus
```

```
std::string address
```

```
std::string subdevice_path
```

MediaLibrary sensor and resolution definitions.

Enums

```
enum SensorType
```

Values:

```
enumerator IMX334
```

```
enumerator IMX664
```

```
enumerator IMX675
```

```
enumerator IMX678
```

```
enumerator IMX715
```

```
enumerator IMX307
```

```
enumerator IMX662
```

```
enum Resolution
```

Values:

```
enumerator FHD
```

```
enumerator UHD_4K
```

```
enumerator FIVE_MP
```

```
enumerator FOUR_MP
```

```
enum CSI_MODE
```

Values:

```
enumerator CSI_MODE_SDR
```

```
enumerator CSI_MODE_MERCURY_ISP_STITCH_HDR
```

```
enumerator CSI_MODE_DEFAULT_HDR
```

```
struct ResolutionInfo
```

Public Members

```
uint32_t width
```

```
uint32_t height
```

```
std::string name
```

```
struct ResolutionInfo::VSMOffsets vsm_offsets
```

```
struct VSMOffsets
```

Public Members

```
uint32_t h_offset
```

```
uint32_t v_offset
```

```
struct SensorModeInfo
```

Public Members

```
int sensor_mode
```

```
int csi_mode
```

```
struct SensorModeKey
```

Public Functions

```
SensorModeKey(Resolution res)
```

```
SensorModeKey(Resolution res, std::optional<hdr\_dol\_t> hdr)
```

```
bool operator==(const SensorModeKey &other) const
```

Public Members

```
Resolution resolution
```

```
std::optional<hdr\_dol\_t> hdr_mode
```

```
struct Hash
```

Public Functions

```
std::size_t operator() (const SensorModeKey &key) const
```

```
struct SensorCapabilities
```

Public Members

```
std::string sensor_name
```

```
std::string sub_dev_prefix
```

```
std::set<Resolution> supported_resolutions
```

```
int pixel_format
```

```
std::unordered_map<SensorModeKey, SensorModeInfo, SensorModeKey::Hash> mode_mappings
```

9. Low Level API Reference

9.1. DSP Utils

namespace `dsp_utils`

Functions

`dsp_status release_device()`

Release the DSP device. This function releases the device using the DSP library. If there are other references to the device, just decrement the refcount and skip the release.

Returns `dsp_status`

`dsp_status acquire_device()`

Acquire the DSP device. This function creates the DSP device using the DSP library once, and then increases the reference count.

Returns `dsp_status`

`dsp_status create_hailo_dsp_buffer(size_t size, void **buffer, bool dma = false)`

Create a buffer on the DSP. This function requests a buffer from the DSP library. The buffer can be used later for DSP operations.

Parameters

- `size` - **[in]** the size of the buffer to create
- `buffer` - **[out]** a pointer to a buffer - DSP library will allocate the buffer

Returns `dsp_status`

`void release_hailo_dsp_buffer(void *buffer)`

Release a buffer allocated by the DSP

Parameters `buffer` - **[in]** the buffer to release

Returns `dsp_status`

`dsp_status perform_resize(hailo_buffer_data_t *input_buffer_data, hailo_buffer_data_t ...)`

Perform DSP Resize. This function calls the DSP library to perform resize on a given buffer. DSP will place the result in the output buffer.

Parameters

- `input_buffer_data` - **[in]** input buffer data
- `output_buffer_data` - **[in]** output buffer data
- `dsp_interpolation_type` - **[in]** interpolation type to use
- `use_letterbox` - **[in]** should letterbox resize be used

Returns `dsp_status`

`dsp_status perform_resize(dsp_image_properties_t *input_image_properties, ...)`

Perform DSP Resize. This function calls the DSP library to perform resize on a given buffer. DSP will place the result in the output buffer.

Parameters

- `input_image_properties` - **[in]** input image properties
- `output_image_properties` - **[in]** output image properties
- `dsp_interpolation_type` - **[in]** interpolation type to use
- `use_letterbox` - **[in]** should letterbox resize be used

Returns `dsp_status`

`dsp_status perform_crop_and_resize(hailo_buffer_data_t *input_buffer_data, ...)`
 Perform DSP crop and resize This function calls the DSP library to perform crop and resize on a given buffer. DSP will place the result in the output buffer.

Parameters

- `input_buffer_data` - **[in]** input buffer data
- `output_buffer_data` - **[out]** output buffer data
- `args` - **[in]** crop and resize arguments
- `dsp_interpolation_type` - **[in]** interpolation type to use

Returns `dsp_status`

`dsp_status perform_crop_and_resize(dsp_image_properties_t *input_image_properties, ...)`
 Perform DSP crop and resize This function calls the DSP library to perform crop and resize on a given buffer. DSP will place the result in the output buffer.

Parameters

- `input_image_properties` - **[in]** input image properties
- `output_image_properties` - **[out]** output image properties
- `args` - **[in]** crop and resize arguments
- `dsp_interpolation_type` - **[in]** interpolation type to use

Returns `dsp_status`

`dsp_status perform_dsp_multi_resize(dsp_multi_crop_resize_params_t ...)`
 Perform multiple crops and resizes on the DSP This function calls the DSP library to perform crops and resizes on a given input buffer. DSP will place the results in the array of output buffer.

Parameters `multi_crop_resize_params` - **[in]** crop and resize metadata

Returns `dsp_status`

`dsp_status perform_dsp_multi_resize(dsp_multi_crop_resize_params_t ...)`
 Apply a privacy mask and perform multiple crops and resizes on the DSP This function calls the DSP library to perform crops and resizes on a given input buffer. DSP will place the results in the array of output buffer.

Parameters

- `multi_crop_resize_params` - **[in]** crop and resize metadata
- `privacy_mask_params` - **[in]** privacy mask metadata

Returns `dsp_status`

`dsp_status perform_dsp_telescopic_multi_resize(dsp_multi_crop_resize_params_t ...)`
 Performs a telescopic multi-resize operation using DSP.

This function takes in parameters for multi-crop and resize operations and performs the telescopic multi-resize using the DSP (Digital Signal Processor).

Parameters `multi_crop_resize_params` - A pointer to a structure containing the parameters for the multi-crop and resize operations.

Returns `dsp_status` The status of the DSP operation.

`dsp_status perform_dsp_telescopic_multi_resize(dsp_multi_crop_resize_params_t ...)`
 Perform a telescopic multi-resize operation with privacy masking.

This function performs a telescopic multi-resize operation on the given multi-crop resize parameters and applies a privacy mask if provided.

Parameters

- `multi_crop_resize_params` - Pointer to the parameters for multi-crop resize.

- `privacy_mask_params` - Pointer to the parameters for the privacy mask.

Returns `dsp_status` Status of the DSP operation.

`dsp_status perform_dsp_telescopic_multi_resize(dsp_multi_crop_resize_params_t ...)`

Parameters

- `multi_crop_resize_params` - Pointer to the parameters for the multi-crop re-size.
- `privacy_mask_params` - Pointer to the parameters for the privacy mask.
- `denoise_params` - Pointer to the parameters for the denoise operation.

Returns `dsp_status` Status of the DSP operation.

`dsp_status perform_dsp_telescopic_multi_resize(dsp_multi_crop_resize_params_t ...)`

Parameters

- `multi_crop_resize_params` - Pointer to the parameters for the multi-crop re-size.
- `denoise_params` - Pointer to the parameters for the denoise operation.

Returns `dsp_status` Status of the DSP operation.

`dsp_status perform_dsp_frontend_process(const dsp_frontend_params_t &frontend_params)`
Performs a telescopic multi-resize operation using DSP.

This function takes in parameters for multi-crop and resize operations and performs the telescopic multi-resize using the DSP (Digital Signal Processor).

Parameters `dsp_frontend_params_t` - A pointer to a structure containing the parameters for the multi-crop and resize operations.

Returns `dsp_status` The status of the DSP operation.

`dsp_status perform_dsp_privacy_mask(hailo_buffer_data_t *input_buffer_data, const ...)`
Apply privacy masks on an image.

The function applies one or more privacy masks on an image.

Parameters

- `image` - pointer to input image to apply the privacy masks on.
- `privacy_mask_params` - Pointer to the required privacy mask parameters.

Returns `dsp_status` The status of the DSP operation.

`dsp_status perform_dsp_dewarp(hailo_buffer_data_t *input_buffer_data, hailo_buffer_data_t ...)`

`dsp_status perform_dsp_dewarp(dsp_image_properties_t *input_image_properties, ...)`

`dsp_status perform_dsp_dewarp(hailo_buffer_data_t *input_buffer_data, hailo_buffer_data_t ...)`

`dsp_status perform_dsp_dewarp(dsp_image_properties_t *input_image_properties, ...)`

`dsp_status perform_dsp_multiblend(hailo_buffer_data_t *input_buffer_data, ...)`
Perform DSP blending using multiple overlays This function calls the DSP library to perform blending between one main buffer and multiple overlay buffers. DSP will blend the overlay buffers onto the image frame in place

Parameters

- `image_buffer_data` - **[in]** pointer to input image to blend on

- **overlay** - **[in]** pointer to input images to overlay with
- **overlays_count** - **[in]** number of overlays to blend

Returns dsp_status

dsp_status perform_dsp_multiblend(dsp_image_properties_t *image_frame, ...)
Perform DSP blending using multiple overlays This function calls the DSP library to perform blending between one main buffer and multiple overlay buffers. DSP will blend the overlay buffers onto the image frame in place

Parameters

- **image_frame** - **[in]** pointer to input image to blend on
- **overlay** - **[in]** pointer to input images to overlay with
- **overlays_count** - **[in]** number of overlays to blend

Returns dsp_status

dsp_status hailo_buffer_data_to_dsp_buffer_data(hailo_buffer_data_t *buffer_data, ...)

dsp_status hailo_buffer_data_to_dsp_image_props(hailo_buffer_data_t *buffer_data, ...)
Convert hailo_buffer_data_t to dsp_image_properties_t Allocates memory - caller is responsible for freeing it with free_image_property_planes

Parameters

- **buffer_data** - **[in]** pointer to the hailo buffer data
- **out_dsp_buffer_props** - **[out]** pointer to the dsp output image properties

Returns dsp_status

void free_overlay_property_planes(dsp_overlay_properties_t *overlay_properties)
Free DSP struct resources

Parameters **overlay_properties** - **[in]** pointer to the properties to free

Returns void

void free_image_property_planes(dsp_image_properties_t *image_properties)
Free DSP struct resources

Parameters **image_properties** - **[in]** pointer to the properties to free

Returns void

size_t get_dsp_desired_stride_from_width(size_t width)
get_dsp_desired_stride_from_width will return the appropriate buffer stride for each resolution DSP operation with these strides are more efficient

Parameters **width** - **[in]** the width of the frame

Returns size_t the desired stride

static dsp_status create_device()
Create a DSP device and store it globally This function requests a device from the DSP library.

Returns dsp_status

hailo_dsp_buffer_data_t hailo_buffer_data_to_dsp_buffer_data(hailo_buffer_data_t ...)

Variables

```
static constexpr int max_blend_overlays = 50
static dsp_device device = NULL
static uint dsp_device_refcount = 0
static const int DSP_VISION_PRIORITY = 100
struct crop_resize_dims_t
#include <dsp_utils.hpp> Crop and resize parameters
```

9.2. Encoder (low level) API-Manual

9.2.1. Acronyms and Conventions

Acronym	Description
1080p	High Definition resolution of 1920 x 1080 progressive video
720p	High Definition resolution of 1280 x 720 progressive video
ACE	AXI Coherency Extensions
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
API	Application Programming Interface
AVC	Advanced Video Coding (aka H.264, MVC)
AVX	Advanced Vector eXtensions
AXI	Advanced eXtensible Interface
bps	Bits Per Second
CABAC	Context Adaptive Binary Arithmetic Coding, HEVC entropy coding method
CB	Coding Block
CBR	Constant BitRate
CIF	Common Interchange Format (352 x 288 pixels)
CIR	Cyclic Intra Refresh
CODEC	COmpressor/DECompressor
CPB	Coded Picture Buffer
CTB	Coding Tree Block
CTU	Coding Tree Unit
CU	Coding Unit
CVS	Coded Video Sequence
DBF	DeBlocking Filter
DC	Direct Current
DPB	Decoded Picture Buffer
DPCM	Differential Pulse Code Modulation
DU	Decoding Unit

Continued on next page

Table 17 – continued from previous page

Acronym	Description
EOS	End Of Sequence
FMA	Fused Multiply Add
fps	Frames Per Second
GDR	Gradual Decoding Refresh
GOP	Group Of Pictures
GPU	Graphics Processing Unit
HD	High Definition (HD=1980 pixels x1080 lines, progressive scan)
HEVC	High Efficiency Video Coding, a video coding standard (JCT-VC) (aka H.265)
HRD	Hypothetical Reference Decoder
IDR	Instantaneous Decoder Refresh
MAD	Mean Absolute Difference
mbps	Mega (1000000) Bits Per Second
MC	Motion Compensation
MV	Motion Vector
NAL	Network Abstraction Layer
NALU	NAL Unit, the smallest decodable unit of a stream
NLE	NonLinear Editing
PAL	Phase Alternate Line (resolution 720 x 576 pixels)
POC	Picture Order Count
PPS	Picture Parameter Set
PU	Prediction Unit
QCIF	Quarter CIF (176 x 144 pixels)
QVGA	Quarter Video Graphics Array (320 x 240 pixels)
QP	Quantization Parameter
QQVGA	Quarter QVGA (160 x 120 pixels)
RC	Rate Control
RGB	Red, Green, Blue color space representation
ROI	Region Of Interest
RPDCM	Residual DPCM
RPS	Reference Picture Set
SAD	Sum of Absolute Difference
SAO	Sample Adaptive Offset
SEI	Supplemental Enhancement Information
SPS	Sequence Parameter Set
sub-QCIF	Sub Quarter CIF video resolution (128 x 96 pixels)
SXGA	Super eXtended Graphics Array (resolution 1280 x 1024 pixels)
TU	Transform Unit
UHD	Ultra High Definition (4K=3840 pixels x 2160 lines; 8K=7680 pixels x 4320 lines)

Continued on next page

Table 17 – continued from previous page

Acronym	Description
VBR	Variable BitRate
VGA	Video Graphics Array (resolution 640 x 480 pixels)
VLC	Variable Length Coding
VPS	Video Parameter Set
VUI	Video Usability Information
WCG	Wide Color Gamut
WPP	Wavefront Parallel Processing
YCbCr	Color space with one luma and two chroma components used for digital encoding
VPS	Video Parameter Set
YUV	Color space with one luma and two chroma component. Originally used for analog encoding, currently often used for either digital or analog encoding.

9.2.2. Overview

This chapter describes the Application Programming Interface (API) of the HEVC/AVC video encoder hardware.

The description of the Video Encoder API are described in the following order:

- Components such as data types, return codes, enumerations and relevant structures.
- The Function syntax and their related descriptions.
- An overview of the programming sequence with examples. The code is written in C and parameter types follow the standard C conventions. This document assumes that the reader understands the fundamentals of C-language and the HEVC (H.265)/AVC (H.264) standards. Currently, there are no deprecated functions in this API.

The code is written in C and parameter types follow standard C conventions. This document assumes that the reader understands the fundamentals of C-language and the HEVC (H.265)/AVC (H.264) standards.

Currently, there are no deprecated functions in this API.

Supported Standards

The API discussed in this version of the document are compatible with the following video encoder standards and profiles. Additional levels may be supported via software.

- HEVC (H.265) - ITU-T Rec. H.265 (04/2013), ISO/IEC 23008-2 9
 - Main Profile, Level 5.1, High Tier
 - Main10 profile, Level 5.1, High Tier
 - Main Still Profile
- AVC (H.264) - ITU-T Rec. H.264 (03/2010) / ISO/IEC 14496-10
 - Main Profile, levels 1 - 5.2
 - High Profile, levels 1 - 5.2
 - High 10 Profile, levels 1 - 5.2

Compatible Hardware:

- VC8000E Video Encoder v6.0.00 or later

Standard References

[ITU-T H.265](#). High Efficiency Video Coding

[ITU-T H.264](#). Advanced video coding for generic audiovisual services

[ITU-R-REC-BT.601](#). Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios.

[ITU-R-REC-BT.709](#) Parameter values for the HDTV standards for production and international programme exchange.

9.2.3. API Components

This section describes the API declared in **hevencapi.h**. Enumerations and structures are listed alphabetically in this document.

9.2.4. Numeric Data Types

The following common numeric data types are declared in **base_type.h**:

Name	Data type
u8	unsigned 8 bit integer
i8	signed 8 bit integer
u16	unsigned 16 bit integer
i16	signed 16 bit integer
u32	unsigned 32 bit
i32	signed 32 bit

9.2.5. Return Codes

VCEncRet Enumeration

Specifies the return values for the API functions.

VCEncRet Values	Description
VCENC_OK	Success.
VCENC_EWL_ERROR	Error, the encoder's system interface failed to initialize.
VCENC_EWL_MEMORY_ERROR	Error, the system interface failed to allocate memory.
VCENC_FRAME_READY	A frame encoding was finished.
VCENC_HW_BUS_ERROR	Error. This can be caused by invalid bus addresses that push the encoder to access an invalid memory area. New frame encoding must be started.
VCENC_HW_RESERVED	Error, the hardware could not be reserved for exclusive access.
VCENC_HW_RESET	Error, the hardware was reset by external means. The whole frame is lost.
VCENC_HW_TIMEOUT	Error, the wait for a hardware finish has timed out. The current frame is lost. New frame encoding must be started.
VCENC_INSTANCE_ERROR	Error, the encoder instance is invalid or corrupted.
VCENC_INVALID_ARGUMENT	Error, one of the arguments was invalid. None of the arguments was set.
VCENC_INVALID_STATUS	Error, the stream was started with Hypothetical Reference Decoder (HRD) enabled and consequently, the rate control parameters cannot be altered.
VCENC_MEMORY_ERROR	Error, the encoder was not able to allocate memory.
VCENC_NULL_ARGUMENT	Error, a pointer argument had an invalid NULL value.
VCENC_OUTPUT_BUFFER_OVERFLOW	Error, the output buffer's size was too small to fit the generated stream. Allocate a bigger buffer and try again.
VCENC_SYSTEM_ERROR	Error, a fatal system error occurred. The encoding can't continue. The encoder instance must be released.

9.2.6. Enumerations Used in the API

VCEncColorConversion Enumeration

Specifies the RGB to YUV conversion type to be used. Used in the VCEncPreProcessingCfg data structure.

VCEncColorConversion Values	Description
VCENC_RGBTOYUV_BT601	Conversion according to ITU-R Recommendation ITU-R-REC-BT.601
VCENC_RGBTOYUV_BT709	Conversion according to ITU-R Recommendation ITU-R-REC-BT.709
VCENC_RGBTOYUV_USER_DEFINED	Conversion using user-defined coefficients coeffA, coeffB, coeffC, coeffE and coeffF in formula: $Y = \frac{(coeffA * R + coeffB * G + coeffC * B)}{65536}$ $Cb = \frac{(coeffE * (B - Y))}{65536 + 128}$ $Cr = \frac{(coeffF * (R - Y))}{65536 + 128}$

VCEncProfile Enumeration

Specifies the HEVC(H.265)/AVC(H.264) profile of the generated stream. Used in the *VCEncConfig* structure.

VCEncProfile Values	Description
VCENC_HEVC_MAIN_PROFILE	Allows for a bit depth of 8 bits per sample with 4:2:0 chroma sampling.
VCENC_HEVC_MAIN_STILL_PICTURE_PROFILE	Allows for a single still picture to be encoded with the same constraints as the Main profile.
VCENC_HEVC_MAIN_10_PROFILE	Allows for a bit depth of 8-bits to 10-bits per sample with 4:2:0 chroma sampling.
VCENC_H264_MAIN_PROFILE	Used for standard-definition digital TV broadcasts that use the MPEG-4 format.
VCENC_H264_HIGH_PROFILE	The primary profile for broadcast and disc storage applications, particularly for high-definition TV applications.
VCENC_H264_HIGH_10_PROFILE	This profile builds on top of the High Profile, adding support for up to 10 bits per sample.

VCEncLevel Enumeration

Specifies the HEVC/H.264 level of the generated stream. Used in the *VCEncConfig* structure.

Table 18. HEVC/AVC Levels

VCEncLevel Values	Encoded Picture Size	Frame Rate (fps)	Max Luma Picture Size (samples)	Max Luma Sample Rate (samples/sec)	Max Bit Rate (Kbps)
HEVC (H265)					
VCENC_HEVC_LEVEL_1	QCIF	15	36,864	552,960	128
VCENC_HEVC_LEVEL_2	CIF	30	122,880	3,686,400	1500
VCENC_HEVC_LEVEL_2_1	Q720p	30	245,760	7,372,800	3000
VCENC_HEVC_LEVEL_3	QHD	30	552,960	16,588,800	6000
VCENC_HEVC_LEVEL_3_1	1280x720	30	983,040	33,177,600	10000
VCENC_HEVC_LEVEL_4	2Kx1080	30	2,228,224	66,846,720	12000
VCENC_HEVC_LEVEL_4_1	2Kx1080	60	2,228,224	133,693,440	20000
VCENC_HEVC_LEVEL_5	4096x2160	30	8,912,896	267,386,880	25000
VCENC_HEVC_LEVEL_5_1	4096x2160	60	8,912,896	534,773,760	40000
VCENC_HEVC_LEVEL_5_2	4096x2160	120	8,912,896	1,069,547,520	60000
VCENC_HEVC_LEVEL_6	8192x4320	30	35,651,584	1,069,547,520	60000
VCENC_HEVC_LEVEL_6_1	8192x4320	60	35,651,584	2,139,095,040	120000
VCENC_HEVC_LEVEL_6_2	8192x4320	120	35,651,584	4,278,190,080	240000
AVC (H264)					
VCENC_H264_LEVEL_1	Sub-QCIF	15	99 (QCIF)	1,485	64
VCENC_H264_LEVEL_1	QCIF	15	99 (QCIF)	1,485	64
VCENC_H264_LEVEL_1_b	Sub-QCIF	15	99 (QCIF)	1,485	128
VCENC_H264_LEVEL_1_b	QCIF	15	99 (QCIF)	1,485	128
VCENC_H264_LEVEL_1_1	QCIF	30	396 (QCIF)	3000	192
VCENC_H264_LEVEL_1_1	QVGA	10	396 (QCIF)	3000	192
VCENC_H264_LEVEL_1_2	QVGA	20	396 (QCIF)	6000	384
VCENC_H264_LEVEL_1_2	CIF	15	396 (QCIF)	6000	384
VCENC_H264_LEVEL_1_3	QVGA	30	396 (QCIF)	11,880	768
VCENC_H264_LEVEL_1_3	CIF	30	396 (QCIF)	11,880	768
VCENC_H264_LEVEL_2	CIF	30	396 (QCIF)	11,880	2000
VCENC_H264_LEVEL_2_1	512x384	25	792	19,800	4000
VCENC_H264_LEVEL_2_2	VGA	15	1,620 (PAL)	20,250	4,000
VCENC_H264_LEVEL_2_2	720x480	15	1,620 (PAL)	20,250	4,000
VCENC_H264_LEVEL_3	VGA	30	1,620 (PAL)	40,500	10,000
VCENC_H264_LEVEL_3	720x576	25	1,620 (PAL)	40,500	10,000
VCENC_H264_LEVEL_3	720x480	30	1,620 (PAL)	40,500	10,000
VCENC_H264_LEVEL_3_1	1280x720	30	3,600 (HD 720p)	108,000	14,000

Continued on next page

Table 18 – continued from previous page

VCEncLevel Values	Encoded Picture Size	Frame Rate (fps)	Max Luma Picture Size (samples)	Max Luma Sample Rate (samples/sec)	Max Bit Rate (Kbps)
VCENC_H264_LEVEL_3_2	1280x1024	30	5,120 (SXGA)	216,000	20,000
VCENC_H264_LEVEL_4	1920x1088	30	8,192 (1080p)	245,760	20,000
VCENC_H264_LEVEL_4_1	1920x1088	30	8,192 (1080p)	245,760	50,000
VCENC_H264_LEVEL_4_2	1920x1088	60	8,192 (1080p)	491,520	50,000
VCENC_H264_LEVEL_5	2048x1088	60	22,080	589,824	135,000
VCENC_H264_LEVEL_5_1	3840x2160	15	36,864	983,040	240,000

VCEncPictureCodingType Enumeration

Specifies the encoding type for the specified picture. Used in the *VCEncIn*, *VCEncOut* and *VCEncGopPicConfig* data structures.

VCEncPictureCodingType Values	Description
VCENC_INTRA_FRAME	The picture should be INTRA coded.
VCENC_PREDICTED_FRAME	The picture should be INTER coded using the previous picture as a predictor.
VCENC_BIDIR_PREDICTED_FRAME	The picture should be INTER coded using the backward and/or forward pictures as predictors.

VCEncPictureRotation Enumeration

Specifies the YUV picture rotation before encoding. Used in the *VCEncPreProcessingCfg* data structure.

VCEncPictureRotation Values	Description
VCENC_ROTATE_0	No rotation.
VCENC_ROTATE_90R	Rotates the picture clockwise by 90 degrees before the encoding.
VCENC_ROTATE_90L	Rotates the picture counter-clockwise by 90 degrees before the encoding.
VCENC_ROTATE_180R	Rotates the picture clockwise by 180 degrees before the encoding.

VCEncPictureType Enumeration

Specifies the input picture format. Used in the *VCEncPreProcessingCfg* data structure.

The formats are described in the *Video Frame Storage Format* section.

VCEncPictureType Values	Description
VCENC_YUV420_PLANAR	
VCENC_YUV420_SEMIPLANAR	
VCENC_YUV420_SEMIPLANAR_VU	
VCENC_YUV422_INTERLEAVED_YUYV	
VCENC_YUV422_INTERLEAVED_UYVY	
VCENC_RGB565	
VCENC_BGR565	
VCENC_RGB555	
VCENC_BGR555	
VCENC_RGB444	
VCENC_BGR444	
VCENC_RGB888	
VCENC_BGR888	
VCENC_RGB101010	
VCENC_BGR101010	
VCENC_YUV420_PLANAR_10BIT_I010	
VCENC_YUV420_PLANAR_10BIT_P010	
VCENC_YUV420_PLANAR_10BIT_PACKED_PLANAR	
VCENC_YUV420_10BIT_PACKED_Y0L2	

VCEncStreamType Enumeration

Specifies the type of the stream generated. Used in the *VCEncConfig* structure.

VCEncStreamType Values	Description
VCENC_BYTE_STREAM	The encoder will produce the stream in byte stream format.
VCENC_NAL_UNIT_STREAM	The encoder will produce the stream in NAL units.

9.2.7. Structures

VCEncApiVersion

A structure containing the API's major and minor version number.

VCEncApiVersion Members	Type	Description
major	u32	The major version number.
minor	u32	The minor version number.

VCEncBuild

A structure containing the encoder's build information.

VCEncBuild Members	Type	Description
swBuild	u32	The internal release version of the encoder software.
hwBuild	u32	The internal release version of the encoder hardware.

VCEncCodingCtrl

A structure containing the encoder's coding parameters.

VCEncCodingCtrl Members	Type	Description
sliceSize	u32	Sets the size of a slice in CTB rows. A zero value disables the use of slices. This parameter can be updated during the encoding process, between any picture encoding. Valid value range: [0, height/ctu_size] Default value is 0.
seiMessages	u32	Enables insertion of picture timing and buffering period SEI messages into the stream in the beginning of every encoded frame. Valid value range: [0, 1] Default value is 0 (disabled).

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
vuivideoFullRange	u32	Input video signal sample range. Valid value range: [0, 1] 0 = Y range in [16, 235], Cb&Cr range in [16, 240] 1 = Y, Cb and Cr range in [0, 255] Default value is 1
disableDeblockingFilter	u32	Value of disable_deblocking_filter_idc. Valid value range: [0, 1] 0 = Inloop deblocking filter enabled (best quality). 1 = Inloop deblocking filter disabled Default value is 0.
tc_Offset	i32	Deblocking filter TC offset. Valid value range: [-6, 6] Default value is -2.
beta_Offset	i32	Deblocking filter beta offset. Valid value range: [-6, 6] Default value is 5.
enableDeblockOverride	u32	Deblocking override enable flag. Valid value range: [0, 1] 0 = Disable override 1 = enable override Default value is 0. (Invalid for H.264; instead, use deblockOverride in slice header)
deblockOverride	u32	Deblocking override flag. Valid value range: [0, 1] 0 = do not override filter parameter 1 = override filter parameter. Default value is 0.

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
enableSao	u32	Disable or enable SAO Filter. Valid value range: [0, 1] 0 = SAO disabled 1 = SAO enabled. Default value is 1. (Invalid for H.264)
enableScalingList	u32	Specifies whether to use average or default scaling list. Valid value range: [0, 1] 0 = average scaling list. 1 = default scaling list Default value is 0.
sampleAspectRatioWidth	u32	Horizontal size of the sample aspect ratio (in arbitrary units), 0 for unspecified. Valid value range: [0, 65535]
sampleAspectRatioHeight	u32	Vertical size of the sample aspect ratio (in arbitrary units), 0 for unspecified. Valid value range: [0, 65535]
enableCabac	u32	[0,1] H.264 entropy coding mode: 1 for CAVLC, 1 for CABAC
cabacInitFlag	u32	Context Adaptive Binary Arithmetic Coding (CABAC) table initial flag. The cabac_init_flag will be 0 regardless of the SW setting. Valid value range: [0, 1] (This feature is currently not supported by the hardware.)

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
cirStart	u32	Sets the first CTB for Cyclic Intra Refresh (CIR). CIR forces some of the CTBs to be intra coded and can be used to improve stream error recovery. Valid value range: [0, ctbPerFrame-1]. Default value is 0.
cirInterval	u32	Sets the CTB interval for Cyclic Intra Refresh. A zero value disables CIR. CIR forces CTBs with numbers: cirStart, cirStart+cirInterval, cirStart+2*cirInterval, ... to be intra coded. Valid value range: [0, ctbPerFrame]. Default value is 0 (CIR disabled).
pcm_enabled_flag	i32	
pcm_sample_bit_depth_luma_minus1	i32	
pcm_sample_bit_depth_chroa_minus1	i32	
pcm_loop_filter_disabled_flag	i32	
intraArea	VCEncPictureArea	Specifies a rectangular area of a Coding Tree Block (CTB) to be forced as intra coded. All CTBs inside the area and including the coordinates will be intra coded. Default value: disabled (intraArea.enable=0).

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
ipcm1Area	VCEncPictureArea	First area for forcing IPCM macroblocks
ipcm2Area	VCEncPictureArea	Second area for forcing IPCM macroblocks
roi1Area	VCEncPictureArea	Specifies a rectangular area of CTBs to be used as a Region-Of-Interest (ROI) with modified Quantization Parameter (QP) value. All CTBs inside the area and including the coordinates will be coded using the roi1DeltaQp value to modify the default picture QP. Default value: disabled (intraArea.enable=0).
roi2Area	VCEncPictureArea	Specifies a rectangular area of CTBs to be used as a second Region-Of-Interest with modified QP value. All CTBs inside the area and including the coordinates will be coded using the roi2DeltaQp value to modify the default picture QP. Default value: disabled (intraArea.enable=0).
roi1DeltaQp	i32	Specifies the QP delta value for the first ROI area. The QP to be used will be: qpHdr + roi1DeltaQp. Valid value range: [-15, 0] Default value is 0 (disabled).

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
roi2DeltaQp	i32	Specifies the QP delta value for the second ROI area. The QP to be used will be: qpHdr + roi2DeltaQp. Valid value range: [-15, 0] Default value is 0 (disabled).
fieldOrder	u32	Specifies the field order for interlaced coding. 0 = bottom field first 1 = top field first Valid value range: [0, 1]. Default value: 0.
chroma_qp_offset	i32	Chroma QP offset. Valid value range: [-12, 12]. Default value: 0.
roiMapDeltaQpEnable	u32	Enable ROI map function. 0 = disable, 1 = enable Valid value range: [0, 1]. Default value: 0.
roiMapDeltaQpBlockUnit	u32	Specifies ROI map block unit size. 0 = 64x64 1 = 32x32 2 = 16x16 3 = 8x8 Valid value range for HEVC: [0, 3]. Valid value range for H.264: [0, 2]. Default value: 0.
noiseReductionEnable	u32	Obsolete, not supported. Default value: 0.

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
noiseLow	u32	Specifies the low boundary for noise estimation. Valid value range: [1, 30]. Default value: 10.
firstFrameSigma	u32	Specifies the initial noise sigma. Valid value range: [1, 30]. Default value: 11.
gdrDuration	u32	Gradual Decoding Refresh (GDR) duration. A value of 0 means that GDR feature is disabled. Nonzero values define the amount of pictures (frames not fields) that it will take to do GDR. The starting point for GDR is the frame which has codingType set to VCENC_INTRA_FRAME. However, this frame will be encoded as VCENC_PREDICTED_FRAME. intraArea and roi1Area are used to implement the GDR function, therefore these features cannot be used if GDR is enabled. GDR function is not supported with B-frames. Valid value range: [0, 0xFFFFFFFF].
codecH264	i32	Enable H.264 encoding. 0 = HEVC 1 = H.264
inputLineBufEn	u32	For low latency: enable input image control signals
inputLineBufLoopBackEn	u32	For low latency: input buffer loopback mode enable

Continued on next page

Table 19 – continued from previous page

VCEncCodingCtrl Members	Type	Description
inputLineBufDepth	u32	For low latency: input buffer depth in MB lines
inputLineBufHwModeEn	u32	For low latency: hardware handshake
inputLineBufCbFunc	EncInputLineBufCallbackFunc	For low latency: callback function
inputLineBufCbData	void *	For low latency: callback function data
vuiColorDescription	VuiColorDescription	The color description in the VUI.
vuiVideoSignalTypePresentFlag	U32	Whether to present the video signal type in the VUI flag. 0 default: do not present 1 present
vuiVideoFormat	U32	The video format in the VUI. 0: component 1: PAL 2: NTSC 3: SECAM 4: MAC 5 (default): UNDEF

VCEncConfig

A structure containing the encoder's initial configuration parameters.

VCEncConfig Members	Type	Description
streamType	VCEncStreamType	The type of the stream generated.
profile	VCEncProfile	The HEVC/H.264 profile of the generated stream.
level	VCEncLevel	The HEVC/H.264 level of the generated stream. For more information refer to the Initialization and release of the encoder section.
width	u32	The width of the encoded image in pixels. Horizontal stride is assumed to be the next multiple of 16 larger than or equal to this value. Refer to limitations listed in the Encoder Limitations section. Valid value range: [130, 4096]
height	u32	The height of the encoded image in pixels. Refer to limitations listed in the Encoder Limitations section. Valid value range: [130, 8192]
frameRateNum	u32	The numerator part of the input frame rate. The frame rate is defined by the frameRateNum/frameRateDenom ratio. This value also defines the time resolution as ticks per second. Valid value range: [1, 1048575]
frameRateDenom	u32	The denominator part of the input frame rate. This value must be equal or less than the numerator part frameRateNum. Valid value range: [1, 65535]
refFrameAmount	u32	Specifies the amount of reference

VCEncGopConfig

A structure which contains all GOP structures that will be used throughout the sequence.

VCEncGopConfig Members	Type	Description
pGopPicCfg	VCEncGopPicConfig	Pointer to an array of structure VCEncGopPicConfig. Each VCEncGopPicConfig structure contains the Group of Pictures (GOP) configuration of one picture.
size	u8	Number of GOP structures VCEncGopPicConfig pointed to by pGopPicCfg.
id	u8	Index of structure VCEncGopPicConfig used by the current picture, ranging from 0 to size-1.
id_next	u8	Index of structure VCEncGopPicConfig used by the next picture, ranging from 0 to size-1. (H.264 only, used for MMO)
delta_poc_to_next	i32	Picture Order Count (POC) difference between the next picture and the current picture. (H.264 only, used for MMO)
ltrInterval	i32	Long term reference picture interval. 0: disable > 0: gap between 2 long term reference pictures in display order

VCEncHEVCCuOutData [HEVC only]

A structure which contains the Coding Unit (CU) information of a picture output by hardware.

VCEncHEVCCuOutData Members	Type	Description
ctuOffset	u32	Pointer to the memory containing the total Coding Unit (CU) number by the end of each Coding Tree Unit (CTU).
cuData	u8	Pointer to the CU information of a picture.

VCEncHEVCCuInfo [HEVC only]

A structure that contains the parsed Coding Unit (CU) information.

VCEncHEVCCuInfo Members	Type	Description
cuLocationX	u8	CU x-coordinate relative to Coding Tree Unit (CTU).
cuLocationY	u8	CU y-coordinate relative to CTU.
cuSize	u8	CU size: 8/16/32/64
cuMode	u8	CU mode: 0:INTER 1:INTRA
cuSadCost	u32	CU Sum of Absolute Difference (SAD) cost
interPredIdc	u8	Prediction direction - only for INTER CU. 0: by list0 1: by list1 2: bi-direction
mv[2]	VCEncHEVCMv	Motion information - only for INTER CU. mv[0] for list0 if it's valid mv[1] for list1 if it's valid
intraPartMode	u8	Partition mode - only for INTRA CU. 0:2Nx2N 1: NxN
intraPredMode[4]	u8	Prediction mode - only for INTRA CU. 0: planar 1: DC 2-34: Angular in HEVC spec intraPredMode[1~3] only valid for NxN

VCEncHEVCMv [HEVC only]

A structure that contains the motion information for the INTER Coding Unit.

VCEncHEVCMv Members	Type	Description
refIdx	u8	Reference index in reference list.
mvX	i16	Horizontal motion in 1/4 pixel.
mvY	i16	Vertical motion in 1/4 pixel.

VCEncIn

This data structure is common for all the API calls which generate a stream. Not all fields are used when starting a stream.

VCEncln Members	Type	Description
busLuma	u32	Bus address of the buffer where the input picture's luminance component for YCbCr420_PLANAR and YCbCr420_SEMIPLANAR formats or the whole input picture for YCbYCr422_INTERLEAVED or RGB formats is located.
busChromaU	u32	Bus address of the buffer where the input picture's first chrominance component (for YCbCr420_PLANAR) or both chrominance components (for YCbCr420_SEMIPLANAR) is located.
busChromaV	u32	Bus address of the buffer where the input picture's second chrominance component (for YCbCr420_PLANAR) is located.
timeIncrement	u32	Time stamp of the frame relative to the last encoded frame. This is given in ticks (one tick equals 1/frameRateNum seconds). A zero value is usually used for the very first frame. The ticks per second were set at the encoder initialization phase (frameRateNum). For correct rate control functioning it is important to set this value appropriately. The typical value for a fixed frame rate video is: timeIncrement=frameRateDenom.
pOutBuf	u32	Pointer to the output buffer which will be used to store the generated stream. Must be a linear, memory residing buffer and 8-byte aligned.
busOutBuf	u32	Bus address of the output buffer, required by the hardware operations.

VCEncOut

This data structure is common for all the API calls which generate a stream. Not all fields are used when starting a stream.

VCEncOut Members	Type	Description
codingType	VCEncPictureCodingType	Encoding type for the specified picture.
streamSize	u32	Actual size of the generated stream in bytes is stored here. If the call was unsuccessful then this value is not relevant and it shall be ignored.
pNaluSizeBuf	u32	Pointer to a buffer where the encoder has stored the sizes of all the created Network Abstraction Layer (NAL) units. A zero value is stored after the last NAL unit size value.
numNalus	u32	Number of NAL units created.
busScaledLuma	u32	Bus address of the buffer where the down-scaled picture is stored. The picture format is YCbYCr422_INTERLEAVED.
scaledPicture	u8	Pointer to a buffer where the down-scaled picture is stored.
cuOutData	VCEncHEVCCuOutData	Coding Unit (CU) information of a picture output by hardware.

VCEncPictureArea

A structure defining the rectangular area of a Coding Tree Block (CTB).

VCEncPictureArea Members	Type	Description
enable	u32	Enable the rectangular area of a CTB. Valid value range: [0,1] 0: Disable 1: Enable
top	u32	Y-coordinate of the upper-left corner of the rectangular area of the CTB. Valid value range: [0, ctbPerColumn-1].
left	u32	X-coordinate of the upper-left corner of the rectangular area of the CTB. Valid value range: [0, ctbPerRow-1].
bottom	u32	Y-coordinate of the lower-right corner of the rectangular area of the CTB. Valid value range: [top, ctbPerColumn-1].
right	u32	X-coordinate of the lower-right corner of the rectangular area of the CTB. Valid value range: [left, ctbPerRow-1].

VCEncPreProcessingCfg

A structure which contains a pre-processing block's parameters.

For more information, refer to Video Pre-Processor Usage Section 4.7. If an invalid configuration value will be set, the cropping block will keep its old configuration.

When the video stabilization is in use, the xOffset and yOffset values hold the latest stabilization result.

VCEncPreProcessingCfg Members	Type	Description
origWidth	u32	Input image's full width. This size must be equal to or larger than the encoded image's width specified during the encoder initialization phase (See VCEncInit()). A zero value disables the cropping. It is also restricted depending on the input image format (See Video Frame Storage Format). Horizontal stride is assumed to be the next multiple of 16 larger than or equal to this value. Valid value range: 0 or [130, 8192]
origHeight	u32	Input image's full height. This size must be larger than or equal to the encoded image's height specified at the encoder initialization phase (See VCEncInit()). A zero value disables the cropping. Valid value range: 0 or [130, 8192]
xOffset	u32	Horizontal offset from the top-left corner of the input image to the top-left corner of the encoded image. Note: the minimum encoded picture width is 130. Valid value range: [0, 8062]
yOffset	u32	Vertical offset from the top-left corner of the input image to the top-left corner of the encoded image. Note: the minimum encoded picture height is 130. Valid value range: [0, 8062]
inputType	VCEncPictureType	Input picture format type.
Page 227 rotation	Release 1.11.0 Confidential and Proprietary Copyright © 2026 – Hailo Technologies Ltd. VCEncPictureRotation	YUV picture rotation before encoding.

VCEncPPSCfg

A structure that contains the user-provided Picture Parameter Set (PPS) parameters.

VCEncPPSCfg Members	Type	Description
chroma_qp_offset	i32	Chroma Quantization Parameter (QP) offset.
tc_Offset	i16	Deblock parameter, tc_offset.
beta_Offset	i16	Deblock parameter, beta_offset.

VCEncRateCtrl

A structure containing the rate control parameters.

VCEncRateCtrl Members	Type	Description
pictureRc	u32	Enables picture level rate control to adjust QP between frames. This should be enabled if target bit rate is set. Valid value range: [0, 1] 0 = disable 1 = enable Default value: 0
ctbRc	u32	Enables CTB level rate control to adjust QP between CTBs within frame. Valid value range: [0, 1] 0 = disable 1 = enable Default value: 0
blockRcSize	u32	Set the unit of block size to adjust QP. Valid value range for HEVC: [0, 2] Valid value range for H.264: [2, 2] 0: 64x64 1: 32x32 2: 16x16
pictureSkip	u32	Allow rate control to skip pictures if not enough bits are available. When Hypothetical Reference Decoder (HRD) is enabled, the rate control may have to skip frames despite this value. Valid value range: [0, 1]
qpHdr	i32	The initial Quantization Parameter (QP) used by the encoder. If the rate control is enabled then this value is used only at the beginning of the encoding process. When the rate control is disabled then this QP value is used all the time. -1 lets RC calculate initial QP. Not recommended to be set lower than 10. Valid value range: -1 or [0, 51] Default value: 26

VCEncSliceReady

A structure which contains information to pass to the slice ready callback function. This callback function is called after a slice encoding has been finished by the hardware.

```
typedef void (*VCEncSliceReadyCallBackFunc)(VCEncSliceReady *);
```

VCEncSliceReady Members	Type	Description
slicesReadyPrev	u32	Number of slices that were ready at the time of previous callback.
slicesReady	u32	Number of slices that are ready and accessible in the output buffer.
nalUnitInfoNum	u32	Number of information Network Abstraction Layer (NAL) units are completed, including all kinds of Supplemental Enhancement Information (SEI).
sliceSizes	u32	Pointer to a buffer where the encoder has stored the sizes (in bytes) of all the finished slices.
pOutBuf	u32	Pointer to the beginning of the output stream buffer storing the generated stream.
pAppData	void	Pointer to application specific data.

VCEncGopPicConfig

A structure which contains the GOP structure of one picture.

VCEncGopPicConfig Members	Type	Description
poc	u32	Picture Order Count (POC): Display order count within a GOP, ranging from 1 to gopSize.
QpOffset	i32	Quantization Parameter (QP) offset is added to the QP parameter to set the final QP value.
QpFactor	double	Weight used during rate distortion optimization. Higher values mean lower quality and less bits.
temporalId	i32	temporal ID of frame in multi-layer stream. (H.264 not supported yet)
codingType	VCEncPictureCodingType	Picture coding type, can be either Predicted or Bidirectional Predicted.
numRefPics	u32	Number of reference pictures kept for this picture, including references pictures used by current and future pictures.
refPics[VCENC_MAX_REF_FRAMES]	VCEncGopPicRps	An array that defines the Reference Picture Sets (RPS) of this picture. Each array member describes one reference picture, so the number of valid array member is numRefPics.

VCEncGopPicRps

A structure which contains one reference picture.

VCEncGopPicRps Members	Type	Description
ref_pic	i32	Delta Picture Order Count (POC) of this reference picture, relative to the POC of the current picture.
used_by_cur	u32	Specifies whether this reference picture is used by the current picture (1) or not (0).

VuiColorDescription

Color description in the vui which coded in the sps(sequence parameter sets).

Only valid when video signal type present flag in the vui is set.

VuiColorDescription Members	Type	Description
vuiColorDescripPresentFlag	8u	Color description present in the vui.0- not present, 1- present
vuiColorPrimaries	8u	Color's Primaries
vuiTransferCharacteristics	U8	Transfer Characteristics
vuiMatrixCoefficients	U8	Matrix Coefficients

9.2.8. Video Encoder Functions

These functions are not listed in alphabetical order, but rather in an order in which they are likely to be used.

VCEncGetApiVersion

Description:

Returns the encoder's API version information.

Syntax:

VCEncApiVersion **VCEncGetApiVersion** (void);

Parameters:

None

Returns

VCEncApiVersion: a structure containing the API's major and minor version number.

VCEncGetBuild

Description:

Returns the hardware and software build information of the encoder.

Does not require encoder initialization.

Syntax:

VCEncBuild **VCEncGetBuild** (void);

Parameters:

None

Returns

VCEncBuild structure

VCEncInit

Description:

Initializes the encoder and returns an instance of it. The configuration contains stream parameters that can't be altered during encoding.

Syntax:

VCEncRet **VCEncInit** (

const VCEncConfig *pEncCfg,

VCEncInst *instAddr

);

Parameters:

pEncCfg	Points to the <i>VCEncConfig</i> structure that contains the encoder's initial configuration parameters.
instAddr	Points to a space where the new encoder instance pointer will be stored.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INVALID_ARGUMENT, VCENC_MEMORY_ERROR, VCENC_EWL_ERROR, VCENC_EWL_MEMORY_ERROR

VCEncRelease

Description:

Releases an encoder instance. This will free all the resources allocated at the encoder initialization phase.

Syntax:

VCEncRet **VCEncRelease** (

 VCEncInst inst

);

Parameters:

inst The encoder instance to be released. This instance was created earlier with a call to [VCEncInit\(\)](#).

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR

VCEncSetCodingCtrl

Description:

Sets the encoder's coding parameters. All of the parameters can be adjusted before the stream is started. The following parameters can also be altered between frames:

- sliceSize
- cirStart
- cirInterval
- intraArea
- roiDeltaQp

If the following parameters need to be altered between frames,

- roiArea
- roiMapDeltaQpEnable
- gdrDuration

cu_qp_delta_enabled_flag in PPS will be reconfigured, so the following steps should be done:

- [VCEncStrmEnd\(\)](#) : to end the last sequence
- [VCEncSetCodingCtrl\(\)](#) : to change parameters
- [VCEncStrmStart\(\)](#) to output a new Picture Parameter Set (PPS).

then, call

- [VCEncStrmEncode\(\)](#) to encode the next frame.

Syntax:

VCEncRet **VCEncSetCodingCtrl** (

```

    VCEncInst inst,
    const VCEncCodingCtrl *pCodeParams
);

```

Parameters:

inst	The instance that defines the encoder in use.
pCodeParams	Pointer to the VCEncCodingCtrl structure that contains the encoder's coding parameters.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

VCEncGetCodingCtrl

Description:

Returns the current coding parameters in use by the encoder.

Syntax:

```

VCEncRet VCEncGetCodingCtrl (
    VCEncInst inst,
    const VCEncCodingCtrl *pCodeParams
);

```

Parameters:

inst	The instance that defines the encoder in use.
pCodeParams	Pointer to the VCEncCodingCtrl structure where the encoder's coding parameters will be saved.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR

VCEncSetRateCtrl

Description:

Sets the rate control parameters of the encoder. The function should be called before starting the stream.

If the Hypothetical Reference Decoder (HRD) is disabled, the function can also be called between frames to set new rate control parameters except the ctbRc parameter. This however will reset the rate control and is recommended to do so at the end of Group of Pictures (GOP). To modify ctbRc between frames, cu_qp_delta_enabled_flag in the Picture Parameter Set (PPS) will be reconfigured, so the following steps should be done:

- [VCEncStrmEnd\(\)](#): to end the last sequence
- [VCEncSetRateCtrl\(\)](#) to change ctbRc
- [VCEncStrmStart\(\)](#) to output a new PPS.

Then, call

- [VCEncStrmEncode\(\)](#): to encode the next frame.

In general, `ctbRc` should not be changed between frames.

Syntax:

```
VCEncRet VCEncSetRateCtrl (
    VCEncInst inst,
    const VCEncRateCtrl *pRateCtrl
);
```

Parameters:

inst	The instance that defines the encoder in use.
pRateCtrl	Pointer to the VCEncRateCtrl structure where the new rate control parameters are set.

Returns

[VCEncRet](#) value: `VCENC_OK`, `VCENC_NULL_ARGUMENT`, `VCENC_INSTANCE_ERROR`, `VCENC_INVALID_ARGUMENT`, `VCENC_INVALID_STATUS`

VCEncGetRateCtrl

Description:

Returns the current rate control parameters in use by the encoder.

Syntax:

```
VCEncRet VCEncGetRateCtrl (
    VCEncInst inst,
    const VCEncRateCtrl *pRateCtrl
);
```

Parameters:

inst	The instance that defines the encoder in use.
pRateCtrl	Pointer to the VCEncRateCtrl structure where the new rate control parameters will be saved.

Returns

[VCEncRet](#) value: `VCENC_OK`, `VCENC_NULL_ARGUMENT`, `VCENC_INSTANCE_ERROR`

VCEncStrmStart

Description:

Starts a new stream and generates the Video Parameter Set (VPS), Sequence Parameter Set (SPS) and Picture Parameter Set (PPS). The VPS is the first Network Abstraction Layer (NAL) unit, SPS is the second NAL unit and PPS is the third NAL unit.

Syntax:

```
VCEncRet VCEncStrmStart (
    VCEncInst inst,
    const VCEncIn *pEncIn,
    VCEncOut *pEncOut
);
```

);

Parameters:

inst	The instance that defines the encoder in use.
pEncIn	Pointer to the VCEncIn structure where the input parameters are provided.
pEncOut	Pointer to the VCEncOut structure where the output parameters will be stored.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT, VCENC_INVALID_STATUS, VCENC_OUTPUT_BUFFER_OVERFLOW

VCEncStrmEncode

Description:

Encodes a video frame.

Syntax:

```
VCEncRet VCEncStrmEncode (
    VCEncInst inst,
    const VCEncIn *pEncIn,
    VCEncOut *pEncOut,
    VCEncSliceReadyCallBackFunc cbFunc,
    void *pAppData )
);
```

Parameters:

inst	The instance that defines the encoder in use.
pEncIn	Pointer to the VCEncIn structure where the input parameters are provided.
pEncOut	Pointer to the VCEncOut structure where the output parameters will be saved.
cbFunc	Pointer to the VCEncSliceReady CallBackFunc callback function that will be called after a slice encoding has been finished by the hardware.
pApp-data	Pointer to application-specific data to be passed on to the callback function.

Returns

[VCEncRet](#) value: VCENC_FRAME_READY, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT, VCENC_INVALID_STATUS, VCENC_OUTPUT_BUFFER_OVERFLOW, VCENC_HW_TIMEOUT, VCENC_HW_BUS_ERROR, VCENC_HW_RESET, VCENC_HW_RESERVED, VCENC_SYSTEM_ERROR

VCEncStrmEnd

Description:

Ends a previously started stream. Stream end consists of a Network Abstraction Layer (NAL) unit of type 'End-of-Sequence'. After a stream is ended, a new encoding sequence must begin with [VCEncStrmStart\(\)](#).

Syntax:

```
VCEncRet VCEncStrmEnd (  
    VCEncInst inst,  
    const VCEncIn *pEncIn,  
    VCEncOut *pEncOut  
);
```

Parameters:

inst	The instance that defines the encoder in use.
pEncIn	Pointer to the VCEncIn structure where the input parameters are provided.
pEncOut	Pointer to the VCEncOut structure where the output parameters will be saved.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT, VCENC_INVALID_STATUS

VCEncSetPreProcessing

Description:

Sets the pre-processing block's parameters.

Syntax:

```
VCEncRet VCEncSetPreProcessing (  
    VCEncInst inst,  
    const VCEncPreProcessingCfg *pProcCfg  
);
```

Parameters:

inst	The instance that defines the encoder in use.
pPreProcCfg	Pointer to the VCEnc PreProcessingCfg structure where the new parameters are set.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT, VCENC_SYSTEM_ERROR

VCEncGetPreProcessing

Description:

Returns the current pre-processing block's parameters in use by the encoder.

Syntax:

```
VCEncRet VCEncGetPreProcessing (
    VCEncInst inst,
    VCEncPreProcessingCfg *pProcCfg
);
```

Parameters:

inst	The instance that defines the encoder in use.
pPreProcCfg	Pointer to the VCEncPreProcessingCfg structure where the new parameters are saved.

Returns

VCEncRet value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR

VCEncSetSeiUserData

Description:

Enables or disables writing user data to the encoded stream. The user data will be written in the stream as a Supplemental Enhancement Information (SEI) message connected to all the following encoded frames. The SEI message payload type is marked as user_data_unregistered.

Syntax:

```
VCEncRet VCEncSetSeiUserData (
    VCEncInst inst,
    const u8 *pUserData,
    u32 userDataSize
);
```

Parameters:

inst	The instance that defines the encoder in use.
pUserData	Pointer to a buffer containing the user data. The encoder stores a pointer to this buffer and reads data from the buffer during encoding of the following frames. Because the encoder reads data straight from this buffer, it must not be freed before disabling the user data writing.
userDataSize	Size of the data in the pUserData buffer in bytes. If a zero value is given, the user data writing is disabled. Invalid value disables user data writing. Valid value range: 0 or [16, 2048]

Returns

VCEncRet value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR

VCEncGetPerformance

Description:

Returns the current picture hardware performance.

Syntax:

```
u32 VCEncGetPerformance (
    VCEncInst inst
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
-------------	---

Returns

The number of clock cycles used to encode the previous frame.

VCEncHEVCCuInfo [HEVC only]

Description:

Returns the encoding information of a Coding Unit(CU) in a Coding Tree Unit (CTU). This function is only valid for HEVC.

Syntax:

```
VCEncRet VCEncHEVCCuInfo (
    VCEncInst inst,
    VCEncHEVCCuOutData *pEncCuOutData,
    VCEncHEVCCuInfo *pEncCuInfo,
    u32 ctuNum,
    u32 cuNum)
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
pEncCuOutData	Pointer to the VCEncHEVCCuOutData structure containing the CTU table and CU information stream output by the hardware.
pEncCuInfo	Pointer to the VCEncHEVCCuInfo structure returning the parsed CU information.
ctuNum	CTU number within picture.
cuNum	CU number within CTU.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_INVALID_ARGUMENT

VCEncSetInputMBLines

Description:

Low latency: sets the valid input MB lines for the encoder to work. This function is only valid for HEVC.

Syntax:

```
VCEncRet VCEncSetInputMBLines (  
    VCEncInst inst,  
    u32 lines)  
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
lines	Valid input MB lines to set.

Returns

[VCEncRet](#) value: VCENC_OK, NULL_ARGUMENT, VCENC_INVALID_ARGUMENT

VCEncGetEncodedMbLines

Description:

Low latency: returns the encoded lines information from the encoder. This function is only valid for HEVC.

Syntax:

```
U32 VCEncGetEncodedMbLines (  
    VCEncInst inst  
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .

Returns

The encoded lines information from the encoder.

VCEncCreateNewPPS

Description:

Creates a new Picture Parameter Set (PPS).

Syntax:

```
VCEncRet VCEncCreateNewPPS (  
    VCEncInst inst,  
    const VCEncPPSCfg *pPPSCfg,  
    i32 *newPPSId
```

);

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
pPPSCfg	Pointer to VCEncPPSCfg structure with user-provided PPS parameters.
newPP-SId	New PPS ID for user.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

VCEncModifyOldPPS

Description:

Modifies an existing Picture Parameter Set (PPS).

Syntax:

```
VCEncRet VCEncModifyOldPPS (
    VCEncInst inst,
    const VCEncPPSCfg *pPPSCfg,
    i32 ppsId
```

);

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
ppp-SCfg	Pointer to VCEncPPSCfg structure with user-provided PPS parameters.
ppsId	Old PPS ID provided by the user.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

VCEncGetPPSData

Description:

Returns the Picture Parameter Set (PPS) data for the user.

Syntax:

```
VCEncRet VCEncGetPPSData (
    VCEncInst inst,
    const VCEncPPSCfg *pPPSCfg,
    i32 ppsId
```

```
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
ppp-SCfg	Pointer to VCEncPPSCfg structure with user-provided PPS parameters returned.
ppsId	PPS ID provided by user.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

VCEncActiveAnotherPPS

Description:

Activates another Picture Parameter Set (PPS) for subsequent frames.

Syntax:

VCEncRet **VCEncActiveAnotherPPS** (

 VCEnclnst inst,

 i32 ppsId

```
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
ppsId	PPS ID provided by user.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

VCEncGetActivePPSId

Description:

Returns the ID of the active Picture Parameter Set (PPS).

Syntax:

VCEncRet **VCEncGetActivePPSId** (

 VCEnclnst inst,

 i32 *ppsId

```
);
```

Parameters:

inst	The encoder instance for which the performance is reported. The instance was created earlier with a call to VCEncInit() .
pp-sld	PPS ID returned to user.

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

1. Additional Test-Related Functions

VCEncInitGopConfigs

Description:

Initializes the Group of Pictures (GOP) configuration.

Syntax:

```
static int VCEncInitGopConfigs (
    int gopSize,
    commandLine_s *cml,
    VCEncGopConfig *gopCfg,
    u8 *gopCfgOffset
);
```

Parameters:

gopSize	GOP size.
cml	
gopCfg	Pointer to the VCEncGopConfig structure.
gopCfgOffset	

Returns

[VCEncRet](#) value: VCENC_OK, VCENC_NULL_ARGUMENT, VCENC_INSTANCE_ERROR, VCENC_INVALID_ARGUMENT

10. AI Analytics API

Hailo AI Analytics is a comprehensive software framework designed to facilitate the development and deployment of AI-powered video analytics applications on Hailo-15 AI Vision Processor Units (VPUs). It provides a robust set of tools, libraries, and APIs that enable developers to build efficient, scalable, and production-ready video analytics pipelines. The main components of Hailo AI Analytics include:

- **HailoRT** – A production-grade, scalable runtime environment that enables high-throughput AI inference on Hailo devices.
- **Media Library AI Analytics API** – A C++ API built on top of HailoRT, enabling developers to efficiently implement AI-powered video analytics functionality.
- **Media Library AI Analytics API reference examples** – Sample code demonstrating how to use the API to develop analytics applications.
- **Reference pipelines and applications** – Pre-built example pipelines and applications that can be used as reference implementations for building complete AI video analytics solutions. This section includes descriptions for the AI Analytics API and reference code examples for it.

10.1. Buffer

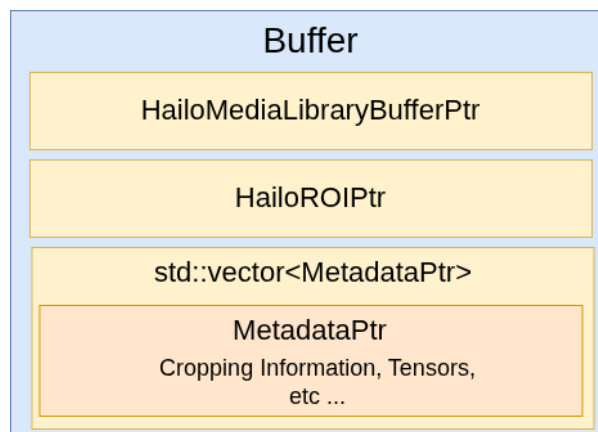
10.1.1. Overview

The `Buffer` class is a fundamental data structure in the Hailo AI Analytics pipeline, representing a single image frame along with its associated metadata and region of interest (ROI). Each `Buffer` encapsulates:

- **Raw Frame Data:** Wrapped in a `HailoMediaLibraryBufferPtr` object
- **Metadata:** Extensible metadata system for attaching additional information (tensors, batch info, size properties, etc.)
- **Region of Interest (ROI):** Analytics results and bounding box information via `HailoROI`

Buffers flow through various stages of the analytics pipeline, accumulating metadata and analytics results from different processing nodes such as inference engines, post-processing modules, and cropping components.

Key Concepts



Frame Representation

A Buffer represents one complete image frame. The underlying `HailoMediaLibraryBuffer` contains the actual pixel data (NV12).

Metadata System

The Buffer class provides an extensible metadata system that allows different pipeline components to attach additional information to a frame without modifying the core Buffer structure. Several metadata types are available:

1. **TensorMetadata:** Associates neural network tensor outputs with the frame
2. **BatchMetadata:** Tracks a buffer's position within a batch (index and total batch size)
3. **SizeMetadata:** Stores size-related information with descriptive labels
4. **BufferMetadata:** Attaches references to other buffers, enabling relationships between frames

This design allows pipeline stages to add context-specific information that downstream components can query and utilize.

Region of Interest (ROI)

Each Buffer contains a `HailoROI` object that stores all analytics results for regions within the frame:

- **Bounding box information** (initially covering the full frame)
- **Detection objects** with bounding boxes, labels, and confidence scores
- **Classification results** for image or region classification
- **Landmarks** for pose estimation and facial feature detection
- **Tracker IDs** for object tracking across frames
- **Segmentation masks** for pixel-level classification
- **Hierarchical structure** for nested analytics results (e.g., detections within detections)

The ROI is the central container for all analytics outputs and is essential for object detection, classification, tracking, pose estimation, segmentation, and other analytics workflows where specific regions of the frame need to be analyzed and annotated.

Typical Workflow

1. **Acquisition:** Buffers are typically acquired from a buffer pool to ensure efficient memory management
2. **Frame Capture:** Raw frame data is captured from a camera or cropped from another image into the buffer
3. **Processing:** The buffer flows through pipeline stages (inference, post-processing, tracking, etc.)
4. **Metadata Accumulation:** Each stage may add metadata (analytics results, cropping roi, etc.)
5. **Analysis:** Analytics components read the buffer content, ROI, and metadata to perform their functions
6. **Export:** Vision frames are encoded for streaming, and analytics results are exported to client devices
7. **Release:** When processing is complete, the buffer is released back to the pool for reuse

10.1.2. Buffer API Guide

This section describes the main classes and methods for working with Buffers in the Hailo AI Analytics pipeline.

Buffer Class

```
class hailo_analytics::pipeline::Buffer
```

Represents a single image frame with associated metadata and region of interest.

The *Buffer* class is the core data structure for passing image frames through the analytics pipeline. Each *Buffer* wraps a HailoMediaLibraryBuffer (the raw frame data) and provides functionality to:

- Attach and manage various types of metadata (tensors, batch info, size info, etc.)
- Store and retrieve region of interest (ROI) information with detection objects
- Enable frame processing and analysis throughout the pipeline

HailoMediaLibraryBuffer objects are typically acquired from a MediaLibraryBufferPool. After wrapping as a *Buffer*, they flow through various pipeline stages where they accumulate metadata from different processing nodes (inference, post-processing, etc.).

Public Functions

```
Buffer( HailoMediaLibraryBufferPtr buffer )
```

Constructs a *Buffer* from a HailoMediaLibraryBuffer.

Initializes a new *Buffer* with a full-frame ROI (bounding box covering the entire frame).

Parameters *buffer* – The underlying media library buffer containing the frame data

```
Buffer( Buffer &other )
```

Copy constructor for *Buffer*.

Creates a shallow copy of the buffer and metadata vector, but creates a deep copy of the ROI.

Parameters *other* – The *Buffer* to copy from

```
Buffer( HailoMediaLibraryBufferPtr buffer, HailoROIPtr roi )
```

Constructs a *Buffer* from a HailoMediaLibraryBuffer and a specified ROI.

Parameters

- *buffer* – The underlying media library buffer containing the frame data
- *roi* – The region of interest associated with this buffer

```
HailoMediaLibraryBufferPtr get_buffer( ) const
```

Gets the underlying media library buffer.

Returns Shared pointer to the HailoMediaLibraryBuffer containing the raw frame data

```
HailoROIPtr get_roi( ) const
```

Gets the region of interest (ROI) associated with this buffer.

Returns Shared pointer to the HailoROI object containing detection objects and bounding box information

```
void add_metadata( MetadataPtr metadata )
```

Adds metadata to this buffer.

Metadata can be used to store additional information like tensor data, batch information, size properties, or references to other buffers.

Parameters *metadata* – Shared pointer to the *Metadata* object to attach

```
void remove_metadata ( MetadataPtr metadata )
    Removes a specific metadata object from this buffer.
```

Parameters metadata – Shared pointer to the [Metadata](#) object to remove

```
std::vector<MetadataPtr> get_metadata_of_type ( MetadataType metadata_type ) const
    Retrieves all metadata of a specific type.
```

Parameters metadata_type – The MetadataType to filter by

Returns Vector of shared pointers to [Metadata](#) objects matching the specified type

Metadata Classes

MetadataType Enum

```
enum hailo_analytics::pipeline::MetadataType
    Enum class representing different types of metadata that can be attached to a Buffer.
```

Values:

```
enumerator UNKNOWN
    Unknown metadata type.
```

```
enumerator BUFFER
    Buffer metadata type.
```

```
enumerator TENSOR
    Tensor metadata type.
```

```
enumerator EXPECTED_CROPS
    Expected crops metadata type.
```

```
enumerator SIZE
    Size metadata type.
```

```
enumerator BATCH
    Batch metadata type.
```

```
enumerator SKIPPED_DETECTIONS
    Skipped detections metadata type (quality gate)
```

Base Metadata Class

```
class hailo_analytics::pipeline::Metadata
    Base class for metadata that can be attached to a Buffer.
```

[Metadata](#) provides additional information about a [Buffer](#) beyond its raw data content. Different metadata types can be attached to track various properties like size, tensors, batches, etc.

Subclassed by [hailo_analytics::pipeline::BatchMetadata](#), [hailo_analytics::pipeline::BufferMetadata](#),
[hailo_analytics::pipeline::CroppingMetadata](#), [hailo_analytics::pipeline::SizeMetadata](#),
[hailo_analytics::pipeline::SkippedDetectionsMetadata](#)

Public Functions

`Metadata ( MetadataType type = MetadataType::UNKNOWN )`
Constructs a `Metadata` object with the specified type.

Parameters `type` – The type of metadata (default: `MetadataType::UNKNOWN`)

`virtual ~Metadata ( ) = default`

`MetadataType get_type ( ) const`
Gets the metadata type.

Returns The `MetadataType` of this metadata object

SizeMetadata Class

`class hailo_analytics::pipeline::SizeMetadata : public hailo_analytics::pipeline::Metadata`
`Metadata` class for storing size information with a label.

This metadata type is used to attach size-related information to a `Buffer`, such as memory allocated or other size properties.

Public Functions

`SizeMetadata ( std::string label, size_t size )`
Constructs a `SizeMetadata` object.

Parameters

- `label` – A descriptive label for the size measurement
- `size` – The size value

`std::string get_label ( )`
Gets the label associated with this size metadata.

Returns The label string

`size_t get_size ( )`
Gets the size value.

Returns The size value

BufferMetadata Class

`class hailo_analytics::pipeline::BufferMetadata : public hailo_analytics::pipeline::Metadata`
`Metadata` class that contains a reference to another `Buffer`.

This metadata type allows attaching a `Buffer` reference to another `Buffer`, enabling relationships between buffers.

Subclassed by `hailo_analytics::pipeline::TensorMetadata`

Public Functions

`BufferMetadata ( BufferPtr buffer, MetadataType type = MetadataType::UNKNOWN )`
Constructs a *BufferMetadata* object.

Parameters

- `buffer` - The *Buffer* to attach as metadata
- `type` - The metadata type (default: *MetadataType::UNKNOWN*)

`BufferPtr get_buffer ( )`
Gets the attached buffer.

Returns Shared pointer to the attached *Buffer*

TensorMetadata Class

`class hailo_analytics::pipeline::TensorMetadata : public`

`hailo_analytics::pipeline::BufferMetadata`
Metadata class for tensor information attached to a *Buffer*.

This metadata type stores a tensor buffer along with its name, useful for tracking neural network tensor data associated with a frame.

Public Functions

`TensorMetadata ( BufferPtr buffer, std::string tensor_name )`
Constructs a *TensorMetadata* object.

Parameters

- `buffer` - The *Buffer* containing the tensor data
- `tensor_name` - The name of the tensor

`std::string get_tensor_name ( )`
Gets the tensor name.

Returns The name of the tensor

BatchMetadata Class

`class hailo_analytics::pipeline::BatchMetadata : public hailo_analytics::pipeline::Metadata`
Metadata class for batch processing information.

This metadata type stores information about a buffer's position within a batch, including the total batch size and the buffer's index within that batch.

Public Functions

`BatchMetadata ( size_t total_size, size_t index )`
Constructs a *BatchMetadata* object.

Parameters

- `total_size` - The total number of buffers in the batch
- `index` - The index of this buffer within the batch (0-based)

`size_t get_total_size ( )`
Gets the total batch size.

Returns The total number of buffers in the batch

`size_t get_index ( )`
Gets the buffer's index within the batch.

Returns The 0-based index of this buffer in the batch

10.1.3. Usage Examples

Creating and Using a Buffer

```
// Acquire a buffer from a media library buffer pool
// Given a MediaLibraryBufferPoolPtr named buffer_pool
HailoMediaLibraryBufferPtr ml_buffer = std::make_shared<hailo_media_library_
    ↳buffer>();
auto acquire_status = buffer_pool->acquire_buffer(ml_buffer);

// Create a Buffer object
BufferPtr buffer = std::make_shared<Buffer>(ml_buffer);

// Get the ROI for adding analytics results
HailoROIPtr roi = buffer->get_roi();

// Add analytics objects to the ROI
// (After inference or other processing)
```

Adding Metadata

```
// Add batch metadata
BatchMetadataPtr batch_metadata = std::make_shared<BatchMetadata>(batch_size, 0
    ↳batch_index);
buffer->add_metadata(batch_metadata);

// Add tensor metadata
BufferPtr tensor_buffer = std::make_shared<Buffer>(tensor_ml_buffer);
TensorMetadataPtr tensor_metadata = std::make_shared<TensorMetadata>(tensor_
    ↳buffer, "tensor_layer_name");
buffer->add_metadata(tensor_metadata);

// Add size metadata
SizeMetadataPtr size_metadata = std::make_shared<SizeMetadata>("original_width", 0
    ↳1920);
buffer->add_metadata(size_metadata);
```

Querying Metadata

```
// Get all tensor metadata
auto tensor_metadatas = buffer->get_metadata_of_type(MetadataType::TENSOR);

for (const auto& metadata : tensor_metadatas) {
    auto tensor_meta = std::dynamic_pointer_cast<TensorMetadata>(metadata);
    std::string tensor_name = tensor_meta->get_tensor_name();
    BufferPtr tensor_buffer = tensor_meta->get_buffer();
    // Process tensor data...
}

// Get batch metadata
auto batch_metadatas = buffer->get_metadata_of_type(MetadataType::BATCH);
if (!batch_metadatas.empty()) {
    auto batch_meta = std::dynamic_pointer_cast<BatchMetadata>(batch_metadatas[0]);
    size_t index = batch_meta->get_index();
```

(continues on next page)

(continued from previous page)

```
size_t total = batch_meta->get_total_size();
}
```

Accessing Buffer Content

```
// Get the underlying media library buffer
HailoMediaLibraryBufferPtr ml_buffer = buffer->get_buffer();

// Access frame properties from the media library buffer
// (See MediaLibraryBuffer API documentation for details)

// Get and work with the ROI
HailoROIPtr roi = buffer->get_roi();

// Get all detections from the ROI
auto detections = roi->get_objects_typed(HAILO_DETECTION);

for (auto& detection : detections) {
    HailoBBox bbox = detection->get_bbox();
    std::string label = detection->get_label();
    float confidence = detection->get_confidence();
    // Process detection...
}

// Get classifications
auto classifications = roi->get_objects_typed(HAILO_CLASSIFICATION);

// Get landmarks (e.g., pose estimation, facial features)
auto landmarks = roi->get_objects_typed(HAILO_LANDMARKS);

// Get tracker ID if tracking is enabled
int tracker_id = roi->get_track_id();

// Access other analytics results as needed
```

10.1.4. Best Practices

Memory Management

- Always acquire buffers from a buffer pool rather than allocating them directly
- Release buffers back to the pool as soon as processing is complete
- Avoid holding buffer references longer than necessary to prevent pool exhaustion

Metadata Usage

- Use specific metadata types (TensorMetadata, BatchMetadata, etc.) rather than the base Metadata class
- Query metadata by type using `get_metadata_of_type()` for efficient filtering
- Consider the lifecycle of metadata - it persists for the lifetime of the Buffer

Thread Safety

- If sharing buffers between threads, implement appropriate synchronization
- Consider using one buffer per thread or a thread-safe queue for passing buffers
- The ROI object is shared, so modifications affect all references to that Buffer

10.2. Queue

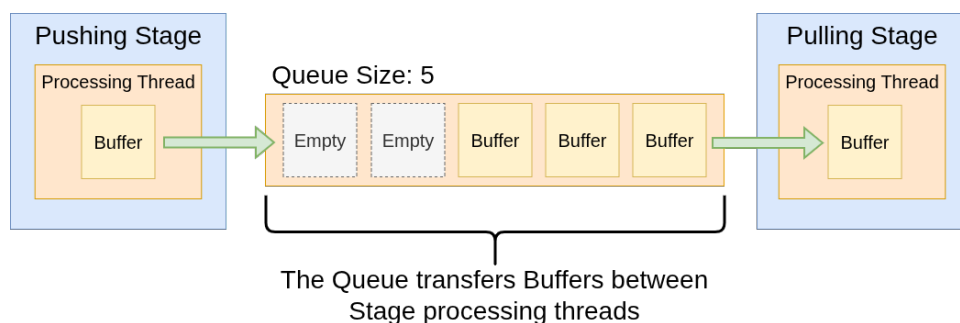
10.2.1. Overview

The `Queue` class provides a thread-safe mechanism for transferring `Buffer` objects between different stages in the Hailo AI Analytics pipeline. Queues act as the connectors between pipeline stages, enabling asynchronous processing where producer stages can continue working while consumer stages process buffers at their own pace.

Key features of the `Queue` class:

- **Thread-safe operations:** All queue operations are protected by mutexes and condition variables
- **Size limiting:** Maximum queue depth can be configured to control memory usage
- **Two operational modes:** Blocking mode (default) and leaky mode for different flow control strategies
- **Performance monitoring:** Built-in tracing support for queue depth tracking

Key Concepts



Queue Ownership and Connectivity

In the Hailo AI Analytics pipeline, queues connect stages and are owned by the subscribing (downstream) stage:

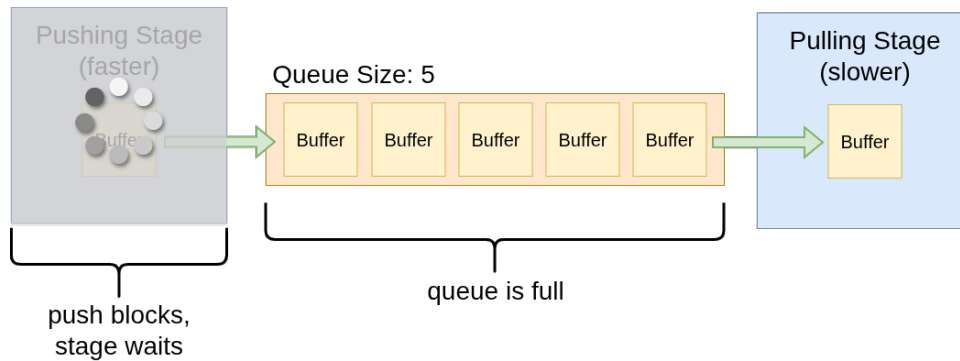
- If **StageB subscribes to StageA**, then **StageB owns the Queue** that receives buffers from StageA
- A stage can own **multiple queues** if it has multiple input sources
- The producer stage (StageA) pushes buffers to the queue
- The consumer stage (StageB) pops buffers from the queue

This ownership model ensures that each stage manages its own input buffering and can control how it receives data from upstream stages.

Blocking Mode vs. Leaky Mode

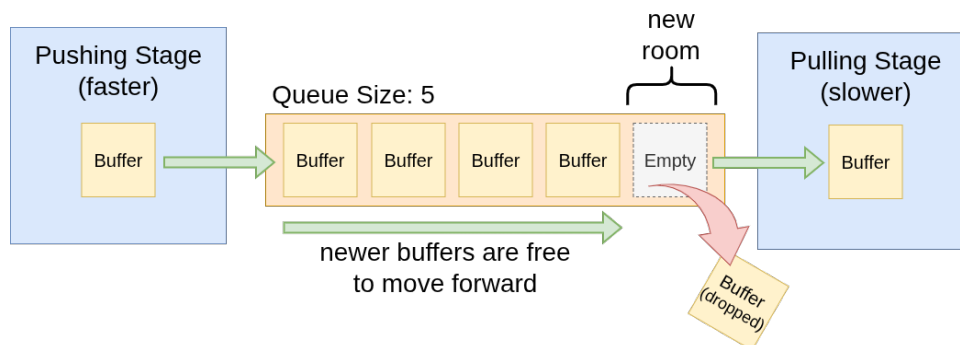
The Queue supports two modes of operation to handle full queue scenarios:

Blocking Mode (default, leaky=false)



- When the queue reaches `max_buffers`, the `push()` call **blocks** until space becomes available
- Ensures no buffers are dropped, maintaining data integrity
- Creates backpressure upstream, slowing down producers when consumers can't keep up
- Best for scenarios where every frame **must** be processed
- Also known as a *non-leaky queue* or *blocking queue*

Leaky Mode (leaky=true)



- When the queue reaches `max_buffers`, the **oldest buffer is dropped** to make room for new buffers
- Always drops the **downstream** buffer (front of queue, oldest buffer)
- Prevents blocking, allowing producers to continue at their own pace
- Best for real-time scenarios where the latest data is more important than processing every frame
- Useful for live camera feeds where dropping old frames is acceptable
- Also known as a *leaky queue* or *non-blocking queue*

Thread Safety and Synchronization

The Queue class uses several synchronization primitives to ensure thread-safe operation:

- **Mutex:** Protects access to the internal queue data structure
- **Condition Variables:** Efficiently signal between threads when buffers are added or removed
- **Atomic Flushing Flag:** Coordinates shutdown and reset operations

Producer threads block in `push()` when the queue is full (in blocking mode), and consumer threads block in `pop()` when the queue is empty. Condition variables ensure threads wake up immediately when the queue state changes, avoiding busy-waiting.

Typical Workflow

1. **Queue Creation:** When connecting stages, the downstream stage creates a Queue with appropriate parameters
2. **Buffer Production:** Upstream stage processes buffers and calls `push()` to the subscribing stage, where it is added to the queue
3. **Buffer Transfer:** Buffers wait in the queue until the downstream stage is ready
4. **Buffer Consumption:** Downstream stage calls `pop()` to retrieve buffers for processing
5. **Flow Control:** Blocking (backpressure) or leaky (dropping) behavior manages queue fullness
6. **Pipeline Shutdown:** `flush()` is called to clear the queue and signal shutdown

10.2.2. Queue API Guide

This section describes the main methods for working with Queues in the Hailo AI Analytics pipeline.

Queue Class

```
class hailo_analytics::pipeline::Queue
    Thread-safe queue for transferring Buffers between pipeline stages.
```

The [Queue](#) class provides a thread-safe mechanism for passing `BufferPtr` objects between different stages in the analytics pipeline. It supports two modes of operation:

- **Blocking mode** (default): When the queue is full, [push\(\)](#) blocks until space is available
- **Leaky mode:** When the queue is full, the oldest buffer is dropped to make room for new buffers

Queues connect pipeline stages, with each queue owned by the subscribing (downstream) stage. For example, if StageB subscribes to StageA, then StageB owns the [Queue](#) that receives buffers from StageA. A stage may own multiple queues if it has multiple input sources.

The [Queue](#) uses condition variables for efficient blocking/signaling and includes built-in tracing support for monitoring queue depth and performance.

Public Functions

`Queue ( std::string parent_name, std::string queue_name, size_t max_buffers, bool leaky = false )`
Constructs a *Queue* with the specified parameters.

The `parent_name` and `queue_name` are used for logging and tracing. In blocking mode, *push()* will block when the queue is full until *pop()* is called. In leaky mode, the oldest buffer (front of queue) is dropped to make room for new buffers.

Parameters

- `parent_name` – The name of the parent stage that owns this queue
- `queue_name` – A descriptive name for this queue instance
- `max_buffers` – The maximum number of buffers this queue can hold
- `leaky` – If true, drops oldest buffers when full; if false (default), blocks on push when full

`~Queue ( )`

`std::string name ( )`
Gets the name of this queue.

Returns The queue name string

`int size ( )`
Gets the current number of buffers in the queue.

This method is thread-safe and provides a snapshot of the queue size at the time of the call.

Returns The current queue size

`void push ( BufferPtr buffer )`
Pushes a buffer onto the queue.

Behavior depends on the queue mode:

- **Blocking mode** (`leaky=false`): If the queue is full, this call blocks until space is available
- **Leaky mode** (`leaky=true`): If the queue is full, the oldest buffer is dropped and the new buffer is added

If the queue is being flushed, the buffer is discarded without blocking.

Parameters `buffer` – The `BufferPtr` to add to the queue

`BufferPtr pop ( )`
Pops a buffer from the queue.

This call blocks until a buffer is available or the queue is being flushed. When a buffer is popped, space is made available for new buffers in blocking mode, and waiting *push()* calls are notified.

Returns `BufferPtr` from the front of the queue, or `nullptr` if the queue is flushing and empty

`uint64_t check_timestamp ( std::optional<std::chrono::milliseconds> timeout = std::nullopt )`
Checks the timestamp of the next buffer in the queue.

This method allows peeking at the timestamp of the next buffer without removing it from the queue. If a timeout is specified, it will wait up to that duration for a buffer to become available.

Parameters `timeout` – Optional timeout duration to wait for a buffer

Returns The timestamp of the next buffer, or 0 if no buffer is available within the timeout

void `flush()`

Flushes all buffers from the queue and enters flushing mode.

Removes all buffers currently in the queue and sets the flushing flag. This causes any blocking `pop()` calls to return `nullptr` and prevents new buffers from being added via `push()`. Used for pipeline shutdown or reset.

void `reset()`

Resets the queue to normal operation.

Clears the flushing flag and removes any remaining buffers. After reset, the queue can accept new buffers via `push()`.

10.2.3. Usage Examples

Creating a Queue

```
// Create a blocking queue (default mode)
// Maximum of 10 buffers, blocks on push when full
auto queue = std::make_shared<Queue>("ParentStage", "input_queue", 10, false);

// Create a leaky queue
// Maximum of 5 buffers, drops oldest buffer when full
auto leaky_queue = std::make_shared<Queue>("RealtimeStage", "camera_input", 5,
→true);
```

Producer: Pushing Buffers

```
// Push to the queue (given queue pointer)
// In blocking mode: this may block if queue is full
// In leaky mode: this never blocks, may drop old buffer
queue->push(buffer);
```

Consumer: Popping Buffers

```
// Pop a buffer from the queue
// This blocks until a buffer is available or queue is flushing
BufferPtr buffer = queue->pop();

if (buffer == nullptr) {
    // Queue is flushing and empty, handle shutdown
    return;
}
```

Monitoring Queue Size

```
// Check current queue depth
int current_size = queue->size();
std::cout << "Queue " << queue->name() << " has "
    << current_size << " buffers" << std::endl;

// This can be useful for monitoring and diagnostics
if (current_size > WARNING_THRESHOLD) {
    LOG_WARNING("Queue backing up: " + queue->name());
}
```

Flushing and Resetting

```
// During shutdown
void shutdown() {
    // Flush all queues to unblock waiting threads
    queue->flush();

    // All pop() calls will now return nullptr
    // All push() calls will be ignored
}

// To restart
void restart() {
    // Reset the queue to normal operation
    queue->reset();

    // Queue is now ready to accept buffers again
}
```

10.2.4. Best Practices

Choosing Queue Size

- **Small queues (1-5 buffers):** Minimize latency, useful for real-time applications
- **Medium queues (5-15 buffers):** Balance between latency and throughput
- **Large queues (15+ buffers):** Maximize throughput, tolerate processing variations
- **Consider memory constraints:** each buffer may contain large image data
- It is strongly recommended to keep queue sizes to a minimum in order to accommodate buffer-pool sizes in the upstream producers. Consider scenarios where the queue fills up, can the buffer-pool still provide buffers to the producers, or are all buffers in use?
- Monitor queue depth during development to find optimal size

Choosing Queue Mode

Use Blocking Mode When:

- Every frame **must** be processed (e.g., inside analytics pipeline)
- Backpressure to slow down producers is acceptable
- Downstream processing is generally fast enough to keep up

Use Leaky Mode When:

- Processing live camera feeds where latest frames are most important
- Downstream processing may have occasional slowdowns
- Dropping frames is acceptable to maintain real-time responsiveness
- Want to prevent one slow stage from blocking the entire pipeline

Thread Safety Considerations

- Queue operations are thread-safe, no external synchronization needed
- Multiple producers can push to the same queue safely
- However, typically one producer pushes to one queue owned by one consumer

Performance Optimization

- Avoid calling `size()` frequently in hot paths (it acquires a lock)
- Use `flush()` during shutdown to unblock all waiting threads cleanly
- Consider queue size vs. latency tradeoff: larger queues increase latency
- Leaky queues prevent pipeline stalls but may drop many frames if too aggressive

Pipeline Shutdown

- Always call `flush()` on all queues during pipeline shutdown
- This ensures any threads blocked in `pop()` or `push()` are released
- After flush, `pop()` returns `nullptr` and `push()` is ignored
- Use `reset()` if you need to restart the pipeline after flushing
- This is already handled if inheriting from `ThreadedStage`, but take note if you have custom implementations

Error Handling

- Check for `nullptr` return from `pop()` to detect flushing/shutdown
- In blocking mode, be aware that `push()` may block indefinitely if consumer stops
- Consider using timeouts or monitoring mechanisms for production systems
- Log queue depths periodically to detect pipeline bottlenecks

10.3. Stage

10.3.1. Overview

The `Stage` class is the fundamental building block of a Hailo AI Analytics pipeline. Stages perform specific tasks such as image capture, inference, post-processing, cropping, encoding, or streaming. Each `Stage` owns and manages a thread to process data asynchronously and in parallel to other `Stages`. This allows for the pipeline to stream buffers and reduce latency.

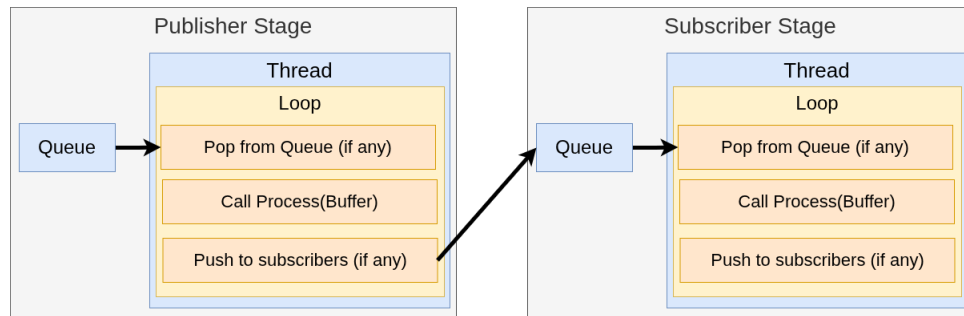
Key features of stages:

- **Modular Design:** Each stage encapsulates a specific processing function
- **Connectivity:** Stages can subscribe to other stages to form processing pipelines
- **Thread-Based Processing:** Each stage typically runs in its own thread for parallel execution
- **Queue Management:** Stages own input queues for receiving buffers from upstream stages
- **Extensibility:** Custom stages can be created by inheriting from base classes
- **Performance Monitoring:** Built-in tracing support for monitoring stage performance

The Hailo AI Analytics API provides two base stage classes:

- **Stage**: Abstract base class defining the stage interface
- **ThreadedStage**: Concrete implementation with thread management and a processing loop

Key Concepts



Stage Connectivity and Ownership

When connecting stages, the **subscriber (downstream) stage owns the queue**:

- If **StageB subscribes to StageA**, then **StageB owns the Queue**
- StageA (publisher) pushes buffers to StageB's queue
- StageB (subscriber) pops buffers from its own queue
- A stage can have **multiple input queues** if it subscribes to multiple publishers
- A stage can have **multiple subscribers** to broadcast buffers to multiple downstream stages

This ownership model ensures each stage controls how it receives data.

Stage Hierarchy

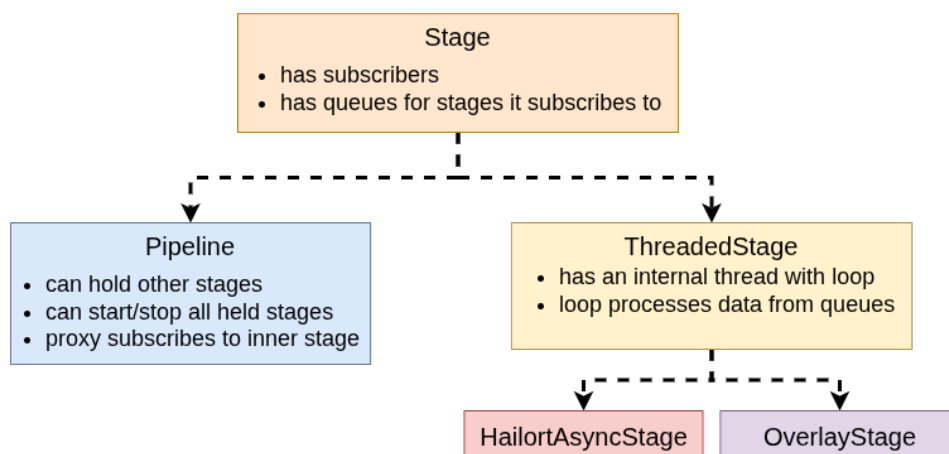


Figure 9. Note what capabilities are inherited from the base classes.

The biggest takeaway from the hierarchy is that Pipeline is a type of Stage. This means that all the capabilities and behaviors defined for Stage also apply to Pipeline, and therefore Pipeline can subscribe to other Stages or Pipelines. Since Pipeline also holds Stage instances, and Pipeline is a Stage itself, it can hold other Pipeline instances.

Stage Lifecycle

Stages follow a defined lifecycle:

1. **Construction:** Stage is created with configuration parameters
2. **Subscription:** Stages are connected using `add_subscriber()`
3. **Start:** `start()` is called to initialize resources and launch the processing thread
4. **Processing:** Stage continuously processes buffers in its thread
5. **Publishing:** Processed buffers are sent to subscribers
6. **Stop:** `stop()` is called to signal shutdown, join thread, and cleanup resources

Note:

All Stage variants offered in Hailo AI Analytics come with accompanying `Builder` classes that provide a higher-level API for constructing and configuring stages. See the `Builder` class documentation for more details.

ThreadedStage Processing Loop

The default `ThreadedStage` processing loop:

1. **Pop** a buffer from the first input queue (blocks until available)
2. **Process** the buffer by calling the `process()` method
3. **Push** the processed buffer to subscribers (optional, based on logic)
4. **Trace** performance metrics
5. **Repeat** until end-of-stream is signaled

Custom stages can override `loop()` for more complex processing patterns, such as:

- Processing from multiple input queues
- Batch processing multiple buffers together
- Time-based processing rather than buffer-driven
- Generating new buffers without input

Thread Safety

Each `ThreadedStage` runs in its own thread, which provides:

- **Parallel execution:** Multiple stages process simultaneously
- **Thread isolation:** Each stage's processing is independent
- **Queue synchronization:** Queues handle thread-safe buffer transfer

Stages don't need to worry about thread synchronization for queue operations, but should ensure thread safety for any shared resources they access.

Creating Custom Stages

To create a custom stage:

1. **Inherit from Stage or ThreadedStage**
2. **Override process():** Implement your processing logic
3. **Optional: Override init():** Initialize resources (models, hardware, etc.)
4. **Optional: Override deinit():** Cleanup resources
5. **Optional: Override loop():** Custom processing loop if needed

The most common pattern is to inherit from ThreadedStage, and override process () while using the default loop. This works well for most single-input, single-output stages.

10.3.2. Stage API Guide

This section describes the main classes, enums, and methods for working with Stages in the Hailo AI Analytics pipeline.

AppStatus Enum

```
enum hailo_analytics::pipeline::AppStatus
    Status codes returned by application operations.
```

Values:

```
enumerator SUCCESS
    Operation completed successfully.
```

```
enumerator INVALID_ARGUMENT
    Invalid argument provided.
```

```
enumerator CONFIGURATION_ERROR
    Configuration error.
```

```
enumerator BUFFER_ALLOCATION_ERROR
    Failed to allocate buffer.
```

```
enumerator HAILORT_ERROR
    HailoRT library error.
```

```
enumerator DSP_OPERATION_ERROR
    DSP operation error.
```

```
enumerator UNINITIALIZED
    Component not initialized.
```

```
enumerator PIPELINE_ERROR
    General pipeline error.
```

```
enumerator DMA_ERROR
    DMA transfer error.
```

```
enumerator MEDIA_LIBRARY_ERROR
    Media library error.
```

StagePoolMode Enum

enum `hailo_analytics::pipeline::StagePoolMode`
Mode for handling buffer pool exhaustion in stages.

Defines behavior when a stage's buffer pool has no available buffers.

Values:

enumerator `FAIL_ON_EMPTY_POOL`
Return an error when pool is empty.

enumerator `LEAKY`
Drop the current frame when pool is empty.

enumerator `BLOCKING`
Wait until a buffer becomes available.

enumerator `USE_AVAILABLE_BUFFERS`
Use whatever buffers are available, even if less than requested.

enumerator `STAGE_POOL_MODE_MAX`
Maximum enum value.

Stage Class

class `hailo_analytics::pipeline::Stage`
Abstract base class for all pipeline stages.

The *Stage* class defines individual processing unit in the Hailo Analytics pipeline. Each stage performs a specific operation on incoming data buffers, such as image capture, inference, post-processing, encoding, and more. Stages can be connected together to form a pipeline where buffers flow from one stage to another through queues.

Each stage has:

- A unique name for identification and logging
- Start/stop lifecycle management
- Subscriber management for connecting to downstream stages
- *Queue* management for receiving buffers from upstream stages
- Performance tracing support

Subclassed by `SimpleStage`, `hailo_analytics::pipeline::Pipeline`, `hailo_analytics::pipeline::ThreadedStage`

Public Functions

`Stage ( std::string name, bool trace_processing_operations = true )`
Constructs a *Stage* with the given name.

Parameters

- `name` – The name of the stage for identification and logging
- `trace_processing_operations` – If true, enables performance tracing for processing operations

`virtual ~Stage ( ) = default`

```
std::string get_name() const
```

Gets the name of this stage.

Returns The stage name string

```
void trace_fps()
```

Traces the frame rate (FPS) of this stage.

Increments the internal frame counter for performance monitoring and tracing.

```
virtual AppStatus start() = 0
```

Starts the stage operation.

Called to begin stage processing. Implementations should initialize resources and start any necessary threads or processing loops.

Returns AppStatus::SUCCESS on success, error code otherwise

```
virtual AppStatus stop() = 0
```

Stops the stage operation.

Called to end stage processing. Implementations should clean up resources and stop any running threads.

Returns AppStatus::SUCCESS on success, error code otherwise

```
virtual void add_subscriber(StagePtr subscriber, std::optional<std::string> stream_id = std::nullopt) = 0
```

Adds a subscriber stage that will receive buffers from this stage.

When a subscriber is added, this stage will push processed buffers to the subscriber's input queue.

Parameters

- `subscriber` - The stage to subscribe to this stage's output
- `stream_id` - Optional stream identifier for multi-stream scenarios

```
virtual void add_queue(std::string publisher_name) = 0
```

Adds an input queue for receiving buffers from a publisher stage.

Creates and registers a queue for receiving buffers from the specified publisher.

Parameters `publisher_name` - The name of the stage that will push buffers to this queue

```
virtual void push(BufferPtr data, std::string publisher_name) = 0
```

Pushes a buffer to this stage from a publisher.

Routes the buffer to the appropriate input queue based on the publisher name.

Parameters

- `data` - The buffer to push
- `publisher_name` - The name of the publisher stage sending the buffer

ThreadedStage Class

`class hailo_analytics::pipeline::ThreadedStage` : public `hailo_analytics::pipeline::Stage`
Concrete stage implementation that processes buffers in a dedicated thread.

ThreadedStage extends *Stage* to provide a complete implementation with thread management, queue handling, and a processing loop. This is the most common base class for custom stages in the analytics pipeline.

Key features:

- **Dedicated thread:** Each *ThreadedStage* runs in its own thread with its processing loop
- **Input queues:** Maintains queues for receiving buffers from upstream publishers
- **Subscriber management:** Can forward processed buffers to multiple downstream subscribers
- **Configurable queues:** *Queue* size and leaky/blocking behavior are configurable
- **Lifecycle hooks:** Provides *init()*, *deinit()*, and *process()* virtual methods for customization

To create a custom stage, inherit from *ThreadedStage* and override:

- *init()*: Initialize resources (optional, default does nothing)
- *process(BufferPtr)*: Process a single buffer (main processing logic)
- *deinit()*: Clean up resources (optional, default does nothing)
- *loop()*: Custom processing loop (optional, default loop pops from queue and calls process)

Subclassed by `CacheStage`, `ClipImagePreprocess`, `CustomStage`, `FaissStorageStage`, `Full-FrameBBBoxInjectorStage`, `TestThreadedStage`, `ThumStorageStage`, `VideoStorageStage`, `hailo_analytics::pipeline::ai::AnalyticsDBStage`, `hailo_analytics::pipeline::ai::HailortAsyncStage`, `hailo_analytics::pipeline::ai::LightweightTrackerStage`, `hailo_analytics::pipeline::ai::PostprocessStage`, `hailo_analytics::pipeline::codecs::AnalyticMetadataPackagerStage`, `hailo_analytics::pipeline::codecs::EncoderStage`, `hailo_analytics::pipeline::cropping::AggregatorStage`, `hailo_analytics::pipeline::cropping::DspBaseCropStage`, `hailo_analytics::pipeline::muxing::DemuxerStage`, `hailo_analytics::pipeline::muxing::MuxerStage`, `hailo_analytics::pipeline::overlay::OverlayStage`, `hailo_analytics::pipeline::routing::CallbackStage`, `hailo_analytics::pipeline::routing::FreezeStage`, `hailo_analytics::pipeline::routing::TeeStage`, `hailo_analytics::pipeline::routing::TrackerTrafficCtrlStage`, `hailo_analytics::pipeline::routing::ValveStage`, `hailo_analytics::pipeline::sinks::AppSinkStage`, `hailo_analytics::pipeline::sinks::GstSinkStage`, `hailo_analytics::pipeline::sinks::Mp4Stage`, `hailo_analytics::pipeline::sinks::RTPConverterStage`, `hailo_analytics::pipeline::sinks::UdpStage`, `hailo_analytics::pipeline::sinks::WebSocketSinkStage`, `hailo_analytics::pipeline::sinks::ZmqCommStage`, `hailo_analytics::pipeline::sources::FileSourceStage`, `hailo_analytics::pipeline::sources::FrontendStage`, `hailo_analytics::pipeline::sources::GstSourceStage`

Public Functions

`ThreadedStage ( std::string name, size_t queue_size, bool leaky = false, bool ... )`
Constructs a *ThreadedStage* with the specified parameters.

The queue parameters apply to all input queues created for this stage.

Parameters

- `name` – The name of the stage for identification and logging
- `queue_size` – Maximum number of buffers each input queue can hold
- `leaky` – If true, queues drop old buffers when full; if false, queues block when full
- `trace_processing_operations` – If true, enables performance tracing for processing operations

virtual *AppStatus* `start ( )` override

Starts the stage by initializing and launching the processing thread.

Calls `init()` for stage-specific initialization, then creates and starts the processing thread running the `loop()` method. Sets the thread name on Linux.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual `AppStatus stop()` override

Stops the stage by signaling end-of-stream and joining the thread.

Sets the end-of-stream flag, waits for the thread to complete, then calls `deinit()` for stage-specific cleanup.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual void `add_subscriber(StagePtr subscriber, std::optional<std::string> stream_id = ...)`

Adds a subscriber and creates the connecting queue.

Registers the subscriber and instructs it to create an input queue for receiving buffers from this stage.

Parameters

- `subscriber` – The stage to subscribe to this stage's output
- `stream_id` – Optional stream identifier (currently unused)

virtual void `add_queue(std::string publisher_name)` override

Creates an input queue for receiving buffers from a publisher.

Creates a new `Queue` with the stage's configured `queue_size` and leaky parameters.

Parameters `publisher_name` – The name of the stage that will push buffers to this queue

virtual void `push(BufferPtr data, std::string publisher_name)` override

Pushes a buffer to this stage's input queue.

Finds the queue associated with the `publisher_name` and pushes the buffer to it.

Parameters

- `data` – The buffer to push
- `publisher_name` – The name of the publisher stage sending the buffer

virtual `AppStatus init()`

Initializes the stage before processing begins.

Called once before the processing thread starts. Override to allocate resources, initialize hardware, load models, etc. Default implementation does nothing.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual `AppStatus deinit()`

Deinitializes the stage after processing ends.

Called once after the processing thread stops. Override to release resources, cleanup hardware, etc. Default implementation does nothing.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual `AppStatus process(BufferPtr buffer)`

Processes a single buffer.

Called by the default `loop()` for each buffer popped from the input queue. Override with your stage's processing logic. Default implementation does nothing.

Parameters `buffer` – The buffer to process

Returns `AppStatus::SUCCESS` on success, error code otherwise

`virtual void loop()`

Main processing loop that runs in the stage's thread.

Default implementation continuously pops buffers from the first input queue, calls `process()` on each buffer, and traces performance. Override for custom loop behavior (e.g., processing from multiple queues, different timing, etc.).

The loop should check `m_end_of_stream` to know when to exit.

`void send_to_subscribers(BufferPtr data)`

Sends a buffer to all subscribers of this stage.

Pushes the buffer to all stages that have subscribed to this stage's output. Call this after processing a buffer to forward it downstream.

Parameters `data` – The buffer to send

`void send_to_specific_subscriber(std::string stage_name, BufferPtr data)`

Sends a buffer to a specific subscriber by name.

Pushes the buffer only to the subscriber with the matching name. Useful for selective routing in multi-subscriber scenarios.

Parameters

- `stage_name` – The name of the subscriber stage to send to
- `data` – The buffer to send

`void set_end_of_stream(bool end_of_stream)`

Sets the end-of-stream flag and manages queue flushing.

When set to true, signals the processing loop to exit and flushes all input queues to unblock any waiting `pop()` calls. When set to false, resets queues to allow the stage to restart.

Parameters `end_of_stream` – If true, flushes all queues; if false, resets all queues

`inline const std::vector<StagePtr> &get_subscribers() const`

Get the list of subscriber stages.

Returns Vector of subscriber stages

10.3.3. Usage Examples

Creating a Simple Custom Stage

```
// Custom stage that processes each buffer
class MyProcessingStage : public hailo_analytics::pipeline::ThreadedStage
{
public:
    MyProcessingStage(std::string name, size_t queue_size, bool leaky = false,
                     bool trace_processing_operations = true)
        : hailo_analytics::pipeline::ThreadedStage(name, queue_size, leaky, trace_
        processing_operations)
    {
```

(continues on next page)

(continued from previous page)

```

}

AppStatus init() override
{
    // Initialize resources (e.g., load model)
    HAILO_ANALYTICS_LOG_INFO("Initializing {}", m_stage_name);
    return AppStatus::SUCCESS;
}

AppStatus process(BufferPtr buffer) override
{
    // Process the buffer
    HailoROIPtr roi = buffer->get_roi();

    // Do some processing...
    // (e.g., run inference, add detections, etc.)

    // Forward to subscribers
    send_to_subscribers(buffer);

    return AppStatus::SUCCESS;
}

AppStatus deinit() override
{
    // Cleanup resources
    HAILO_ANALYTICS_LOG_INFO("Deinitializing {}", m_stage_name);
    return AppStatus::SUCCESS;
}
};

```

Subscribing Stages

```

// Connect stages (build the pipeline graph)
// given stages source_stage, processing_stage, and sink_stage
source_stage->add_subscriber(processing_stage);
processing_stage->add_subscriber(sink_stage);

// Start pipeline (start in reverse order)
sink_stage->start();
processing_stage->start();
source_stage->start();

// Let it run...
std::this_thread::sleep_for(std::chrono::seconds(10));

// Stop pipeline (stop in forward order)
source_stage->stop();
processing_stage->stop();
sink_stage->stop();

```

Custom Processing Loop

```
// Stage with custom loop that processes from multiple queues
class MultiInputStage : public hailo_analytics::pipeline::ThreadedStage
{
public:
    MultiInputStage(std::string name, size_t queue_size, bool leaky = false,
                    bool trace_processing_operations = true)
        : hailo_analytics::pipeline::ThreadedStage(name, queue_size, leaky, trace_
        ↪processing_operations)
    {
    }

    void loop() override
    {
        while (!m_end_of_stream)
        {
            // Custom logic: alternate between queues
            if (m_queues.size() < 2)
            {
                std::this_thread::sleep_for(std::chrono::milliseconds(10));
                continue;
            }

            // Pop from first queue
            BufferPtr buffer1 = m_queues[0]->pop();
            if (buffer1 == nullptr) break;

            // Pop from second queue
            BufferPtr buffer2 = m_queues[1]->pop();
            if (buffer2 == nullptr) break;

            // Process both buffers together
            process_dual_input(buffer1, buffer2);

            trace_fps();
        }
    }

private:
    void process_dual_input(BufferPtr buf1, BufferPtr buf2)
    {
        // Custom processing with two inputs
        // ...
    }
};
```

Selective Subscriber Routing

```
class RouterStage : public hailo_analytics::pipeline::ThreadedStage
{
public:
    RouterStage(std::string name, size_t queue_size, bool leaky = false,
                bool trace_processing_operations = true)
        : hailo_analytics::pipeline::ThreadedStage(name, queue_size, leaky, trace_
        ↪processing_operations)
    {
    }

    AppStatus process(BufferPtr buffer) override
```

(continues on next page)

(continued from previous page)

```
{
    // Route based on some condition
    HailoROIPtr roi = buffer->get_roi();
    auto detections = roi->get_objects_typed(HAILO_DETECTION);

    if (detections.size() > 0)
    {
        // Send to analytics stage if detections found
        send_to_specific_subscriber("AnalyticsStage", buffer);
    }
    else
    {
        // Send to bypass stage if no detections
        send_to_specific_subscriber("BypassStage", buffer);
    }

    return AppStatus::SUCCESS;
}
};
```

Leaky vs Blocking Queue Configuration

```
// Blocking stage: ensures every frame is processed
// Queue will block producers when full
auto blocking_stage = std::make_shared<ProcessingStage>(
    "blocking_processor",
    10,    // queue_size
    false, // leaky = false (blocking mode)
    true   // trace_processing_operations
);

// Leaky stage: drops old frames if consumer is slow
// Good for real-time camera feed processing
auto leaky_stage = std::make_shared<ProcessingStage>(
    "realtime_processor",
    5,    // queue_size
    true,  // leaky = true (leaky mode)
    true   // trace_processing_operations
);
```

Broadcasting to Multiple Subscribers

```
// Given stages source, subscriber1, subscriber2, and subscriber3

// Source broadcasts to all three subscribers
source->add_subscriber(subscriber1);
source->add_subscriber(subscriber2);
source->add_subscriber(subscriber3);

// Inside the source stage's process() method:
send_to_subscribers(buffer); // Goes to all three
```

Error Handling in Processing

```
class RobustStage : public hailo_analytics::pipeline::ThreadedStage
{
public:
    RobustStage(std::string name, size_t queue_size, bool leaky = false,
                bool trace_processing_operations = true)
        : hailo_analytics::pipeline::ThreadedStage(name, queue_size, leaky, trace_
→processing_operations)
    {
    }

    AppStatus process(BufferPtr buffer) override
    {
        try
        {
            // Attempt processing
            auto status = do_processing(buffer);

            if (status != AppStatus::SUCCESS)
            {
                HAILO_ANALYTICS_LOG_ERROR("Processing failed in {}", m_stage_name);
                return status;
            }

            // Forward to subscribers
            send_to_subscribers(buffer);
            return AppStatus::SUCCESS;
        }
        catch (const std::exception& e)
        {
            HAILO_ANALYTICS_LOG_ERROR("Exception in {}: {}", m_stage_name, e.what());
            return AppStatus::PIPELINE_ERROR;
        }
    }

private:
    AppStatus do_processing(BufferPtr buffer)
    {
        // Your processing logic
        return AppStatus::SUCCESS;
    }
};
```

10.3.4. Best Practices

Stage Design

- **Single Responsibility:** Each stage should have one clear purpose
- **Error Handling:** Always handle errors gracefully and return appropriate AppStatus
- **Resource Management:** Use RAII principles, allocate in init(), cleanup in deinit()
- **Performance:** Minimize processing time in process() to maintain throughput

Queue Configuration

- **Queue Size:** Balance between latency and memory usage
 - Smaller queues (1-10): Lower latency, less buffering
 - Larger queues (10-50): Better throughput, longer latency
- **Leaky Mode:** Use for real-time scenarios where dropping frames is acceptable
- **Blocking Mode:** Use when every frame must be processed

Pipeline Construction

- **Start Order:** Start stages in reverse topological order (sinks before sources)
- **Stop Order:** Stop stages in forward topological order (sources before sinks)
- **Connection Validation:** Ensure all stages are properly connected before starting
- **Resource Cleanup:** Always call stop() to ensure proper cleanup

Threading Considerations

- **Thread Count:** Each ThreadedStage creates one thread; consider CPU cores
- **Blocking Operations:** Avoid blocking operations in process() that could stall the pipeline
- **Thread Safety:** Ensure any shared resources accessed by stages are thread-safe
- **Thread Names:** ThreadedStage automatically sets thread names on Linux for debugging

Performance Optimization

- **Minimize Copies:** Avoid unnecessary buffer or data copies, note we work exclusively with pointers
- **Use Metadata:** Store additional information as metadata rather than copying buffers
- **Profile Stages:** Use built-in tracing to identify bottlenecks
- **Balance Queues:** Monitor queue depths to find imbalances in the pipeline
- **Optimize Hot Paths:** Focus optimization on the most frequently called code (process())

Error Recovery

- **Graceful Degradation:** Continue processing even if one buffer fails
- **Logging:** Log errors with sufficient context for debugging
- **Status Codes:** Return appropriate AppStatus codes for different error types
- **Cleanup:** Ensure resources are released even when errors occur

Pipeline Shutdown

- **Signal End-of-Stream:** Use `set_end_of_stream()` to signal graceful shutdown
- **Flush Queues:** Queues are automatically flushed when end-of-stream is set
- **Join Threads:** `stop()` automatically joins the processing thread
- **Cleanup Order:** Stop sources first to prevent new buffers, then stop downstream stages

Testing Custom Stages

- **Unit Test `process()`:** Test the processing logic independently
- **Integration Test:** Test the stage in a minimal pipeline
- **Load Test:** Verify performance under expected load
- **Error Cases:** Test error handling and recovery
- **Resource Leaks:** Check for memory leaks with long-running tests

10.4. Pipeline

10.4.1. Overview

“A **Pipeline** holds **Stages** connected by **Queues** that process **Buffers**”

The `Pipeline` class is a container and manager for organizing multiple stages into a cohesive processing graph. It provides lifecycle management for collections of stages, ensuring proper startup and shutdown ordering, and enables treating a group of stages as a single logical unit.

Key features of the `Pipeline` class:

- **Stage Organization:** Groups stages by type (source, general, sink) for proper ordering
- **Lifecycle Management:** Automatically starts and stops stages in the correct order
- **Composite Pattern:** Pipelines can act as stages, allowing nested pipeline architectures
- **Stage Lookup:** Find stages by name for runtime configuration and inspection
- **Input/Output Designation:** Specify which stages serve as pipeline entry and exit points
- **Connection Management:** Connect pipelines to other stages or pipelines

The `Pipeline` class inherits from `Stage`, which means:

- Pipelines can be connected to other stages using `add_subscriber()`
- Pipelines can be nested within other pipelines
- Complex multi-pipeline systems can be built hierarchically

Key Concepts

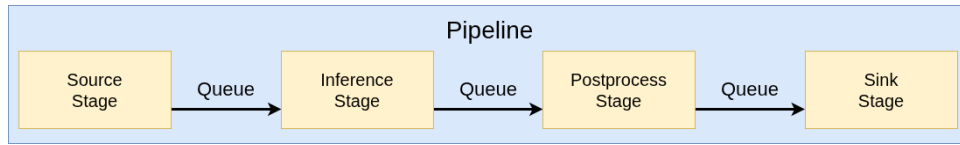


Figure 10. A simple single stream pipeline.

Stage Types and Ordering

Stages within a pipeline are categorized into three types:

SOURCE Stages

- Generate or capture data (e.g., camera capture, file reader, network receiver)
- Started **last** during pipeline startup
- Stopped **first** during pipeline shutdown
- This prevents data flow before downstream stages are ready

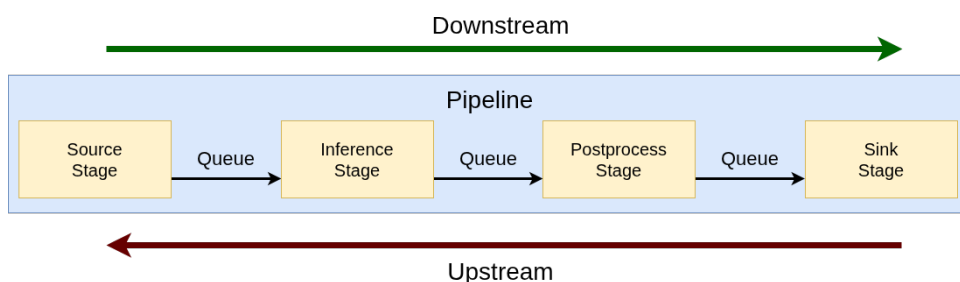
GENERAL Stages

- Process data in the middle of the pipeline (e.g., inference, post-processing, tracking)
- Started **middle** during pipeline startup
- Stopped **middle** during pipeline shutdown

SINK Stages

- Consume or output data (e.g., UDP, analytics exporter, file writer)
- Started **first** during pipeline startup
- Stopped **last** during pipeline shutdown
- This ensures they're ready to receive data when sources start

Pipeline Directions: Upstream / Downstream



It is common to refer to pipelines like a flowing stream of water: with data flowing from **upstream** to **downstream**.

Startup Order

When `start()` is called on a pipeline:

1. SINK stages start (downstream consumers ready)
2. GENERAL stages start (middle processing ready)
3. SOURCE stages start (upstream producers begin)

This ensures that each stage is ready to receive data before the upstream stage begins sending it.

Shutdown Order

When `stop()` is called on a pipeline:

1. SOURCE stages stop (stop producing new data)
2. GENERAL stages stop (finish processing existing data)
3. SINK stages stop (finish consuming remaining data)

This ensures graceful shutdown where data flows cleanly to completion before stages terminate.

Input and Output Stages

Pipelines can designate specific stages as entry and exit points:

- **Input Stage:** The stage that receives buffers from external sources
 - Set using `set_in_stage()`
 - When other stages/pipelines push to this pipeline, buffers go to the input stage
 - Typically a stage near the beginning of the pipeline
- **Output Stage:** The stage that sends buffers to external sinks
 - Set using `set_out_stage()`
 - When this pipeline has subscribers, buffers are pushed from the output stage
 - Typically a stage near the end of the pipeline

This abstraction allows pipelines to be connected together without exposing internal structure.

Pipelines as Stages (Sub-Pipelines)

Because Pipeline inherits from Stage, pipelines can be used anywhere a stage can be used:

- Connect a pipeline to a stage: `stage->add_subscriber(pipeline)`
- Connect a stage to a pipeline: `pipeline->add_subscriber(stage)`
- Connect pipelines together: `pipeline1->add_subscriber(pipeline2)`
- Nest pipelines within pipelines: `outer_pipeline->add_stage(inner_pipeline)`

This enables hierarchical composition of complex processing graphs.

Stage Lookup

Pipelines maintain references to all contained stages and provide lookup by name:

- `get_stage_by_name()` returns a specific stage
- Useful for runtime configuration, inspection, or control
- Enables dynamic pipeline behavior based on stage state

Typical Workflow

1. **Pipeline Creation:** Create a Pipeline instance with a name
2. **Add Stages:** Add stages to the pipeline with appropriate types (SOURCE, GENERAL, SINK)
3. **Connect Stages:** Connect stages to each other using `add_subscriber()`
4. **Set I/O Stages:** Designate input and output stages if the pipeline will be connected externally
5. **Start Pipeline:** Call `start()` to begin processing in correct order
6. **Processing:** Buffers flow through the pipeline stages
7. **Stop Pipeline:** Call `stop()` to gracefully shutdown in correct order

Note:

Steps 1-3 in the above workflow can be simplified using the `PipelineBuilder` class, which provides a higher-level API for constructing pipelines. See the `PipelineBuilder` documentation for more details.

10.4.2. Pipeline API Guide

This section describes the main classes, enums, and methods for working with Pipelines in the Hailo AI Analytics system.

StageType Enum

```
enum hailo_analytics::pipeline::StageType
    Categorization of stages within a pipeline.
```

Stage types help the *Pipeline* manage startup and shutdown ordering. Source stages are started last and stopped first to control data flow.

Values:

```
enumerator GENERAL
    General processing stage (e.g., inference, post-processing)
```

```
enumerator SOURCE
    Source stage that generates or captures data (e.g., camera, file reader)
```

```
enumerator SINK
    Sink stage that outputs or consumes data (e.g., encoder, display, network sender)
```

Pipeline Class

```
class hailo_analytics::pipeline::Pipeline : public hailo_analytics::pipeline::Stage
    Container and manager for a collection of connected stages.
```

The *Pipeline* class organizes multiple *Stage* objects into a processing chain and manages their lifecycle. It provides:

- Automatic startup/shutdown ordering based on stage types
- *Stage* registration and retrieval by name
- Input/output stage designation for inter-pipeline connections
- The ability to treat a pipeline as a single stage (to encapsulate complex processing)

Pipelines can be nested, allowing complex multi-pipeline systems where entire pipelines act as stages within a larger pipeline graph.

Stage Ordering:

- **Start Order:** Sinks → General → Sources (downstream to upstream)
- **Stop Order:** Sources → General → Sinks (upstream to downstream)

This ordering ensures that:

- Downstream stages are ready before upstream stages start producing data
- Upstream stages stop producing data before downstream stages shut down

Public Functions

```
Pipeline (std::string name, bool trace_processing_operations = true)
    Constructs a Pipeline with the given name.
```

Parameters

- **name** – The name of the pipeline for identification and logging
- **trace_processing_operations** – If true, enables performance tracing for the pipeline

```
void add_stage (StagePtr stage, StageType type = StageType::GENERAL)
    Adds a stage to the pipeline with the specified type.
```

Stages are organized by type to control startup and shutdown ordering. All stages are also added to the main stages list for lookup and management.

Parameters

- **stage** – The stage to add to the pipeline
- **type** – The category of the stage (GENERAL, SOURCE, or SINK)

```
void set_in_stage (StagePtr stage)
    Sets the input stage of the pipeline.
```

When the pipeline is connected to another stage or pipeline, this stage will be the entry point for incoming buffers.

Parameters **stage** – The stage that will receive data from external sources

```
void set_out_stage (StagePtr stage)
    Sets the output stage of the pipeline.
```

When the pipeline has subscribers, this stage will push buffers to them.

Parameters stage - The stage that will send data to external sinks

virtual `AppStatus start ( )` override
Starts all stages in the pipeline in the correct order.

Stages are started in this order:

- i. Sink stages (downstream consumers)
- ii. General stages (middle processing)
- iii. Source stages (upstream producers)

This ensures downstream stages are ready before upstream stages start producing data.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual `AppStatus stop ( )` override
Stops all stages in the pipeline in the correct order.

Stages are stopped in this order:

- i. Source stages (stop producing data)
- ii. General stages (finish processing)
- iii. Sink stages (finish consuming data)

This ensures data flow stops cleanly from upstream to downstream.

Returns `AppStatus::SUCCESS` on success, error code otherwise

virtual void `add_subscriber ( StagePtr subscriber, std::optional<std::string> stream_id = ... )`
Subscribes another stage to this pipeline's output.

Delegates to the output stage's `add_subscriber` method. The subscriber will receive buffers from the pipeline's designated output stage.

Parameters

- `subscriber` - The stage that will receive buffers from this pipeline
- `stream_id` - Optional stream identifier for multi-stream scenarios

virtual void `add_queue ( std::string publisher_name )` override
Creates an input queue for receiving buffers from a publisher.

Delegates to the input stage's `add_queue` method.

Parameters `publisher_name` - The name of the stage that will push buffers to this pipeline

virtual void `push ( BufferPtr data, std::string publisher_name )` override
Pushes a buffer to this pipeline's input stage.

Delegates to the input stage's `push` method.

Parameters

- `data` - The buffer to push
- `publisher_name` - The name of the publisher stage sending the buffer

StagePtr get_stage_by_name(std::string stage_name)
Retrieves a stage from the pipeline by name.

Searches through all stages in the pipeline and returns the first match. Useful for runtime configuration or inspection of pipeline components.

Parameters stage_name - The name of the stage to find

Returns Shared pointer to the stage if found, nullptr otherwise

const std::vector<StagePtr> &get_stages() const
Get all stages in the pipeline.

Returns Vector of all stages

StagePtr get_in_stage() const
Get the input stage of the pipeline.

Returns Shared pointer to the input stage, or nullptr if not set

StagePtr get_out_stage() const
Get the output stage of the pipeline.

Returns Shared pointer to the output stage, or nullptr if not set

10.4.3. Usage Examples

Note: In practice, pipelines are typically built using the **PipelineBuilder** class, which provides a more convenient and robust API for pipeline construction. The examples here demonstrate the underlying Pipeline class API for educational purposes. See the PipelineBuilder documentation for the recommended approach to building pipelines.

Creating a Basic Pipeline

```
using namespace hailo_analytics::pipeline;

// Create a pipeline
auto pipeline = std::make_shared<Pipeline>("my_pipeline");

// Create stages
auto frontend_stage = std::make_shared<sources::FrontendStage>("frontend", 10);
auto inference_stage = std::make_shared<ai::HailortAsyncStage>("inference", 10);
auto postprocess_stage = std::make_shared<ai::PostprocessStage>("postprocess", 5);

// Configure stages as needed ...

// Add stages to pipeline with appropriate types
pipeline->add_stage(frontend_stage, StageType::SOURCE);
pipeline->add_stage(inference_stage, StageType::GENERAL);
pipeline->add_stage(postprocess_stage, StageType::SINK);

// Connect stages together
frontend_stage->add_subscriber(inference_stage);
inference_stage->add_subscriber(postprocess_stage);

// Start and stop
pipeline->start(); // Starts: frontend  inference  postprocess
// ... processing ...
pipeline->stop(); // Stops: postprocess  inference  frontend
```

Setting Input and Output Stages

```
using namespace hailo_analytics::pipeline;

// Create a pipeline that can be connected to other pipelines
auto processing_pipeline = std::make_shared<Pipeline>("processor");

// Create internal stages
auto postprocess_stage = std::make_shared<ai::PostprocessStage>("postprocess", 10);
auto overlay_stage = std::make_shared<overlay::OverlayStage>("overlay", 10);
auto encoder_stage = std::make_shared<codecs::EncoderStage>("encoder", 10);

// Configure stages as needed ...

// Add and connect stages
processing_pipeline->add_stage(postprocess_stage, StageType::GENERAL);
processing_pipeline->add_stage(overlay_stage, StageType::GENERAL);
processing_pipeline->add_stage(encoder_stage, StageType::SINK);

postprocess_stage->add_subscriber(overlay_stage);
overlay_stage->add_subscriber(encoder_stage);

// Designate input and output for external connections
processing_pipeline->set_in_stage(postprocess_stage);
processing_pipeline->set_out_stage(encoder_stage);

// Now the pipeline can be connected like a stage
// External stages push to postprocess_stage
// External subscribers receive from encoder_stage
```

Connecting Pipelines Together

```
using namespace hailo_analytics::pipeline;

// Create two pipelines
auto capture_pipeline = std::make_shared<Pipeline>("capture");
auto analytics_pipeline = std::make_shared<Pipeline>("analytics");

// Setup capture pipeline
auto frontend_stage = std::make_shared<sources::FrontendStage>("frontend", 10);
capture_pipeline->add_stage(frontend_stage, StageType::SOURCE);
capture_pipeline->set_out_stage(frontend_stage);

// Setup analytics pipeline
auto inference_stage = std::make_shared<ai::HailortAsyncStage>("inference", 10);
auto postprocess_stage = std::make_shared<ai::PostprocessStage>("postprocess", 10);
analytics_pipeline->add_stage(inference_stage, StageType::GENERAL);
analytics_pipeline->add_stage(postprocess_stage, StageType::GENERAL);
inference_stage->add_subscriber(postprocess_stage);
analytics_pipeline->set_in_stage(inference_stage);
analytics_pipeline->set_out_stage(postprocess_stage);

// Connect pipelines together
capture_pipeline->add_subscriber(analytics_pipeline);

// Start both pipelines
analytics_pipeline->start();
capture_pipeline->start();
```

Nested Pipelines

```
using namespace hailo_analytics::pipeline;

// Create an outer pipeline that contains inner pipelines
auto main_pipeline = std::make_shared<Pipeline>("main");

// Create sub-pipelines (treating them as compound stages)
auto inference_pipeline = std::make_shared<Pipeline>("inference_processing");
auto output_pipeline = std::make_shared<Pipeline>("output_processing");

// ... configure internal structures of sub-pipelines ...

// Add sub-pipelines as stages in the main pipeline
main_pipeline->add_stage(inference_pipeline, StageType::GENERAL);
main_pipeline->add_stage(output_pipeline, StageType::GENERAL);
inference_pipeline->add_subscriber(output_pipeline);

// Starting main pipeline starts all sub-pipelines
main_pipeline->start();
```

10.4.4. Best Practices

Stage Type Assignment

- **Categorize** stages correctly when known to ensure proper startup/shutdown ordering
- **SOURCE**: Only stages that generate new buffers (cameras, file readers, synthetic generators)
- **SINK**: Only stages that terminate buffer flow (UDP, file writers, analytics exporters)
- **GENERAL**: All processing stages in between
- When in doubt, use GENERAL - it's safe but may not be optimal for ordering

Pipeline Lifecycle

- **Start Order**: Start pipelines from downstream to upstream (sinks to sources)
- **Stop Order**: Stop pipelines from upstream to downstream (sources to sinks)
- **Always stop pipelines**: Ensure `stop()` is called during Cleanup
- Since Pipeline manages stage lifecycles, avoid starting/stopping individual stages manually

Input/Output Stage Selection

- **Set input/output stages** when the pipeline will be connected externally
- **Choose logical boundaries**: Input stage is where external data enters, output is where it exits
- **Not required for standalone pipelines**: Only needed when treating pipeline as a stage
- **Can be the same stage**: If appropriate for your design

Pipeline Nesting

- **Use nesting for modularity:** Group related stages into sub-pipelines
- **Consider performance:** Each pipeline adds a layer of indirection
- **Balance complexity:** Too much nesting can make debugging difficult
- **Document structure:** Clearly document the purpose of each nested pipeline

Stage Lookup

- **Use descriptive names:** Make stages easy to find and identify
- **Avoid frequent lookups:** Cache stage pointers if accessed repeatedly
- **Check for nullptr:** Always verify that `get_stage_by_name()` returns a valid pointer
- **Use for configuration:** Stage lookup is ideal for runtime configuration and control

Error Handling

- **Check start() return:** Verify that `start()` returns SUCCESS
- **Handle stage failures:** If a stage fails to start, the pipeline may be in an inconsistent state
- **Stop on error:** If startup fails, call `stop()` to cleanup stages that did start
- **Log pipeline state:** Include pipeline name in log messages for multi-pipeline systems

Performance Considerations

- **Pipeline overhead:** Each pipeline adds minimal overhead for lifecycle management
- **Stage count:** More stages provide flexibility but may impact latency
- **Queue sizes:** Balance between latency and throughput when configuring stage queues
- **Nesting depth:** Limit nesting depth to 2-3 levels for maintainability

Multi-Pipeline Systems

- **Isolate concerns:** Use separate pipelines for independent processing paths
- **Share resources carefully:** Be cautious about sharing stages between pipelines
- **Coordinate startup:** Ensure dependent pipelines start in the correct order
- **Monitor independently:** Track performance metrics per pipeline
- **Plan shutdown:** Stop pipelines in dependency order during system shutdown

Testing Pipelines

- **Unit test stages:** Test individual stages before adding to pipeline
- **Integration test:** Test complete pipeline with all stages connected
- **Test startup/shutdown:** Verify correct ordering and no resource leaks
- **Test error cases:** Ensure pipeline handles stage failures gracefully
- **Load test:** Verify performance under expected data rates

10.5. PipelineBuilder

10.5.1. Overview

The `PipelineBuilder` class provides a fluent interface for constructing pipelines with automatic validation. It simplifies pipeline creation by:

- **Supporting flexible interleaving** of stage additions and connections
- **Providing diagnostic warnings** for disconnected stages
- **Supporting method chaining** for concise pipeline definitions
- **Providing compile-time type safety** with template methods

The builder pattern separates pipeline configuration from execution, making complex pipeline construction more readable and less error-prone.

10.5.2. Key Concepts

Builder Pattern

`PipelineBuilder` uses the builder pattern to incrementally construct a pipeline. This separates the construction logic from the pipeline object itself:

```
// Instead of manually creating and wiring a pipeline:
auto pipeline = std::make_shared<Pipeline>("my_pipeline");
pipeline->add_stage(stage1, StageType::GENERAL);
pipeline->add_stage(stage2, StageType::GENERAL);
stage1->add_subscriber(stage2);

// Use the builder for cleaner, validated construction:
PipelineBuilder builder;
builder.add_stage(stage1)
    .add_stage(stage2);
builder.connect("stage1", "stage2");
auto pipeline = builder.build("my_pipeline");
```

Fluent API

All builder methods return `*this`, allowing method chaining for concise pipeline definitions:

```
builder.add_stage(frontend)
    .add_stage(inference)
    .add_stage(postprocess)
    .connect("frontend", "inference")
    .connect("inference", "postprocess")
    .build("detection_pipeline");
```

Flexible Construction Order

`PipelineBuilder` allows flexible interleaving of `add_stage()` and `connect()` calls. You can add all stages first and then connect them, or interleave additions and connections:

```
// Add all stages first, then connect
builder.add_stage(stage1)
    .add_stage(stage2)
    .add_stage(stage3)
    .connect("stage1", "stage2")
    .connect("stage2", "stage3")
    .build("pipeline");

// Or interleave additions and connections
builder.add_stage(stage1)
    .add_stage(stage2)
    .connect("stage1", "stage2")
    .add_stage(stage3)
    .connect("stage2", "stage3")
    .build("pipeline");
```

Both approaches are valid and produce the same result.

10.5.3. API Guide

PipelineBuilder Class

`class hailo_analytics::pipeline::PipelineBuilder`
Builder class for constructing pipelines with automatic validation.

[PipelineBuilder](#) provides a fluent interface for building pipelines with compile-time and runtime validation. It allows flexible interleaving of stage additions and connections to support complex pipeline topologies, including pipelines that contain other pipelines as stages.

Key Features:

- Fluent API with method chaining
- Automatic stage name extraction from [Stage](#) objects
- Type-safe stage addition with compile-time checking
- Validates all stages are used in connections at build time
- Supports both generic connections and frontend multi-stream connections
- Single-stage pipelines require no connections
- Flexible: supports interleaved add/connect operations

Typical Usage Pattern:

```
PipelineBuilder builder;
builder.add_stage(inference_stage)
    .add_stage(postprocess_stage)
    .connect("inference", "postprocess")
    .build("my_pipeline");
```

Public Functions

PipelineBuilder &add_stage (const std::string &name, StagePtr stage, *StageType* type = *StageType::GENERAL*)
Adds a stage to the pipeline with explicit name and type.

The template parameter is validated at compile-time to ensure it derives from *Stage*. *Stage* names must be unique within a pipeline. Can be called at any time before *build()*.

Template Parameters T – The stage type (must derive from *Stage*)

Parameters

- name – The name to register this stage under
- stage – Shared pointer to the stage
- type – The stage category (SOURCE, GENERAL, or SINK)

Throws std::invalid_argument – if stage is null or name already exists

Returns Reference to this builder for method chaining

PipelineBuilder &add_stage (StagePtr stage, *StageType* type = *StageType::GENERAL*)
Adds a stage to the pipeline using the stage's own name.

The stage name is extracted using stage->get_name(). This is the preferred method when the stage already has a meaningful name. Can be called at any time before *build()*.

Parameters

- stage – Shared pointer to the stage
- type – The stage category (SOURCE, GENERAL, or SINK)

Throws std::invalid_argument – if stage is null or stage name already exists

Returns Reference to this builder for method chaining

PipelineBuilder &connect (const std::string &sourceName, const std::string &targetName)
Connects a source stage's output to a target stage's input.

Uses the standard add_subscriber mechanism for generic stage connections. Can be called at any time before *build()*, allowing flexible interleaving with *add_stage()*.

Parameters

- sourceName – The name of the stage sending buffers
- targetName – The name of the stage receiving buffers

Throws std::invalid_argument – if either stage name doesn't exist

Returns Reference to this builder for method chaining

PipelineBuilder &connect_frontend (const std::string &frontendName, const std::string &streamId, ...)
Connects a FrontendStage's specific stream output to a target stage.

FrontendStages can produce multiple output streams. This method connects a specific stream to a target stage. Can be called at any time before *build()*, allowing flexible interleaving with *add_stage()*.

Parameters

- `frontendName` - The name of the FrontendStage
- `streamId` - The stream identifier string
- `targetName` - The name of the target stage receiving this stream

Throws `std::invalid_argument` - if either stage name doesn't exist

Returns Reference to this builder for method chaining

`std::shared_ptr<Pipeline> build( std::string name, bool trace_processing_operations = true )`
Builds and returns the configured pipeline.

Build Process:

- Creates [Pipeline](#) and adds all stages with their types
- Establishes all generic connections
- Establishes all frontend subscriptions

The builder supports flexible pipeline topologies including:

- Single-stage pipelines (no connections required)
- Multi-stage connected pipelines
- Pipelines with disconnected stages (if needed by the application)
- Pipelines containing other pipelines as stages

Parameters

- `name` - The name for the created pipeline
- `trace_processing_operations` - If true, enables performance tracing

Returns Shared pointer to the constructed [Pipeline](#)

[PipelineBuilder](#) &export_to_dot(const std::string &filename)
Export the pipeline structure to a DOT file for visualization.

Creates a GraphViz DOT file representing the pipeline structure.

Parameters `filename` - Path to the output DOT file

Returns Reference to this [PipelineBuilder](#) for method chaining

10.5.4. Usage Examples

Simple Two-Stage Pipeline

Basic pipeline with inference and postprocessing:

```
#include "hailo_analytics/pipeline/core/pipeline_builder.hpp"
#include "hailo_analytics/pipeline/ai/hailort_async_stage.hpp"
#include "hailo_analytics/pipeline/postprocess/postprocess_stage.hpp"

using namespace hailo_analytics::pipeline;
namespace ai_stages = hailo_analytics::pipeline::ai;

// Create stages
auto inference_stage = ai_stages::HailortAsyncStageBuild::create()
    .set_stage_name("inference")
    .set_hef_path("/path/to/hef");
```

(continues on next page)

(continued from previous page)

```
auto postprocess_stage = ai_stages::PostprocessStageBuild::create()
    .set_so_path("/path/to/postprocess.so")
    .set_function_name_opt("postprocess");

// Build pipeline
PipelineBuilder builder;
auto pipeline = builder.add_stage(inference_stage)
    .add_stage(postprocess_stage)
    .connect("inference", "postprocess")
    .build("detection");
```

Multi-Stream Frontend Pipeline

Pipeline with frontend producing multiple output streams:

```
using namespace hailo_analytics::pipeline;

// Create frontend stage
sources::FrontendStageBuild::Builder frontend_builder = sources::
    ↪FrontendStageBuild::create();
frontend_builder.set_stage_name("frontend")
    .set_queue_size_opt(1)
    .set_leaky_opt(false);
std::shared_ptr<sources::FrontendStage> frontend_stage = frontend_builder.
    ↪buildptr();

// Configure frontend (assumes media_library->m_frontend exists)
frontend_stage->configure(media_library->m_frontend);

// Create processing stages for different streams
auto process0 = /* ... create stage named "process0" ... */;
auto process1 = /* ... create stage named "process1" ... */;

// Build pipeline with frontend connections
PipelineBuilder builder;
auto pipeline = builder.add_stage(frontend_stage, StageType::SOURCE)
    .add_stage(process0)
    .add_stage(process1)
    .connect_frontend("frontend", "stream0", "process0")
    .connect_frontend("frontend", "stream1", "process1")
    .build("multi_stream");
```

Encoder and UDP Output Pipeline

Pipeline with encoding and UDP vision output:

```
using namespace hailo_analytics::pipeline;

// Create encoder stage
codecs::EncoderStageBuild::Builder encoder_builder = codecs::EncoderStageBuild::
    ↪create();
encoder_builder.set_stage_name("encoder")
    .set_queue_size_opt(1)
    .set_leaky_opt(false);
std::shared_ptr<codecs::EncoderStage> encoder_stage = encoder_builder.buildptr();

// Configure encoder (assumes encoder MediaLibraryEncoderPtr exists)
```

(continues on next page)

(continued from previous page)

```
encoder_stage->configure(encoder);

// Create UDP sink stage
sinks::UdpStageBuild::Builder udp_builder = sinks::UdpStageBuild::create();
udp_builder.set_stage_name("udp")
    .set_queue_size_opt(1)
    .set_leaky_opt(false);
std::shared_ptr<sinks::UdpStage> udp_stage = udp_builder.buildptr();

// Configure UDP (host, port, encoding type)
udp_stage->configure("192.168.1.100", "5000", EncodingType::H264);

// Build pipeline
PipelineBuilder builder;
auto pipeline = builder.add_stage(encoder_stage, StageType::SINK)
    .add_stage(udp_stage, StageType::SINK)
    .connect("encoder", "udp")
    .build("output_pipeline");
```

Pipeline with Overlay Stage

Adding overlay for visualization:

```
using namespace hailo_analytics::pipeline;
namespace overlay_stage = hailo_analytics::pipeline::overlay;

// Create overlay stage
overlay_stage::OverlayStageBuild::Builder overlay_builder = overlay_stage::
    ↳OverlayStageBuild::create();
overlay_builder.set_stage_name("overlay")
    .set_queue_size(5)
    .set_leaky_opt(false)
    .set_skip_opt(false);
std::shared_ptr<overlay_stage::OverlayStage> overlay_stage_ptr = overlay_builder.
    ↳buildptr();

// Combine with other stages
auto postprocess_stage = /* ... create postprocess stage ... */;
auto encoder_stage = /* ... create encoder stage ... */;

PipelineBuilder builder;
auto pipeline = builder.add_stage(postprocess_stage)
    .add_stage(overlay_stage_ptr)
    .add_stage(encoder_stage, StageType::SINK)
    .connect("postprocess", "overlay")
    .connect("overlay", "encoder")
    .build("overlay_pipeline");
```

10.5.5. Best Practices

Use Stage Names Consistently

Prefer using the stage's own name via the single-parameter `add_stage()`:

```
// Good: Uses stage's internal name
builder.add_stage(inference_stage)
    .connect("inference", "postprocess"); // Must match stage->get_name()
```

Specify Stage Types

Explicitly set `StageType` for SOURCE and SINK stages to ensure proper startup/shutdown ordering:

```
builder.add_stage(frontend, StageType::SOURCE) // Start first
    .add_stage(processing) // StageType::GENERAL (default)
    .add_stage(encoder, StageType::SINK); // Start last, stop first
```

Diagnostic Warnings

The builder logs diagnostic warnings for stages that are not connected to any other stage in multi-stage pipelines. This helps catch potential configuration issues:

```
PipelineBuilder builder;
builder.add_stage(stage1)
    .add_stage(stage2)
    .add_stage(stage3) // Not connected
    .connect("stage1", "stage2")
    .build("pipeline"); // Logs warning: stage3 not connected
```

Note: Single-stage pipelines do not require connections and will not generate warnings. Disconnected stages in multi-stage pipelines may be intentional depending on your design.

Use Chaining for Readability

Chain all builder calls in a single expression for compact, readable pipeline definitions:

```
auto pipeline = PipelineBuilder()
    .add_stage(frontend, StageType::SOURCE)
    .add_stage(ai_stage)
    .add_stage(post_stage)
    .add_stage(output_stage, StageType::SINK)
    .connect("frontend", "ai_stage")
    .connect("ai_stage", "post_stage")
    .connect("post_stage", "output_stage")
    .build("my_pipeline", true); // Enable tracing
```

10.6. Available Stages

A variety of pre-defined stages are available in Hailo AI Analytics to facilitate common processing tasks. These stages can be easily integrated into your analytics pipelines to perform functions such as AI inference, video encoding, and network transmission.

10.6.1. Ai Stages

Ai Stage

API Reference

HailortAsyncStage Class

```
class hailo_analytics::pipeline::ai::HailortAsyncStage: public
hailo_analytics::pipeline::ThreadedStage
```

Class representing an asynchronous HailoRT stage in a connected stage pipeline.

This class is responsible for managing the HailoRT inference model, setting up buffer pools, and executing asynchronous inference jobs.

Subclassed by [hailo_analytics::pipeline::ai::HailortAsyncStageBuild](#)

Public Functions

HailortAsyncStage(std::string name, std::string hef_path, size_t queue_size, int output_pool_size, ...))
Construct a new [HailortAsyncStage](#) object.

Parameters

- **name** – Name of the stage.
- **hef_path** – Path to the Hailo Execution File (HEF).
- **queue_size** – Size of the processing queue.
- **output_pool_size** – Size of the output buffer pool.
- **group_id** – Group ID for the HailoRT device.
- **batch_size** – Batch size for inference.
- **scheduler_threshold** – Threshold for the scheduler.
- **scheduler_timeout** – Timeout for the scheduler.
- **print_fps** – Whether to print frames per second information.

virtual [AppStatus](#) **init**() override
Initialize the HailoRT stage.

Returns AppStatus Status of the initialization.

virtual [AppStatus](#) **deinit**() override
Deinitialize the HailoRT stage.

Returns AppStatus Status of the deinitialization.

[AppStatus](#) **set_pix_buf**(const HailoMediaLibraryBufferPtr buffer)
Set pixel buffer for the inference input.

Parameters **buffer** – [Buffer](#) containing the pixel data.

Returns AppStatus Status of setting the pixel buffer.

[AppStatus](#) **acquire_and_set_tensor_buffers**(std::unordered_map<std::string, BufferPtr> ...)
Acquire and set tensor buffers for the inference output.

Parameters `tensor_buffers` - Map of tensor buffers to be acquired and set.

Returns `AppStatus` Status of acquiring and setting the tensor buffers.

AppStatus `infer ( BufferPtr input_buffer, const std::unordered_map<std::string, BufferPtr> ... )`
Perform inference on the input buffer by triggering an async infer job. This job calls the given callback on completion.

Parameters

- `input_buffer` - *Buffer* containing the input data.
- `tensor_buffers` - Map of tensor buffers for the inference output.

Returns `AppStatus` Status of the inference.

virtual *AppStatus* `process ( BufferPtr data )` override
Process the data in the buffer.

Parameters `data` - *Buffer* containing the data to be processed.

Returns `AppStatus` Status of the processing.

HailortAsyncStageBuild Class

`class hailo_analytics::pipeline::ai::HailortAsyncStageBuild: public
hailo_analytics::pipeline::ai::HailortAsyncStage`

Public Static Functions

static *Builder* `create ( )`

class `Builder`

Public Functions

Builder &`set_stage_name ( std::string name )`

Builder &`set_hef_path ( std::string path )`

Builder &`set_queue_size ( size_t size )`

Builder &`set_output_pool_size ( int size )`

Builder &`set_group_id ( std::string id )`

Builder &`set_batch_size ( int size )`

Builder &`set_job_limit ( size_t size )`

Builder &`set_scheduler_threshold_opt ( int threshold )`

Builder &`set_dynamic_threshold_opt ( bool activate )`

Builder &`set_scheduler_timeout_opt ( std::chrono::milliseconds timeout )`

Builder &`set_printfps_opt ( bool activate )`

```

Builder &set_pool_mode_opt ( StagePoolMode mode )

Builder &set_nms_score_threshold ( float32_t score_threshold )

Builder &set_nms_classes_filter_mask ( std::vector<bool> mask )

Builder &set_nms_max_accumulated_mask_size_multiplier ( size_t multiplier )

Builder &set_trace_opt ( bool activate )

Builder &set_use_hailort_service ( bool use_service )

std::shared_ptr<HailortAsyncStage> buildptr ( ) const

```

Postprocess Stage

API Reference

PostprocessStage Class

```

class hailo_analytics::pipeline::ai::PostprocessStage : public
hailo_analytics::pipeline::ThreadedStage

```

Class representing a post-processing stage in the connected stage pipeline.

This class is responsible for loading a shared object library, initializing it, and applying post-processing functions to the data.

Subclassed by [hailo_analytics::pipeline::ai::PostprocessStageBuild](#)

Public Functions

```

PostprocessStage ( std::string name, std::string so_path, std::string function_name = ... )
Construct a new Postprocess Stage object.

```

Parameters

- `name` – Name of the stage.
- `so_path` – Path to the shared object file.
- `function_name` – Name of the function to be executed from the shared object file.
- `config_path` – Path to the configuration file.
- `queue_size` – Size of the processing queue.
- `leaky` – Whether the queue is leaky.
- `print_fps` – Whether to print frames per second information.

```

virtual AppStatus init ( ) override

```

Initialize the post-processing stage. by loading the provided so file with dlsym. If the binary has an init function to load parameters then it is called.

Returns AppStatus Status of the initialization.

```

virtual AppStatus deinit ( ) override

```

Deinitialize the post-processing stage loaded library.

Returns AppStatus Status of the deinitialization.

virtual *AppStatus* process (BufferPtr data) override
Process the data in the buffer using the loaded library.

Parameters data – *Buffer* containing the data to be processed.

Returns AppStatus Status of the processing.

PostprocessStageBuild Class

```
class hailo_analytics::pipeline::ai::PostprocessStageBuild: public
hailo_analytics::pipeline::ai::PostprocessStage
```

Public Static Functions

```
static Builder create( )
```

```
class Builder
```

Public Functions

```
Builder &set_stage_name( std::string name )
```

```
Builder &set_so_path( std::string path )
```

```
Builder &set_function_name_opt( std::string func_name )
```

```
Builder &set_config_path_opt( std::string path )
```

```
Builder &set_queue_size_opt( size_t size )
```

```
Builder &set_leaky_opt( bool activate )
```

```
Builder &set_trace_opt( bool activate )
```

```
std::shared_ptr<PostprocessStage> buildptr( ) const
```

Analytics DB Stage

API Reference

AnalyticsDBStage Class

```
class hailo_analytics::pipeline::ai::AnalyticsDBStage: public
hailo_analytics::pipeline::ThreadedStage
```

Stage for processing AI analytics data and storing it in the Analytics Database.

This stage processes different types of analytics data (detection, instance segmentation, semantic segmentation) from AI pipeline buffers and stores them in the Analytics Database for later retrieval and processing. The stage extracts relevant metadata, converts coordinates, and maintains tensor buffer references to ensure data validity.

Supported analytics types:

- Detection: Processes bounding boxes and stores detection results

- Instance Segmentation: Processes segmentation masks with bounding boxes
- Semantic Segmentation: Processes semantic masks with class labels

Subclassed by [hailo_analytics::pipeline::ai::AnalyticsDBStageBuild](#)

Public Functions

[AnalyticsDBStage](#)(const std::string &name, size_t queue_size, bool leaky, const std::string ...)
 Constructor for [AnalyticsDBStage](#).

Parameters

- name – [Stage](#) name for identification
- queue_size – Size of the processing queue
- leaky – Whether the queue should drop old frames when full
- analytics_data_id – Identifier for the analytics data in the database
- type – Type of analytics to process (default: INSTANCE_SEGMENTATION)
- trace_processing_operations – Enable tracing for processing operations (default: true)

virtual [AppStatus](#) process(BufferPtr data) override
 Process buffer and add analytics data to database.

Parameters data – [Buffer](#) containing analytics data to process

Returns AppStatus indicating success or failure

AnalyticsDBStageBuild Class

class hailo_analytics::pipeline::ai::AnalyticsDBStageBuild: public
[hailo_analytics::pipeline::ai::AnalyticsDBStage](#)
[Builder](#) pattern implementation for [AnalyticsDBStage](#).

Provides a fluent interface for constructing [AnalyticsDBStage](#) instances with configurable parameters.

Public Static Functions

static [Builder](#) create()
 Create a new [Builder](#) instance.

Returns [Builder](#) instance for constructing [AnalyticsDBStage](#)

class Builder
[Builder](#) class for constructing [AnalyticsDBStage](#) instances.

Public Functions

[Builder](#) &set_stage_name(std::string name)
 Set the stage name.

Parameters name – Name for the stage

Returns [Builder](#) reference for chaining

[Builder](#) &set_queue_size(size_t size)
 Set the queue size.

Parameters size – Size of the processing queue

Returns [Builder](#) reference for chaining

[Builder](#) &set_leaky_opt(bool activate)
 Set the leaky option.

Parameters activate – If true, queue drops old frames when full

Returns [Builder](#) reference for chaining

Builder &set_analytics_data_id(const std::string &analytics_data_id)

Set the analytics data ID.

Parameters analytics_data_id - Identifier for analytics data in database

Returns *Builder* reference for chaining

Builder &set_type(*AnalyticsType* type)

Set the analytics type.

Parameters type - Type of analytics to process

Returns *Builder* reference for chaining

Builder &set_trace_opt(bool activate)

Set the trace option.

Parameters activate - If true, enables tracing for processing operations

Returns *Builder* reference for chaining

std::shared_ptr<*AnalyticsDBStage*> buildptr() const

Build and return shared pointer to *AnalyticsDBStage*.

Throws std::runtime_error - if required parameters are missing

Returns Shared pointer to constructed *AnalyticsDBStage*

Lightweight Tracker Stage

API Reference

LightweightTrackerStage Class

class hailo_analytics::pipeline::ai::LightweightTrackerStage: public

hailo_analytics::pipeline::ThreadedStage

Stage for tracking detections across frames using a lightweight tracker.

This stage applies object tracking to detections in the pipeline, maintaining consistent tracking IDs for detected objects across frames. It supports per-class tracker configurations and can filter out non-tracked classifications.

Features:

- Per-class tracker instances with independent parameters
- Configurable tracking parameters (IOU threshold, history size, etc.)
- Optional blocking of non-tracked object classes
- Lightweight implementation suitable for real-time processing

Subclassed by *hailo_analytics::pipeline::ai::LightweightTrackerStageBuild*

Public Functions

LightweightTrackerStage(std::string name, std::map<int, TrackerParams> tracker_params, ...)

Constructor for *LightweightTrackerStage*.

Parameters

- name - *Stage* name for identification
- tracker_params - Map of class IDs to tracker parameters
- queue_size - Size of the processing queue (default: TRACKER_QUEUE_SIZE_DEFAULT)
- leaky - Whether the queue should drop old frames when full (default: false)
- classification_ids - Vector of class IDs to track (default: empty)
- block_non_tracked_class - If true, removes detections not in classification_ids (default: false)

- `trace_processing_operations` - Enable tracing for processing operations (default: true)

virtual `AppStatus` `process` (`BufferPtr` data) override
Process buffer and apply tracking to detections.

Parameters data - `Buffer` containing detection data to track

Returns `AppStatus` indicating success or failure

LightweightTrackerStageBuild Class

`class hailo_analytics::pipeline::ai::LightweightTrackerStageBuild: public
hailo_analytics::pipeline::ai::LightweightTrackerStage
Builder` pattern implementation for `LightweightTrackerStage`.

Provides a fluent interface for constructing `LightweightTrackerStage` instances with configurable tracker parameters. Supports per-class configuration or global configuration (when `class_id = -1`) that applies to all tracked classes.

Public Static Functions

static `Builder` `create` ()
Create a new `Builder` instance.

Returns `Builder` instance for constructing `LightweightTrackerStage`

`class Builder`
`Builder` class for constructing `LightweightTrackerStage` instances.

Public Functions

`Builder` &`set_stage_name` (`std::string` name)
Set the stage name.

Parameters name - Name for the stage

Returns `Builder` reference for chaining

`Builder` &`set_queue_size_opt` (`size_t` size)
Set the queue size.

Parameters size - Size of the processing queue

Returns `Builder` reference for chaining

`Builder` &`set_leaky_opt` (`bool` activate)
Set the leaky option.

Parameters activate - If true, queue drops old frames when full

Returns `Builder` reference for chaining

`Builder` &`set_classification_ids` (`const std::vector<int>` &ids)
Set the classification IDs to track.

Parameters ids - Vector of class IDs to track

Returns `Builder` reference for chaining

`Builder` &`set_trace_opt` (`bool` activate)
Set the trace option.

Parameters activate - If true, enables tracing for processing operations

Returns `Builder` reference for chaining

`Builder` &`set_history_size` (`size_t` history_size, `int` class_id = -1)
Set the history size for tracker.

Parameters

- `history_size` - Number of frames to keep in tracking history
- `class_id` - Class ID to configure, or -1 for all classes (default: -1)

Returns `Builder` reference for chaining

Builder &set_smooth_alpha (float smooth_alpha, int class_id = -1)

Set the smoothing alpha parameter.

Parameters

- smooth_alpha - Alpha value for exponential smoothing (0.0 to 1.0)
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_iou_threshold (float iou_threshold, int class_id = -1)

Set the IOU threshold for matching detections to tracks.

Parameters

- iou_threshold - Minimum IOU value for matching (0.0 to 1.0)
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_weighted_average_decay (float weighted_average_decay, int class_id = -1)

Set the weighted average decay parameter.

Parameters

- weighted_average_decay - Decay factor for weighted averaging
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_grid_size (size_t grid_size, int class_id = -1)

Set the grid size for spatial hashing.

Parameters

- grid_size - Size of grid for spatial hashing optimization
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_grace_period (int grace_period, int class_id = -1)

Set the grace period for tracks.

Parameters

- grace_period - Number of frames to keep tracks alive without detection
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_add_tracking_id (bool add_tracking_id, int class_id = -1)

Set whether to add tracking IDs to detections.

Parameters

- add_tracking_id - If true, adds tracking ID to detection objects
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_copy_nested_objects (bool copy_nested_objects, int class_id = -1)

Set whether to copy nested objects during tracking.

Parameters

- copy_nested_objects - If true, copies nested objects from detections
- class_id - Class ID to configure, or -1 for all classes (default: -1)

Returns *Builder* reference for chaining

Builder &set_block_non_tracked_classification_id (bool block)

Set whether to block non-tracked classification IDs.

Parameters block - If true, removes detections not in classification_ids list

Returns *Builder* reference for chaining

std::shared_ptr<*LightweightTrackerStage*> buildptr () const

Build and return shared pointer to *LightweightTrackerStage*.

Throws std::runtime_error - if required parameters are missing

Returns Shared pointer to constructed *LightweightTrackerStage*

10.6.2. Overlay Stages

Overlay Stage

API Reference

HailoOverlay Struct

```
struct hailo_analytics::pipeline::overlay::HailoOverlay
```

Structure containing configuration parameters for overlay.

Public Members

```
int line_thickness
```

Line thickness for overlay.

```
int font_thickness
```

Font thickness for overlay.

```
float landmark_point_radius
```

Radius for landmark points.

```
bool face_blur
```

Enable or disable face blur.

```
bool show_confidence
```

Enable or disable confidence display.

```
bool local_gallery
```

Enable or disable local gallery usage.

```
uint mask_overlay_n_threads
```

Number of threads for mask overlay.

OverlayStage Class

```
class hailo_analytics::pipeline::overlay::OverlayStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class responsible for handling overlay stage that will do the drawing.

Subclassed by [hailo_analytics::pipeline::overlay::OverlayStageBuild](#)

Public Functions

```
OverlayStage (std::string name, bool skip = false, bool partial_landmarks = false, ...)
```

Constructor for [OverlayStage](#).

Parameters

- `name` - The name of the stage.
- `queue_size` - Size of the queue for this stage.
- `leaky` - Indicates if the queue is leaky.
- `print_fps` - Flag to enable or disable printing FPS information.

```
virtual AppStatus init () override
```

Initialize the overlay stage.

Returns Status of the initialization.

virtual *AppStatus* deinit () override
Deinitialize the overlay stage.

Returns Status of the deinitialization.

virtual *AppStatus* process (BufferPtr data) override
Process the given data buffer and apply overlay.

Parameters data - The data buffer to process.

Returns Status of the processing.

void set_skip (bool skip)
Set the overlay skip flag.

Parameters skip - Flag to set the skip state.

bool get_skip ()
Get the overlay skip flag.

Returns Current skip state.

OverlayStageBuild Class

class hailo_analytics::pipeline::overlay::OverlayStageBuild: public

hailo_analytics::pipeline::overlay::OverlayStage

Builder-based overlay stage for simplified construction.

Provides a builder pattern interface for creating overlay stages with configurable parameters for drawing customization.

Public Static Functions

static *Builder* create ()
Create a new *Builder* for *OverlayStage* construction.

Returns *Builder* instance.

class Builder

Builder class for *OverlayStage* construction.

Public Functions

Builder &set_stage_name (std::string name)
Set the stage name.

Parameters name - Name for the overlay stage.

Returns Reference to this builder for chaining.

Builder &set_skip_opt (bool skip)
Set whether to skip overlay drawing (optional).

Parameters skip - True to skip drawing.

Returns Reference to this builder for chaining.

Builder &set_partial_landmarks (bool partial_landmarks)
Set whether to draw partial landmarks.

Parameters partial_landmarks - True to enable partial landmark drawing.

Returns Reference to this builder for chaining.

Builder &set_landmark_indices_to_draw (std::unordered_set<size_t> indices)
Set which landmark indices to draw.

Parameters indices - Set of landmark indices to render.

Returns Reference to this builder for chaining.

Builder &set_queue_size (size_t size)
Set the queue size.

Parameters size - [Queue](#) size.

Returns Reference to this builder for chaining.

[Builder](#) &set_leaky_opt (bool activate)
Set whether the queue is leaky (optional).

Parameters activate - True to enable leaky mode.

Returns Reference to this builder for chaining.

[Builder](#) &set_class_ids_to_draw (std::unordered_set<int> class_ids_to_draw)
Set which class IDs to draw.

Parameters class_ids_to_draw - Set of class IDs to render.

Returns Reference to this builder for chaining.

[Builder](#) &set_color_selector (std::function<cv::Scalar(const HailoDetectionPtr&> ...)
Set a custom color selector function.

Parameters color_selector - Function that returns a color based on detection.

Returns Reference to this builder for chaining.

[Builder](#) &set_trace_opt (bool activate)
Set whether to enable tracing (optional).

Parameters activate - True to enable tracing.

Returns Reference to this builder for chaining.

std::shared_ptr<[OverlayStage](#)> buildptr () const
Build and return the [OverlayStage](#).

Returns Shared pointer to the constructed [OverlayStage](#).

10.6.3. Codec Stages

Encoder Stage

API Reference

EncoderStage Class

```
class hailo_analytics::pipeline::codecs::EncoderStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class representing an encoder stage in the pipeline.

This stage encodes video frames using the media library encoder. It receives buffers from upstream stages, encodes them, and passes the encoded data to downstream stages.

Subclassed by [hailo_analytics::pipeline::codecs::EncoderStageBuild](#)

Public Functions

EncoderStage (std::string name, size_t queue_size = ENCODER_QUEUE_SIZE_DEFAULT, bool leaky = ...)
Construct a new [EncoderStage](#) object.

Parameters

- name - Name of the stage.
- queue_size - Size of the processing queue.
- leaky - If true, drops oldest buffer when queue is full.
- trace_processing_operations - If true, enables performance tracing.

[AppStatus](#) create ([MediaLibraryEncoder](#) &encoder)
Create the encoder stage with an encoder instance.

Parameters encoder - Media library encoder pointer.

Returns AppStatus Status of the creation.

virtual *AppStatus* init () override
Initialize the encoder stage.

Returns *AppStatus* Status of the initialization.

virtual *AppStatus* deinit () override
Deinitialize the encoder stage.

Returns *AppStatus* Status of the deinitialization.

AppStatus configure (*MediaLibraryEncoder* &encoder)
Configure the encoder stage with an encoder instance.

Parameters encoder – Media library encoder pointer.

Returns *AppStatus* Status of the configuration.

virtual *AppStatus* process (BufferPtr data)
Process a buffer by encoding it.

Parameters data – *Buffer* containing the frame to encode.

Returns *AppStatus* Status of the processing.

EncoderStageBuild Class

class hailo_analytics::pipeline::codecs::EncoderStageBuild: public

hailo_analytics::pipeline::codecs::EncoderStage
Builder-based encoder stage for simplified construction.

Provides a builder pattern interface for creating encoder stages with configurable parameters.

Public Static Functions

static *Builder* create ()
Create a new *Builder* for *EncoderStage* construction.

Returns *Builder* instance.

class Builder
Builder class for *EncoderStage* construction.

Public Functions

Builder &set_stage_name (std::string name)
Set the stage name.
Parameters name – Name for the encoder stage.
Returns Reference to this builder for chaining.

Builder &set_queue_size_opt (size_t size)
Set the queue size (optional).
Parameters size – *Queue* size.
Returns Reference to this builder for chaining.

Builder &set_leaky_opt (bool activate)
Set whether the queue is leaky (optional).
Parameters activate – True to enable leaky mode.
Returns Reference to this builder for chaining.

Builder &set_trace_opt (bool activate)
Set whether to enable tracing (optional).
Parameters activate – True to enable tracing.
Returns Reference to this builder for chaining.

std::shared_ptr<*EncoderStage*> buildptr () const
Build and return the *EncoderStage*.
Returns Shared pointer to the constructed *EncoderStage*.

Analytic Metadata Packager Stage

API Reference

AnalyticMetadataPackagerStage Class

```
class hailo_analytics::pipeline::codecs::AnalyticMetadataPackagerStage: public
hailo_analytics::pipeline::ThreadedStage
```

Stage for packaging analytic metadata into JSON format for transmission.

This stage processes AI analytics data (detections, landmarks) from the pipeline and packages them into a structured JSON format. The JSON is then serialized to MessagePack binary format (default) or plain JSON string for efficient transmission. The metadata includes frame information and all detected objects with their coordinates transformed to native frame dimensions.

Features:

- Converts detections and landmarks to JSON format
- Transforms coordinates to native frame space
- Supports hierarchical object structures (nested objects)
- Optimized with buffer reuse to minimize allocations
- Outputs MessagePack binary (default) or JSON string format

Subclassed by *hailo_analytics::pipeline::codecs::AnalyticMetadataPackagerStageBuild*

Public Functions

```
AnalyticMetadataPackagerStage( std::string name, Format format = Format::MSGPACK, ... )
    Constructor for AnalyticMetadataPackagerStage.
```

Parameters

- *name* – *Stage* name for identification
- *format* – Output serialization format (default: MSGPACK)
- *queue_size* – Size of the processing queue (default: ANALYTIC_METADATA_QUEUE_SIZE_DEFAULT)
- *leaky* – Whether the queue should drop old frames when full (default: false)
- *trace_processing_operations* – Enable tracing for processing operations (default: true)

```
virtual AppStatus process( BufferPtr data ) override
    Process incoming data buffer and package analytic metadata into JSON format.
```

Parameters *data* – Input buffer containing AI analytics data

Returns AppStatus indicating success or failure of processing

AnalyticMetadataPackagerStageBuild Class

```
class hailo_analytics::pipeline::codecs::AnalyticMetadataPackagerStageBuild
: public hailo_analytics::pipeline::codecs::AnalyticMetadataPackagerStage
    Builder pattern implementation for AnalyticMetadataPackagerStage.
```

Provides a fluent interface for constructing *AnalyticMetadataPackagerStage* instances with configurable parameters.

Public Static Functions

static *Builder* create()
Create a new *Builder* instance.

Returns *Builder* instance for constructing *AnalyticMetadataPackagerStage*

class *Builder*
Builder class for constructing *AnalyticMetadataPackagerStage* instances.

Public Functions

Builder &set_stage_name (std::string name)
Set the stage name.

Parameters name – Name for the stage

Returns *Builder* reference for chaining

Builder &set_format_opt (Format format)
Set the output serialization format.

Parameters format – Output format (MSGPACK or JSON)

Returns *Builder* reference for chaining

Builder &set_queue_size_opt (size_t size)
Set the queue size.

Parameters size – Size of the processing queue

Returns *Builder* reference for chaining

Builder &set_leaky_opt (bool activate)
Set the leaky option.

Parameters activate – If true, queue drops old frames when full

Returns *Builder* reference for chaining

Builder &set_trace_opt (bool activate)
Set the trace option.

Parameters activate – If true, enables tracing for processing operations

Returns *Builder* reference for chaining

std::shared_ptr<*AnalyticMetadataPackagerStage*> buildptr () const
Build and return shared pointer to *AnalyticMetadataPackagerStage*.

Throws std::runtime_error – if required parameters are missing

Returns Shared pointer to constructed *AnalyticMetadataPackagerStage*

10.6.4. Cropping Stages

DSP Cropping Base

API Reference

DspBaseCropStage Class

class hailo_analytics::pipeline::cropping::DspBaseCropStage : public
hailo_analytics::pipeline::ThreadedStage
Base class for DSP-accelerated cropping stages.

This abstract base class provides the core functionality for cropping video frames using DSP acceleration. It manages buffer pools, coordinates, scaling modes, and the cropping pipeline. Derived classes implement the specific cropping logic (e.g., bounding box crops, tiling).

Features:

- DSP-accelerated cropping and resizing
- Configurable scaling modes (letterbox or stretch)

- Frame skip support (crop every N frames)
- *Buffer* pool management
- Subscriber pattern for main and sub-outputs

Subclassed by *hailo_analytics::pipeline::cropping::BBoxCropStage*, *hailo_analytics::pipeline::cropping::TilingCropStage*

Public Functions

`DspBaseCropStage (std::string name, int output_pool_size, int input_width, int input_height, int ...)`
 Constructor to initialize the stage with specified parameters.

Parameters

- `name` - Name of the stage.
- `output_pool_size` - Size of the output buffer pool.
- `input_width` - Width of the input data.
- `input_height` - Height of the input data.
- `output_width` - Width of the output data.
- `output_height` - Height of the output data.
- `main_sub_name` - Name of the main subscriber.
- `sub_sub_name` - Name of the sub-subscriber.
- `queue_size` - Size of the queue.
- `leaky` - Boolean flag for leaky behavior.
- `print_fps` - Boolean flag for printing FPS.
- `crop_every_n_frames` - Crop every n frames (default 1)

`void set_crop_every_x_frames (int crop_every_x_frames)`
 Sets the crop frequency (how often frames are cropped).

This can be called at runtime to adjust the inference rate without recreating the pipeline.

Parameters `crop_every_x_frames` - Crop every N frames (1 = every frame).

`virtual void prepare_single_crop_dim (HailoBBox bbox, std::vector<dsp_crop_api_t> ...)`
 Prepares cropping dimensions for a single bounding box.

Parameters

- `bbox` - Bounding box for cropping.
- `crop_resize_dims` - Vector to store crop resize dimensions.
- `input_width` - Input buffer width.
- `input_height` - Input buffer height.

`virtual void prepare_crops (BufferPtr input_buffer, std::vector<dsp_crop_api_t> &crop_resize_dims) = 0`
 Prepares crop dimensions for the input buffer.

Parameters

- `input_buffer` - Input buffer.
- `crop_resize_dims` - Vector to store crop dimensions.

`virtual HailoBBox get_crop_bbox (int index) = 0`
 Gets the bounding box for a specific crop.

Parameters `index` - Index of the crop.

Returns Bounding box of the crop.

virtual void `post_crop`(BufferPtr input_buffer) = 0
Performs post-processing after cropping.

Parameters `input_buffer` - Input buffer.

virtual void `pre_crop`(BufferPtr input_buffer) = 0
Performs pre-processing before cropping.

Parameters `input_buffer` - Input buffer.

virtual HailoROIPtr `get_crop_roi`(int index) = 0
Gets the ROI for a specific crop.

Parameters `index` - Index of the crop.

Returns ROI of the crop.

virtual [AppStatus](#) `process`(BufferPtr data) override
Processes the data buffer, performing cropping and resizing.

Parameters `data` - Data buffer.

Returns Status of the operation.

BBox Crop Stage

API Reference

BBoxCropStage Class

class `hailo_analytics::pipeline::cropping::BBoxCropStage`: public

`hailo_analytics::pipeline::cropping::DspBaseCropStage`

Derived class of [DspBaseCropStage](#) for handling cropping based on detected bounding boxes.

Subclassed by [hailo_analytics::pipeline::cropping::BBoxCropStageBuild](#)

Public Functions

`BBoxCropStage`(std::string name, int output_pool_size, int input_width, int input_height, int ...)
Constructor to initialize the stage with specified parameters.

Parameters

- `name` - Name of the stage.
- `output_pool_size` - Size of the output buffer pool.
- `input_width` - Width of the input data.
- `input_height` - Height of the input data.
- `output_width` - Width of the output data.
- `output_height` - Height of the output data.
- `main_sub_name` - Name of the main subscriber.
- `sub_sub_name` - Name of the sub-subscriber.
- `labels` - Target labels for filtering detections.
- `queue_size` - Size of the queue.
- `leaky` - Boolean flag for leaky behavior.
- `trace_processing_operations` - Boolean flag for tracing processing operations.
- `pool_mode` - Pool mode for buffer management.

- `crop_every_x_frames` - Crop every x frames.
- `use_letterbox` - Enable letterbox support.
- `letterbox_alignment` - Letterbox alignment (only used if `use_letterbox=true`).
- `letterbox_color` - Letterbox color (only used if `use_letterbox=true`).

virtual [AppStatus](#) `init()` override
Initializes the buffer pool.

Returns Status of the operation.

virtual void `prepare_crops`(BufferPtr input_buffer, std::vector<dsp_crop_api_t> &crop_resize_dims) ...)
Prepares crop dimensions based on detected bounding boxes.

Parameters

- `input_buffer` - Input buffer.
- `crop_resize_dims` - Vector to store crop dimensions.

virtual HailoBBox `get_crop_bbox`(int index) override
Gets the bounding box for a specific detection.

Parameters `index` - Index of the detection.

Returns Bounding box of the detection.

virtual HailoROIPtr `get_crop_roi`(int index) override
Gets the ROI for a specific detection.

Parameters `index` - Index of the detection.

Returns ROI of the detection.

virtual void `pre_crop`(BufferPtr input_buffer) override
Performs pre-processing before cropping.

Parameters `input_buffer` - Input buffer.

virtual void `post_crop`(BufferPtr input_buffer) override
Clears detection data after cropping.

Parameters `input_buffer` - Input buffer.

virtual [AppStatus](#) `process`(BufferPtr data) override
Override process to add letterbox support when enabled.

Parameters `data` - Input buffer to process.

Returns Status of the operation.

BBoxCropStageBuild Class

```
class hailo_analytics::pipeline::cropping::BBoxCropStageBuild: public
hailo_analytics::pipeline::cropping::BBoxCropStage
    Builder pattern implementation for BBoxCropStage.
```

Provides a fluent interface for constructing [BBoxCropStage](#) instances with configurable parameters.

Public Static Functions

static *Builder* create()
Create a new *Builder* instance.

Returns *Builder* instance for constructing *BBoxCropStage*

class *Builder*
Builder class for constructing *BBoxCropStage* instances.

Public Functions

Builder &set_stage_name(std::string name)
Set the stage name.

Parameters name - Name for the stage

Returns *Builder* reference for chaining

Builder &set_output_pool_size(int size)
Set the output pool size.

Parameters size - Size of the output buffer pool

Returns *Builder* reference for chaining

Builder &set_input_width(int size)
Set the input width.

Parameters size - Width of the input data in pixels

Returns *Builder* reference for chaining

Builder &set_input_height(int size)
Set the input height.

Parameters size - Height of the input data in pixels

Returns *Builder* reference for chaining

Builder &set_output_width(int size)
Set the output width.

Parameters size - Width of the output data in pixels

Returns *Builder* reference for chaining

Builder &set_output_height(int size)
Set the output height.

Parameters size - Height of the output data in pixels

Returns *Builder* reference for chaining

Builder &set_main_sub_name(std::string name)
Set the main subscriber name.

Parameters name - Name of the main subscriber

Returns *Builder* reference for chaining

Builder &set_sub_sub_name(std::string name)
Set the sub-subscriber name.

Parameters name - Name of the sub-subscriber

Returns *Builder* reference for chaining

Builder &set_labels(std::vector<std::string> labels)
Set the target labels for filtering detections.

Parameters labels - Vector of label strings

Returns *Builder* reference for chaining

Builder &set_queue_size(size_t size)
Set the queue size.

Parameters size - Size of the processing queue

Returns *Builder* reference for chaining

Builder &set_leaky_opt(bool activate)
Set the leaky option.

Parameters activate - If true, queue drops old frames when full

Returns *Builder* reference for chaining

Builder &set_trace_opt (bool activate)

Set the trace option.

Parameters activate – If true, enables tracing for processing operations

Returns *Builder* reference for chaining

Builder &set_pool_mode_opt (*StagePoolMode* mode)

Set the pool mode.

Parameters mode – Pool mode for buffer management

Returns *Builder* reference for chaining

Builder &set_crop_every_x_frames (size_t crop_every_x_frames)

Set the crop frequency.

Parameters crop_every_x_frames – Crop every x frames (1 = every frame)

Returns *Builder* reference for chaining

Builder &set_letterbox_opt (dsp_letterbox_alignment_t alignment = ...)

Enable letterbox mode with alignment and color.

Parameters

- alignment – Letterbox alignment (default: DSP_LETTERBOX_MIDDLE)
- color – Letterbox padding color (default: black)

Returns *Builder* reference for chaining

std::shared_ptr<*BBoxCropStage*> buildptr () const

Build and return shared pointer to *BBoxCropStage*.

Throws std::runtime_error – if required parameters are missing

Returns Shared pointer to constructed *BBoxCropStage*

Tiling Crop Stage

API Reference

TilingCropStage Class

class hailo_analytics::pipeline::cropping::TilingCropStage: public

hailo_analytics::pipeline::cropping::DspBaseCropStage

Derived class of *DspBaseCropStage* for handling specific cropping logic based on predefined tiles.

Subclassed by *hailo_analytics::pipeline::cropping::TilingCropStageBuild*

Public Functions

TilingCropStage (std::string name, int output_pool_size, int input_width, int input_height, int ...)

Constructor to initialize the stage with specified parameters.

Parameters

- name – Name of the stage.
- output_pool_size – Size of the output buffer pool.
- input_width – Width of the input data.
- input_height – Height of the input data.
- output_width – Width of the output data.
- output_height – Height of the output data.
- main_sub_name – Name of the main subscriber.
- sub_sub_name – Name of the sub-subscriber.
- queue_size – Size of the queue.
- leaky – Boolean flag for leaky behavior.

virtual [AppStatus](#) init () override
Initializes the buffer pool and tile ROIs.

Returns Status of the operation.

virtual void prepare_crops (BufferPtr input_buffer, std::vector<dsp_crop_api_t> &crop_resize_dims) ...)
Prepares crop dimensions based on tiles.

Parameters

- input_buffer - Input buffer.
- crop_resize_dims - Vector to store crop dimensions.

virtual HailoBBox get_crop_bbox (int index) override
Gets the bounding box for a specific tile.

Parameters index - Index of the tile.

Returns Bounding box of the tile.

virtual void post_crop (BufferPtr input_buffer) override
Performs post-processing after cropping.

Parameters input_buffer - Input buffer.

virtual void pre_crop (BufferPtr input_buffer) override
Performs pre-processing before cropping.

Parameters input_buffer - Input buffer.

virtual HailoROIPtr get_crop_roi (int index) override
Gets the ROI for a specific crop.

Parameters index - Index of the crop.

Returns ROI of the crop.

TilingCropStageBuild Class

class hailo_analytics::pipeline::cropping::TilingCropStageBuild: public
hailo_analytics::pipeline::cropping::TilingCropStage
[Builder](#) pattern implementation for [TilingCropStage](#).

Provides a fluent interface for constructing [TilingCropStage](#) instances with configurable parameters.

Public Static Functions

static [Builder](#) create ()
Create a new [Builder](#) instance.

Returns [Builder](#) instance for constructing [TilingCropStage](#)

class Builder
[Builder](#) class for constructing [TilingCropStage](#) instances.

Public Functions

[Builder](#) &set_stage_name (std::string name)
Set the stage name.

Parameters name - Name for the stage

Returns [Builder](#) reference for chaining

[Builder](#) &set_output_pool_size (int size)
Set the output pool size.

Parameters size - Size of the output buffer pool

Returns [Builder](#) reference for chaining

Builder &set_input_width(int size)
Set the input width.
Parameters size - Width of the input data in pixels
Returns *Builder* reference for chaining

Builder &set_input_height(int size)
Set the input height.
Parameters size - Height of the input data in pixels
Returns *Builder* reference for chaining

Builder &set_output_width(int size)
Set the output width.
Parameters size - Width of the output data in pixels
Returns *Builder* reference for chaining

Builder &set_output_height(int size)
Set the output height.
Parameters size - Height of the output data in pixels
Returns *Builder* reference for chaining

Builder &set_main_sub_name(std::string name)
Set the main subscriber name.
Parameters name - Name of the main subscriber
Returns *Builder* reference for chaining

Builder &set_sub_sub_name(std::string name)
Set the sub-subscriber name.
Parameters name - Name of the sub-subscriber
Returns *Builder* reference for chaining

Builder &set_bbox_tiles(const std::vector<HailoBBox> &bbox_tiles)
Set the tile bounding boxes.
Parameters bbox_tiles - Vector of bounding boxes defining tile regions
Returns *Builder* reference for chaining

Builder &set_queue_size(size_t size)
Set the queue size.
Parameters size - Size of the processing queue
Returns *Builder* reference for chaining

Builder &set_leaky_opt(bool activate)
Set the leaky option.
Parameters activate - If true, queue drops old frames when full
Returns *Builder* reference for chaining

Builder &set_trace_opt(bool activate)
Set the trace option.
Parameters activate - If true, enables tracing for processing operations
Returns *Builder* reference for chaining

Builder &set_pool_mode_opt(*StagePoolMode* mode)
Set the pool mode.
Parameters mode - Pool mode for buffer management
Returns *Builder* reference for chaining

Builder &set_crop_every_x_frames(size_t crop_every_x_frames)
Set the crop frequency.
Parameters crop_every_x_frames - Crop every x frames (1 = every frame)
Returns *Builder* reference for chaining

std::shared_ptr<*TilingCropStage*> buildptr() const
Build and return shared pointer to *TilingCropStage*.
Throws std::runtime_error - if required parameters are missing
Returns Shared pointer to constructed *TilingCropStage*

Aggregator Stage

API Reference

AggregatorStage Class

```
class hailo_analytics::pipeline::cropping::AggregatorStage : public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Stage for aggregating cropped subframes back into the main frame.

This stage aggregates detection results from multiple cropped subframes (e.g., tiles or bounding box crops) back into the coordinate space of the main frame. It handles:

- Coordinate transformation from subframe to main frame space
- NMS (Non-Maximum Suppression) across subframes
- Border detection removal to avoid duplicate detections at tile boundaries
- Multi-scale aggregation support
- *Metadata* migration from subframes to main frame

Subclassed by *hailo_analytics::pipeline::cropping::AggregatorStageBuild*, *hailo_analytics::pipeline::cropping::SyncAggregatorStage*

Public Functions

```
AggregatorStage ( std::string name, std::string main_inlet_name, size_t main_queue_size, bool ... )
```

Constructor for *AggregatorStage*.

Parameters

- *name* – *Stage* name for identification
- *main_inlet_name* – Name of the main frame inlet
- *main_queue_size* – Size of the main frame queue
- *main_queue_leaky* – If true, main queue drops old frames when full
- *sub_inlet_name* – Name of the subframe inlet
- *sub_queue_size* – Size of the subframe queue
- *sub_queue_leaky* – If true, subframe queue drops old frames when full
- *multi_scale* – Enable multi-scale aggregation (default: false)
- *iou_threshold* – IOU threshold for NMS (default: 0.3)
- *m_border_threshold* – Border threshold for removing edge detections (default: 0.1)
- *skip_migration* – Skip metadata migration (default: false)
- *trace_processing_operations* – Enable tracing (default: true)
- *static_sub_frames* – Fixed number of subframes, if known (default: nullopt)
- *copy_sub_frame_tensor_to_metadata* – Copy tensor metadata from subframes (default: false)

```
~AggregatorStage ( )
```

Destructor.

```
virtual void add_queue ( std::string name ) override
```

Add a queue to the stage.

Parameters *name* – Name of the queue to add

HailoBBBox create_flattened_bbox (const HailoBBBox &bbox, const HailoBBBox &parent_bbox)
Create flattened bounding box in parent coordinate space.

Parameters

- bbox – Bounding box in local coordinates
- parent_bbox – Parent bounding box

Returns Flattened bounding box

void flatten_hailo_roi (HailoROIPtr roi, HailoROIPtr parent_roi, hailo_object_t filter_type)
Flatten ROI hierarchy to parent ROI.

Parameters

- roi – ROI to flatten
- parent_roi – Parent ROI
- filter_type – Object type to filter

void remove_exceeded_bboxes (HailoROIPtr hailo_tile_roi, float border_threshold)
Remove detections close to the boundary of the tile. Not including tile borders that located on one of the borders of the full frame.

Parameters

- hailo_tile_roi – **[in]** HailoTileROIPtr taken from the buffer.
- border_threshold – **[in]** float. threshold - 0 - 1 value of 'close to border' ratio.

Returns void.

float iou_calc (const HailoBBBox &box_1, const HailoBBBox &box_2)
Calculate Intersection over Union between two bounding boxes.

Parameters

- box_1 – First bounding box
- box_2 – Second bounding box

Returns IOU value (0.0 to 1.0)

void nms (HailoROIPtr hailo_roi, const float iou_thr)
Perform IOU based NMS on detection objects of HailoRoi.

Parameters

- hailo_roi – HailoROIPtr The HailoROI contains detections to perform NMS on.
- iou_thr – float Threshold for IOU filtration

int count_subframes (BufferPtr main_buffer)
Count expected number of subframes for a main buffer.

Parameters main_buffer – Main frame buffer

Returns Number of subframes

void stamp_and_send (BufferPtr buffer)
Stamp buffer with timestamp and send to subscribers.

Parameters buffer – *Buffer* to stamp and send

void migrate_metadata (BufferPtr main_buffer, std::vector<BufferPtr> &subframes)
Migrate metadata from subframes to main buffer.

Parameters

- main_buffer – Main frame buffer
- subframes – Vector of subframe buffers

virtual tl::expected<std::vector<BufferPtr>, SubframeStatus> get_subframes (BufferPtr main_buffer, ...)
Get subframes for a main buffer.

Parameters

- `main_buffer` – Main frame buffer
- `num_subframes` – Number of subframes to retrieve

Returns Expected containing vector of subframes or status

virtual void loop() override
Main processing loop.

virtual *AppStatus* deinit() override
Deinitialize the stage.

Returns *AppStatus* indicating success or failure

AggregatorStageBuild Class

class hailo_analytics::pipeline::cropping::AggregatorStageBuild: public
hailo_analytics::pipeline::cropping::AggregatorStage
Builder pattern implementation for *AggregatorStage*.

Provides a fluent interface for constructing *AggregatorStage* instances with configurable parameters.

Public Static Functions

static *Builder* create()
Create a new *Builder* instance.

Returns *Builder* instance for constructing *AggregatorStage*

class Builder
Builder class for constructing *AggregatorStage* instances.

Public Functions

Builder &set_stage_name(std::string name)
Set the stage name.

Parameters `name` – Name for the stage

Returns *Builder* reference for chaining

Builder &set_static_subframes_opt(int num)
Set static number of subframes.

Parameters `num` – Fixed number of subframes to expect

Returns *Builder* reference for chaining

Builder &set_main_inlet_name(std::string name)
Set the main inlet name.

Parameters `name` – Name of the main frame inlet

Returns *Builder* reference for chaining

Builder &set_main_queue_size(size_t size)
Set the main queue size.

Parameters `size` – Size of the main frame queue

Returns *Builder* reference for chaining

Builder &set_main_leaky(bool leaky)
Set the main queue leaky option.

Parameters `leaky` – If true, main queue drops old frames when full

Returns *Builder* reference for chaining

Builder &set_sub_inlet_name(std::string name)
Set the subframe inlet name.

Parameters `name` – Name of the subframe inlet

Returns *Builder* reference for chaining

Builder &set_sub_queue_size(size_t size)
Set the subframe queue size.

Parameters size – Size of the subframe queue

Returns *Builder* reference for chaining

Builder &set_sub_leaky(bool leaky)
Set the subframe queue leaky option.

Parameters leaky – If true, subframe queue drops old frames when full

Returns *Builder* reference for chaining

Builder &set_multiscale_opt(bool multi_scale)
Set the multi-scale option.

Parameters multi_scale – If true, enables multi-scale aggregation

Returns *Builder* reference for chaining

Builder &set_skip_migration_opt(bool skip)
Set the skip migration option.

Parameters skip – If true, skips metadata migration

Returns *Builder* reference for chaining

Builder &set_trace_opt(bool activate)
Set the trace option.

Parameters activate – If true, enables tracing for processing operations

Returns *Builder* reference for chaining

Builder &set_iou_threshold_opt(float threshold)
Set the IOU threshold for NMS.

Parameters threshold – IOU threshold value (0.0 to 1.0)

Returns *Builder* reference for chaining

Builder &set_border_threshold_opt(float threshold)
Set the border threshold for edge detection removal.

Parameters threshold – Border threshold value (0.0 to 1.0)

Returns *Builder* reference for chaining

Builder &set_copy_sub_frame_tensor_to_metadata_opt(bool copy)
Set whether to copy tensor metadata from subframes.

Parameters copy – If true, copies tensor metadata from subframes

Returns *Builder* reference for chaining

std::shared_ptr<AggregatorStage> buildptr() const
Build and return shared pointer to *AggregatorStage*.

Throws std::runtime_error – if required parameters are missing

Returns Shared pointer to constructed *AggregatorStage*

Sync Aggregator Stage

API Reference

SyncAggregatorStage Class

```
class hailo_analytics::pipeline::cropping::SyncAggregatorStage: public
hailo_analytics::pipeline::cropping::AggregatorStage
```

Synchronized aggregator stage with timeout and adaptive synchronization.

Extends *AggregatorStage* to handle synchronized streams with timeout management and adaptive timeout adjustment based on drop rate. This stage waits for subframes with a configurable timeout and can automatically adjust the timeout based on frame drop statistics to optimize performance.

Features:

- Configurable timeout for subframe synchronization
- Adaptive timeout adjustment based on drop rate
- Min/max timeout bounds
- Drop rate monitoring and blocking

Subclassed by [hailo_analytics::pipeline::cropping::SyncAggregatorStageBuild](#)

Public Functions

`SyncAggregatorStage ( std::string name, std::string main_inlet_name, size_t main_queue_size, bool ... )`
 Constructor for [SyncAggregatorStage](#).

Parameters

- `name` - [Stage](#) name for identification
- `main_inlet_name` - Name of the main frame inlet
- `main_queue_size` - Size of the main frame queue
- `main_queue_leaky` - If true, main queue drops old frames when full
- `sub_inlet_name` - Name of the subframe inlet
- `sub_queue_size` - Size of the subframe queue
- `sub_queue_leaky` - If true, subframe queue drops old frames when full
- `multi_scale` - Enable multi-scale aggregation (default: false)
- `iou_threshold` - IOU threshold for NMS (default: 0.3)
- `m_border_threshold` - Border threshold for removing edge detections (default: 0.1)
- `skip_migration` - Skip metadata migration (default: false)
- `trace_processing_operations` - Enable tracing (default: true)
- `static_sub_frames` - Fixed number of subframes, if known (default: nullopt)
- `timeout` - Timeout for waiting for subframes (default: nullopt)
- `min_timeout` - Minimum timeout value for adaptive adjustment (default: nullopt)
- `max_timeout` - Maximum timeout value for adaptive adjustment (default: nullopt)
- `timeout_adjustment_period` - Period for timeout adjustment (default: 1000ms)
- `drop_rate_threshold` - Threshold for drop rate monitoring (default: 0.2)
- `timeout_step_size` - Step size for timeout adjustments (default: 10ms)
- `drop_rate_block` - Block on high drop rate (default: false)

`void timeout_adjustment ( )`
 Adjust timeout based on drop rate statistics.

`virtual tl::expected<std::vector<BufferPtr>, SubframeStatus> get_subframes ( BufferPtr main_buffer, ... )`
 Get subframes for a main buffer with timeout.

Parameters

- `main_buffer` - Main frame buffer
- `num_subframes` - Number of subframes to retrieve

Returns Expected containing vector of subframes or status

SyncAggregatorStageBuild Class

```
class hailo_analytics::pipeline::cropping::SyncAggregatorStageBuild: public
hailo_analytics::pipeline::cropping::SyncAggregatorStage
```

Builder pattern implementation for *SyncAggregatorStage*.

Provides a fluent interface for constructing *SyncAggregatorStage* instances with configurable parameters including timeout management.

Public Static Functions

```
static Builder create()
    Create a new Builder instance.
```

Returns *Builder* instance for constructing *SyncAggregatorStage*

```
class Builder
    Builder class for constructing SyncAggregatorStage instances.
```

Public Functions

```
Builder &set_stage_name(std::string name)
```

```
Builder &set_static_subframes_opt(int num)
```

```
Builder &set_main_inlet_name(std::string name)
```

```
Builder &set_main_queue_size(size_t size)
```

```
Builder &set_main_leaky(bool leaky)
```

```
Builder &set_sub_inlet_name(std::string name)
```

```
Builder &set_sub_queue_size(size_t size)
```

```
Builder &set_sub_leaky(bool leaky)
```

```
Builder &set_multiscale_opt(bool multi_scale)
```

```
Builder &set_skip_migration_opt(bool skip)
```

```
Builder &set_trace_opt(bool activate)
```

```
Builder &set_iou_threshold_opt(float threshold)
```

```
Builder &set_border_threshold_opt(float threshold)
```

```
Builder &set_timeout_opt(std::chrono::milliseconds timeout)
    Set the subframe wait timeout.
```

Parameters *timeout* - Timeout duration for waiting for subframes

Returns *Builder* reference for chaining

```
Builder &set_min_timeout_opt(std::chrono::milliseconds min_timeout)
    Set the minimum timeout for adaptive adjustment.
```

Parameters *min_timeout* - Minimum timeout value

Returns *Builder* reference for chaining

Builder &set_max_timeout_opt (std::chrono::milliseconds max_timeout)
Set the maximum timeout for adaptive adjustment.

Parameters max_timeout – Maximum timeout value

Returns *Builder* reference for chaining

Builder &set_timeout_adjustment_period_opt (std::chrono::milliseconds period)
Set the period for timeout adjustments.

Parameters period – Period duration between adjustments

Returns *Builder* reference for chaining

Builder &set_drop_rate_threshold_opt (float threshold)
Set the drop rate threshold.

Parameters threshold – Drop rate threshold (0.0 to 1.0)

Returns *Builder* reference for chaining

Builder &set_timeout_step_size_opt (std::chrono::milliseconds step_size)
Set the timeout step size for adjustments.

Parameters step_size – Step size for timeout changes

Returns *Builder* reference for chaining

Builder &set_drop_rate_block_opt (bool block)
Set whether to block on high drop rate.

Parameters block – If true, blocks when drop rate exceeds threshold

Returns *Builder* reference for chaining

std::shared_ptr<*SyncAggregatorStage*> buildptr() const
Build and return shared pointer to *SyncAggregatorStage*.

Throws std::runtime_error – if required parameters are missing

Returns Shared pointer to constructed *SyncAggregatorStage*

10.6.5. Muxing Stages

Muxer Stage

API Reference

MuxerStage Class

```
class hailo_analytics::pipeline::muxing::MuxerStage: public
hailo_analytics::pipeline::ThreadedStage
```

Stage that multiplexes multiple video streams into a single output stream.

The *MuxerStage* combines two separate input streams (main and sub) into a single output stream by embedding the sub-stream buffer as metadata (*BufferMetadata*) within the main stream buffer. This is useful for scenarios where multiple related video streams need to be synchronized and processed together in the pipeline.

The stage maintains two separate input queues (main and sub) and blocks on both, ensuring that buffers from each stream are properly paired before being sent downstream. The sub-buffer is attached to the main buffer as metadata, allowing downstream stages to access both streams.

Subclassed by *hailo_analytics::pipeline::muxing::MuxerStageBuild*

Public Functions

MuxerStage (std::string name, std::string main_inlet_name, size_t main_queue_size, bool ...)

Constructs a [MuxerStage](#).

Constructs a [MuxerStage](#) with the specified configuration.

Initializes both the main and sub input queues with their respective sizes and leaky behaviors.

Parameters

- name - The name of the stage
- main_inlet_name - The name of the main input inlet
- main_queue_size - The size of the main input queue
- main_queue_leaky - If true, drops oldest buffers when main queue is full
- sub_inlet_name - The name of the sub input inlet
- sub_queue_size - The size of the sub input queue
- sub_queue_leaky - If true, drops oldest buffers when sub queue is full
- trace_processing_operations - Enable tracing for performance analysis (default: true)

virtual void add_queue (std::string name) override

Adds a queue to the stage (no-op for [MuxerStage](#)).

No-op for [MuxerStage](#) as it has fixed queues.

[MuxerStage](#) has fixed queues (main and sub) that are created in the constructor, so this method does nothing.

[MuxerStage](#) creates its two fixed queues (main and sub) in the constructor, so additional queue creation is not supported.

Parameters name - The name of the queue (ignored)

virtual void loop () override

Main processing loop that multiplexes main and sub streams.

Main processing loop that synchronizes and multiplexes the two input streams.

Continuously reads from both main and sub queues, waits for buffers from each stream, embeds the sub-buffer as metadata in the main buffer, and sends the combined buffer to subscribers. Stops when either stream reaches end-of-stream.

The processing flow:

- i. Reads a buffer from the main queue (blocking)
- ii. Reads a buffer from the sub queue (blocking)
- iii. Creates [BufferMetadata](#) containing the sub-buffer
- iv. Attaches the sub-buffer metadata to the main buffer
- v. Sends the combined buffer to all subscribers

The loop continues until either stream reaches end-of-stream or is flushing. Both streams must provide buffers for each iteration, ensuring synchronization.

MuxerStageBuild Class

```
class hailo_analytics::pipeline::muxing::MuxerStageBuild: public
```

```
hailo_analytics::pipeline::muxing::MuxerStage
```

Builder class for constructing *MuxerStage* instances.

Provides a fluent interface for configuring and building *MuxerStage* objects with various options. This follows the builder pattern to ensure all required parameters are set before construction.

Public Static Functions

```
static Builder create()
```

Creates a new *Builder* instance for constructing a *MuxerStage*.

Returns A new *Builder* instance

```
class Builder
```

Builder for *MuxerStage* configuration.

Allows step-by-step configuration of a *MuxerStage* before building. Required parameters: stage name, main inlet name, and sub inlet name.

Public Functions

```
Builder &set_stage_name(std::string name)
```

Sets the stage name (required).

Parameters name - The name for the muxer stage

Returns Reference to this builder for method chaining

```
Builder &set_main_inlet_name(std::string name)
```

Sets the main inlet name (required).

Parameters name - The name for the main input inlet

Returns Reference to this builder for method chaining

```
Builder &set_main_queue_size(size_t size)
```

Sets the main queue size.

Parameters size - The size of the main input queue (default: 10)

Returns Reference to this builder for method chaining

```
Builder &set_main_leaky(bool leaky)
```

Configures the main queue leaky behavior.

Parameters leaky - If true, drops oldest buffers when main queue is full (default: false)

Returns Reference to this builder for method chaining

```
Builder &set_sub_inlet_name(std::string name)
```

Sets the sub inlet name (required).

Parameters name - The name for the sub input inlet

Returns Reference to this builder for method chaining

```
Builder &set_sub_queue_size(size_t size)
```

Sets the sub queue size.

Parameters size - The size of the sub input queue (default: 10)

Returns Reference to this builder for method chaining

```
Builder &set_sub_leaky(bool leaky)
```

Configures the sub queue leaky behavior.

Parameters leaky - If true, drops oldest buffers when sub queue is full (default: false)

Returns Reference to this builder for method chaining

```
Builder &set_trace_opt(bool activate)
```

Enables or disables tracing for performance analysis.

Parameters activate - If true, enables tracing (default: true)

Returns Reference to this builder for method chaining

```
std::shared_ptr<MuxerStage> buildptr() const
```

Builds and returns a shared pointer to the configured *MuxerStage*.

Throws `std::invalid_argument` - if required parameters are not set
Returns Shared pointer to the newly created [MuxerStage](#)

Demuxer Stage

API Reference

DemuxerStage Class

```
class hailo_analytics::pipeline::muxing::DemuxerStage: public
hailo_analytics::pipeline::ThreadedStage
    Stage that demultiplexes a single video stream into multiple output streams.
```

The [DemuxerStage](#) processes buffers containing embedded sub-buffers (stored in [BufferMetadata](#)) and separates them into two distinct output streams: a main outlet and a sub outlet. This is useful for scenarios where a single buffer contains multiple data streams that need to be processed separately.

The stage extracts sub-buffers from [BufferMetadata](#) within the incoming buffer and sends them to the sub outlet, while the main buffer (with metadata removed) is sent to the main outlet. Optionally, HailoROI metadata can be copied from the main buffer to sub-buffers.

Subclassed by [hailo_analytics::pipeline::muxing::DemuxerStageBuild](#)

Public Functions

[DemuxerStage](#) (`std::string` name, `std::string` main_outlet_name, `std::string` sub_outlet_name, `size_t` ...)
 Constructs a [DemuxerStage](#).

Constructs a [DemuxerStage](#) with the specified configuration.

Parameters

- `name` - The name of the stage
- `main_outlet_name` - The name of the main output outlet
- `sub_outlet_name` - The name of the sub output outlet
- `queue_size` - The size of the internal processing queue (default: 10)
- `leaky` - If true, drops oldest buffers when queue is full (default: false)
- `trace_processing_operations` - Enable tracing for performance analysis (default: true)
- `copy_roi_metadata` - If true, copies HailoROI metadata from main to sub buffers (default: false)

virtual [AppStatus](#) `process` (`BufferPtr` buffer) override
 Processes incoming buffers by demultiplexing them into main and sub streams.

Processes the buffer by extracting sub-buffers and routing to appropriate outlets.

Extracts [BufferMetadata](#) from the incoming buffer, sends the contained sub-buffer to the sub outlet, removes the metadata from the main buffer, and sends the main buffer to the main outlet. If `copy_roi_metadata` is enabled, copies HailoROI objects from the main buffer to the sub-buffer before sending.

The processing flow:

- i. Searches for [BufferMetadata](#) in the incoming buffer
- ii. Extracts the first sub-buffer found in the metadata
- iii. Optionally copies HailoROI metadata to the sub-buffer

- iv. Sends the sub-buffer to the sub outlet
- v. Removes the metadata from the main buffer
- vi. Sends the main buffer to the main outlet

Parameters `buffer` – The input buffer to process

Returns `AppStatus::SUCCESS` on successful processing

DemuxerStageBuild Class

```
class hailo_analytics::pipeline::muxing::DemuxerStageBuild: public
```

```
hailo_analytics::pipeline::muxing::DemuxerStage
```

Builder class for constructing *DemuxerStage* instances.

Provides a fluent interface for configuring and building *DemuxerStage* objects with various options. This follows the builder pattern to ensure all required parameters are set before construction.

Public Static Functions

```
static Builder create()
```

Creates a new *Builder* instance for constructing a *DemuxerStage*.

Returns A new *Builder* instance

```
class Builder
```

Builder for *DemuxerStage* configuration.

Allows step-by-step configuration of a *DemuxerStage* before building. Required parameters: stage name, main outlet name, and sub outlet name.

Public Functions

```
Builder &set_stage_name(std::string name)
```

Sets the stage name (required).

Parameters `name` – The name for the demuxer stage

Returns Reference to this builder for method chaining

```
Builder &set_main_outlet_name(std::string name)
```

Sets the main outlet name (required).

Parameters `name` – The name for the main output outlet

Returns Reference to this builder for method chaining

```
Builder &set_sub_outlet_name(std::string name)
```

Sets the sub outlet name (required).

Parameters `name` – The name for the sub output outlet

Returns Reference to this builder for method chaining

```
Builder &set_queue_size(size_t size)
```

Sets the internal queue size.

Parameters `size` – The size of the processing queue (default: 10)

Returns Reference to this builder for method chaining

```
Builder &set_leaky_opt(bool leaky)
```

Configures the leaky queue behavior.

Parameters `leaky` – If true, drops oldest buffers when queue is full (default: false)

Returns Reference to this builder for method chaining

```
Builder &set_trace_opt(bool activate)
```

Enables or disables tracing for performance analysis.

Parameters `activate` – If true, enables tracing (default: true)

Returns Reference to this builder for method chaining

```
Builder &set_copy_roi_metadata_opt(bool copy_roi)
```

Configures HailoROI metadata copying.

Parameters `copy_roi` - If true, copies ROI metadata from main to sub buffers (default: false)

Returns Reference to this builder for method chaining

`std::shared_ptr<DemuxerStage> buildptr() const`
Builds and returns a shared pointer to the configured [DemuxerStage](#).

Throws `std::invalid_argument` - if required parameters are not set

Returns Shared pointer to the newly created [DemuxerStage](#)

10.6.6. Routing Stages

Callback Stage

API Reference

CallbackStage Class

`class hailo_analytics::pipeline::routing::CallbackStage : public`

`hailo_analytics::pipeline::ThreadedStage`

[Stage](#) that executes a user-defined callback function on each buffer.

The [CallbackStage](#) allows users to inject custom processing logic into the pipeline by providing a callback function that is invoked for each buffer passing through. After executing the callback, the buffer is forwarded to all subscribers.

This is useful for custom logging, debugging, metrics collection, or any other user-defined processing that needs to be performed without modifying the buffer flow.

Subclassed by [hailo_analytics::pipeline::routing::CallbackStageBuild](#)

Public Functions

`CallbackStage ( std::string name, size_t queue_size, bool leaky = false, std::function<void(BufferPtr)> ... )`
Constructs a [CallbackStage](#).

Constructs a [CallbackStage](#) with the specified configuration.

Parameters

- `name` - The name of the stage
- `queue_size` - The size of the internal processing queue
- `leaky` - If true, drops oldest buffers when queue is full (default: false)
- `callback` - The callback function to execute on each buffer (default: NULL)
- `trace_processing_operations` - Enable tracing for performance analysis (default: true)

`virtual AppStatus process ( BufferPtr data ) override`

Processes a buffer by executing the callback and forwarding to subscribers.

Processes a buffer by executing the callback (if set) and forwarding to subscribers.

Parameters `data` - The buffer to process

Returns `AppStatus::SUCCESS` on successful processing

`void set_callback ( std::function<void(BufferPtr)> callback )`
Sets or updates the callback function.

Sets or updates the callback function to be executed on each buffer.

Parameters `callback` - The new callback function to execute on each buffer

CallbackStageBuild Class

```
class hailo_analytics::pipeline::routing::CallbackStageBuild: public
```

```
hailo_analytics::pipeline::routing::CallbackStage
Builder class for constructing CallbackStage instances.
```

Provides a fluent interface for configuring and building *CallbackStage* objects.

Public Static Functions

```
static Builder create()
Creates a new Builder instance for constructing a CallbackStage.
```

Returns A new *Builder* instance

```
class Builder
Builder for CallbackStage configuration.
```

Public Functions

```
Builder &set_stage_name(std::string name)
Sets the stage name (required).
```

Parameters *name* - The name for the callback stage
Returns Reference to this builder for method chaining

```
Builder &set_queue_size_opt(size_t size)
Sets the internal queue size.
```

Parameters *size* - The size of the processing queue (default: 1)
Returns Reference to this builder for method chaining

```
Builder &set_leaky_opt(bool activate)
Configures the leaky queue behavior.
```

Parameters *activate* - If true, drops oldest buffers when queue is full (default: false)
Returns Reference to this builder for method chaining

```
Builder &set_trace_opt(bool activate)
Enables or disables tracing for performance analysis.
```

Parameters *activate* - If true, enables tracing (default: true)
Returns Reference to this builder for method chaining

```
std::shared_ptr<CallbackStage> buildptr() const
Builds and returns a shared pointer to the configured CallbackStage.
```

Throws *std::invalid_argument* - if required parameters are not set
Returns Shared pointer to the newly created *CallbackStage*

Freeze Stage

API Reference

FreezeStage Class

```
class hailo_analytics::pipeline::routing::FreezeStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
Stage that can freeze the data flow.
```

When freeze is enabled, the stage will output the last received buffer instead of forwarding new incoming buffers.

Subclassed by *hailo_analytics::pipeline::routing::FreezeStageBuild*

Public Functions

`FreezeStage ( std::string name, size_t queue_size, bool leaky = false, bool print_fps = false )`
Construct a new *FreezeStage*.

Parameters

- `name` - *Stage* name.
- `queue_size` - Input queue size.
- `leaky` - Whether the queue is leaky.
- `print_fps` - Whether to print FPS statistics.

`virtual AppStatus process ( BufferPtr data ) override`
Process a single buffer.

If freeze is enabled, this will output the last saved buffer. Otherwise, the incoming buffer is saved and forwarded.

Parameters `data` - Input buffer.

Returns `AppStatus` Status code.

`bool is_freeze ( )`
Check whether the stage is currently frozen.

Returns True if frozen.

`void set_freeze ( bool freeze )`
Enable/disable freeze mode.

Parameters `freeze` - True to freeze.

`void set_saved_buffer ( BufferPtr buffer )`
Set the saved buffer that will be output while frozen.

Parameters `buffer` - *Buffer* to save.

`BufferPtr get_saved_buffer ( )`
Get the currently saved buffer.

Returns Saved buffer.

`void clear_saved_buffer ( )`
Clear the saved buffer.

FreezeStageBuild Class

`class hailo_analytics::pipeline::routing::FreezeStageBuild : public`
`hailo_analytics::pipeline::routing::FreezeStage`
Builder wrapper for *FreezeStage*.

Public Static Functions

`static Builder create ( )`
Create a builder with default values.

Returns *Builder* instance.

`class Builder`
FreezeStage builder.

Public Functions

Builder &set_stage_name (std::string name)
Set the stage name.

Parameters name – *Stage* name.

Returns *Builder* reference.

Builder &set_queue_size (size_t size)
Set the stage queue size.

Parameters size – *Queue* size.

Returns *Builder* reference.

Builder &set_leaky_opt (bool activate)
Enable/disable leaky queue behavior.

Parameters activate – True to enable.

Returns *Builder* reference.

Builder &set_printfps_opt (bool activate)
Enable/disable FPS printing.

Parameters activate – True to enable.

Returns *Builder* reference.

std::shared_ptr<*FreezeStage*> buildptr () const
Build a *FreezeStage* instance.

Returns Shared pointer to the stage.

Tee Stage

API Reference

TeeStage Class

class hailo_analytics::pipeline::routing::TeeStage: public

hailo_analytics::pipeline::*ThreadedStage*

Stage that duplicates the input buffer to multiple outputs (no copy).

Subclassed by *hailo_analytics::pipeline::routing::TeeStageBuild*

Public Functions

TeeStage (std::string name, size_t queue_size, bool leaky = false, bool trace_processing_operations = true)
Construct a new *TeeStage*.

Parameters

- name – *Stage* name.
- queue_size – Input queue size.
- leaky – Whether the queue is leaky.
- trace_processing_operations – Whether to trace processing operations.

virtual *AppStatus* process (BufferPtr data) override
Process a single buffer and forward it to all outputs.

Parameters data – Input buffer.

Returns AppStatus Status code.

TeeStageBuild Class

```
class hailo_analytics::pipeline::routing::TeeStageBuild: public
```

```
hailo_analytics::pipeline::routing::TeeStage
```

Builder wrapper for *TeeStage*.

Public Static Functions

```
static Builder create()
```

Create a builder with default values.

Returns *Builder* instance.

```
class Builder
```

TeeStage builder.

Public Functions

```
Builder &set_stage_name(std::string name)
```

Set the stage name.

Parameters name - *Stage* name.

Returns *Builder* reference.

```
Builder &set_queue_size(size_t size)
```

Set the stage queue size.

Parameters size - *Queue* size.

Returns *Builder* reference.

```
Builder &set_leaky_opt(bool activate)
```

Enable/disable leaky queue behavior.

Parameters activate - True to enable.

Returns *Builder* reference.

```
Builder &set_trace_opt(bool activate)
```

Enable/disable tracing.

Parameters activate - True to enable.

Returns *Builder* reference.

```
std::shared_ptr<TeeStage> buildptr() const
```

Build a *TeeStage* instance.

Returns Shared pointer to the stage.

Tracker Traffic Control Stage

API Reference

TrackerTrafficCtrlStage Class

```
class hailo_analytics::pipeline::routing::TrackerTrafficCtrlStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Stage that controls traffic of tracked objects based on classification and rate limiting.

The *TrackerTrafficCtrlStage* manages the flow of detected and tracked objects through the pipeline by applying various filtering and rate-limiting strategies. It can block untracked objects, throttle the frequency of tracked objects (both classified and unclassified), and enforce a maximum objects-per-second limit.

Key features:

- Block all untracked objects (optional)
- Control frame pass-through rate for unclassified tracked objects
- Control frame pass-through rate for classified tracked objects

- Enforce maximum objects per second rate limit
- Automatic cleanup of stale tracking data

Subclassed by [hailo_analytics::pipeline::routing::TrackerTrafficCtrlStageBuild](#)

Public Functions

`TrackerTrafficCtrlStage (std::string name, bool block_all_untracked = false, size_t ...)`

Constructs a [TrackerTrafficCtrlStage](#).

Parameters

- `name` - The name of the stage
- `block_all_untracked` - If true, blocks all untracked objects (default: false)
- `tracked_unclassified_frame_block_cnt` - Number of frames to block before passing unclassified tracked objects (default: 1)
- `tracked_classified_frame_block_cnt` - Number of frames to block before passing classified tracked objects (default: 1)
- `tracked_max_objects_per_second` - Maximum objects per second allowed (0 = no limit, default: 0)
- `queue_size` - The size of the internal processing queue (default: 5)
- `leaky` - If true, drops oldest buffers when queue is full (default: false)
- `trace_processing_operations` - Enable tracing for performance analysis (default: true)

virtual [AppStatus](#) `init ()` override

Initializes the stage.

Returns AppStatus indicating success or failure

virtual [AppStatus](#) `deinit ()` override

Deinitializes the stage and cleans up resources.

Returns AppStatus indicating success or failure

virtual [AppStatus](#) `process (BufferPtr data)`

Processes a buffer by filtering tracked objects based on configured rules.

Parameters `data` - The buffer to process

Returns AppStatus indicating success or failure

[AppStatus](#) `set_unclassified_fps_to_block (size_t count)`

Sets the number of frames to block for unclassified tracked objects.

Parameters `count` - Number of frames to block

Returns AppStatus indicating success or failure

`size_t get_unclassified_fps_to_block ()`

Gets the current number of frames being blocked for unclassified objects.

Returns Number of frames being blocked

TrackerTrafficCtrlStageBuild Class

```
class hailo_analytics::pipeline::routing::TrackerTrafficCtrlStageBuild: public
hailo_analytics::pipeline::routing::TrackerTrafficCtrlStage
```

Builder class for constructing *TrackerTrafficCtrlStage* instances.

Provides a fluent interface for configuring and building *TrackerTrafficCtrlStage* objects with various traffic control options.

Public Static Functions

```
static Builder create()
```

Creates a new *Builder* instance for constructing a *TrackerTrafficCtrlStage*.

Returns A new *Builder* instance

```
class Builder
```

Builder for *TrackerTrafficCtrlStage* configuration.

Allows step-by-step configuration of a *TrackerTrafficCtrlStage* before building.

Public Functions

```
Builder &set_stage_name( std::string name )
```

Sets the stage name (required).

Parameters name - The name for the tracker traffic control stage

Returns Reference to this builder for method chaining

```
Builder &set_queue_size_opt( size_t size )
```

Sets the internal queue size.

Parameters size - The size of the processing queue (default: 5)

Returns Reference to this builder for method chaining

```
Builder &set_leaky_opt( bool activate )
```

Configures the leaky queue behavior.

Parameters activate - If true, drops oldest buffers when queue is full (default: false)

Returns Reference to this builder for method chaining

```
Builder &set_block_untracked_obj( bool activate )
```

Configures whether to block all untracked objects.

Parameters activate - If true, blocks all untracked objects (default: false)

Returns Reference to this builder for method chaining

```
Builder &set_classified_fps_to_block( size_t count )
```

Sets the number of frames to block for classified tracked objects.

Parameters count - Number of frames to block (default: 1)

Returns Reference to this builder for method chaining

```
Builder &set_tracked_max_objects_per_second( size_t max_objects )
```

Sets the maximum objects per second rate limit.

Parameters max_objects - Maximum objects per second (0 = no limit, default: 0)

Returns Reference to this builder for method chaining

```
Builder &set_unclassified_fps_to_block( size_t count )
```

Sets the number of frames to block for unclassified tracked objects.

Parameters count - Number of frames to block (default: 1)

Returns Reference to this builder for method chaining

```
Builder &set_trace_processing_operations( bool activate )
```

Enables or disables tracing for performance analysis.

Parameters activate - If true, enables tracing (default: true)

Returns Reference to this builder for method chaining

```
std::shared_ptr<TrackerTrafficCtrlStage> buildptr() const
```

Builds and returns a shared pointer to the configured *TrackerTrafficCtrlStage*.

Throws std::invalid_argument - if required parameters are not set

Returns Shared pointer to the newly created [TrackerTrafficCtrlStage](#)

ObjectRateLimiter Class

```
class hailo_analytics::pipeline::routing::ObjectRateLimiter
```

Rate limiter for controlling the frequency of object processing.

The [ObjectRateLimiter](#) maintains a sliding window of timestamps to track the number of objects processed within the last second. It can be used to throttle object processing to prevent overwhelming downstream systems.

Public Functions

```
ObjectRateLimiter(uint64_t max_objects_per_second)
```

Constructs an [ObjectRateLimiter](#).

Constructs an [ObjectRateLimiter](#) with the specified maximum objects per second.

Parameters `max_objects_per_second` - Maximum objects allowed per second (0 = no limit)

```
uint64_t add_object()
```

Attempts to add an object to the rate limiter.

Attempts to add an object, returning wait time if rate limit exceeded.

Returns 0 if object added successfully or no limit set, otherwise milliseconds to wait if limit exceeded

```
uint64_t get_current_count()
```

Gets the current count of objects in the sliding window.

Returns Current number of objects processed in the last second

```
void reset()
```

Resets the limiter by clearing all timestamps.

Valve Stage

API Reference

ValveStage Class

```
class hailo_analytics::pipeline::routing::ValveStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class representing a valve stage in the pipeline.

This stage can block or allow buffers to pass through based on a valve state. When the valve is open (true), buffers pass through to subscribers. When the valve is closed (false), buffers are dropped. Typically used as a GENERAL stage in the middle of a pipeline.

Subclassed by [hailo_analytics::pipeline::routing::ValveStageBuild](#)

Public Functions

`ValveStage (std::string name, size_t queue_size = VALVE_QUEUE_SIZE_DEFAULT, bool leaky = false, ...)`
Construct a new *ValveStage* object.

Parameters

- `name` – Name of the stage.
- `queue_size` – Size of the processing queue.
- `leaky` – If true, drops oldest buffer when queue is full.
- `trace_processing_operations` – If true, enables performance tracing.

virtual *AppStatus* `process (BufferPtr data)` override
Process a buffer by either passing it through or dropping it.

Parameters `data` – *Buffer* to process.

Returns *AppStatus* Status of the processing.

void `set_valve (bool valve)`
Set the valve state.

Parameters `valve` – True to open the valve (allow buffers), false to close (block buffers).

ValveStageBuild Class

`class hailo_analytics::pipeline::routing::ValveStageBuild: public`
`hailo_analytics::pipeline::routing::ValveStage`

Builder-based valve stage for simplified construction.

Provides a builder pattern interface for creating valve stages with configurable parameters.

Public Static Functions

static *Builder* `create ()`
Create a new *Builder* for *ValveStage* construction.

Returns *Builder* instance.

`class Builder`
Builder class for *ValveStage* construction.

Public Functions

Builder &`set_stage_name (std::string name)`
Set the stage name.

Parameters `name` – Name for the valve stage.

Returns Reference to this builder for chaining.

Builder &`set_queue_size_opt (size_t size)`
Set the queue size (optional).

Parameters `size` – *Queue* size.

Returns Reference to this builder for chaining.

Builder &`set_leaky_opt (bool activate)`
Set whether the queue is leaky (optional).

Parameters `activate` – True to enable leaky mode.

Returns Reference to this builder for chaining.

Builder &`set_trace_opt (bool activate)`
Set whether to enable tracing (optional).

Parameters `activate` – True to enable tracing.

Returns Reference to this builder for chaining.

```
std::shared_ptr<ValveStage> build_ptr() const
    Build and return the ValveStage.
Returns Shared pointer to the constructed ValveStage.
```

10.6.7. Sink Stages

App Sink Stage

API Reference

AppSinkStage Class

```
class hailo_analytics::pipeline::sinks::AppSinkStage: public
hailo_analytics::pipeline::ThreadedStage
```

Class representing an application sink stage in the pipeline.

This stage allows applications to provide a custom processing function that will be called for each buffer. Useful for integrating custom application logic at the end of a pipeline. Typically used as a SINK stage at the end of a pipeline.

Subclassed by *hailo_analytics::pipeline::sinks::AppSinkStageBuild*

Public Functions

```
AppSinkStage (std::string name, size_t queue_size = APP_QUEUE_SIZE_DEFAULT, bool leaky = false, ...)
    Construct a new AppSinkStage object.
```

Parameters

- *name* – Name of the stage.
- *queue_size* – Size of the processing queue.
- *leaky* – If true, drops oldest buffer when queue is full.
- *trace_processing_operations* – If true, enables performance tracing.

```
virtual AppStatus init() override
    Initialize the app sink stage.
```

Returns AppStatus Status of the initialization.

```
virtual AppStatus deinit() override
    Deinitialize the app sink stage.
```

Returns AppStatus Status of the deinitialization.

```
AppStatus configure (std::function<void(BufferPtr)> process_func)
    Configure the app sink stage with a custom processing function.
```

Parameters *process_func* – Function to call for each buffer.

Returns AppStatus Status of the configuration.

```
virtual AppStatus process (BufferPtr data) override
    Process a buffer by calling the custom processing function.
```

Parameters *data* – *Buffer* to process.

Returns AppStatus Status of the processing.

AppSinkStageBuild Class

```
class hailo_analytics::pipeline::sinks::AppSinkStageBuild: public
```

```
hailo_analytics::pipeline::sinks::AppSinkStage
```

Builder-based app sink stage for simplified construction.

Provides a builder pattern interface for creating app sink stages with configurable parameters.

Public Static Functions

```
static Builder create()
```

Create a new *Builder* for *AppSinkStage* construction.

Returns *Builder* instance.

```
class Builder
```

Builder class for *AppSinkStage* construction.

Public Functions

```
Builder &set_stage_name(std::string name)
```

Set the stage name.

Parameters name - Name for the app sink stage.

Returns Reference to this builder for chaining.

```
Builder &set_queue_size_opt(size_t size)
```

Set the queue size (optional).

Parameters size - *Queue* size.

Returns Reference to this builder for chaining.

```
Builder &set_leaky_opt(bool activate)
```

Set whether the queue is leaky (optional).

Parameters activate - True to enable leaky mode.

Returns Reference to this builder for chaining.

```
Builder &set_trace_opt(bool activate)
```

Set whether to enable tracing (optional).

Parameters activate - True to enable tracing.

Returns Reference to this builder for chaining.

```
Builder &set_process_func(std::function<void(BufferPtr)> func)
```

Set the custom processing function.

Parameters func - Function to call for each buffer.

Returns Reference to this builder for chaining.

```
std::shared_ptr<AppSinkStage> buildptr() const
```

Build and return the *AppSinkStage*.

Returns Shared pointer to the constructed *AppSinkStage*.

RTP Converter Stage

API Reference

RTPConverterStage Class

```
class hailo_analytics::pipeline::sinks::RTPConverterStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class representing an RTP converter stage in the pipeline.

This stage converts encoded video data to RTP packets and sends them to a receiver. It receives encoded buffers from upstream stages and converts them to RTP format. Typically used as a SINK stage at the end of a pipeline.

Subclassed by *hailo_analytics::pipeline::sinks::RTPConverterStageBuild*

Public Functions

RTPConverterStage (std::string name, std::shared_ptr<RTPReceiver> receiver, size_t queue_size = ...)
Construct a new *RTPConverterStage* object.

Parameters

- name - Name of the stage.
- receiver - Shared pointer to *RTPReceiver* for handling RTP packets.
- queue_size - Size of the processing queue.
- leaky - If true, drops oldest buffer when queue is full.
- trace_processing_operations - If true, enables performance tracing.
- session_name - Name for the RTP session.

~RTPConverterStage() = default

AppStatus create (EncodingType type)
Create the RTP converter stage with encoding type.

Parameters type - Encoding type of the data to convert.

Returns AppStatus Status of the creation.

virtual *AppStatus* init() override
Initialize the RTP converter stage.

Returns AppStatus Status of the initialization.

virtual *AppStatus* deinit() override
Deinitialize the RTP converter stage.

Returns AppStatus Status of the deinitialization.

AppStatus configure (EncodingType encoding_type)
Configure the RTP converter stage with encoding type.

Parameters encoding_type - Encoding type of the data to convert.

Returns AppStatus Status of the configuration.

virtual *AppStatus* process (BufferPtr data)
Process a buffer by converting it to RTP packets.

Parameters data - *Buffer* containing the encoded data to convert.

Returns AppStatus Status of the processing.

class RTPReceiver
Abstract base class for receiving RTP packets.
Subclassed by WebRTCStreamerExt, webserver::resources::WebRtcResource

Public Functions

RTPReceiver() = default

virtual ~RTPReceiver() = default

virtual void on_rtp_packet (GstSample *sample, rtp_session_id_t stream_name) = 0

virtual rtp_session_id_t start (std::string session_name) = 0


```
virtual void stop( std::string stream_name ) = 0
```

RTPConverterStageBuild Class

```
class hailo_analytics::pipeline::sinks::RTPConverterStageBuild: public
```

```
hailo_analytics::pipeline::sinks::RTPConverterStage
```

Builder-based RTP converter stage for simplified construction.

Provides a builder pattern interface for creating RTP converter stages with configurable parameters.

Public Static Functions

```
static Builder create()
```

Create a new *Builder* for *RTPConverterStage* construction.

Returns *Builder* instance.

```
class Builder
```

Builder class for *RTPConverterStage* construction.

Public Functions

```
Builder &set_stage_name( std::string name )
```

Set the stage name.

Parameters *name* - Name for the RTP converter stage.

Returns Reference to this builder for chaining.

```
Builder &set_session_name( std::string session_name )
```

Set the session name.

Parameters *session_name* - Name for the RTP session.

Returns Reference to this builder for chaining.

```
Builder &set_rtp_receiver( std::shared_ptr< RTPConverterStage::RTPReceiver > receiver )
```

Set the RTP receiver.

Parameters *receiver* - Shared pointer to RTPReceiver.

Returns Reference to this builder for chaining.

```
Builder &set_encoding_type( EncodingType type )
```

Set the encoding type.

Parameters *type* - Encoding type.

Returns Reference to this builder for chaining.

```
Builder &set_queue_size_opt( size_t size )
```

Set the queue size (optional).

Parameters *size* - *Queue* size.

Returns Reference to this builder for chaining.

```
Builder &set_leaky_opt( bool activate )
```

Set whether the queue is leaky (optional).

Parameters *activate* - True to enable leaky mode.

Returns Reference to this builder for chaining.

```
Builder &set_trace_opt( bool activate )
```

Set whether to enable tracing (optional).

Parameters *activate* - True to enable tracing.

Returns Reference to this builder for chaining.

```
std::unique_ptr< RTPConverterStage > buildptr() const
```

Build and return the *RTPConverterStage*.

Returns Shared pointer to the constructed *RTPConverterStage*.

UDP Stage

API Reference

UdpStage Class

```
class hailo_analytics::pipeline::sinks::UdpStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class representing a UDP sink stage in the pipeline.

This stage sends encoded video data over UDP to a specified host and port. It receives encoded buffers from upstream stages and transmits them via UDP. Typically used as a SINK stage at the end of a pipeline.

Subclassed by [hailo_analytics::pipeline::sinks::UdpStageBuild](#)

Public Functions

`UdpStage ( std::string name, size_t queue_size = UDP_QUEUE_SIZE_DEFAULT, bool leaky = false, bool ... )`
Construct a new [UdpStage](#) object.

Parameters

- `name` – Name of the stage.
- `queue_size` – Size of the processing queue.
- `leaky` – If true, drops oldest buffer when queue is full.
- `trace_processing_operations` – If true, enables performance tracing.
- `print_fps` – If true, prints FPS information.

[AppStatus](#) `create ( std::string host, std::string port, EncodingType type )`
Create the UDP stage with connection parameters.

Parameters

- `host` – Target host address (e.g., "192.168.1.100").
- `port` – Target port number (e.g., "5000").
- `type` – Encoding type of the data being sent.

Returns `AppStatus` Status of the creation.

virtual [AppStatus](#) `init ( )` override
Initialize the UDP stage.

Returns `AppStatus` Status of the initialization.

virtual [AppStatus](#) `deinit ( )` override
Deinitialize the UDP stage.

Returns `AppStatus` Status of the deinitialization.

[AppStatus](#) `configure ( std::string host, std::string port, EncodingType type )`
Configure the UDP stage with connection parameters.

Parameters

- `host` – Target host address (e.g., "192.168.1.100").
- `port` – Target port number (e.g., "5000").
- `type` – Encoding type of the data being sent.

Returns `AppStatus` Status of the configuration.

virtual [AppStatus](#) `process ( BufferPtr data )`
Process a buffer by sending it over UDP.

Parameters `data` – [Buffer](#) containing the encoded data to send.

Returns AppStatus Status of the processing.

UdpStageBuild Class

```
class hailo_analytics::pipeline::sinks::UdpStageBuild: public
```

```
hailo_analytics::pipeline::sinks::UdpStage
```

Builder-based UDP stage for simplified construction.

Provides a builder pattern interface for creating UDP stages with configurable parameters.

Public Static Functions

```
static Builder create()
```

Create a new *Builder* for *UdpStage* construction.

Returns *Builder* instance.

```
class Builder
```

Builder class for *UdpStage* construction.

Public Functions

```
Builder &set_stage_name(std::string name)
```

Set the stage name.

Parameters name - Name for the UDP stage.

Returns Reference to this builder for chaining.

```
Builder &set_queue_size_opt(size_t size)
```

Set the queue size (optional).

Parameters size - *Queue* size.

Returns Reference to this builder for chaining.

```
Builder &set_leaky_opt(bool activate)
```

Set whether the queue is leaky (optional).

Parameters activate - True to enable leaky mode.

Returns Reference to this builder for chaining.

```
Builder &set_printfps_opt(bool activate)
```

Set whether to print FPS information (optional).

Parameters activate - True to enable FPS printing.

Returns Reference to this builder for chaining.

```
Builder &set_trace_opt(bool activate)
```

Set whether to enable tracing (optional).

Parameters activate - True to enable tracing.

Returns Reference to this builder for chaining.

```
std::shared_ptr<UdpStage> buildptr() const
```

Build and return the *UdpStage*.

Returns Shared pointer to the constructed *UdpStage*.

ZMQ Communication Stage

API Reference

ZmqCommStage Class

```
class hailo_analytics::pipeline::sinks::ZmqCommStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Stage that provides ZeroMQ (ZMQ) communication for publishing or receiving messages.

The *ZmqCommStage* supports two modes of operation:

- PUBLISHER: Publishes buffer data as messages to a ZMQ publisher socket
- RECEIVER: Subscribes to and receives messages from a ZMQ subscriber socket

This stage is useful for inter-process communication, distributed systems, or integrating the analytics pipeline with external systems using ZeroMQ. Typically used as a SINK stage at the end of a pipeline for publishing data, or as a SOURCE stage for receiving data.

Public Types

enum Mode

Operating mode for the ZMQ communication stage.

Values:

enumerator PUBLISHER

Publish messages to a ZMQ socket.

enumerator RECEIVER

Receive messages from a ZMQ socket.

Public Functions

ZmqCommStage (std::string name, *Mode* mode, const std::string &pub_address, const std::string ...)
Constructs a *ZmqCommStage*.

Parameters

- name – The name of the stage
- mode – Operating mode (PUBLISHER or RECEIVER)
- pub_address – Publisher socket address (used in PUBLISHER mode)
- sub_address – Subscriber socket address (used in RECEIVER mode)
- queue_size – The size of the internal processing queue (default: 10)
- leaky – If true, drops oldest buffers when queue is full (default: false)
- trace_processing_operations – Enable tracing for performance analysis (default: true)
- print_fps – If true, prints FPS statistics (default: false)

virtual *AppStatus* init () override

Initializes the ZMQ communication stage.

Sets up the appropriate ZMQ socket (publisher or subscriber) based on the mode. In RECEIVER mode, also starts the subscriber thread.

Returns AppStatus indicating success or failure

virtual *AppStatus* deinit () override

Deinitializes the ZMQ communication stage.

Closes ZMQ sockets and stops the subscriber thread if running.

Returns AppStatus indicating success or failure

virtual *AppStatus* process (BufferPtr input_buffer) override

Processes a buffer by publishing it as a ZMQ message.

In PUBLISHER mode, serializes and publishes the buffer data. In RECEIVER mode, may attach received messages to the buffer.

Parameters input_buffer – The buffer to process

Returns AppStatus indicating success or failure

ZmqCommStageBuild Class

class hailo_analytics::pipeline::sinks::ZmqCommStageBuild
Builder class for constructing *ZmqCommStage* instances.

Provides a fluent interface for configuring and building *ZmqCommStage* objects with various ZeroMQ communication options.

Public Static Functions

static *Builder* create()
 Creates a new *Builder* instance for constructing a *ZmqCommStage*.

Returns A new *Builder* instance

class Builder
Builder for *ZmqCommStage* configuration.

Allows step-by-step configuration of a *ZmqCommStage* before building.

Public Functions

Builder &set_stage_name(const std::string &name)
 Sets the stage name (required).
Parameters name - The name for the ZMQ communication stage
Returns Reference to this builder for method chaining

Builder &set_queue_size(size_t size)
 Sets the internal queue size.
Parameters size - The size of the processing queue (default: 10)
Returns Reference to this builder for method chaining

Builder &set_leaky(bool leaky)
 Configures the leaky queue behavior.
Parameters leaky - If true, drops oldest buffers when queue is full (default: false)
Returns Reference to this builder for method chaining

Builder &set_trace_opt(bool activate)
 Enables or disables tracing for performance analysis.
Parameters activate - If true, enables tracing (default: true)
Returns Reference to this builder for method chaining

Builder &set_print_fps(bool print_fps)
 Enables or disables FPS printing.
Parameters print_fps - If true, prints FPS statistics (default: false)
Returns Reference to this builder for method chaining

Builder &set_mode(*ZmqCommStage::Mode* mode)
 Sets the operating mode.
Parameters mode - Operating mode (PUBLISHER or RECEIVER, default: PUBLISHER)
Returns Reference to this builder for method chaining

Builder &set_pub_address(const std::string &addr)
 Sets the publisher socket address.
Parameters addr - Address to bind the publisher socket to (e.g., "tcp://0.0.0.0:5555")
Returns Reference to this builder for method chaining

Builder &set_sub_address(const std::string &addr)
 Sets the subscriber socket address.
Parameters addr - Address to connect the subscriber socket to (e.g., "tcp://localhost:5555")
Returns Reference to this builder for method chaining

std::shared_ptr<*ZmqCommStage*> buildptr() const
 Builds and returns a shared pointer to the configured *ZmqCommStage*.
Throws std::invalid_argument - if required parameters are not set

Returns Shared pointer to the newly created [ZmqCommStage](#)

10.6.8. Source Stages

Frontend Stage

API Reference

FrontendStage Class

```
class hailo_analytics::pipeline::sources::FrontendStage: public
```

```
hailo_analytics::pipeline::ThreadedStage
```

Class representing a frontend source stage in the pipeline.

This stage captures video frames from various sources (camera, file, etc.) using the media library frontend. It can produce multiple output streams, each of which can be subscribed to by different downstream stages. Typically used as a SOURCE stage at the beginning of a pipeline.

Subclassed by [hailo_analytics::pipeline::sources::FrontendStageBuild](#), [hailo_analytics::pipeline::sources::FrontendStageFromFile](#)

Public Functions

```
FrontendStage( std::string name, size_t queue_size = FRONTEND_QUEUE_SIZE_DEFAULT, bool leaky ... )
```

Construct a new [FrontendStage](#) object.

Parameters

- `name` - Name of the stage.
- `queue_size` - Size of the processing queue.
- `leaky` - If true, drops oldest buffer when queue is full.
- `trace_processing_operations` - If true, enables performance tracing.

```
~FrontendStage() override
```

Destructor for [FrontendStage](#).

```
AppStatus create( MediaLibraryFrontend &frontend )
```

Create the frontend stage with a frontend instance.

Parameters `frontend` - Media library frontend pointer.

Returns `AppStatus` Status of the creation.

```
virtual void add_subscriber( StagePtr subscriber, std::optional<std::string> stream_id = ... )
```

Add a subscriber to this stage's output.

Note: Subscription is done by stream ID as frontend has multiple output streams.

Parameters

- `subscriber` - [Stage](#) to subscribe to this stage's output.
- `stream_id` - Optional stream ID for multi-stream frontends.

```
AppStatus subscribe_to_stream( output_stream_id_t stream_id, hailo_analytics::pipeline::StagePtr ... )
```

Subscribe a stage to a specific output stream.

Parameters

- `stream_id` - The stream identifier.
- `subscriber` - [Stage](#) to receive buffers from this stream.

Returns `AppStatus` Status of the subscription.

```
AppStatus subscribe_output_streams( )
```

Subscribe all configured output streams to their respective subscribers.

Returns AppStatus Status of the subscriptions.

virtual AppStatus stop() override
Stop the frontend stage.

Returns AppStatus Status of the stop operation.

virtual AppStatus init() override
Initialize the frontend stage.

Returns AppStatus Status of the initialization.

virtual AppStatus deinit() override
Deinitialize the frontend stage.

Returns AppStatus Status of the deinitialization.

AppStatus configure(MediaLibraryFrontend &frontend)
Configure the frontend stage with a frontend instance.

Parameters frontend – Media library frontend pointer.

Returns AppStatus Status of the configuration.

virtual void loop() override
Main processing loop for the frontend stage.

Continuously captures frames from the frontend and sends them to subscribers.

tl::expected<std::vector<frontend_output_stream_t>, media_library_return> get_outputs_streams()
Get the available output streams from the frontend.

Returns Expected containing vector of output streams or error code.

inline const std::map<output_stream_id_t, std::vector<StagePtr>> &get_stream_subscribers() const
Get the map of stream subscribers.

Returns Map of stream IDs to their subscriber stages

FrontendStageBuild Class

class hailo_analytics::pipeline::sources::FrontendStageBuild: public

hailo_analytics::pipeline::sources::FrontendStage
Builder-based frontend stage for simplified construction.

Provides a builder pattern interface for creating frontend stages with configurable parameters.

Public Static Functions

static Builder create()
Create a new Builder for FrontendStage construction.

Returns Builder instance.

class Builder
Builder class for FrontendStage construction.

Public Functions

Builder &set_stage_name (std::string name)

Set the stage name.

Parameters name – Name for the frontend stage.

Returns Reference to this builder for chaining.

Builder &set_queue_size_opt (size_t size)

Set the queue size (optional).

Parameters size – *Queue* size.

Returns Reference to this builder for chaining.

Builder &set_leaky_opt (bool activate)

Set whether the queue is leaky (optional).

Parameters activate – True to enable leaky mode.

Returns Reference to this builder for chaining.

Builder &set_trace_opt (bool activate)

Set whether to enable tracing (optional).

Parameters activate – True to enable tracing.

Returns Reference to this builder for chaining.

std::shared_ptr<*FrontendStage*> buildptr() const

Build and return the *FrontendStage*.

Returns Shared pointer to the constructed *FrontendStage*.

11. Reference Pipelines and Applications

This section includes reference pipelines and applications based on Hailo AI Analytics framework. They demonstrated efficient ways to utilize the Hailo AI Analytics and Media Library, to combine video capture, AI inference, post-processing, encoding, and streaming into end-to-end AI vision analytics pipelines.

11.1. Analytic Draw Client

11.1.1. Overview

The `app-analytic-draw-client.py` is a Python-based video player that runs on the host machine and is designed to synchronize real-time video streams with external analytic metadata. It receives H.264 video over UDP, extracts SEI (Supplemental Enhancement Information) NAL units for frame-accurate timing, and synchronizes this with object detection data received via ZeroMQ (ZMQ).

The application uses **Cairo** to draw bounding boxes and labels directly onto the video frames.

This tool is designed to work with backend applications running on the Hailo-15 platform, including:

- [Single Stream Object Detection Reference Application](#) - Displays object detection results with bounding boxes and labels
- [Face Landmarks Reference Application](#) - Displays face detection boxes and facial landmark points

The analytic viewer receives video streams and metadata from these backend applications and provides real-time visualization on the host machine.

11.1.2. Features

- **UDP Video Receiver:** Listens for H15 H.264 RTP streams.
- **SEI Metadata Parsing:** Extracts ISP timestamps embedded in the video bitstream.
- **ZMQ Synchronization:** Matches H15 analytics metadata with specific video frames using isp timestamps.
- **Cairo Overlay:** Renders dynamic bounding boxes and labels.

11.1.3. Client Side Environment Setup

Describes the required setup for both Win10/11 and Ubuntu to run the client app.

Win 10/11

1. Download and Install MSYS2.
2. Open MSYS2 UCRT64 Terminal.
3. `pacman -Syu` to update core system.
4. Install dependencies:

```
pacman -S mingw-w64-ucrt-x86_64-python \
  mingw-w64-ucrt-x86_64-python-gobject \
  mingw-w64-ucrt-x86_64-python-cairo \
  mingw-w64-ucrt-x86_64-python-pyzmq \
  mingw-w64-ucrt-x86_64-gstreamer \
  mingw-w64-ucrt-x86_64-gst-python \
  mingw-w64-ucrt-x86_64-gst-plugins-base \
  mingw-w64-ucrt-x86_64-gst-plugins-good \
```

(continues on next page)

(continued from previous page)

```
mingw-w64-ucrt-x86_64-gst-plugins-bad \
mingw-w64-ucrt-x86_64-gst-plugins-ugly \
mingw-w64-ucrt-x86_64-gst-libav \
mingw-w64-ucrt-x86_64-python-pydantic \
mingw-w64-ucrt-x86_64-python-msgpack
```

5. Now ready to run the client app.

Ubuntu

Only 22.04 and 24.04 is supported.

1. Recommend to create python 3.1x environment.
2. `sudo apt update`
3. Install system dependencies:

```
sudo apt install \
python3-dev \
python3-pip \
libcairo2-dev \
libgirepository1.0-dev \
pkg-config \
gstreamer1.0-tools \
gstreamer1.0-libav \
gstreamer1.0-plugins-base \
gstreamer1.0-plugins-good \
gstreamer1.0-plugins-bad \
gstreamer1.0-plugins-ugly \
gir1.2-gstreamer-1.0 \
gir1.2-gst-plugins-base-1.0 \
gir1.2-gst-plugins-bad-1.0 \
gir1.2-gtk-4.0
```

4. Create python virtual environment and activate it

```
python3 -m venv .venv
source .venv/bin/activate
```

5. While in python virtual environment install Python packages via pip:

```
cd hailo-analytics/apps/analytic_viewer
pip install -r requirements.txt
```

6. Now ready to run the client app.

11.1.4. Usage

Basic Usage

To run the application with default settings (UDP port 5000, minimum landmarks):

```
cd hailo-analytics/apps/analytic_viewer
python app_analytic_draw_client.py
```

Command-Line Arguments

The application supports the following command-line arguments:

- `-p, --port PORT`: UDP port to receive video stream (default: 5000)
- `--analytic-data-ip IP`: IP address for ZMQ analytic data connection (default: 10.0.0.1)
- `--analytic-data-port PORT`: Port for ZMQ analytic data connection (default: 7000)
- `--face-landmark-filter LEVEL`: Face landmark filter level (default: 2)
 - 0 = maximum (all face landmark points)
 - 1 = standard (facial outline, eyes, eyebrows, nose, lips)
 - 2 = minimum (eyes, nose, lips only)
- `--debug-perf`: Enable performance timing for drawing operations
- `--perf-print-freq N`: Print performance statistics every N frames (default: 30)

Examples

To specify a custom UDP port:

```
python app_analytic_draw_client.py --port 5005
```

To specify custom ZMQ analytic data source:

```
python app_analytic_draw_client.py --analytic-data-ip 10.0.0.3 --analytic-data-  
↪port 7000
```

To draw all face landmark points (maximum detail):

```
python app_analytic_draw_client.py --face-landmark-filter 0
```

To enable performance debugging with statistics printed every 60 frames:

```
python app_analytic_draw_client.py --debug-perf --perf-print-freq 60
```

To combine multiple options:

```
python app_analytic_draw_client.py --port 5005 --analytic-data-ip 10.0.0.3 --face-  
↪landmark-filter 1 --debug-perf
```

11.1.5. Technical Logic

1. **ZMQ Listener**: Runs in a background thread, collecting metadata into a queue.
2. **SEI Probe**: A GStreamer probe monitors the incoming H.264 stream for SEI units containing timestamps.
3. **Synchronization**: When a timestamp is found in the video, the script searches the ZMQ queue for a matching entry. If found, it is added to a draw registry with the frame PTS as key.
4. **Cairo Overlay**: The `on_draw` callback retrieves the metadata from the registry using the frame's presentation timestamp and renders the graphics.

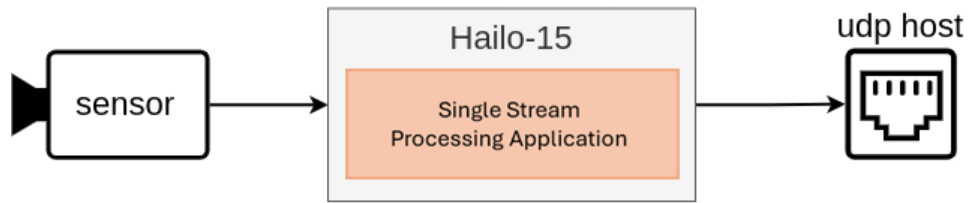
11.1.6. Sample Metadata

```
{
  "detections": [
    {
      "bbox": {
        "xmax": 1253.876220703125,
        "xmin": 1106.83251953125,
        "ymax": 867.47216796875,
        "ymin": 721.2953491210938
      },
      "confidence": 0.8466451168060303,
      "label": "person"
    },
    {
      "bbox": {
        "xmax": 1187.091064453125,
        "xmin": 1139.53955078125,
        "ymax": 800.7326049804688,
        "ymin": 736.2702026367188
      },
      "confidence": 0.9045873880386353,
      "label": "face",
      "landmarks": [
        {
          "pairs": [],
          "points": [
            1166.1790771484375,
            777.7117919921875,
            1.0,
            1164.6505126953125,
            771.7915649414062,
            1.0,
            1165.0872802734375,
            773.567626953125,
            1.0
          ],
          "points_format": "x,y,conf",
          "points_stride": 3
        }
      ]
    }
  ],
  "frame_height": 1080,
  "frame_width": 1920,
  "isp_timestamp_ns": 9022934317000
}
```

11.2. Single Stream Processing Reference Application

11.2.1. Overview

This reference application shows a simple single stream vision application. The key takeaway from this reference pipeline is how to access a video feed from the Media Library and encode that video using an encoder for streaming.



11.2.2. Running the Reference Application

The Hailo-15 Vision Processor Software package includes the Single stream processing pipeline pre-compiled and ready to run.

To run the single stream processing pipeline, follow these steps:

1. On the host machine, set up and run the Analytic Draw Client to receive and display the video stream with overlays.

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Then, navigate to the analytic viewer directory and run the application:

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py
```

The single stream processing reference application will listen on UDP port 5000 (default) for the video stream and display it with inference overlays.

You can customize the port and other settings using command-line arguments. For example:

```
$ python app_analytic_draw_client.py --port 5000 --analytic-data-ip 10.0.0.1
```

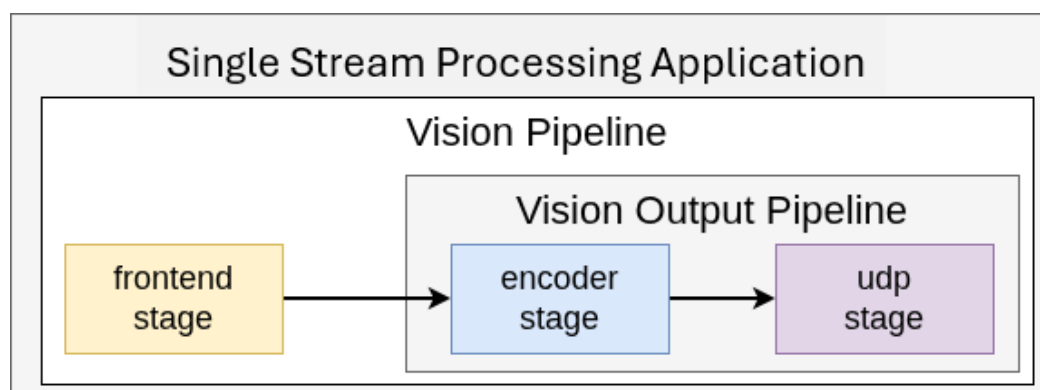
2. On the Hailo15 platform, run the executable located at the following path:

```
$ ./apps/case_studies/single_stream/single_stream_case_study
```

You should now be able to see the video feed with the inference overlay on the host machine screen.

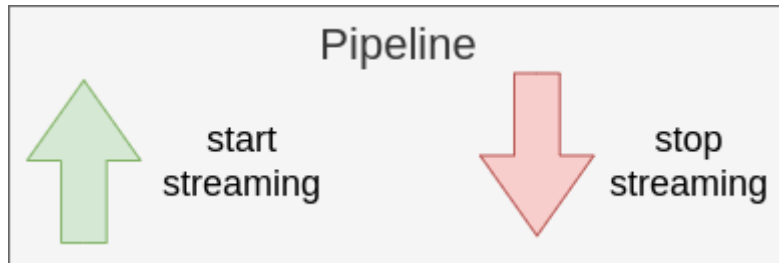
11.2.3. Reference Application at a Glance

So how is this pipeline actually built? You can see how the classes from the Hailo AI Analytics API are used to build the pipeline here:



We build a media pipeline using discrete components called stages, where each stage performs a specific task—such as encoding or adding overlays. Each stage runs in its own thread, allowing the pipeline to process data efficiently in parallel, stay responsive under load, and remain modular and easy to extend.

As you can see, you can build a streaming pipeline using minimal components. To house and manage the different stages used, we create a **Pipeline** instance. This class manages the enclosed stages and allows the user to *start* and *stop* streaming.



Nested Pipelines

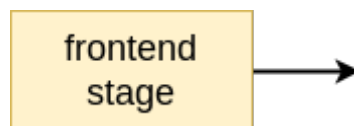
Note that in the single stream processing reference application includes nested pipelines. Since **Pipeline** inherits from **Stage**, we can use pipelines as stages and nest them within other pipelines. This method allows us to build complex applications while keeping the code modular and organized. Additionally, this lets us reuse prebuilt pipelines as components in other applications.

For example, the streaming pipeline in this example was quickly built using a generator function, and can be reused in other applications that require streaming functionality.

Lets look at the different stages used in this pipeline in the order they operate:

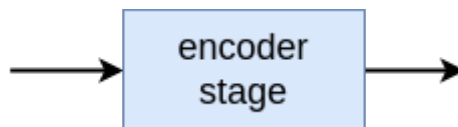
1. **Frontend Stage:** The frontend stage is used to access the video feed from the Media Library. It is responsible for capturing frames from the camera and passing them to the next stage in the pipeline.

Besides capturing frames, the Frontend also performs some basic image adjustment, such as **dewarping** and **resizing**. these operations are performed on the **DSP** for hardware acceleration.



Hardware components involved: **ISP, DSP**

2. **Encoder Stage:** The encoder stage is used to encode raw video feed into an encoded format (H264/H265). Encoding allows the video to be streamed over the network efficiently - higher-quality transmission can be transmitted at lower bandwidth because the bytes required are compressed.

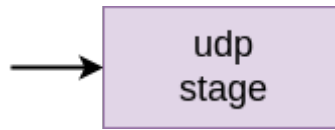


Hardware components involved: **Encoder**

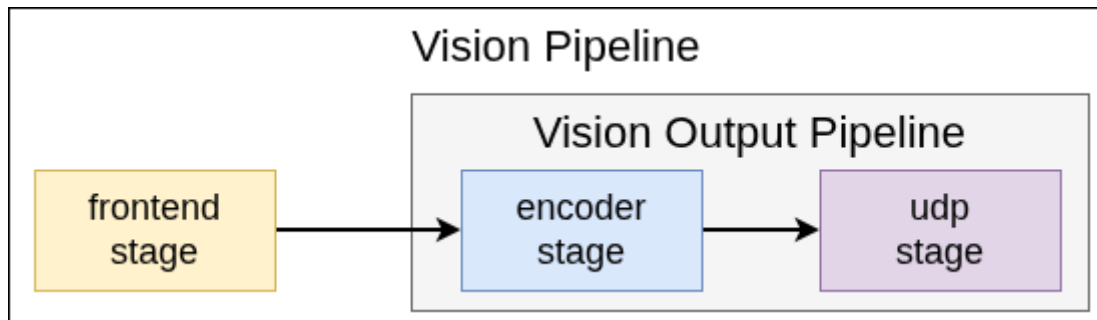
It is important to note that the Hailo-15 includes an on-chip hardware encoder, which allows for real-time encoding of video streams. This also reduces the workload required for compression from the CPU, allowing for a more efficient use of resources.

The Hailo Media Library provides a C++ interface to access the hardware encoder. This API further provides the **EncoderStage** class, which wraps this interface so that it may be easily used in a pipeline.

3. **UDP Stage:** The last stage in this pipeline is the UDP stage. This stage is responsible for sending the encoded video stream over the network using the UDP protocol. The UDP stage takes the encoded video frames from the encoder stage and sends them to a specified IP address and port.



Putting it all together, we now have what is commonly referred to as a **"Vision Pipeline"**: it facilitates streaming from the camera to the host machine.



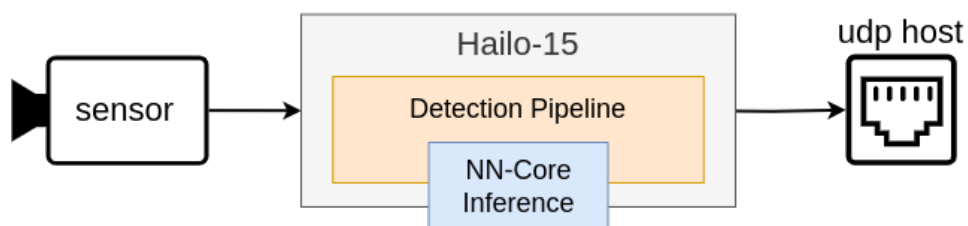
A vision pipeline contains a frontend stage and output streams (encoder + UDP).

11.3. Single Stream Object Detection Reference Application

11.3.1. Overview

This example shows a simple detection pipeline. The key takeaway from this example is how to run basic inference on a single stream using the Hailo-15 hardware accelerated NN-core. This app specifically shows how to run basic inference.

This application runs on the Hailo-15 backend and streams video along with detection metadata over the network. The results can be visualized on the host machine using the [Analytic Draw Client](#) tool, which receives the video stream via UDP and detection metadata via ZeroMQ.



11.3.2. Running the Application

The Hailo-15 Vision Processor Software package includes the Single stream detection application pre-compiled and ready to run on the Hailo15 platform as part of the release image.

To run the application, follow these steps:

1. On the host machine, set up and run the Analytic Draw Client to receive and display the video stream with overlays.

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Then, navigate to the analytic viewer directory and run the application:

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py
```

The application will listen on UDP port 5000 (default) for the video stream and display it with inference overlays.

You can customize the port and other settings using command-line arguments. For example:

```
$ python app_analytic_draw_client.py --port 5000 --analytic-data-ip 10.0.0.1
```

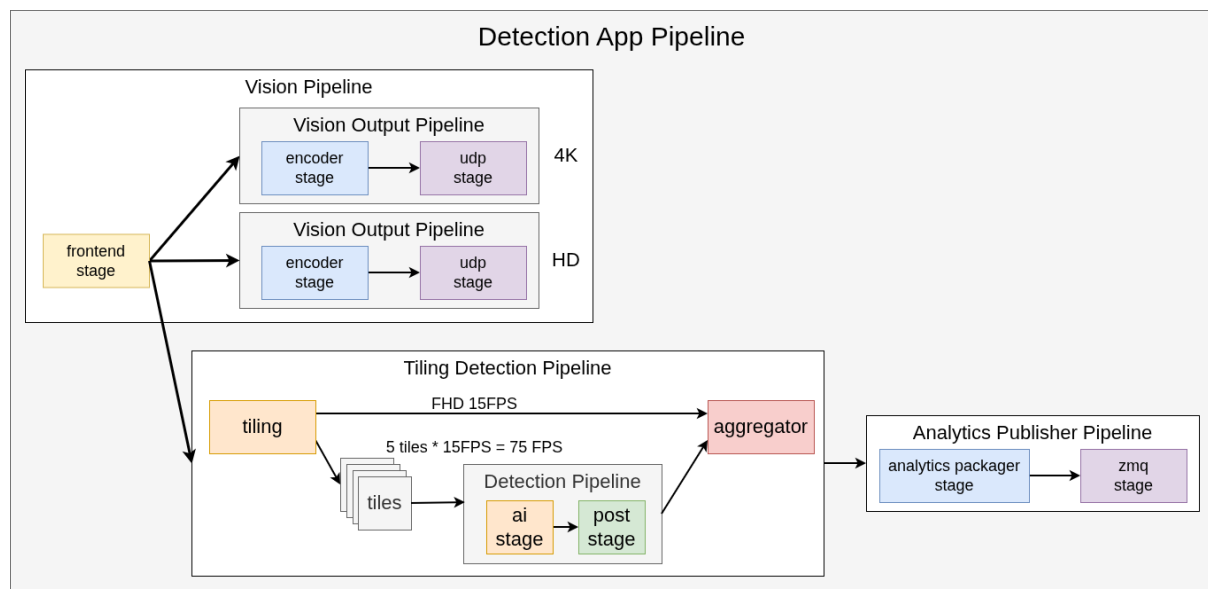
2. On the Hailo15 platform, run the executable located at the following path:

```
$ ./apps/case_studies/detection/detection_case_study
```

You should now be able to see the video feed with the inference overlay on the host machine screen.

11.3.3. Application at a Glance

You can see how the classes from the Hailo AI Analytics API are used to build the pipeline here:

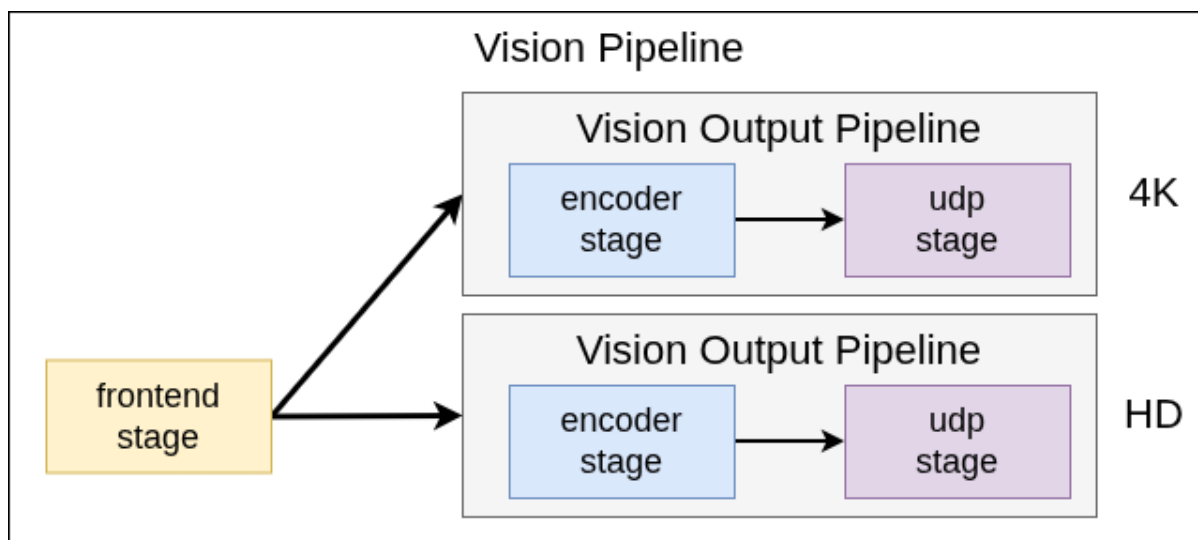


At first it may look like there are a lot of new stages compared to the [Single Stream Processing Reference Application](#), but we leverage **subpipelines** to manage complexity.

Lets look at the different subpipelines used in this application in the order they operate:

1. **Vision Pipeline: The Vision Pipeline is a subpipeline that orchestrates capture of video frames from the sensor. It is respon**

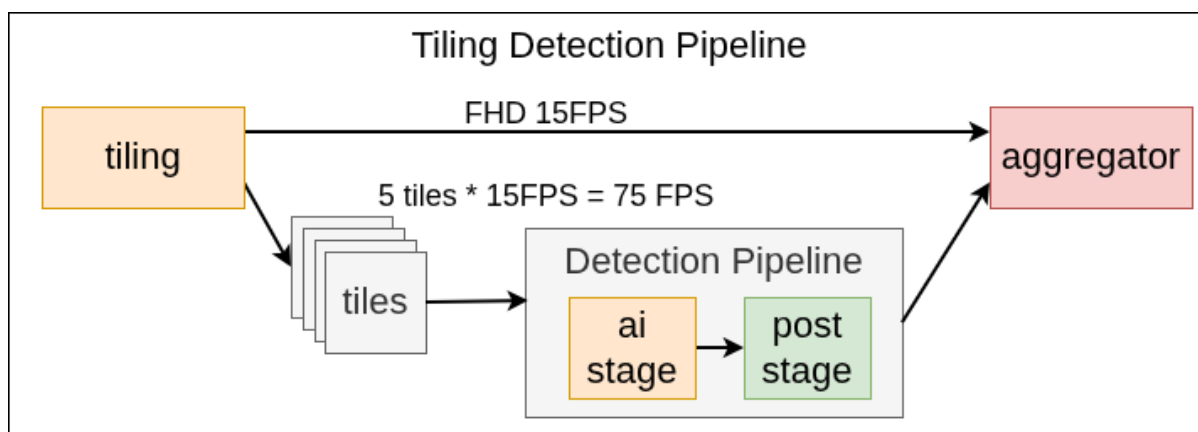
Note that in this application, the vision pipeline has two streaming outputs: 4K and HD. You can configure this behavior through the Media Library Profiles.



Hardware components involved: **ISP, DSP, NN-Core, Encoder**

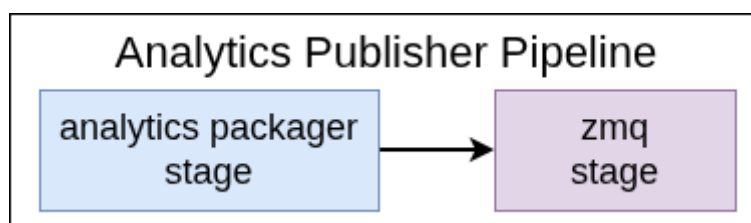
2. Tiling Detection Pipeline: The Tiling Detection Pipeline is a subpipeline that performs tiling and runs the detection model on

The tiling is performed by the DSP for hardware acceleration, and the detection model runs on the NN-Core. This subpipeline comes prebuilt, allowing for easy integration into the application.



Hardware components involved: **NN-Core, DSP**

3. Analytics Publisher Pipeline: The Analytics Publisher Pipeline is responsible for taking the detection results (bounding boxes, labels) and sending them to the host machine via ZeroMQ. This allows the host-side analytic viewer to draw overlays on the video stream based on the inference results.



Hardware components involved: **CPU**

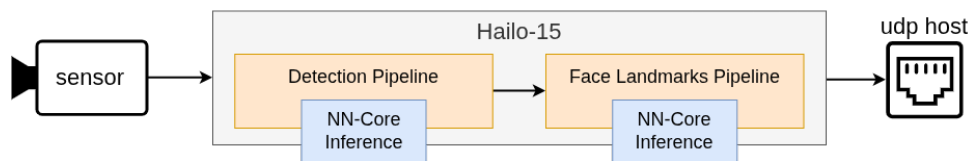
Putting it all together, we have a pipeline that takes video feed from the sensor, runs inference on it, and streams the results over the network.

11.4. Face Landmarks Reference Application

11.4.1. Overview

This example shows a face landmarks detection pipeline that builds upon the basic detection pipeline. The key take-away from this example is how to chain multiple AI stages together, where the output of one AI model (person/face detection) feeds into another AI model (facial landmarks detection). This app demonstrates a common pattern in computer vision: using detection results as input regions for more specialized analysis.

This application runs on the Hailo-15 backend and streams video along with face detection and landmark metadata over the network. The results can be visualized on the host machine using the [Analytic Draw Client](#) tool, which receives the video stream via UDP and metadata via ZeroMQ, displaying face detection boxes and facial landmark points.



11.4.2. Running the Application

The Hailo-15 Vision Processor Software package includes the Face Landmark application pre-compiled and ready to run.

To run the face_landmarks application, follow these steps:

1. On the host machine, set up and run the Analytic Draw Client to receive and display the video stream with overlays.

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Then, navigate to the analytic viewer directory and run the application:

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py
```

The application will listen on UDP port 5000 (default) for the video stream and display it with inference overlays.

You can customize the port and other settings using command-line arguments. For example:

```
$ python app_analytic_draw_client.py --udp-port 5000 --analytic-data-ip 10.0.0.1
```

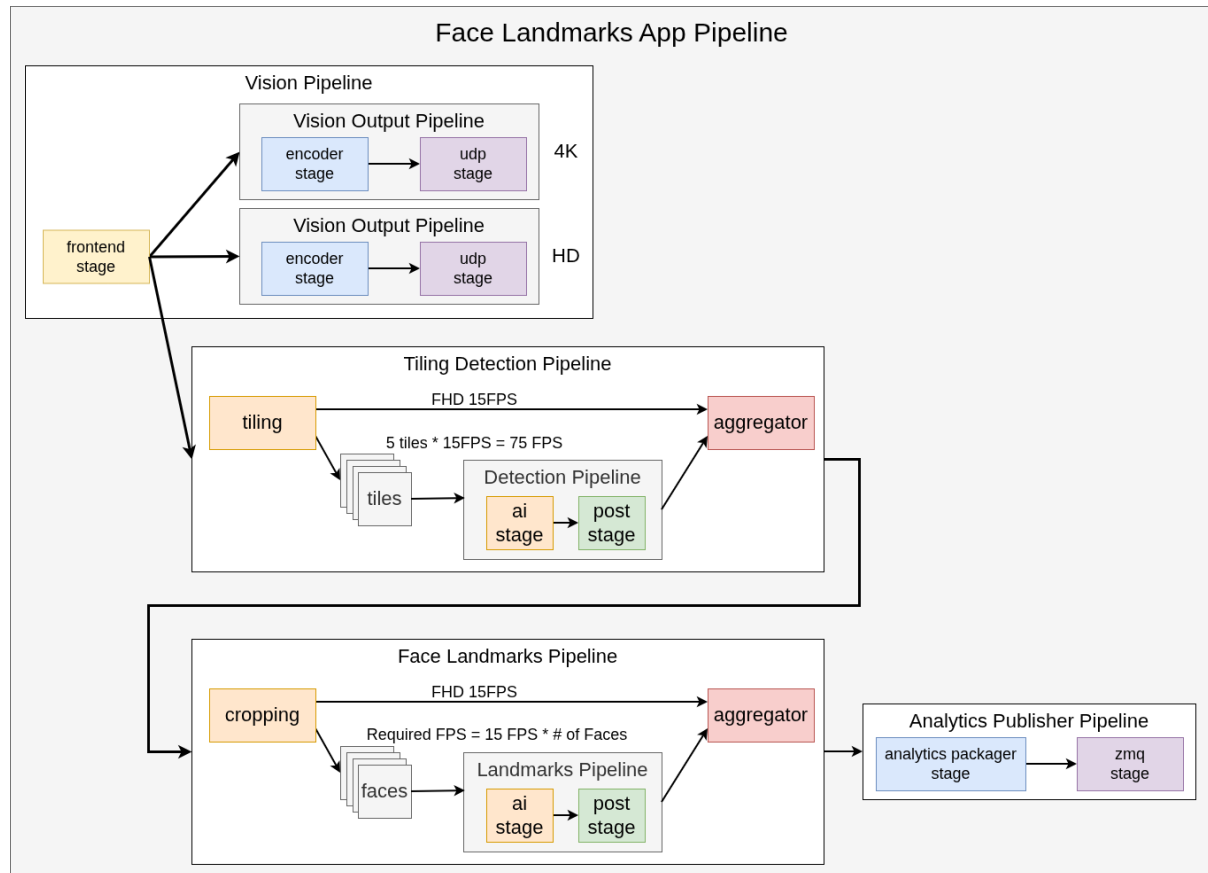
2. On the Hailo15 platform, run the executable located at the following path:

```
$ ./apps/face_landmarks/face_landmarks_app
```

You should now be able to see the video feed with face detection boxes and facial landmark points overlaid on the host machine screen.

11.4.3. Application at a Glance

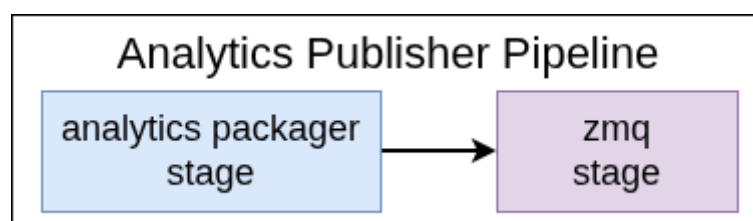
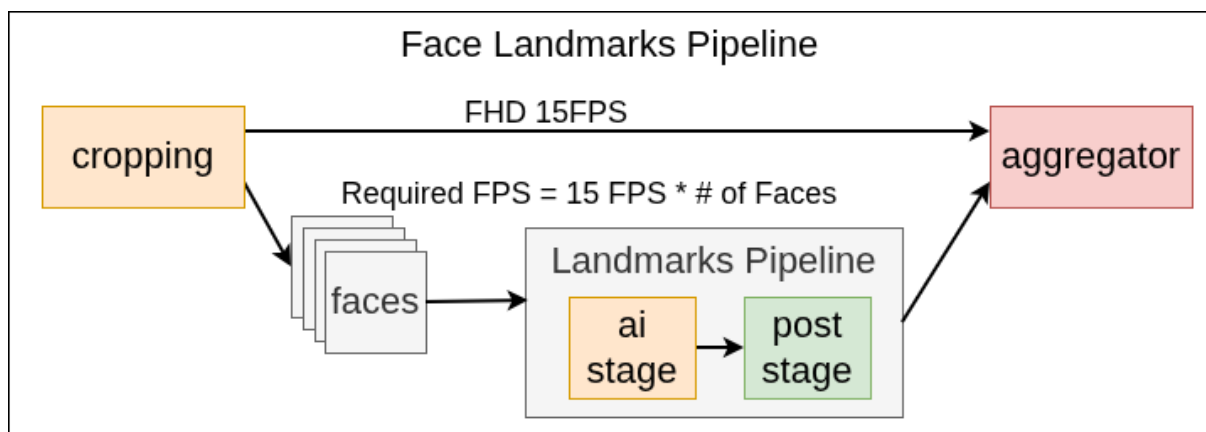
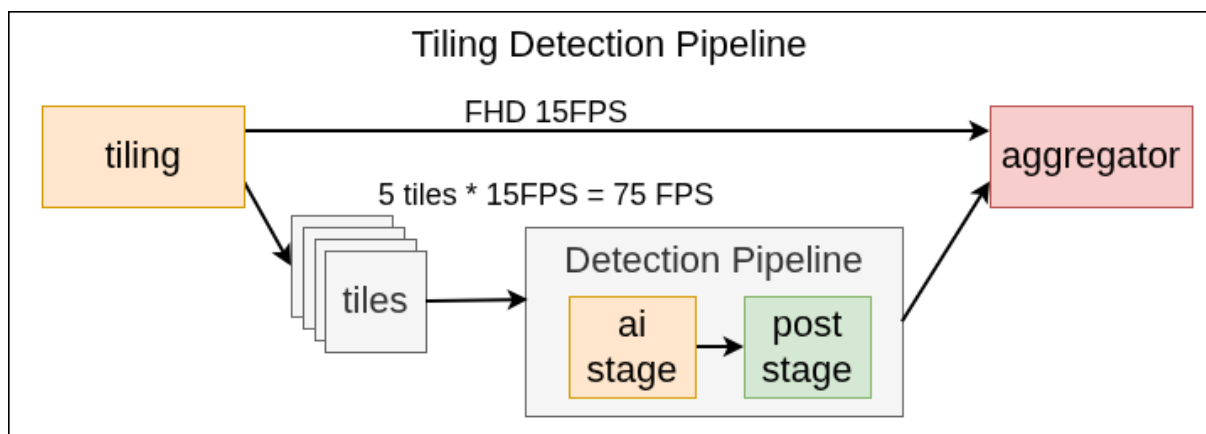
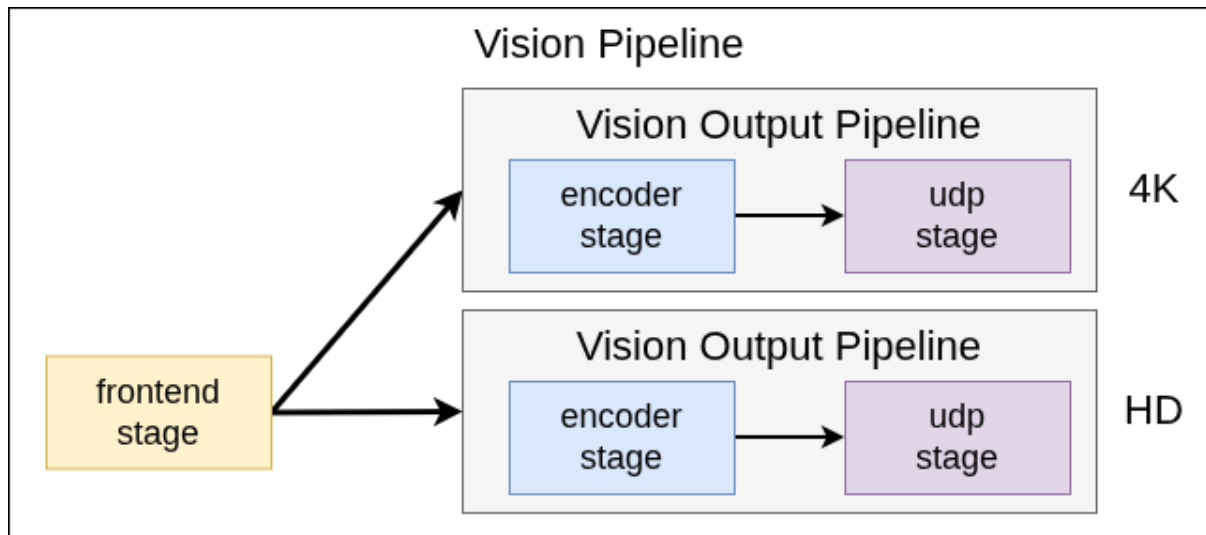
You can see how the classes from the Hailo AI Analytics API are used to build the pipeline here:



This application extends the detection pipeline by adding a **Face Landmarks** stage that performs additional AI inference on detected faces. The pipeline demonstrates a cascaded AI architecture where detection results guide subsequent specialized analysis.

Lets look at the different stages used in this pipeline in the order they operate:

- 1. Vision Pipeline: The Vision Pipeline is a subpipeline that orchestrates capture of video frames from the sensor. It is responsible for capturing video frames from the sensor and encoding them for streaming.**
 Note that in this application, the vision pipeline has two streaming outputs: 4K and HD. You can configure this behavior through the Media Library Profiles.
 Hardware components involved: **ISP, DSP, NN-Core, Encoder**
- 2. Tiling Detection Pipeline: The Tiling Detection Pipeline is a subpipeline that performs tiling and runs the detection model on the tiled frames.**
 The tiling is performed by the DSP for hardware acceleration, and the detection model runs on the NN-Core. This subpipeline comes prebuilt, allowing for easy integration into the application.
 Hardware components involved: **NN-Core, DSP**
- 3. Face Landmarks Pipeline: The Face Landmarks Pipeline is a subpipeline that takes the detected face bounding boxes from the detection pipeline and performs face landmark detection.**
 Cropping is handled by the DSP for hardware acceleration, and the landmarks model runs on the NN-Core. This subpipeline also comes prebuilt for easy integration.
 Hardware components involved: **NN-Core, DSP**
- 4. Analytics Publisher Pipeline: The Analytics Publisher Pipeline is responsible for taking the detection results (bounding boxes, labels) and sending them to the host machine via ZeroMQ. This allows the host-side analytic viewer to draw overlays on the video stream based on the inference results.**
 Hardware components involved: **CPU**



Putting it all together, we have a pipeline that takes video feed from the sensor, runs cascaded AI inference (detection followed by landmarks), and streams the results with metadata over the network.

11.4.4. Cascaded AI Pipeline Pattern

This application demonstrates an important architectural pattern: **Cascaded AI Inference**.

Key Concepts:

- **Region-based Processing:** The landmarks model only processes face regions detected by the first model, not the entire frame
- **Modularity:** Each AI stage is independent and can be swapped or extended without affecting others
- **Metadata Flow:** Detection results (bounding boxes) flow through the pipeline as metadata, guiding subsequent stages

This pattern is commonly used in many computer vision applications:

- Face detection → Facial landmarks
- Person detection → Pose estimation
- Vehicle detection → License plate recognition

The Hailo AI Analytics API makes it easy to build such cascaded pipelines by allowing AI stages to consume and produce metadata that flows alongside the video frames.

11.5. Dynamic Privacy Mask Case Study

11.5.1. Overview

This example shows a dynamic privacy masking pipeline that applies real-time privacy masks to detected objects directly on the Hailo-15 device. The key takeaway from this example is how to combine AI inference (detection and segmentation) with the hardware-accelerated privacy mask blender in the encoder, producing privacy-masked video output without any host-side post-processing.

Unlike the detection and face landmarks case studies which stream metadata for host-side overlay, this application applies masking directly in the encoder on-device. AI detection and segmentation results are stored in the Analytics Database and the encoder's privacy mask blender queries this database each frame to apply masks to the encoded video stream.

The application supports configurable object labels (person, vehicle, face, license_plate), two masking modes (solid color overlay or pixelization), and automatically adapts resource usage to the platform (Hailo-15H or Hailo-15L).

Optionally, detection metadata can also be streamed over ZeroMQ for visualization on the host machine using the [Analytic Draw Client](#) tool.

11.5.2. Running the Application

The application will come pre-compiled and ready to run on the Hailo15 platform as part of the release image.

To run the dynamic privacy mask application, follow these steps:

1. On the Hailo15 platform, run the executable:

```
$ ./apps/case_studies/dynamic_privacy_mask/dynamic_privacy_mask_app
```

The application will start streaming privacy-masked video over UDP to the host machine.

- (Optional) To visualize detection metadata on the host machine, enable ZMQ output and run the Analytic Draw Client:

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Then run the application with a ZMQ port:

```
$ ./apps/case_studies/dynamic_privacy_mask/dynamic_privacy_mask_app -
  ↪ -zmq-port 7000
```

And on the host machine:

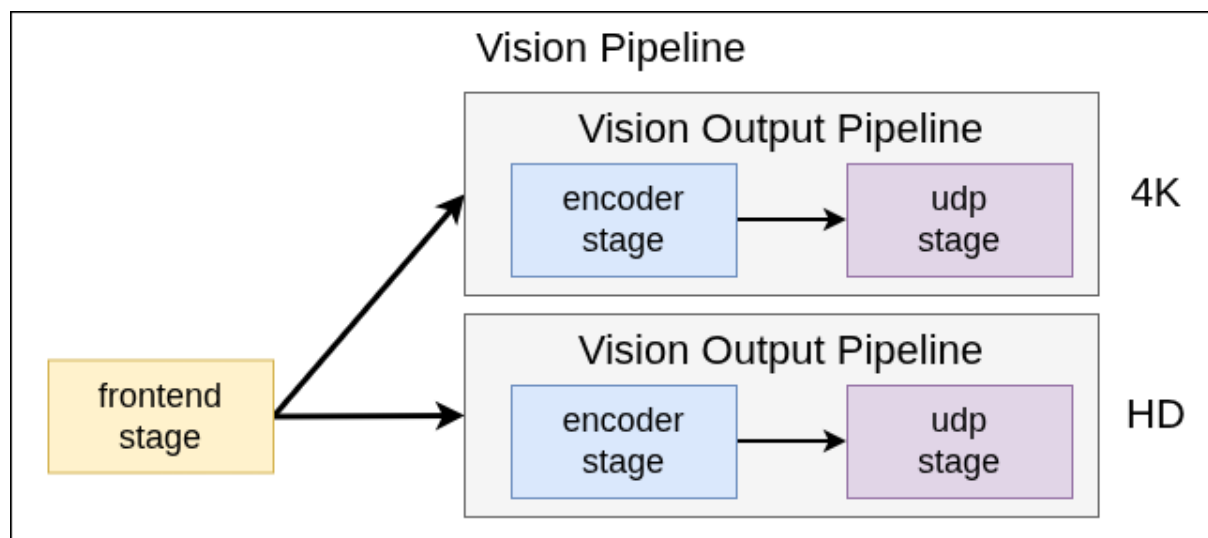
```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py --udp-port 5000 --analytic-data-
  ↪ ip 10.0.0.1
```

11.5.3. Application at a Glance

This application builds two main pipelines: a **Vision Pipeline** for video capture and streaming, and a **DPM AI Pipeline** for detection, segmentation, and analytics storage. The AI pipeline results feed back into the encoder's privacy mask blender, creating a closed-loop system where the output video stream already has privacy masks applied.

Lets look at the different subpipelines used in this application in the order they operate:

- Vision Pipeline:** The Vision Pipeline captures video frames from the sensor, encodes them, and streams over UDP. The encoder in this pipeline includes the **Privacy Mask Blender**, which queries the Analytics Database each frame to retrieve segmentation masks and apply them to the encoded output. This is where the actual masking happens — the AI pipeline produces the data, and the encoder consumes it.



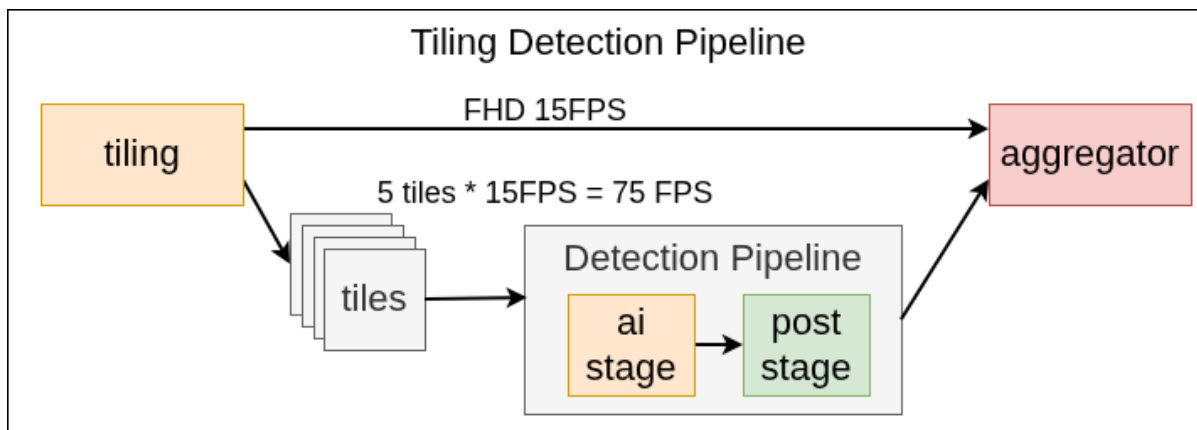
Hardware components involved: **ISP, DSP, Encoder**

- Tiling Detection Pipeline:** The Tiling Detection Pipeline splits high-resolution frames into smaller tiles, runs YOLOv8n object detection on each tile using the NN-Core, and merges the results. Tiling allows accurate detection on high-resolution inputs without downscaling artifacts. The DSP handles the tiling operations for hardware acceleration.

Hardware components involved: **NN-Core, DSP**

- Lightweight Tracker:** An IOU-based tracker that maintains object identity across frames. This reduces jitter in the privacy masks by providing consistent tracking IDs, ensuring that masks follow objects smoothly rather than flickering as detections appear and disappear between frames.

Hardware components involved: **CPU**



4. **Detection Limiter:** Filters detections by label and caps the number of detections per frame to stay within resource limits (12 on Hailo-15H, 6 on Hailo-15L). The limiter uses a label hierarchy system where enabling a parent label automatically includes its children:

- **person** automatically includes **face**
- **vehicle** automatically includes **license_plate**

Hardware components involved: **CPU**

5. **DPM Segmentation Pipeline:** For each detected bounding box, the DSP crops the region from the frame, and the LinkNet model (128x128 input) runs semantic segmentation on the NN-Core to produce a per-pixel mask. An aggregator combines the individual crop results back into a unified set of segmentation masks for the frame.

Hardware components involved: **NN-Core, DSP**

6. **Analytics Database:** Stores the semantic segmentation results with the ID "semantic_segmentation". The encoder's privacy mask blender queries this database every frame to retrieve the latest masks and render them onto the encoded video. This is the bridge between the AI pipeline and the encoder.

Hardware components involved: **CPU**

7. **(Optional) Metadata Sender:** When a ZMQ port is specified, this stage publishes detection metadata over ZeroMQ. This allows the host-side [Analytic Draw Client](#) to display detection boxes alongside the privacy-masked video stream for debugging and visualization purposes.

Hardware components involved: **CPU**

Putting it all together, we have a pipeline that captures video from the sensor, detects and segments objects of interest, stores the segmentation masks in the Analytics Database, and applies those masks in real-time directly in the encoder — producing a privacy-compliant video stream at the edge.

11.5.4. On-Device Privacy Masking Pattern

This application demonstrates an important architectural pattern: **On-Device Privacy Masking**.

Key Concepts:

- **Closed-Loop AI-to-Encoder:** AI inference results feed back into the video encoder via the Analytics Database, enabling privacy masking directly on the device without external processing or host-side post-processing.
- **Cascaded Detection + Segmentation:** Object detection narrows the regions of interest, and segmentation produces precise per-pixel masks only within those regions. This two-stage approach is more efficient than running full-frame segmentation.
- **Label Hierarchy:** Parent-child label relationships (person->face, vehicle->license_plate) allow coarse label selection while automatically including fine-grained sub-labels for comprehensive masking.
- **Platform-Aware Resource Management:** Detection limits automatically adapt to platform capabilities (Hailo-15H supports more concurrent detections than Hailo-15L), ensuring stable real-time performance.

- **Configurable Mask Appearance:** Supports both solid color overlay and pixelization modes, configurable at runtime through the Privacy Mask API or JSON configuration.

This pattern is commonly used in privacy-sensitive applications:

- GDPR-compliant video surveillance
- Privacy-preserving traffic monitoring
- Edge-based anonymization for video analytics

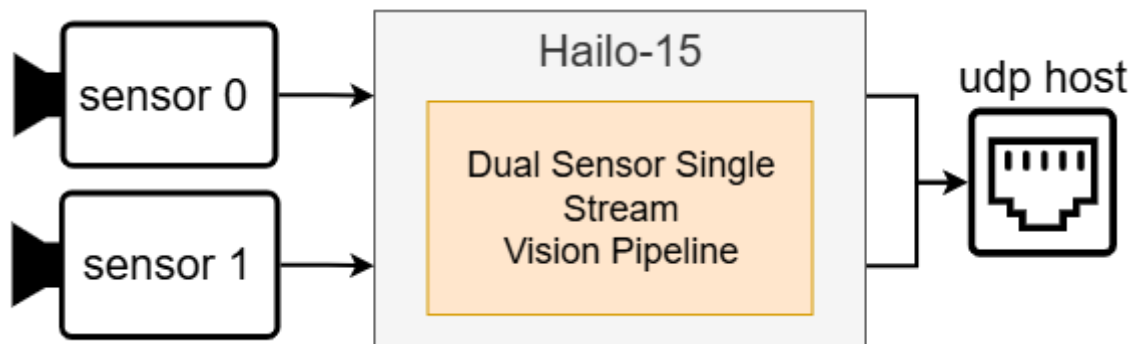
The Hailo Analytics API makes it easy to build such privacy masking pipelines by providing prebuilt generators for tiling detection, segmentation, and analytics database integration. For details on the Privacy Mask API and configuration, see [Privacy Mask](#).

11.6. Dual Sensor Processing Reference Application

11.6.1. Overview

This example shows a dual sensor vision pipeline where each sensor produces a single stream. The key takeaway from this example is how to manage multiple sensors simultaneously, accessing video feeds from two sensors via the Media Library and encoding those video streams using separate encoders for streaming.

This application demonstrates how to: - Configure and manage two sensors independently - Handle separate media library configurations for each sensor - Create distinct pipelines for each sensor while avoiding port conflicts - Stream multiple video feeds simultaneously



11.6.2. Prerequisites

Before running the application, you need to configure the device tree overlay:

```
$ fw_setenv dtb_overlays '#conf-hailo_hailo15-sbc-sensor-1.dtbo'
$ reboot
```

After the reboot, the system will be ready to run the dual sensor application.

11.6.3. Running the Application

The Hailo-15 Vision Processor Software package includes the Dual Sensor application pre-compiled and ready to run.

To run this application, follow these steps:

1. On the host machine, set up and run the Analytic Draw Client to receive and display video feeds from both sensors.

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Since this application handles two sensors, it will be necessary to run separate instances of the client in different terminals:

Terminal 1 (Sensor 0):

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py --port 5000 --analytic-data-ip 10.0.0.1
```

Terminal 2 (Sensor 1):

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py --port 5100 --analytic-data-ip 10.0.0.1 --
  ↳ analytic-data-port 7001
```

Note that Sensor 1 uses port 5100 (5000 + 100 offset) to avoid conflicts with Sensor 0.

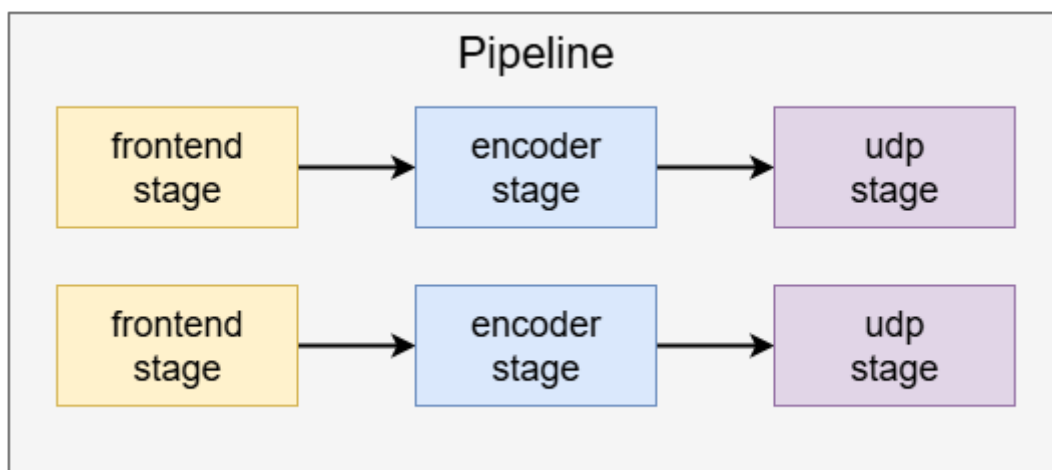
2. On the Hailo-15H platform, run the executable located at the following path:

```
$ ./apps/case_studies/dual_sensor_single_stream/dual_sensor_single_
  ↳ stream_case_study
```

It will now be possible to see feeds from both sensors with inference overlays on the host machine.

11.6.4. Application at a Glance

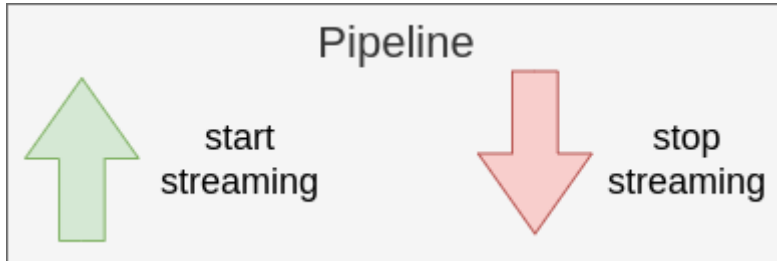
So how is this dual sensor application actually built? The classes from the Hailo AI Analytics API are used to build the dual sensor pipeline here:



The dual sensor media pipeline is built using discrete components called stages, where each stage performs a specific task such as encoding or adding overlays. Each stage runs in its own thread, allowing the pipeline to process data from both sensors efficiently in parallel, stay responsive under load, and remain modular and easy to extend.

Unlike the single sensor case study, this application manages **two independent sensor pipelines** within a single application framework. Each sensor has its own: - Media Library configuration - Frontend Stage - Encoder Stage(s) - UDP Stage(s)

To house and manage the different stages used, a single **Pipeline** instance is created that coordinates both sensor pipelines. This class manages all enclosed stages from both sensors and allows the user to *start* and *stop* streaming for the entire system simultaneously.



11.6.5. Key Architectural Differences for Dual Sensor

Resource Management:

- Two separate Media Library instances (one per sensor)
- Separate configuration files for each sensor:
 - /etc/imaging/cfg/medialib_configs/case_studies/dual_sensor_single_stream_medialib_config_sensor_0.json
 - /etc/imaging/cfg/medialib_configs/case_studies/dual_sensor_single_stream_medialib_config_sensor_1.json

Naming Conventions: - Frontend stages: frontend_stage_sensor_0, frontend_stage_sensor_1
 - Encoder stages: enc_sensor_0_[stream_id], enc_sensor_1_[stream_id] - UDP stages: udp_sensor_0_[stream_id], udp_sensor_1_[stream_id]

Port Management: - Sensor 0: Uses base ports (5000+) - Sensor 1: Uses offset ports (5100+) to avoid conflicts - Port calculation: $\text{base_port} + \text{stream_offset} + (\text{sensor_index} * 100)$

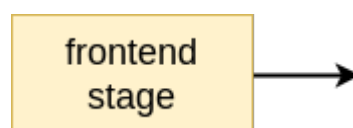
Command Line Arguments:

- --config-file-path-sensor-0: Configuration path for sensor 0
- --config-file-path-sensor-1: Configuration path for sensor 1
- --profile-sensor-0: Profile name for sensor 0
- --profile-sensor-1: Profile name for sensor 1
- --host-ip: Target IP address for UDP streams from both sensors

Let's look at the different stages used in each sensor pipeline:

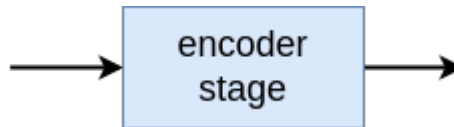
1. **Frontend Stage (Per Sensor):** Each sensor has its own frontend stage responsible for accessing the video feed from that specific sensor via the Media Library. Each frontend captures frames from its respective camera and passes them to the next stage in its pipeline.

Besides capturing frames, each Frontend also performs sensor-specific image adjustments, such as **dewarping** and **resizing**. These operations are performed on the **DSP** for hardware acceleration.



Hardware components involved: **ISP, DSP**

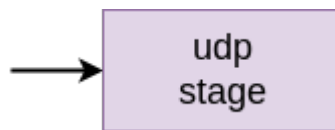
2. **Encoder Stage (Per Sensor Per Stream):** Each sensor has its own encoder stage(s) to encode raw video feed into an encoded format (H264/H265). This allows both sensor streams to be encoded simultaneously and independently.



Hardware components involved: **Encoder**

The Hailo-15's on-chip hardware encoder can handle multiple streams simultaneously, allowing for real-time encoding of both sensor video streams. This design ensures efficient resource usage while maintaining independent processing for each sensor.

3. **UDP Stage (Per Sensor Per Stream):** Each sensor has its own UDP stage(s) responsible for sending the encoded video stream over the network using different UDP ports to avoid conflicts. Each UDP stage takes encoded video frames from its corresponding encoder stage and sends them to the specified IP address and sensor-specific port.



Putting it all together, what is commonly referred to as a “**Dual Sensor Vision Pipeline**” is now available: it facilitates simultaneous streaming from both cameras to the host machine with proper resource management and conflict avoidance.

11.7. Dual Sensor Face Landmarks Application

11.7.1. Overview

This application demonstrates how to run a face landmarks AI pipeline on one sensor while simultaneously streaming simple video feeds from a second sensor. It combines the [Face Landmarks Reference Application](#) cascaded AI pipeline pattern with the [Dual Sensor Processing Reference Application](#) dual sensor management approach.

- **Sensor 0:** Runs the full face landmarks AI pipeline (person/face detection followed by facial landmarks detection) with analytics metadata published over ZeroMQ
- **Sensor 1:** Runs a simple streaming pipeline without AI processing, providing a direct video feed

The results from Sensor 0 can be visualized on the host machine using the [Analytic Draw Client](#) tool, which receives the video stream via UDP and metadata via ZeroMQ, displaying face detection boxes and facial landmark points.

11.7.2. Prerequisites

Before running the application, the device tree overlay needs to be configured to enable the second sensor:

```
$ fw_setenv dtb_overlays '#conf-hailo_hailo15-sbc-sensor-1.dtbo'
$ reboot
```

After the reboot, the system will be ready to run the dual sensor application.

11.7.3. Running the Application

The application will come pre-compiled and ready to run on the Hailo-15 platform as part of the release image.

To run the application, follow these steps:

1. On the host machine, set up and run the Analytic Draw Client to receive and display video feeds from both sensors.

First, ensure you have completed the environment setup as described in the [Analytic Draw Client](#) documentation.

Since this application handles two sensors, run separate instances of the client for each sensor:

Terminal 1 (Sensor 0 - Face Landmarks AI Pipeline):

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py --udp-port 5000 --analytic-data-ip 10.0.0.1
↪1
```

Terminal 2 (Sensor 1 - Simple Stream):

```
$ cd tools/analytic_viewer/
$ python app_analytic_draw_client.py --udp-port 5100 --analytic-data-ip 10.0.0.1 --analytic-data-port 7001
↪1
```

The use of `--analytic-data-port 7001` is intentional: the analytic viewer defaults to subscribing on ZMQ port 7000, which is where Sensor 0 publishes its face landmarks metadata. Pointing Sensor 1's viewer to an unused port ensures Sensor 0's overlays are not drawn on Sensor 1's video feed.

Note that Sensor 0 uses base port 5000, while Sensor 1 uses port 5100 (offset by +100) to avoid conflicts.

2. On the Hailo-15 platform, run the executable located at the following path:

```
$ ./apps/face_landmarks/face_landmarks_dual_sensor_app/face_landmarks_dual_sensor_app
```

It should now be possible to see the video feed from Sensor 0 with face detection boxes and facial landmark points on Terminal 1, and a plain video stream from Sensor 1 on Terminal 2.

11.7.4. Application at a Glance

This application combines two pipeline patterns into a single application:

Sensor 0 - Face Landmarks AI Pipeline:

The Sensor 0 pipeline follows the same cascaded AI architecture as the [Face Landmarks Reference Application](#) application:

1. **Vision Pipeline:** Captures video frames from Sensor 0 and encodes the non-AI streams (4K and HD) for UDP streaming. The AI stream (`sink2`) is excluded from encoding and instead routed to the AI chain.

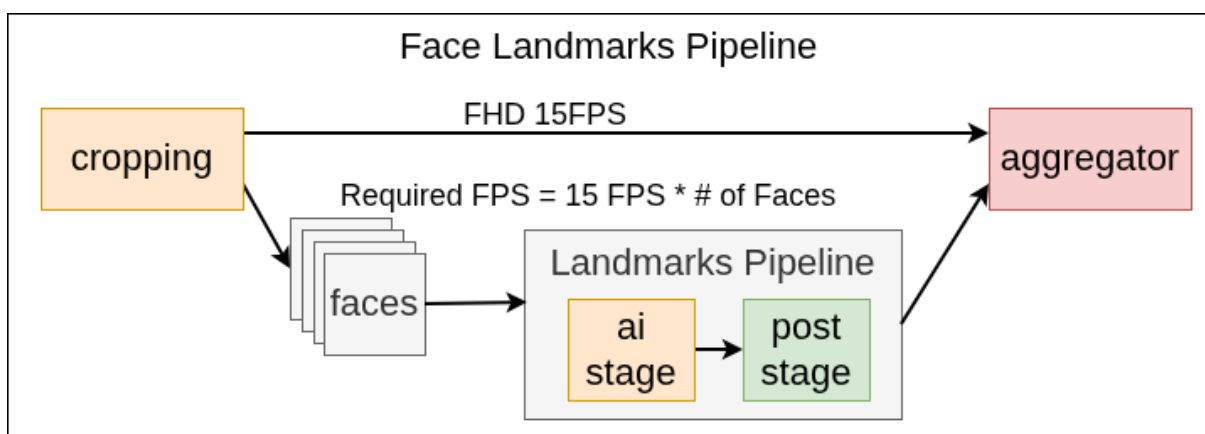
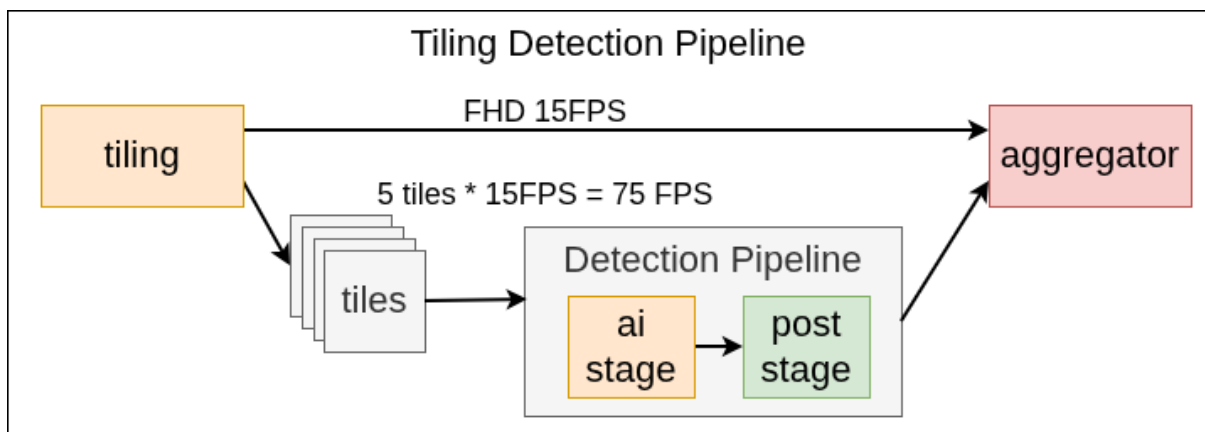
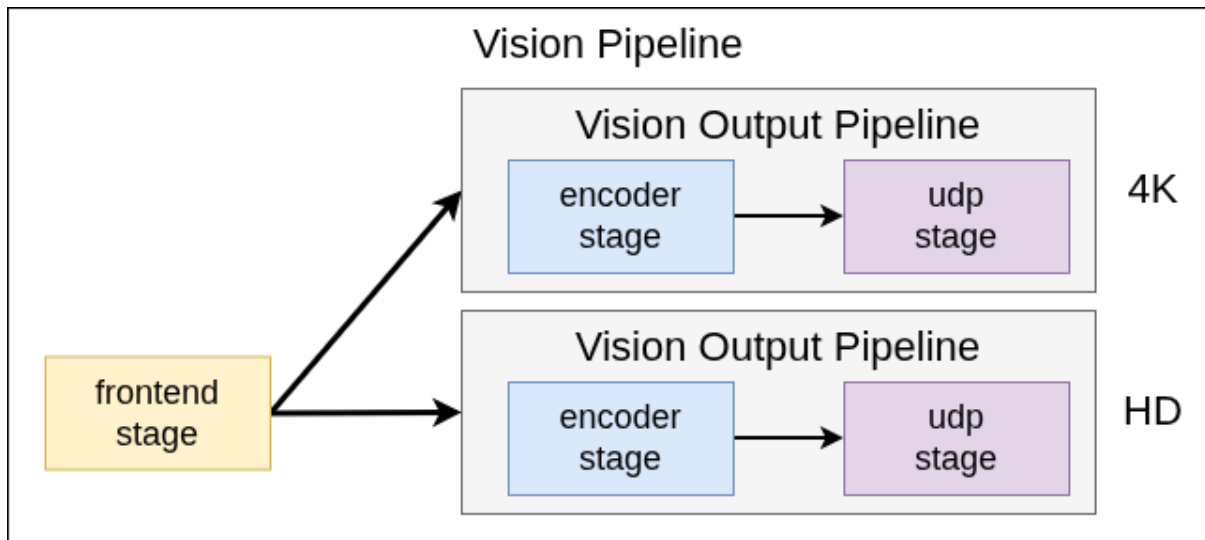
Hardware components involved: **ISP, DSP, Encoder**

2. **Tiling Detection Pipeline:** Performs person/face detection using YOLOv8n on tiled regions of the AI stream. Tiling allows running detection on high-resolution inputs by splitting them into smaller tiles, running inference on each tile, and merging the results.

Hardware components involved: **NN-Core, DSP**

3. **Face Landmarks Pipeline:** Takes detected face bounding boxes from the detection stage and runs a facial landmarks model on those regions. Produces landmark points as output.

Hardware components involved: **NN-Core, DSP**



4. **Analytics Publisher Pipeline:** Packages detection results and landmark metadata and sends them to the host machine via ZeroMQ for visualization by the [Analytic Draw Client](#).



Hardware components involved: **CPU**

Sensor 1 - Simple Vision Pipeline:

Sensor 1 runs a simple vision pipeline that captures video, encodes it, and streams it over UDP. No AI processing is performed on this sensor.

11.7.5. Architecture Notes

Dual Sensor Resource Management:

- Two separate Media Library instances are created, one for each sensor, allowing independent configuration and control
- A single Pipeline instance coordinates stages from both sensors, enabling synchronized start/stop operations

Port Management:

- Sensor 0: Uses base ports (5000+) for video streams and port 7000 for ZMQ metadata
- Sensor 1: Uses offset ports (5100+) to avoid conflicts
- Both base ports are configurable via command-line arguments

Independent Configuration:

- Each sensor has its own media library configuration file
- Each sensor can use a different profile (e.g., different dewarp settings or rotation angles)
- AI pipeline parameters (ZMQ port) only apply to Sensor 0

11.8. GStreamer Vision Analytics Reference Application

11.8.1. Overview

This example shows how to combine GStreamer elements (`gsthailovision`, `gsthailoencoder`) with a Hailo AI Analytics tiling detection and overlay pipeline in a single application.

The key takeaway from this example is how to bridge GStreamer-based video I/O with analytics processing — using GStreamer for camera capture and H.264 encoding, while running tiling-based object detection and overlay drawing through the Hailo AI Analytics pipeline framework.

This example runs object detection on the device, draws the detection overlays directly onto the video frames, and streams the annotated video over UDP as a standard H.264 stream. No separate metadata channel is needed — the host simply receives and displays the video with the overlays already rendered.

This example also extends the [GStreamer API Elements](#) example (`gst_vision_encoder`), which uses GStreamer elements for capture and encoding but does not include any analytics processing.

11.8.2. Running the Application

The Hailo-15 Vision Processor Software package includes the application pre-compiled and ready to run.

To run the application, follow these steps:

1. On the Hailo15 platform, run the executable:

```
$ ./apps/case_studies/gst_vision_analytics_encoder/gst_vision_
  ↳analytics_encoder_case_study
```

2. On the host machine, receive and display the UDP stream. Since the detection overlays are already drawn on the frames on the device, the host only needs to decode and display the H.264 stream:

```
$ gst-launch-1.0 udpsrc port=5000 address=10.0.0.2 \
! application/x-rtp,encoding-name=H264 \
! queue max-size-buffers=30 max-size-bytes=0 max-size-time=0 leaky=no \
! rtpjitterbuffer mode=0 \
! queue max-size-buffers=30 max-size-bytes=0 max-size-time=0 leaky=no \
! rtph264depay \
! queue max-size-buffers=30 max-size-bytes=0 max-size-time=0 leaky=no \
! h264parse ! avdec_h264 \
! queue max-size-buffers=30 max-size-bytes=0 max-size-time=0 \
  ↳leaky=downstream \
! videoconvert n-threads=8 \
! queue max-size-buffers=30 max-size-bytes=0 max-size-time=0 leaky=no \
! fpsdisplaysink fps-update-interval=2000 name=hailo_display text-
  ↳overlay=false sync=false
```

You should now be able to see the video feed with detection overlays on the host machine screen.

Command-Line Options

Option	Default	Description
-t, --timeout	60	Time to run in seconds
-c, --config-file-path	/etc/ imaging/ cfg/ medialib_configs/ ai_example_medialib_config. json	Media Library configuration file path
-o, --host-ip	10.0.0.2	Host IP address for UDP output
-s, --switch-profile	(none)	Profile name to switch to mid-run
-d, --switch-delay	10	Seconds to wait before switching profile

11.8.3. Application at a Glance

This application is built from three cooperating pipelines that pass video frames between them:

```
GStreamer Input Pipeline
gsthailovision  sink0      queue  appsink (to analytics)
                sink1      queue  fakesink

Analytics Pipeline
GstSourceStage  TilingDetectionPipeline  OverlayStage
                GstSinkStage

GStreamer Output Pipeline
appsrc  gsthailoencoder  rtph264pay  udpsink
```

GStreamer Input Pipeline

The input pipeline uses `gsthailovision` to capture video from the camera sensor. It exposes two output streams:

- **sink0** — routed to an `appsink`, which feeds frames into the analytics pipeline for detection and overlay processing.
- **sink1** — routed to a `fakesink` (discarded). This secondary stream is configured in the Media Library profile but not used by the analytics path.

Analytics Pipeline

The analytics pipeline is built using the Hailo AI Analytics pipeline framework and consists of these stages:

1. **GstSourceStage** — Pulls frames from the GStreamer `appsink` into the analytics pipeline.
2. **TilingDetectionPipeline** — A prebuilt subpipeline that performs tiling-based object detection. It splits high-resolution frames into tiles, runs inference on each tile using the NN-Core, and merges the detection results. Hardware components involved: **DSP, NN-Core**.
3. **OverlayStage** — Draws detection bounding boxes and labels directly onto the video frames.
4. **GstSinkStage** — Pushes the annotated frames back into a GStreamer `appsrc` for encoding.

GStreamer Output Pipeline

The output pipeline takes the annotated frames from the `appsrc`, encodes them to H.264 using `gsthailoencoder`, packetizes with `rtph264pay`, and sends the stream over UDP to the host machine.

Shared MediaLibrary Instance

A critical design detail: both GStreamer pipelines (input and output) are created with the **same pipeline name** ("vision-analytics-encoder"). The `gsthailovision` and `gsthailoencoder` elements use the pipeline name as a key to look up and share a single `MediaLibrary` instance. This ensures the encoder can find the stream configuration set up by the vision element.

Despite sharing the `MediaLibrary`, the input and output are separate `GstPipeline` objects to avoid deadlocks during GStreamer state transitions.

Profile Switching

The application supports runtime profile switching via the `--switch-profile` and `--switch-delay` command-line options. After the specified delay, the `gsthailovision` element's `profile-name` property is updated, triggering a reconfiguration of the entire shared `MediaLibrary` pipeline.

12. Debugging and Visualization

This section covers debugging techniques and visualization tools for Hailo AI Analytics pipelines.

12.1. Pipeline Viewer

12.1.1. Overview

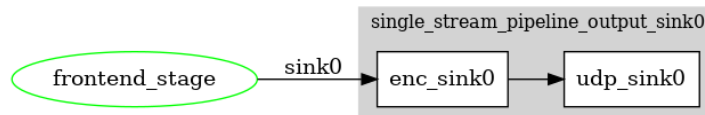


Figure 11. A simple pipeline DOT graph

Hailo AI Analytics provides built-in support for exporting pipeline graphs to DOT format, which can be visualized using various GraphViz-compatible tools. This is extremely useful for:

- Understanding pipeline topology and data flow
- Debugging connection issues between stages
- Documenting pipeline architecture
- Verifying that your pipeline is structured as intended

12.1.2. Enabling DOT Export

To automatically export your pipeline graph to a DOT file, set the following environment variable before running your application:

```
export HAILO_ANALYTICS_DUMP_DOT=1
```

By default, the DOT file will be saved to `/tmp/<pipeline_name>.dot`, where `<pipeline_name>` is the name you provided when building the pipeline.

12.1.3. Customizing Output Directory

To specify a custom output directory for the DOT files, set the `HAILO_ANALYTICS_DUMP_DOT_DIR` environment variable:

```
export HAILO_ANALYTICS_DUMP_DOT=1
export HAILO_ANALYTICS_DUMP_DOT_DIR=/path/to/output/directory
```

The pipeline graph will be saved as `/path/to/output/directory/<pipeline_name>.dot`.

12.1.4. Example Usage

```
# Enable DOT export with custom directory
export HAILO_ANALYTICS_DUMP_DOT=1
export HAILO_ANALYTICS_DUMP_DOT_DIR=$HOME/pipeline_graphs

# Run your application
./my_hailo_app

# The pipeline graph will be saved to ~/pipeline_graphs/<pipeline_name>.dot
```

12.1.5. Visualizing DOT Files

Once you have generated a DOT file, you can visualize it using several methods:

12.1.6. Interactive Viewing with xdot

xdot is an interactive viewer for DOT files that allows panning, zooming, and exploring the graph:

```
# Install xdot (if not already installed)
sudo apt-get install xdot

# View the DOT file interactively
xdot /tmp/my_pipeline.dot
```

12.1.7. Converting to PNG

You can convert the DOT file to a PNG image for easy sharing and documentation:

```
# Install GraphViz (if not already installed)
sudo apt-get install graphviz

# Convert to PNG
dot -Tpng /tmp/my_pipeline.dot -o pipeline_graph.png

# View the PNG
eog pipeline_graph.png # or any image viewer
```

12.1.8. Troubleshooting

12.1.9. DOT File Not Generated

If the DOT file is not being generated:

1. Verify that HAILO_ANALYTICS_DUMP_DOT=1 is set in your environment
2. Check that the output directory exists and is writable
3. Look for error messages in the application logs

12.1.10. Permission Denied Errors

If you get permission denied errors:

```
# Ensure the output directory is writable
mkdir -p $HAILO_ANALYTICS_DUMP_DOT_DIR
chmod 755 $HAILO_ANALYTICS_DUMP_DOT_DIR
```